Programación

None

Cristian Garcia

None

programación

1. Bienvenid@ al módulo de Programación 3
   1.1 Bolsa de proyectos 3
2. Sprint 1. Introducción a la programación 7
   2.1 Semana 1. Estructura, variables y tipos de datos 7
   2.2 Semana 2. Operadores, expresiones y casting 14
   2.3 Semana 3. Constantes, formato y salida printf 29
   2.4 Cierre del Sprint 1 42
3. Sprint 2. Utilización de objetos y clases 43
   3.1 Semana 4. POO, instanciación y memoria Stack/Heap 43
   3.2 Semana 5. Clase Math, wrappers y métodos estáticos 64
   3.3 Semana 6. Métodos String y tiempo con java.time 84
   3.4 Cierre del Sprint 2 112
4. Sprint 3. Uso de estructuras de control 113
   4.1 Semana 7. Selección: if-else, switch y try-catch 113
   4.2 Semana 8. Estructuras de repetición e iteración (bucles `while` y `do-while`) 149
   4.3 Semana 9. Bucles for, control por banderas y depuración 189

1. Bienvenid@ al módulo de Programación

El equipo de estudiantes debe elegir uno de los proyectos que se muestran a continuación.

1.1 Bolsa de proyectos

1.1.1 Ciberseguridad

1. Simulador de phishing — juego interactivo de detección de mensajes fraudulentos; clasificación de mensajes por tipología (Email, SMS, Red Social) con pistas contextuales, gestión de usuarios/sesiones, estadísticas de aciertos e historial de partidas.
2. Bóveda de contraseñas — gestor de credenciales con evaluación de fortaleza mediante políticas de seguridad (Básica, Estricta, Corporativa); control de caducidad, filtrado de cuentas guardadas y persistencia del almacén seguro.

1.1.2 Python

1. Calculadora paso a paso — evaluador de expresiones matemáticas con desglose paso a paso; tokenización, precedencia y cálculo con operadores polimórficos, interfaz con visor del proceso de resolución e histórico de cálculos.
2. Generador de datos de prueba — herramienta de síntesis de datasets sintéticos a partir de plantillas; campos polimórficos (Texto, Número, Fecha, Email), generación de lotes de registros aleatorios y exportación a ficheros CSV.

1.1.3 Videojuegos y realidad virtual

1. Aventura conversacional — motor de narrativa interactiva ramificada con toma de decisiones; gestión de escenas, inventario del jugador, interacción con diferentes tipos de personajes (Aliado, Enemigo, Neutral) y guardado del estado de la partida.
2. Simulador de físicas 2D — entorno de simulación cinemática con detección de colisiones y rebotes; escenario visual con múltiples bolas simultáneas de comportamiento polimórfico (Normal, Pesada, Rebote especial) y exportación de estadísticas de la simulación.

1.1.4 Productos software en contenedores

1. Reparto de contenedores en nodos — planificador de asignación de carga en clúster según capacidad de CPU y memoria; gestión de nodos especializados (Alta Memoria, Alta CPU), interfaz gráfica con barras de ocupación y persistencia de configuraciones de reparto.
2. Catálogo de servicios + generador de config — asistente paso a paso (wizard) para la definición y parametrización de servicios (Web, BD, Worker); generación polimórfica de ficheros de configuración, catálogo de servicios e historial de despliegues.

1.1.5 Inteligencia artificial y big data

1. Motor de recomendación — sistema de recomendación basado en afinidad entre gustos y catálogos de ítems categorizados (Película, Música, Libro); cálculo de puntuaciones de coincidencia por etiquetas, gestión de perfiles y persistencia de valoraciones.
2. Analizador de reseñas — analizador de sentimiento léxico sobre comentarios; procesamiento de palabras clave mediante diccionarios específicos según el tipo de producto, estadísticas agregadas por producto y persistencia del catálogo de reseñas.

1.1.6 Recursos y servicios en la nube

1. Cotizador cloud — presupuestador interactivo de infraestructura en la nube; cálculo polimórfico de costes según el tipo de recurso (Cómputo, Almacenamiento, Red), visor para montar y comparar presupuestos y persistencia de ofertas para clientes.

2. Dashboard de monitorización — panel de control visual para métricas y telemetría de sistemas; simulación de lecturas de sensores (CPU, Memoria, Red), series temporales, detección de umbrales con disparo de alertas e histórico persistido de incidencias.

# Proyecto (especialización)	S1 · RA1	S2 · RA3	S3 · RA2	S4 · RA4	S5 · RA6
1. Simulador de phishing (Ciberseguridad)	Clasificar 1 mensaje fijo sí/no	Menú que recorre varios mensajes, cuenta aciertos	`Random` para elegir mensaje al azar; `String` (`contains`, `toLowerCase`) para buscar palabras sospechosas; `Scanner` para la respuesta del usuario	Clases `Mensaje`, `Usuario`, `Sesión`	Lista de mensajes + estadísticas de aciertos
2. Bóveda de contraseñas (Ciberseguridad)	Evaluar longitud de 1 contraseña	Reglas de fortaleza (mayúsc./dígitos/símbolos)	`String` (`length`, `matches` con regex) para evaluar la contraseña; `LocalDate` para fecha de caducidad	Clases `Credencial`, `PoliticaSeguridad`	Lista de credenciales guardadas, filtrado
3. Calculadora paso a paso (Python)	Evaluar suma/resta simple	Precedencia de operadores, tokenizar con bucles	`StringBuilder` para construir la expresión; clases envoltorio `Integer`/`Double` (`parseInt`, `parseDouble`); `Math` (métodos estáticos) para operaciones	Clases `Expresión`, `Token`, `Operador`	Lista de pasos de resolución (histórico)
4. Generador de datos de prueba (Python)	Generar 1 dato aleatorio de lista fija	Bucle para generar N datos	`Random` para valores aleatorios; `LocalDate` para fechas; `String.format` para dar formato al dato generado	Clases `Plantilla`, `Campo`, `Registro`	Colección de plantillas y lotes generados
5. Aventura conversacional (Videojuegos)	Texto de intro + 1 decisión	Ramificación de la historia con control de flujo	`Scanner` para las decisiones del jugador; `Random` para eventos aleatorios; `String` para comparar respuestas	Clases `Personaje`, `Escena`, `Objeto`	Inventario del jugador, lista de escenas
6. Simulador de físicas 2D (VR/Videojuegos)	Mover 1 bola con variables	Bucle de simulación con	`Math` (`sqrt`, `pow`) para cálculos de	Clases `Bola`, `Escenario`	Colección de bolas simultáneas

# Proyecto (especialización)	S1 · RA1	S2 · RA3	S3 · RA2	S4 · RA4	S5 · RA6
	posición/ velocidad	colisión en bordes	velocidad/ colisión; `Random` para posición/ velocidad inicial		
7. Reparto de contenedores en nodos (Contenedores)	Comprobar si 1 contenedor cabe en 1 nodo	Bucle de reparto con reglas simples	`Math` para comprobar capacidad restante; `Random` para generar contenedores de prueba	Clases `Contenedor`, `Nodo`, `Cluster`	Listas de contenedores/ nodos, algoritmo de reparto
8. Catálogo de servicios + generador de config (Contenedores)	Pedir datos de 1 servicio	Asistente (wizard) paso a paso	`StringBuilder` para construir el fichero de configuración; `LocalDateTime` para la marca de tiempo de la config	Clases `Servicio`, `Configuración`	Catálogo de servicios + historial de configs
9. Motor de recomendación (IA/Big Data)	Mostrar 1 recomendación fija	Bucle que compara gustos con catálogo pequeño	`Random` para simular gustos iniciales; `Math` para calcular puntuación de coincidencia	Clases `Item`, `Usuario`, `Etiqueta`	Catálogo + gustos + cálculo de coincidencias
10. Analizador de reseñas (IA/ Big Data)	Contar palabras +/- en 1 texto	Bucle que analiza varias reseñas	`String` (`split`, `contains`, `toLowerCase`) para contar palabras clave en la reseña	Clases `Producto`, `Reseña`, `Autor`	Reseñas por producto + estadísticas
11. Cotizador cloud (Nube)	Coste de 1 recurso (fórmula fija)	Elegir tipo de recurso y aplicar su fórmula	`Math` y clases envoltorio (`Double`) para cálculo de coste; `String.format` para mostrar el importe	Clases `Recurso`, `Presupuesto`	Recursos de un presupuesto + lista de presupuestos
12. Dashboard de monitorización (Nube)	Mostrar 1 valor aleatorio de CPU	Serie temporal + detección de umbral	`Random` para simular lecturas de sensor; `LocalDateTime` para la marca temporal de cada medición	Clases `Sensor`, `Métrica`, `Alerta`	Histórico de métricas + lista de alertas

2. Sprint 1. Introducción a la programación

2.1 Semana 1. Estructura, variables y tipos de datos

A lo largo de este sprint desarrollaremos y evolucionaremos **un único archivo de pseudocódigo (`ControlAccesoQR.psc`) y una única clase Java (`ControlAccesoQR.java`)** para el caso guía del **IES El Caminàs**, mientras cada estudiante hace evolucionar de forma paralela el archivo único de **su proyecto elegido de la bolsa de proyectos**.

2.1.1 Día 1 - 2 sesiones

1. Caso guía en AzaharTech

Son las tres de la tarde en la sede de **AzaharTech** en Castellón de la Plana. **Laia Claramunt** conecta su portátil al proyector principal. En pantalla aparece el entorno de desarrollo IntelliJ IDEA completamente vacío:

*«Equipo, comenzamos el desarrollo del sistema de acceso para el **IES El Caminàs**; hoy abrimos el archivo oficial de la aplicación: **`ControlAccesoQR`**.*

Una aplicación profesional crece capa a capa. El equipo directivo del instituto necesita que el terminal empiece registrando los parámetros físicos del vestíbulo: el identificador del terminal que controla la pantalla y la temperatura ambiente del sensor térmico.

Hoy crearemos el esqueleto del programa, aprenderemos qué ocurre en la memoria RAM al reservar variables numéricas (`int` y `double`) y capturaremos los primeros datos desde el teclado».

2. Fundamento teórico: estructura del programa y variables numéricas en RAM

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        EL MODELO DE MEMORIA DEL TERMINAL                 │
├──────────────────┬──────────────┬──────────────────┬─────────────────────┤
│ Variable         │ Tipo en Java │ Espacio en RAM   │ Valor almacenado    │
├──────────────────┼──────────────┼──────────────────┼─────────────────────┤
│ terminalId       │ int          │ 32 bits (4 bytes)│ 101 (Número entero) │
│ tempVestibulo    │ double       │ 64 bits (8 bytes)│ 21.5 (Coma flotante)│
└──────────────────┴──────────────┴──────────────────┴─────────────────────┘
```

1. **La clase como contenedor maestro.** En Java, todo código pertenece a una clase (`public class ControlAccesoQR`). El archivo en disco debe llamarse exactamente igual: `ControlAccesoQR.java`.
2. **El método de entrada (`main`).** La JVM busca la instrucción `public static void main(String[] args)` para iniciar la ejecución secuencial de arriba hacia abajo.
3. **Variables numéricas primitivas.**

- **`int`:** Almacena números enteros sin decimales (de -2.147 a $+2.147$ millones).
- **`double`:** Almacena números reales con precisión decimal de 64 bits.

4. **Captura con `Scanner`.** Creamos un canal de lectura interactivo (`Scanner teclado = new Scanner(System.in);`) que captura enteros con `nextInt()` y decimales con `nextDouble()`.

3. Algorítmica y traducción a Java de la app `ControlAccesoQR v0.1`

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.1)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica:        ", tempVestibulo, " ºC"
FinAlgoritmo
```

PASO B. TRADUCCIÓN A JAVA (`pr/src/ControlAccesoQR.java` — V0.1)

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica:       " + tempVestibulo + " ºC");

        teclado.close();
    }
}
```

---

4. Trabajo del estudiante: nacimiento de su proyecto propio

Cada estudiante crea en su carpeta `pr/pseudocodigo/` el archivo maestro de su proyecto propio y en `pr/src/` la clase `.java` v0.1:

- Declara dos variables numéricas (`int` y `double`) a su proyecto:
- En Aventura conversacional: puntos de vida (int puntosVida) y multiplicador de daño (double factorDano).
- En Motor de recomendación: identificador de ítem (int idItem) y valoración media (double valoracionMedia).
- En Simulador de físicas 2D: número de colisiones (int totalColisiones) y masa del cuerpo (double masaKg).
- En Bóveda de contraseñas: días para la caducidad (int diasCaducidad) y porcentaje de fortaleza (double nivelSeguridad).»
- Compila y verifica la ejecución en la consola de IntelliJ.

2.1.2 Día 2 - 2 sesiones

---

1. Caso guía en AzaharTech

Es martes por la tarde. **Pau Ferrer** ejecuta `ControlAccesoQR` v0.1 y muestra la pantalla:

«El terminal ya arranca y guarda el número de terminal `101` y la temperatura `21.5`. Pero cuando una persona acerca el móvil a la pantalla del vestíbulo, el sistema no sabe a quién pertenece ese escaneo».

Alba Torres toma el teclado y abre el archivo de ayer:

«No vamos a crear un programa nuevo. Vamos a evolucionar `ControlAccesoQR.java`. Añadiremos los campos para el nombre de la persona, su DNI, el tipo de relación con el instituto (estudiante, docente, visita) y bandera lógica que indique si está entrando o saliendo.

Pero atención a la trampa de Java: al leer números antes que textos, el buffer del teclado guarda un salto de línea invisible (`\n`) que debemos limpiar para que el programa no se salte la lectura del nombre».

---

2. Fundamento teórico: caracteres, cadenas y el buffer del Scanner

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      EL PROBLEMA DEL BUFFER RESIDUAL EN SCANNER          │
├──────────────────────────────────────────────────────────────────────────┤
│ Usuario teclea: [ 1 ][ 0 ][ 1 ][ \n (Enter) ]                            │
│ nextInt() lee:  [ 1 ][ 0 ][ 1 ]          ──► Deja el [ \n ] flotando en el buffer.  │
│ nextLine() lee: [ \n ]                   ──► Cree que el usuario pulsó Enter vacío. │
│                                                                          │
│ SOLUCIÓN:       teclado.nextLine();      ──► Limpia el buffer antes del texto real. │
└──────────────────────────────────────────────────────────────────────────┘
```

1. `char` **frente a** `String`**:**

- `char letraGrupo = 'B';`: Un único carácter delimitado obligatoriamente por comillas simples (`' '`).
- `String nombre = "Juan Pérez";`: Objeto que gestiona texto de cualquier longitud entre comillas dobles (`" "`).

2. `boolean` **(Estado del sistema):** Almacena `true` o `false`. Ideal para variables de control como `matriculaActiva`.
3. **Limpieza del buffer:** Siempre que se invoque `nextInt()` o `nextDouble()` y la siguiente instrucción sea `nextLine()`, se debe intercalar una llamada a `teclado.nextLine();` para vaciar el salto de línea residual.

---

3. Evolución a `ControlAccesoQR v0.2`

Observa cómo el código de ayer se amplía añadiendo los bloques de identidad sin eliminar lo anterior.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.2)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " ºC"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso:  Entrada (", esEntrada, ")"
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.2)

Abrimos el archivo `ControlAccesoQR.java` en IntelliJ y lo modificamos directamente:

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();
        teclado.nextLine();
```

```
        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita):");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " ºC");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");

        teclado.close();
    }
}
```

4. Trabajo del estudiante: evolución de su proyecto propio

Cada estudiante abre sus archivos `.psc` y `.java`:

- Añade los campos alfanuméricos de su proyecto:
- En Aventura conversacional: nombre del héroe (String), clase de personaje 'G' de Guerrero, 'M' de Mago, 'P' de Pícaro (char) y si la partida está activa (boolean estaVivo).
- En Motor de recomendación: título de la película/libro (String), tipo de contenido 'P' de Película, 'M' de Música, 'L' de Libro (char) y si está marcado como favorito (boolean esFavorito).
- En Simulador de físicas 2D: etiqueta del cuerpo (String), tipo de partícula 'N' de Normal, 'P' de Pesada, 'R' de Rebote especial (char) y si tiene la gravedad activada (boolean gravedadActiva).
- En Bóveda de contraseñas: nombre del servicio o web (String), nivel de política 'B' de Básica, 'E' de Estricta, 'C' de Corporativa (char) y si requiere doble factor (boolean requiere2FA).»
- Aplica la limpieza del buffer con `teclado.nextLine()`.
- Ejecuta pruebas verificando que se pueden introducir nombres con espacios sin saltos inesperados.

2.1.3 Día 3 - 2 sesiones

1. Caso guía en AzaharTech

Es miércoles por la tarde. **Laia Claramunt** revisa la versión v0.2:

«El sistema ya sabe qué terminal lee, la temperatura del vestíbulo y quién pasa. Ahora el IES El Caminàs nos pide procesar el tiempo que la persona permanece en el instituto: para cada acceso debemos calcular los minutos que hay entre la hora de entrada y la hora de salida.

Hoy aprenderemos a operar matemáticamente sobre las variables de nuestro programa y a componer un mensaje unificado donde convivan números y texto sin que el operador `+` distorsione los cálculos».

2. Fundamento teórico: aritmética y concatenación segura

1. **La sobrecarga del operador `+`:**

- Si ambos operandos son numéricos: realiza una **suma matemática** (`50 + 50 = 100`).
- Si al menos un operando es una cadena de texto: realiza una **concatenación** (unión de textos).

2. **Evaluación de izquierda a derecha:**

```
System.out.println("Minutos: " + 50 + 50);   // Imprime "Minutos: 5050" (¡ERROR!)
System.out.println("Minutos: " + (50 + 50)); // Imprime "Minutos: 100"  (CORRECTO)
```

3. **Multiplicación y prioridad:** La multiplicación (`*`) se evalúa antes que la suma (`+`), a menos que usemos paréntesis `()` .

---

3. Evolución a `ControlAccesoQR v0.3`

Ampliamos el código de la versión v0.2 añadiendo el bloque de cálculo de sesiones lectivas y la composición del mensaje de confirmación.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.3)

```
Algoritmo ControlAccesoQR
      Definir terminalId Como Entero
      Definir tempVestibulo Como Real

      Definir nombrePersona, dniPersona Como Cadena
      Definir perfilPersona Como Caracter
      Definir esEntrada Como Logico

      Definir horaEntrada, minutoEntrada Como Entero
      Definir horaSalida, minutoSalida Como Entero
      Definir minutosTotalesEntrada Como Entero
      Definir minutosTotalesSalida Como Entero
      Definir minutosEstanciaTotal Como Entero
      Definir tokenResumen Como Cadena

      Escribir "ID del terminal:"
      Leer terminalId
      Escribir "Temperatura del sensor (ºC):"
      Leer tempVestibulo

      Escribir "DNI de la persona:"
      Leer dniPersona
      Escribir "Nombre completo:"
      Leer nombrePersona
      Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
      Leer perfilPersona

      esEntrada <- Verdadero

      Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
      Leer horaEntrada
      Leer minutoEntrada

      Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
      Leer horaSalida
      Leer minutoSalida

      minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
      minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
      minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

      tokenResumen <- dniPersona + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)

      Escribir "Terminal configurado: #", terminalId
      Escribir "Lectura térmica: ", tempVestibulo, " ºC"
      Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
      Escribir "Perfil: ", perfilPersona
      Escribir "Sentido del paso:  Entrada (", esEntrada, ")"
      Escribir "Token:      ", tokenResumen
      Escribir "Horario:    Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
      Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.3)

Abrimos nuestro archivo `ControlAccesoQR.java` y lo evolucionamos:

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada;
```

```
        int minutoEntrada;
        int horaSalida;
        int minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();
        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita):");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        tokenResumen = dniPersona + "-ESTANCIA-" + minutosEstanciaTotal;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " ºC");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");
        System.out.println("Token: " + tokenResumen);
        System.out.println("Horario:    Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");

        teclado.close();
    }
}
```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java`:

- Incorpora el cálculo aritmético secuencial adaptado a su proyecto.
- En Aventura conversacional: calcular el daño total (danoBase * factorDano) y restar la vida restante (puntosVida - danoTotal).
- En Motor de recomendación: sumar puntuaciones por afinidad (afinidadGenero + afinidadDirector) y calcular la diferencia respecto a la media de la comunidad (totalAfinidad - mediaComunidad).
- En Simulador de físicas 2D: calcular el desplazamiento lineal restando la posición inicial a la posición final calculada (posFinal - posInicial).
- En Bóveda de contraseñas: calcular los días de vigencia restantes restando los días transcurridos al límite de caducidad (diasCaducidad - diasTranscurridos).»
- Utiliza paréntesis `()` para proteger una operación dentro de los mensajes de salida.
- Compila y verifica que la concatenación no genera errores numéricos.

2.1.4 Día 4 - 2 sesiones

1. Caso guía en AzaharTech

Es jueves por la tarde. Concluyen las primeras 8 horas de Programación. **Laia Claramunt** revisa la versión v0.3 en el repositorio:

*«Fijaos en lo que habéis logrado en solo cuatro días: tenemos **una aplicación viva que ya sabe capturar datos del hardware, identificar al usuario y calcular tiempos de estancia en el centro educativo**.*

*Hoy cada uno de vosotros va a dedicar estas dos horas a auditar, limpiar y documentar la versión v0.3 de su **proyecto propio de la bolsa de proyectos**. Aplicaremos la indentación oficial, cerraremos recursos y realizaremos el commit formal en GitHub».*

---

2. Estándares de calidad de código en AzaharTech

1. **Autoformateo en IntelliJ.** Todo código debe pasar por el formateador automático del IDE pulsando ** `Ctrl + Alt + L` ** (en GNU/Linux). La indentación debe ser homogénea de 4 espacios.
2. **Nombres descriptivos.** Las variables deben reflejar su propósito (`minutosTotalesLectivos` , no `mtl`).
3. **Cierre de recursos.** Toda clase que utilice `Scanner` debe cerrarlo explícitamente con `.close()` al final del `main` .

---

3. La versión v0.3 del proyecto propio del estudiante

Cada estudiante verifica que su archivo único `pr/src/MiProyecto.java` cumple con el nivel de consolidación alcanzado en el caso guía:

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada;
        int minutoEntrada;
        int horaSalida;
        int minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL DE ACCESO VESTÍBULO     ");
        System.out.println("   Cliente: IES El Caminàs (Curso 2026/2027)     ");
        System.out.println("================================================");
        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();
        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();
```

```
        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;

        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        tokenResumen = dniPersona + "-ESTANCIA-" + minutosEstanciaTotal;

        System.out.println("-------------------------------------------");
        System.out.println("Terminal configurado:        #" + terminalId);
        System.out.println("Lectura térmica:              " + tempVestibulo + " ºC");
        System.out.println("Persona:                     " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil:                      " + perfilPersona);
        System.out.println("Sentido del paso:            Entrada (" + esEntrada + ")");
        System.out.println("Token:                       " + tokenResumen);
        System.out.println("Horario:                     Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");

        teclado.close();
    }
}
```

---

4. (Hacerlo únicamente si el grupo de estudiantes ya ha sincronizado con GitHub en ED) Cierre formal en git y sincronización con GitHub

El estudiante confirma los avances de la primera semana en su repositorio:

1. Pulsa el atajo `Ctrl + K` (o haz clic en el icono verde de verificación **Commit** en la barra lateral izquierda).
2. En el panel de Commit, marca las casillas de los archivos modificados dentro de `pr/` (`MiProyecto.java` y `MiProyecto.psc`).
3. En la caja de texto para el mensaje, escribe siguiendo el estándar convencional:
   `feat(pr): consolidar version v0.3 con captura de datos, calculos aritmeticos y limpieza de codigo`
4. Despliega el botón azul inferior y selecciona **«Commit and Push»**.
5. En la ventana de confirmación que aparece, pulsa **Push** para enviar los cambios a GitHub.

2.2 Semana 2. Operadores, expresiones y casting

Continuamos trabajando sobre el **mismo archivo maestro** que dejamos al final de la Semana 1. Tomamos la versión `ControlAccesoQR v0.3` y la evolucionamos progresivamente hasta la versión `v0.6` integrando operadores compuestos, descomposición con módulo (`%`) y conversiones de tipo (*casting*), mientras cada estudiante replica esta misma evolución en la clase única de **su proyecto elegido de la bolsa de proyectos**.

2.2.1 Día 5 - 2 sesiones

---

1. Caso guía en AzaharTech

Es lunes por la tarde. En la sala técnica de **AzaharTech**, **Pau Ferrer** abre el archivo `ControlAccesoQR.java` tal y como quedó el jueves anterior (versión v0.3). Quiere añadir un contador para saber cuántos fichajes procesa el terminal a lo largo de la mañana y ha escrito:

```
personasEnCentro =personasEnCentro +1;
aforoDisponible =aforoDisponible -1;
idUltimoFichaje =idUltimoFichaje +1;
tokenResumen =tokenResumen +"-REG"+idUltimoFichaje;
```

Alba Torres se acerca a su monitor, señala la pantalla y le explica:

*«Pau, en un código profesional no duplicamos el nombre de la variable a ambos lados del signo igual. Para acumular valores o contar eventos utilizamos **operadores de asignación compuesta (`+=`)** y el **operador de incremento (`++`)**.*

Hacen que el código sea más compacto, reducen el riesgo de erratas al escribir nombres largos y permiten al compilador generar un código intermedio más optimizado.

*Hoy abriremos nuestro archivo `ControlAccesoQR` y lo refactorizaremos a la versión **v0.4**: sustituiremos las asignaciones largas por operadores compuestos y aprenderemos a distinguir el pre-incremento del post-incremento para evitar efectos secundarios en la memoria».*

---

2. Fundamento teórico: asignación compuesta, incremento y mutabilidad de memoria

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                      OPERADORES DE ASIGNACIÓN COMPUESTA EN JAVA              │
├──────────────────────┬──────────────────────────┬────────────────────────────┤
│ Expresión compacta   │ Expresión equivalente    │ Acción sobre la celda de memoria RAM │
├──────────────────────┼──────────────────────────┼────────────────────────────┤
│ total += valor;      │ total = total + valor;   │ Suma 'valor' al dato actual de total │
├──────────────────────┼──────────────────────────┼────────────────────────────┤
│ texto += " nuevo";   │ texto = texto + valor;   │ Concatena 'texto' al final de cadena │
├──────────────────────┼──────────────────────────┼────────────────────────────┤
│ tiempo -= pausa;     │ tiempo = tiempo - pausa; │ Resta 'pausa' al dato actual │
├──────────────────────┼──────────────────────────┼────────────────────────────┤
│ aforo *= factor;     │ aforo = aforo * factor;  │ Multiplica el contenido por 'factor' │
├──────────────────────┼──────────────────────────┼────────────────────────────┤
│ cupo /= divisor;     │ cupo = cupo / divisor;   │ Divide el contenido entre 'divisor' │
└──────────────────────┴──────────────────────────┴────────────────────────────┘
```

A. OPERADORES UNARIOS DE INCREMENTO (`++`) Y DECREMENTO (`--`)

Aumentan o disminuyen el valor de una variable entera exactamente en una unidad:

- **Post-incremento (`variable++`).** El valor actual de la variable se utiliza en la expresión donde se encuentra y, **justo después de ser leído**, la variable se incrementa en 1 en la memoria RAM.
- **Pre-incremento (`++variable`).** La variable se incrementa en 1 en la memoria RAM **antes** de que su valor sea leído o utilizado en la expresión circundante.

```
// Ejemplo de análisis de memoria en AzaharTech:
int accesos = 10;
System.out.

println(accesos++); // Imprime 10 en consola. En memoria RAM pasa a valer 11.
System.out.

println(accesos);   // Imprime 11.
System.out.

println(++accesos); // En memoria RAM sube a 12 y luego imprime 12.
```

---

3. Refactorización a `ControlAccesoQR v0.4`

Tomamos el archivo único de la Semana 1 y lo refactorizamos incorporando el contador global del terminal y asignaciones compuestas.

PASO A. ACTUALIZACIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.4)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena
```

```
    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 120
    aforoDisponible <- 80
    idUltimoFichaje <- 1042

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " ºC"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso:  Entrada (", esEntrada, ")"
    Escribir "Token:      ", tokenResumen
    Escribir "Horario:    Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
    Escribir "Estado actual del aforo:"
    Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
    Escribir "- plazas libres: ", aforoDisponible, " (-1)"
FinAlgoritmo
```

PASO B. REFACTORIZACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.4)

Abrimos nuestro archivo maestro `ControlAccesoQR.java` y aplicamos directamente la refactorización:

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 120;
        int aforoDisponible = 80;
        int idUltimoFichaje = 1042;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();
```

```
        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " ºC");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");
        System.out.println("Token:      " + tokenResumen);
        System.out.println("Horario:    Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
        System.out.println("Estado actual del aforo:");
        System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
        System.out.println("- plazas libres: " + aforoDisponible + " (-1)");

        teclado.close();
    }
}
```

---

4. Trabajo del estudiante: refactorización de su proyecto propio

El estudiante abre su archivo `.java` (que dejó en v0.3 el jueves anterior):

- Aplica los operadores unarios de incremento (`++`) y decremento (`--`) sobre contadores de eventos o estados numéricos de su sistema:
- En **Aventura conversacional**: incrementar el contador de turnos de la escena y decrementar los puntos de energía o pociones disponibles.
- En **Motor de recomendación**: incrementar el contador de recomendaciones emitidas y decrementar las consultas gratuitas restantes del perfil.
- En **Simulador de físicas 2D**: incrementar el contador de impactos o colisiones detectadas y decrementar las partículas activas en pantalla.
- En **Bóveda de contraseñas**: incrementar el identificador correlativo de la nueva credencial y decrementar los intentos de autenticación permitidos.
- Utiliza el operador de asignación compuesta (`+=`) para construir paso a paso la cadena de texto identificativa (token, resumen o registro de auditoría):
- En **Aventura conversacional**: ir componiendo el registro de la partida sobre la misma variable.
- En **Motor de recomendación**: construir el código de afinidad concatenando partes.
- En **Simulador de físicas 2D**: acumular los datos en la traza de telemetría.
- En **Bóveda de contraseñas**: ensamblar el registro de auditoría de la credencial.
- Compila y ejecuta en IntelliJ comprobando que los contadores actualizan su valor en memoria y que la cadena compuesta con `+=` se muestra completa en la consola.

2.2.2 Día 6 - 2 sesiones

1. Caso guía en AzaharTech

Es martes por la tarde. En la pantalla de control, **Pau Ferrer** muestra que el reloj interno del vestíbulo del IES El Caminàs devuelve el tiempo de actividad del terminal en **segundos brutos acumulados**: `7540` segundos.

Pau comenta:

«En la pantalla no podemos mostrarle al conserje que el terminal lleva '7540 segundos encendido'. El cliente necesita ver cuántas horas completas, minutos sobrantes y segundos exactos representa ese número.

He intentado dividir `7540 / 60`, pero me da 125 minutos y no sé cómo extraer las horas y los segundos sin escribir un algoritmo larguísimo con restas».

Alba Torres toma el mando en la pizarra:

*«No necesitas restas ni condicionales. La combinación de la **división entera (`/`)** y el **operador módulo o resto (`%`)** resuelve este problema en tres líneas de código puramente secuenciales.*

*Hoy abriremos `ControlAccesoQR.java` y lo evolucionaremos a la versión **v0.5**: convertiremos nuestro programa en un motor capaz de descomponer cualquier magnitud temporal de forma exacta».*

2. La división entera frente al operador residuo (`%`)

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                      DESCOMPOSICIÓN MATEMÁTICA DE MAGNITUDES                 │
├──────────────────────────────────────────────────────────────────────────────┤
│ Total: 7540 segundos                                                         │
│ 1. Horas completas:        7540 / 3600 = 2 horas (División entera descarta decimales)  │
│ 2. Segundos restantes:     7540 % 3600 = 340 segundos sobrantes que no llegan a 1 hora │
│ 3. Minutos de ese resto:   340 / 60    = 5 minutos                           │
│ 4. Segundos finales:       7540 % 60   = 40 segundos restantes               │
│                                                                              │
│ RESULTADO EXACTO:          2 horas, 5 minutos y 40 segundos.                 │
└──────────────────────────────────────────────────────────────────────────────┘
```

1. **La división entre enteros en Java (`int / int`):**
   - Realiza un truncamiento automático hacia cero, descartando cualquier parte fraccionaria.
   - `7540 / 3600` da exactamente `2`.
2. **El operador módulo (`%`):**
   - Devuelve el residuo que no ha podido ser absorbido por la división entera.
   - `7540 % 60` da exactamente `40` (porque $60 \times 125 = 7500$; sobran 40).
3. **Poder algorítmico secuencial:** Permite desglosar monedas, tiempos, paquetes o ciclos de turnos sin requerir ninguna estructura condicional `if`.

3. Evolución a `ControlAccesoQR v0.5`

Ampliamos el archivo maestro `ControlAccesoQR` incorporando la lectura de segundos de actividad del terminal y su descomposición horaria.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.5)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
```

```
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 120
    aforoDisponible <- 80
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / 3600)
    minutosUptime <- trunc((segundosActividadTerminal MOD 3600) / 60)
    segundosUptime <- segundosActividadTerminal MOD 60

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " ºC"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso:  Entrada (", esEntrada, ")"
    Escribir "Token:      ", tokenResumen
    Escribir "Horario:    Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
    Escribir "Estado actual del aforo:"
    Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
    Escribir "- plazas libres: ", aforoDisponible, " (-1)"
    Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.5)

Abrimos nuestro archivo `ControlAccesoQR.java` y lo evolucionamos a v0.5:

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 120;
```

```
        int aforoDisponible = 80;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / 3600;
        minutosUptime = (segundosActividadTerminal % 3600) / 60;
        segundosUptime = segundosActividadTerminal % 60;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " ºC");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");
        System.out.println("Token:      " + tokenResumen);
        System.out.println("Horario:    Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
        System.out.println("Estado actual del aforo:");
        System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
        System.out.println("- plazas libres: " + aforoDisponible + " (-1)");
        System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");

        teclado.close();
    }
}
```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java` (que estaba en v0.4):

- Aplica la división entera `/` y el operador módulo `%` para descomponer una magnitud adaptada a su proyecto:
- En **Aventura conversacional**: descomponer el tiempo total de travesía en días completos y horas restantes, o las monedas totales en monedas de plata y cobre sobrante.
- En **Motor de recomendación**: descomponer la duración total de un contenido multimedia en horas completas y minutos restantes.
- En **Simulador de físicas 2D**: descomponer el tiempo de simulación en segundos completos y milisegundos restantes, o fotogramas en segundos y frames sueltos.
- En **Bóveda de contraseñas**: descomponer los días de vigencia restantes de una clave en semanas completas y días sueltos.
- Muestra el desglose por consola y comprueba con la calculadora que la reconstrucción matemática es perfecta.

2.2.3 Día 7 - 2 sesiones

---

1. Caso guía en AzaharTech

Es miércoles por la tarde. En el laboratorio de pruebas, **Pau Ferrer** muestra dos problemas que ha intentado resolver en `ControlAccesoQR`:

1. **El fallo del porcentaje de ocupación.** Ha intentado calcular el porcentaje de aforo del recinto (que cuenta con un aforo máximo de 2000 personas). Al dividir los alumnos presentes entre el aforo (`1201 / 2000 * 100`), la consola imprime:

```
Porcentaje de ocupación: 0.0 %
```

2. **El fallo de la comprobación de pérdida de evaluación continua.** Para verificar si un alumno acumula el límite de 15 faltas injustificadas y debe activarse la pérdida de evaluación continua en el sistema, Pau ha escrito:

```
int faltas = 5;
boolean perdidaEvaluacionContinua = (faltas = 15);
```

Y el compilador ha detenido el programa con un error: «*incompatible types: int cannot be converted to boolean*».

Pau se lleva las manos a la cabeza:

«Por un lado, con más de 1200 personas de 2000 plazas Java dice cero coma cero. Y por otro, solo quería comprobar si el alumno alcanzaba las quince faltas para marcar la pérdida de evaluación continua y me da error de tipos incompatibles».

Alba Torres y **Laia Claramunt** se acercan a la pantalla. Laia explica:

«Has caído en dos trampas clásicas de las expresiones en Java:

*Primero, has dividido dos variables enteras (`int / int`), por lo que Java trunca los decimales a cero antes de multiplicar. Para solucionarlo debemos aplicar un **casting explícito** `(double)`.*

*Segundo, has usado el operador de asignación (`=`) en lugar del **operador de igualdad (`==`)**. En Java, una asignación devuelve el propio número entero asignado (`15`), y un entero jamás puede meterse dentro de un booleano.*

*Además, cuando combinemos condiciones con los operadores lógicos `&&` **(AND)** y `||` **(OR)**, debemos prestar mucha atención a la **evaluación en cortocircuito**: Java se salta la segunda parte de la expresión si la primera ya define el resultado, lo que puede congelar incrementos (`++`) si no tenemos cuidado.*

*Hoy evolucionaremos `ControlAccesoQR` a la versión **v0.6**: aprenderemos la jerarquía completa de operadores, dominaremos el casting y evaluaremos condiciones lógicas puras de forma secuencial sin usar ningún `if`».*

---

2. Precedencia de operadores, lógica booleana y *casting* en Java

Prioridad	Operadores	Dirección de evaluación
1. Paréntesis	( )	De dentro hacia afuera
2. Unarios	++ , -- , + , - , (tipo) [Casting]	De derecha a izquierda
3. Aritmética	* , / , %	De izquierda a derecha
4. Aritmética	+ , -	De izquierda a derecha
5. Relacional	< , > , <= , >=	De izquierda a derecha
6. Igualdad	== (igual que) , != (distinto de)	De izquierda a derecha
7. Lógico AND	&& (Evaluación en cortocircuito)	De izquierda a derecha
8. Lógico OR	|| (Evaluación en cortocircuito)	De izquierda a derecha
9. Asignación	= , += , -= , *= , /= , %=	De derecha a izquierda

A. LA TRAMPA CLÁSICA DE JAVA: ASIGNACIÓN (`=`) FRENTE A IGUALDAD (`==`)

En Java debemos distinguir con absoluta precisión dos operadores que se parecen visualmente pero hacen cosas radicalmente distintas:

1. **El operador de asignación (`=`):**
   Guarda un valor en una variable y devuelve ese mismo valor.

- `int x = 5;` -> Guarda 5 en x.
- La expresión `(x = 10)` modifica `x` y devuelve el número entero `10`.

2. **El operador de comparación de igualdad (`==`):**
   Compara dos valores y devuelve un resultado estrictamente lógico (`true` o `false`).

- `boolean sonIguales = (x == 10);` -> Evalúa si x es igual a 10 y guarda `false`.

Peligro en expresiones booleanas:
Java es un lenguaje de tipado estricto. Un número entero jamás puede convertirse automáticamente en un booleano:

```
int faltas = 5;

// ERROR DE COMPILACIÓN:
// (faltas = 15) asigna 15 y devuelve el número entero 15.
// Un int no se puede meter dentro de una variable boolean.
boolean perdidaEvaluacionContinua = (faltas = 15); // Incompatible types: int cannot be converted to boolean

// CORRECTO (Uso del doble igual == para comparar):
boolean perdidaEvaluacionContinuaCorrecto = (faltas == 15); // Compila y guarda 'false'
```

B. OPERADORES LÓGICOS Y LA EVALUACIÓN EN CORTOCIRCUITO (*SHORT-CIRCUIT*)

Los operadores lógicos nos permiten combinar varias condiciones para obtener un resultado booleano: * `&&` **(AND / Y lógico):** Devuelve `true` si y solo si **ambas** condiciones son verdaderas. * `||` **(OR / O lógico):** Devuelve `true` si **al menos una** de las condiciones es verdadera.

El mecanismo del cortocircuito:
Java evalúa las expresiones lógicas de izquierda a derecha y aplica evaluación perezosa para ahorrar tiempo de proceso: 1. En un `&&` **(AND)**: Si el término de la izquierda es `false`, la expresión completa será necesariamente falsa. Por tanto, **Java se salta por completo la evaluación del término de la derecha**. 2. En un `||` **(OR)**: Si el término de la izquierda es `true`, la expresión completa será necesariamente verdadera. Por tanto, tampoco evalúa la parte derecha.

C. LAS CONVERSIONES DE TIPO EN LA MEMORIA RAM

1. **Conversión implícita (ensanchamiento / *widening*):**

- Ocurre de forma automática y segura cuando un tipo de menor tamaño se almacena en uno mayor (`int` $\rightarrow$ `double`). No hay riesgo de pérdida de información.

2. **Conversión explícita (estrechamiento / *narrowing* / *casting*):**

- Es obligatoria cuando el programador fuerza la conversión de un tipo de mayor precisión en uno menor (`double` $\rightarrow$ `int`).
- **Se produce truncamiento:** Los decimales se eliminan por completo (no se redondean).
- Se antepone el tipo destino entre paréntesis:

```
double porcentaje = 60.05;
int porcentajeEntero = (int) porcentaje; // Almacena 60 (pierde el .05)
```

D. LA SOLUCIÓN TÉCNICA MEDIANTE CASTING EN DIVISIONES

Para calcular el porcentaje sin perder decimales en la división entera, forzamos que al menos una variable sea tratada como decimal:

```
double porcentajeOcupacion = ((double) personasEnCentro / aforoTotal) * 100.0;
```

Al aplicar `(double)` sobre `personasEnCentro`, la división pasa a ser de tipo `double / int`, lo que promociona toda la operación a coma flotante y devuelve el **`60.05 %`** exacto.

---

3. Evolución a `ControlAccesoQR v0.6`

Ampliamos el archivo incorporando el aforo total de 2000 plazas, el cálculo de porcentaje exacto con casting, el truncamiento a valor entero y la evaluación de expresiones booleanas con igualdad (`==`) y cortocircuito (`&&`).

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.6)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Definir esJornadaCompletaExacta Como Logico
    Definir alertaAforoExcedido Como Logico
    Definir contadorAlarmas Como Entero
    contadorAlarmas <- 0

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
```

```
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / 3600)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / 60)
    segundosUptime <- segundosActividadTerminal MOD 60

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * 100.0) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    esJornadaCompletaExacta <- (minutosEstanciaTotal == 360)
    alertaAforoExcedido <- (personasEnCentro > aforoTotal)

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " ºC"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso:  Entrada (", esEntrada, ")"
    Escribir "Token:      ", tokenResumen
    Escribir "Horario:    Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
    Escribir "Jornada exacta de 6h (operador ==): ", esJornadaCompletaExacta
    Escribir "Diagnóstico del terminal (uptime):"
    Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
    Escribir "Estado del aforo (capacidad: ", aforoTotal, " plazas):"
    Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
    Escribir "- plazas libres: ", aforoDisponible, " (-1)"
    Escribir "- ocupación exacta: ", porcentajeOcupacionReal, " %"
    Escribir "- ocupación en panel: ", porcentajeOcupacionEntero, " %"
    Escribir "- alerta sobreaforo: ", alertaAforoExcedido
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.6)

Abrimos nuestro archivo maestro `ControlAccesoQR.java` y lo evolucionamos a v0.6:

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        boolean esJornadaCompletaExacta;
        boolean alertaSobreaforo;
        int contadorAlarmas = 0;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();
```

```
        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / 3600;
        minutosUptime = (segundosActividadTerminal % 3600) / 60;
        segundosUptime = segundosActividadTerminal % 60;

        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * 100.0;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        esJornadaCompletaExacta = (minutosEstanciaTotal == 360);

        alertaSobreaforo = (personasEnCentro > aforoTotal) && (++contadorAlarmas >= 3);

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " ºC");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");
        System.out.println("Token:      " + tokenResumen);
        System.out.println("Horario:    Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
        System.out.println("Jornada exacta de 6h (operador ==): " + esJornadaCompletaExacta);
        System.out.println("Diagnóstico del terminal (uptime):");
        System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");
        System.out.println("Estado del aforo (capacidad: " + aforoTotal + " plazas):");
        System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
        System.out.println("- plazas libres: " + aforoDisponible + " (-1)");
        System.out.println("- ocupación exacta: " + porcentajeOcupacionReal + " %");
        System.out.println("- ocupación en panel: " + porcentajeOcupacionEntero + " %");
        System.out.println("- alerta sobreaforo: " + alertaSobreaforo + " (alarmas disparadas: " + contadorAlarmas + ")");

        teclado.close();
    }
}
```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java` (en versión v0.5):

- **Cálculo porcentual y casting:** Incorpora el cálculo de un ratio o porcentaje que divida dos variables enteras adaptadas a su proyecto, aplicando casting explícito `(double)` y casting inverso a entero `(int)`:
- En **Aventura conversacional**: calcular el porcentaje exacto de salud restante respecto al total máximo `((double) puntosVida / vidaMaxima) * 100.0` y truncar a `(int)`.
- En **Motor de recomendación**: calcular el porcentaje de afinidad sobre la puntuación máxima teórica `((double) puntosAfinidad / afinidadMaxima) * 100.0` y truncar a `(int)`.
- En **Simulador de físicas 2D**: calcular el porcentaje de energía cinética conservada tras el rebote `((double) velocidadFinal / velocidadInicial) * 100.0` y truncar a `(int)`.
- En **Bóveda de contraseñas**: calcular el porcentaje de vigencia consumido de la credencial respecto a su vida útil total `((double) diasConsumidos / diasCaducidadTotal) * 100.0` y extraer la parte entera con `(int)`.
- **Expresiones booleanas y operador `==`:** Evalúa una condición de igualdad pura sin usar `if` asignando el resultado directamente a una variable `boolean` de su sistema:
- En **Aventura conversacional**: evaluar si el héroe está ileso (`boolean estaIleso = (puntosVida == vidaMaxima);`).
- En **Motor de recomendación**: evaluar si la afinidad es perfecta (`boolean afinidadPlena = (puntosAfinidad == afinidadMaxima);`).
- En **Simulador de físicas 2D**: evaluar si la velocidad ha llegado al reposo (`boolean enReposo = (velocidadFinal == 0.0);`).
- En **Bóveda de contraseñas**: evaluar si la clave ha caducado exactamente hoy (`boolean caducaHoy = (diasConsumidos == diasCaducidadTotal);`).
- Comprueba con la calculadora que la división decimal no trunca a cero y que la parte entera descarta los decimales correctamente.

2.2.4 Día 8 - 2 sesiones

---

1. Caso guía en AzaharTech

Es jueves por la tarde. Concluyen las primeras dos semanas del sprint. **Laia Claramunt** revisa en la pantalla de la sala el avance global:

«Hoy cerramos formalmente la Semana 2. En la Semana 1 creamos el esqueleto y la identidad de nuestro software. Durante estos últimos cuatro días habéis convertido el programa en un auténtico motor matemático: dominái los operadores compuestos (`+=`), la descomposición con módulo (`%`) y las conversiones de tipo con casting.

*Hoy dedicaremos estas dos horas a auditar y afianzar la versión **v0.6 de vuestro proyecto propio de la bolsa de proyectos**. No debe quedar ni un solo cálculo ambiguo ni una división entera accidental.*

Al sonar el timbre, la versión v0.6 de vuestra aplicación debe estar confirmada y subida a GitHub».

---

2. Lista de comprobación técnico de calidad del código para la versión v0.6

Antes de realizar el commit, cada estudiante debe verificar los siguientes 5 puntos en IntelliJ:

1. **Ejecución secuencial.** El código se ejecuta de forma estrictamente secuencial de principio a fin.
2. **Casting explícito justificado.** Al menos una división entre enteros cuenta con `(double)` para preservar la precisión decimal.
3. **Uso del operador módulo (`%`).** Al menos una magnitud se descompone o calcula mediante el residuo de una división.
4. **Protección con paréntesis `()`.** Las fórmulas complejas están agrupadas con paréntesis para garantizar la precedencia matemática sin ambigüedades.
5. **Indentación automática.** Se ha pulsado `Ctrl + Alt + L` para ordenar el código según el estándar de estilo oficial.

---

3. La versión v0.6 del proyecto propio del estudiante

Cada estudiante comprueba que su clase única `pr/src/MiProyecto.java` ha evolucionado de forma acumulativa e incremental tanto en pseudocódigo como en Java:

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / 3600)
    minutosUptime <- trunc((segundosActividadTerminal MOD 3600) / 60)
    segundosUptime <- segundosActividadTerminal MOD 60
```

```
    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * 100.0) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " ºC"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso:  Entrada (", esEntrada, ")"
    Escribir "Token:      ", tokenResumen
    Escribir "Horario:    Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
    Escribir "Diagnóstico del terminal (uptime):"
    Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
    Escribir "Estado del aforo (capacidad: ", aforoTotal, " plazas):"
    Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
    Escribir "- plazas libres: ", aforoDisponible, " (-1)"
    Escribir "- ocupación exacta: ", porcentajeOcupacionReal, " %"
    Escribir "- ocupación en panel: ", porcentajeOcupacionEntero, " %"
FinAlgoritmo
```

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;
```

```
        tokenResumen = dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / 3600;
        minutosUptime = (segundosActividadTerminal % 3600) / 60;
        segundosUptime = segundosActividadTerminal % 60;

        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * 100.0;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " ºC");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");
        System.out.println("Token:      " + tokenResumen);
        System.out.println("Horario:    Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
        System.out.println("Diagnóstico del terminal (uptime):");
        System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");
        System.out.println("Estado del aforo (capacidad: " + aforoTotal + " plazas):");
        System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
        System.out.println("- plazas libres: " + aforoDisponible + " (-1)");
        System.out.println("- ocupación exacta: " + porcentajeOcupacionReal + " %");
        System.out.println("- ocupación en panel: " + porcentajeOcupacionEntero + " %");

        teclado.close();
    }
}
```

---

4. Cierre formal en Git y sincronización con GitHub

El estudiante actualiza su repositorio con los avances de la segunda semana desde la interfaz de IntelliJ IDEA:

1. Abre el panel lateral **Commit** de IntelliJ (`Alt + 0` o `Ctrl + K`).
2. Marca la casilla de los archivos modificados (`pr/pseudocodigo/MiProyecto.psc` y `pr/src/MiProyecto.java`) para pasarlos al área de preparación (*Staged*).
3. Escribe en el cuadro de texto el mensaje siguiendo el estándar convencional:

```
feat(pr): evolucionar aplicacion propia a v0.6 con motor matematico, modulo y casting
```

4. Despliega el botón de confirmación y selecciona **Commit and Push…** (o pulsa `Ctrl + Alt + K`).
5. En la ventana emergente, pulsa **Push** para enviar los cambios al repositorio remoto en GitHub.
6. Abre el navegador web y comprueba que el commit y ambos archivos aparecen actualizados en tu repositorio.

2.3 Semana 3. Constantes, formato y salida printf

Llegamos a la semana final del Sprint 1. Partiendo de la versión `ControlAccesoQR v0.6` (que ya cuenta con captura completa, descomposición con módulo y casting), evolucionamos nuestro archivo maestro eliminando números mágicos (`v0.7`), maquetando con secuencias de escape (`v0.8`), formateando con `printf` (`v0.9`) y registrando la versión definitiva `v1.0` secuencial con comentarios formales, mientras el estudiante concluye paralelamente su archivo `MiProyecto.java`.

2.3.1 Día 9 - 2 sesiones

---

1. Caso guía en AzaharTech

Es lunes por la tarde. En la sala técnica de **AzaharTech**, **Alba Torres** proyecta en el monitor principal la clase `ControlAccesoQR.java` tal y como quedó el jueves anterior (versión v0.6). Con el cursor, resalta varios números y textos dispersos por las líneas de cálculo:

```
int minutosTotalesLectivos = (totalSesiones * 50) + minutosExtraGuardia;
horasUptime =segundosActividadTerminal /3600;
minutosUptime =(segundosActividadTerminal %3600)/60;
porcentajeOcupacionReal =((double)contadorFichajesTerminal /aforoMaximoVestibulo)*100.0;
```

Alba se dirige a **Pau Ferrer** y al estudiante:

*«Fijaos en esos números: `50`, `3600`, `60`, `100.0`. En la ingeniería de software profesional los llamamos **números mágicos (magic numbers)**: valores fijos incrustados directamente en mitad de las operaciones matemáticas.*

¿Qué ocurre si la dirección del IES El Caminàs decide cambiar la duración de las sesiones lectivas a 55 minutos el curso que viene? Tendríamos que buscar ese `50` en mitad de las fórmulas, arriesgándonos a cambiar un dato por error.

*Hoy abriremos nuestro archivo `ControlAccesoQR` y lo refactorizaremos a la versión **v0.7**: extraeremos todos los valores fijos y los convertiremos en **constantes inmutables protegidas por el compilador con la palabra clave `final`** al inicio del método».*

---

2. Fundamento teórico: constantes inmutables y literales tipados

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      ELIMINACIÓN DE NÚMEROS MÁGICOS EN MEMORIA           │
├──────────────────────────┬──────────────────────────┬────────────────────┤
│ Código Frágil (Hardcoded)│ Código Profesional (final)│ Ventaja de Ingeniería │
├──────────────────────────┼──────────────────────────┼────────────────────┤
│ total * 50;              │ total * MINUTOS_SESION;   │ Si cambia el valor, solo se │
│ segundos / 3600;         │ segundos / SEGUNDOS_HORA; │ modifica en una única línea de │
│ total * 100.0;           │ total * FACTOR_PORCENTAJE;│ configuración al inicio.  │
└──────────────────────────┴──────────────────────────┴────────────────────┘
```

1. **La palabra reservada `final`.** Le indica al compilador que la posición de memoria es de solo lectura. Cualquier intento de reasignar su valor provocará un error de compilación.
2. **Convención `UPPER_SNAKE_CASE`.** Todas las letras en mayúsculas separadas por guiones bajos (`_`). Permite que cualquier miembro del equipo identifique al instante que se trata de un valor inmutable.
3. **Literales numéricos tipados.** El sufijo `L` para enteros largos (`long`) y `F` para decimales simples (`float`).

---

3. Refactorización a `ControlAccesoQR v0.7`

Abrimos el archivo y extraemos todos los números fijos a la cabecera de la clase.

PASO A. REFACTORIZACIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.7)

```
Algoritmo ControlAccesoQR
    Definir NOMBRE_CENTRO, PREFIJO_CENTRO Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0

    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Definir aforoTotal Como Entero
```

```
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- PREFIJO_CENTRO + "-" + dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir "================================================="
    Escribir "CENTRO:               ", NOMBRE_CENTRO
    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica:      ", tempVestibulo, " ºC"
    Escribir "Fichaje emitido:      #", idUltimoFichaje
    Escribir "Persona:              ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil:               ", perfilPersona
    Escribir "Sentido del paso:     Entrada (", esEntrada, ")"
    Escribir "Token:                ", tokenResumen
    Escribir "Horario:      Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "Permanencia en centro:", minutosEstanciaTotal, " minutos."
    Escribir "Diagnóstico del terminal (uptime):"
    Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
    Escribir "Estado del aforo (capacidad: ", aforoTotal, " plazas):"
    Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
    Escribir "- plazas libres:         ", aforoDisponible, " (-1)"
    Escribir "- ocupación exacta:      ", porcentajeOcupacionReal, " %"
    Escribir "- ocupación en panel:    ", porcentajeOcupacionEntero, " %"
    Escribir "================================================="
FinAlgoritmo
```

PASO B. REFACTARIZACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.7)

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumen;
```

```
        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos acumulados de actividad (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        System.out.println("CENTRO:               " + NOMBRE_CENTRO);
        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica:      " + tempVestibulo + " ºC");
        System.out.println("Fichaje emitido:      #" + idUltimoFichaje);
        System.out.println("Persona:              " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil:               " + perfilPersona);
        System.out.println("Sentido del paso:     Entrada (" + esEntrada + ")");
        System.out.println("Token QR:             " + tokenResumen);
        System.out.println("Horario registrado:   Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia en centro:" + minutosEstanciaTotal + " minutos.");
        System.out.println("Diagnóstico del terminal (uptime):");
        System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");
        System.out.println("Estado del aforo (capacidad: " + aforoTotal + " plazas):");
        System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
        System.out.println("- plazas libres:         " + aforoDisponible + " (-1)");
        System.out.println("- ocupación exacta:      " + porcentajeOcupacionReal + " %");
        System.out.println("- ocupación en panel:    " + porcentajeOcupacionEntero + " %");

        teclado.close();
    }
}
```

4. Trabajo del estudiante: refactorización de su proyecto propio

El estudiante abre su archivo maestro `.java` (versión v0.6):

- Extrae todos los números mágicos y cadenas de texto fijas a constantes `final` al inicio de la clase, aplicando la convención UPPER_SNAKE_CASE:
- En Aventura conversacional: declarar constantes para el nombre del reino o mundo, la salud máxima inicial, la capacidad de la mochila y el factor de daño desarmado.
- En Motor de recomendación: declarar constantes para la plataforma, el umbral de afinidad alta, la puntuación máxima y el factor porcentual.
- En Simulador de físicas 2D: declarar constantes para la aceleración gravitatoria, el coeficiente de fricción estándar y los límites del escenario.
- En Bóveda de contraseñas: declarar constantes para el nombre del almacén, el periodo de caducidad estándar en días, la longitud mínima de clave y los días de una semana.
- Sustituye en todas las operaciones del programa los literales sueltos por los nuevos identificadores de constantes.
- Compila y comprueba que el programa se ejecuta de forma idéntica, pero con una mantenibilidad y legibilidad muy superiores.

2.3.2 Día 10 - 2 sesiones

---

1. Caso guía en AzaharTech

Es martes por la tarde. **Pau Ferrer** muestra la consola de IntelliJ con la versión v0.7 en ejecución:

«El cálculo con constantes funciona de maravilla, pero la salida sigue teniendo un aspecto descuidado: para separar bloques tengo que poner `System.out.println("");` repetidas veces y las palabras 'TOKEN', 'CENTRO' y 'PERMANENCIA' no están alineadas porque cada palabra tiene una longitud distinta».

Laia Claramunt y **Alba Torres** le indican la barra invertida (`\`):

*«Hoy evolucionaremos `ControlAccesoQR.java` a la versión **v0.8**. Utilizaremos **secuencias de escape**: introduciremos tabuladores horizontales (`\t`) para crear columnas alineadas, saltos de línea (`\n`) para separar bloques en una sola instrucción y escaparemos comillas dobles (`\"`) para citar el protocolo oficial del centro educativo».*

---

2. Fundamento teórico: secuencias de escape en cadenas Java

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      SECUENCIAS DE ESCAPE EN CADENAS DE TEXTO            │
├──────────────┬──────────────────────────┬────────────────────────────────┤
│ Secuencia    │ Nombre técnico           │ Efecto en la consola de salida │
├──────────────┼──────────────────────────┼────────────────────────────────┤
│ \n           │ Salto de línea           │ Pasa a la siguiente línea inmediatamente.      │
├──────────────┼──────────────────────────┼────────────────────────────────┤
│ \t           │ Tabulación horizontal    │ Avanza hasta la siguiente parada de columna.   │
├──────────────┼──────────────────────────┼────────────────────────────────┤
│ \"           │ Comilla doble            │ Permite imprimir comillas dentro de un String. │
├──────────────┼──────────────────────────┼────────────────────────────────┤
│ \\           │ Barra invertida          │ Imprime el carácter literal de barra \         │
└──────────────┴──────────────────────────┴────────────────────────────────┘
```

---

3. Evolución a `ControlAccesoQR v0.8`

Modificamos el bloque de salida del archivo maestro `ControlAccesoQR` incorporando las secuencias de escape.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.8)

```
Algoritmo ControlAccesoQR
    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
```

```
    Definir FACTOR_PORCENTAJE Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\\caminas\\terminal\\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0

    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v0.8)"
    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- PREFIJO_CENTRO + "-" + dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE)
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir "CENTRO:\t", NOMBRE_CENTRO
    Escribir "SISTEMA:\t\"Control de acceso por QR\""
    Escribir "UBICACION:\tVestibulo principal \\ Edificio A"
    Escribir "FICHAJE N.:\t#", idUltimoFichaje, "\t\tESTADO:\tVALIDO"
    Escribir "PERSONA:\t", nombrePersona
    Escribir "IDENTIFICADOR:\t", dniPersona, "\tPERFIL:\t", perfilPersona
    Escribir "SENTIDO:\tEntrada (", esEntrada, ")"
    Escribir "TOKEN QR:\t", tokenResumen
    Escribir "HORARIO:\tEntrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:\t", minutosEstanciaTotal, " minutos en el centro."
    Escribir "AFORO (", aforoTotal, "):\t", porcentajeOcupacionReal, " % (Panel: ", porcentajeOcupacionEntero, " %)"
    Escribir "ACTIVO:\t", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "REGISTRO LOG:\t", RUTA_LOGS
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.8)

Actualizamos directamente el bloque de salida de nuestro archivo maestro `ControlAccesoQR.java` :

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        System.out.println("=== AZAHARTECH: TERMINAL " + NOMBRE_CENTRO + " (v0.8) ===");
        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos acumulados de actividad (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;
```

```
        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        System.out.println("CENTRO:\t" + NOMBRE_CENTRO);
        System.out.println("SISTEMA:\t\"Control de acceso por QR\"");
        System.out.println("UBICACIÓN:\tVestíbulo principal \\ Edificio A");
        System.out.println("FICHAJE N.º:\t#" + idUltimoFichaje + "\t\tESTADO:\tVALIDADO");
        System.out.println("PERSONA:\t" + nombrePersona);
        System.out.println("IDENTIFICADOR:\t" + dniPersona + "\tPERFIL:\t" + perfilPersona);
        System.out.println("SENTIDO:\tEntrada (" + esEntrada + ")");
        System.out.println("TOKEN QR:\t" + tokenResumen);
        System.out.println("HORARIO:\tEntrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("PERMANENCIA:\t" + minutosEstanciaTotal + " minutos en el centro.");
        System.out.println("AFORO (" + aforoTotal + "):\t" + porcentajeOcupacionReal + " % (Panel: " + porcentajeOcupacionEntero + " %)");
        System.out.println("ACTIVO:\t" + horasUptime + "h " + minutosUptime + "m " + segundosUptime + "s");
        System.out.println("REGISTRO LOG:\t" + RUTA_LOGS);

        teclado.close();
    }
}
```

---

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo maestro .java (versión v0.7):

- Maqueta la salida por consola utilizando secuencias de escape (\n, \t, \", \) para dar formato a la presentación de datos:
- En Aventura conversacional: maquetar la cabecera de la ficha del héroe con tabulaciones para alinear atributos, comillas en el título de la misión activa y barras en la ruta de guardado.
- En Motor de recomendación: maquetar el informe de afinidad con tabulaciones alineadas, comillas en las etiquetas de género y ruta del catálogo de datos.
- En Simulador de físicas 2D: maquetar el panel de telemetría cinemática con columnas tabuladas, comillas en el material y ruta de exportación de telemetría.
- En Bóveda de contraseñas: maquetar la ficha de credencial con tabulaciones limpias, y barra escapada en la ruta de la base cifrada.
- Comprueba que la consola muestra los bloques visuales ordenados sin desalineaciones tipográficas y sin concatenaciones superfluas.

2.3.3 Día 11 - 2 sesiones

---

1. Caso guía en AzaharTech

Es miércoles por la tarde. **Laia Claramunt** revisa la versión v0.8:

«La tabulación con `\t` es un avance, pero tiene un punto débil: si el nombre de un alumno es muy largo (como ' Constantinopla'), el tabulador salta a la siguiente parada y rompe la columna. Además, los decimales del porcentaje siguen mostrando hasta quince dígitos.

*Hoy alcanzaremos la versión **v0.9**: eliminaremos los `println` del ticket y utilizaremos **`System.out.printf()`**. Definiremos anchos de campo fijos, alinearemos textos a la izquierda y números a la derecha, y redondearemos los decimales automáticamente a exactamente dos posiciones».*

---

2. Fundamento teórico: especificadores de formato en `System.out.printf()`

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      PLANTILLA DE FORMATO PROFESIONAL PRINTF             │
├──────────────────┬──────────────────────────────┬────────────────────────┤
| Patrón           | Función técnica              | Ejemplo                |
├──────────────────┼──────────────────────────────┼────────────────────────┤
| %-25s            | Texto alineado a la IZQUIERDA en | %-25s -> "Juan Pérez          " |
|                  | un campo de 25 caracteres.   |                                  |
```

```
┌──────────────────┬──────────────────────────────────┬──────────────────────────────────┐
│ %04d             │ Entero rellenado con ceros a la  │ %04d  -> "0007"                  │
│                  │ izquierda hasta 4 dígitos.       │                                  │
├──────────────────┼──────────────────────────────────┼──────────────────────────────────┤
│ %6.2f            │ Decimal con 2 decimales fijos en │ %6.2f -> " 98.75"                │
│                  │ un ancho total de 6 caracteres.  │                                  │
├──────────────────┼──────────────────────────────────┼──────────────────────────────────┤
│ %n               │ Salto de línea independiente     │ %n                               │
└──────────────────┴──────────────────────────────────┴──────────────────────────────────┘
```

3. Evolución a `ControlAccesoQR v0.9`

Actualizamos el bloque final de impresión de `ControlAccesoQR` sustituyendo los textos concatenados por una plantilla con `printf`.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.9)

```
Algoritmo ControlAccesoQR
    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\\caminas\\terminal\\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0

    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v0.9)"
    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    tokenResumen <- PREFIJO_CENTRO + "-" + dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
```

```
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir "CENTRO:        ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :  #", idUltimoFichaje, " | TOKEN: ", tokenResumen
    Escribir "PERSONA:       ", nombrePersona, " | PERFIL: ", perfilPersona
    Escribir "IDENTIFICADOR: ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "HORARIO:       Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:   ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "AFORO (", aforoTotal, "):  ", redon(porcentajeOcupacionReal * 100) / 100, " % (Panel: ", porcentajeOcupacionEntero, " %)"
    Escribir "ACTIVO:        ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s (Sensor: ", tempVestibulo, " C)"
    Escribir "REGISTRO LOG:  ", RUTA_LOGS
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V0.9)

```java
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        System.out.println("AZAHARTECH: TERMINAL " + NOMBRE_CENTRO + " (v0.9)");
        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
```

```
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:        %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:  #%05d | TOKEN: %s%n", idUltimoFichaje, tokenResumen);
        System.out.printf("PERSONA:       %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR: %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:       Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:   %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("AFORO (%04d):  %6.2f %% (Panel: %03d %%)%n", aforoTotal, porcentajeOcupacionReal, porcentajeOcupacionEntero);
        System.out.printf("ACTIVO:        %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempVestibulo);
        System.out.printf("REGISTRO LOG:  %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo maestro .java (versión v0.8):

- Sustituye las instrucciones de salida concatenada por plantillas formateadas con System.out.printf (), aplicando especificadores de texto alineado a la izquierda (%-25s), enteros tabulados o con ceros a la izquierda (%04d), decimales acotados (%.2f) y saltos independientes (%n):
- En Aventura conversacional: maquetar la tabla de resumen del estado del héroe con columnas fijas para el identificador del jugador, nombre de personaje, nivel y porcentaje de salud formateado a dos decimales.
- En Motor de recomendación: generar el informe oficial de coincidencias con ancho fijo para el título del ítem, año de lanzamiento, puntuación de afinidad exacta con dos decimales y ranking.
- En Simulador de físicas 2D: generar el cuadro de telemetría cinemática con columnas alineadas para el ID de partícula, material, coordenadas espaciales con dos decimales y velocidad instantánea.
- En Bóveda de contraseñas: generar el informe de auditoría de credenciales con columnas alineadas para el ID de clave, servicio web, días de vigencia restantes y porcentaje de vida útil consumida con dos decimales.
- Ejecuta pruebas introduciendo cadenas de distinta longitud para verificar que las columnas permanecen perfectamente rectas y alineadas verticalmente.

2.3.4 Día 12 - 2 sesiones

1. Caso guía en AzaharTech

Es jueves 1 de octubre. Mañana viernes concluye formalmente el **Sprint 1**. En la sala de juntas de **AzaharTech**, **Laia Claramunt** convoca a todo el equipo de desarrollo frente al proyector:

*«Equipo, contemplad lo que hemos construido en doce días de trabajo: tenemos **un software vivo, robusto y profesional que ha crecido día a día**.*

*Hoy alcanzamos la versión **v1.0**: añadiremos la cabecera formal de documentación Javadoc, limpiaremos cualquier advertencia del compilador y dejaremos sellado el código de vuestro proyecto propio en GitHub para la evaluación de mañana».*

---

2. Fundamento teórico: documentación técnica y autoformateo

1. **Comentarios Javadoc (`/** ... */`).** Permiten a las herramientas de ingeniería extraer manuales técnicos automáticos en formato HTML:
- `@author`. Desarrollador o equipo de trabajo responsable.
- `@version`. Número de versión semántica del software.
2. **Autoformateo en IntelliJ (`Ctrl + Alt + L`).** Reorganiza el código para que respete las directrices internacionales de indentación (4 espacios) y separación de operadores.

---

3. El código definitivo del Sprint 1: `ControlAccesoQR v1.0`

Esta es la versión íntegra y definitiva del caso guía:

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.0 (Definitiva Sprint 1):
 * Programa secuencial integral que captura parámetros de terminal y usuario,
 * realiza el cómputo horario de estancia neta, descomposiciones temporales de servicio (uptime)
 * y cálculos porcentuales de aforo con casting explícito, emitiendo un informe
 * oficial formateado profesionalmente mediante printf.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.0 (Octubre 2026)
 * @since JDK 21 LTS
 */

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // ------------------------------------------------------------------
        // 1. CONSTANTES INMUTABLES DEL SISTEMA (Configuración corporativa)
        // ------------------------------------------------------------------
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        // ------------------------------------------------------------------
        // 2. DECLARACIÓN DE VARIABLES DE MEMORIA
        // ------------------------------------------------------------------
        Scanner teclado = new Scanner(System.in);

        // Variables de hardware y terminal
        int terminalId;
        double tempVestibulo;

        // Variables de identidad y perfil de la persona
        String nombrePersona;
        String dniPersona;
        char perfilPersona;
        boolean esEntrada;

        // Variables de cómputo horario y estancia
        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
```

```
        int minutosEstanciaTotal;
        String tokenResumen;

        // Variables de aforo del recinto y numeración de ticket
        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        // Variables de diagnóstico y tiempo de servicio (uptime)
        int segundosActividadTerminal;
        int horasUptime;
        int minutosUptime;
        int segundosUptime;

        // Variables de aforo consolidado y porcentajes
        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        // ------------------------------------------------------------------
        // 3. CAPTURA INTERACTIVA DE DATOS
        // ------------------------------------------------------------------
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.0 Oficial (Programa Secuencial Base)  ");
        System.out.println("================================================");
        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine(); // Limpieza obligatoria del buffer de teclado

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ------------------------------------------------------------------
        // 4. PROCESAMIENTO SECUENCIAL Y OPERACIONES MATEMÁTICAS
        // ------------------------------------------------------------------
        // Conversión horaria a minutos y cálculo de permanencia neta
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        // Operadores unarios de incremento y decremento de estado
        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        // Composición acumulativa del token con operador +=
        tokenResumen = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        // Descomposición temporal exacta mediante división entera y módulo (%)
        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // Cálculo de porcentaje con casting explícito a (double) para evitar división a 0
        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal; // Casting a entero para panel

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA PROFESIONAL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n================================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                  ");
```

```
        System.out.println("========================================================================");
        System.out.printf("CENTRO:        %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:  #%05d | TOKEN: %s%n", idUltimoFichaje, tokenResumen);
        System.out.printf("PERSONA:       %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR: %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:       Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:   %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("AFORO (%04d):  %6.2f %% (Panel: %03d %%)%n", aforoTotal, porcentajeOcupacionReal, porcentajeOcupacionEntero);
        System.out.printf("ACTIVO:        %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempVestibulo);
        System.out.printf("REGISTRO LOG:  %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        // Cierre preventivo del recurso de entrada
        teclado.close();
    }
}
```

---

4. Trabajo del estudiante: la versión v1.0 de su proyecto propio

Cada estudiante finaliza su archivo `pr/src/MiProyecto.java`:

1. Incorpora la cabecera Javadoc con sus datos.
2. Aplica el formateador de código en IntelliJ (`Ctrl + Alt + L`).
3. Comprueba que el programa compila limpiamente y que la salida por consola con `printf` es una tabla perfectamente alineada.
4. Pulsa el atajo Ctrl + K (o haz clic en el icono verde de verificación Commit en la barra lateral izquierda).
5. En el panel de Commit, marca las casillas de los archivos modificados dentro de pr/ (MiProyecto.java y MiProyecto.psc).
6. En la caja de texto para el mensaje, escribe siguiendo el estándar convencional:
   | feat (pr): consolidar version 1.0 del programa secuencial completo para sprint 1
7. Despliega el botón azul inferior y selecciona «Commit and Push».
8. En la ventana de confirmación que aparece, pulsa Push para enviar los cambios a GitHub.

2.4 Cierre del Sprint 1

- **Entrega lista.** El estudiante tiene su proyecto propio en GitHub en versión `v1.0`, listo para ser registrado mañana viernes bajo la etiqueta **`v0.1.0-sprint1`**.

3. Sprint 2. Utilización de objetos y clases

3.1 Semana 4. POO, instanciación y memoria Stack/Heap

Esta semana marca la entrada formal en los fundamentos de la Programación Orientada a Objetos (POO). A lo largo de estas sesiones dejaremos atrás la visión lineal del código para comprender cómo la Máquina Virtual de Java estructura y gestiona la memoria real, diferenciando las referencias en la pila (Stack) de los objetos dinámicos alojados en el montón (Heap). Apoyándonos en clases predefinidas de la biblioteca estándar como Random para dotar de seguridad e impredecibilidad, aprenderemos a dar vida a objetos mediante el operador new y sus constructores, elevando la complejidad técnica y la robustez tanto del caso guía como de los proyectos propios de cada equipo.

3.1.1 Día 13 - 2 sesiones

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 5 de octubre. En la sala técnica de **AzaharTech**, **Alba Torres** proyecta en grande el código de `ControlAccesoQR.java` (versión v1.0) que cerramos el viernes pasado:

«En el Sprint 1 logramos que nuestro software fuera un motor secuencial exacto. Pero el director del IES El Caminàs nos ha trasladado una vulnerabilidad de seguridad evidente: el token QR que mostramos en la pantalla del vestíbulo utiliza como identificador de registro un número correlativo simple (`#1043`, `#1044`...). Cualquier estudiante avispado puede predecir el siguiente código y falsificar su asistencia desde el móvil sin estar físicamente en el instituto.

*Para blindar el acceso necesitamos que el token contenga un **código criptográfico pseudoaleatorio impredecible** que cambie en cada emisión.*

*No vamos a reinventar la rueda programando generadores matemáticos complejos desde cero. Vamos a dar el salto a la **Programación Orientada a Objetos (POO)**: aprenderemos a instanciar y utilizar clases predefinidas de la biblioteca de Java como `Random`, y entenderemos qué ocurre físicamente bajo el capó en la memoria RAM (*Stack* frente a *Heap*) cuando creamos un objeto con el operador `new`»*.

2. Fundamento teórico: clases, objetos y el modelo de memoria física (*Stack* vs. *Heap*)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        ARQUITECTURA DE LA MEMORIA RAM EN JAVA            │
├──────────────────────────────────┬───────────────────────────────────────┤
│ MEMORIA STACK (Pila de ejecución) │ MEMORIA HEAP (Montón de objetos dinámicos)  │
├──────────────────────────────────┼───────────────────────────────────────┤
│ • Rápida, estructurada y estricta.│ • Área de memoria dinámica y flexible.      │
│ • Almacena variables primitivas   │ • Aquí residen los OBJETOS REALES creados con new. │
│   (int, double, boolean).         │ • Viven los atributos internos y estructuras.     │
│ • Almacena las REFERENCIAS        │ • Gestionada automáticamente por el recolector de  │
│   (punteros/direcciones de memoria)│   basura (Garbage Collector).               │
└──────────────────────────────────┴───────────────────────────────────────┘
```

A. CLASE (MOLDE) FRENTE A OBJETO (INSTANCIA)

- **Clase (`java.util.Random`):** Es el plano o diseño conceptual. Define qué datos puede almacenar y qué métodos sabe ejecutar, pero no ocupa memoria como entidad viva.
- **Objeto:** Es el ejemplar concreto que creamos en memoria a partir de esa clase utilizando el operador `new`.

```
Random generador = new Random(); // Instanciación
```

B. QUÉ OCURRE EXACTAMENTE EN MEMORIA AL INSTANCIAR

Cuando ejecutamos la instrucción `Random generador = new Random();`: 1. `Random generador`: Se reserva en la memoria **Stack** una variable de referencia (un puntero que guardará una dirección física de memoria, por ejemplo `0x4f2a`). 2. `new`: Pide al sistema operativo que reserve un bloque de memoria en el **Heap** para alojar el objeto. 3. `Random()` (Constructor): Inicializa el estado interno del objeto en el Heap (configura la semilla del reloj). 4. Asignación (`=`): Guarda la dirección de memoria del Heap en la variable `generador` del Stack.



C. EL VALOR `null` Y EL TEMIDO `NullPointerException`

Si declaramos una variable de referencia pero no creamos el objeto con `new`:

```
Random generador = null; // No apunta a ninguna dirección de memoria
int numero = generador.nextInt(); // ¡ERROR FATAL!: NullPointerException
```

La excepción `NullPointerException` se produce cuando intentamos invocar un método a través de una variable de referencia que apunta a la nada (`null`).

---

3. Evolución del caso guía a `ControlAccesoQR v1.1` (PSeInt y Java)

Ampliamos el código guía sustituyendo el código fijo por la generación de un token pseudoaleatorio de seguridad con la clase predefinida `Random`.

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.1)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.1 (Evolucion: generacion aleatoria de seguridad)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0

    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    // [NUEVO DÍA 13] Variable para el token de seguridad aleatorio
    Definir codigoSeguridadAleatorio Como Entero

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.1 - Seguridad QR)"
    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
```

```
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    // [NUEVO DÍA 13] Generación de un número aleatorio de seguridad de 4 cifras (1000 a 9999)
    codigoSeguridadAleatorio <- Aleatorio(1000, 9999)

    tokenResumen <- PREFIJO_CENTRO + "-" + dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-SEC#" + ConvertirATexto(codigoSeguridadAleatorio)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:        ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :  #", idUltimoFichaje, " | TOKEN QR DINAMICO: ", tokenResumen
    Escribir "PERSONA:       ", nombrePersona, " | PERFIL: ", perfilPersona
    Escribir "IDENTIFICADOR: ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-------------------------------------------------------------------------"
    Escribir "HORARIO:       Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:   ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "AFORO (", aforoTotal, "):  ", redon(porcentajeOcupacionReal * 100) / 100, " % (Panel: ", porcentajeOcupacionEntero, " %)"
    Escribir "ACTIVO:        ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s (Sensor: ", tempVestibulo, " C)"
    Escribir "REGISTRO LOG:  ", RUTA_LOGS
    Escribir "========================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.1)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.1:
 * Evolución a POO mediante la instanciación de clases predefinidas (Random) en el Heap.
 * Se incorpora un token criptográfico pseudoaleatorio para evitar la predicción de códigos.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.1 (Octubre 2026)
 * @since JDK 21 LTS
 */

import java.util.Scanner;
import java.util.Random; // [NUEVO DÍA 13] Importación de la clase predefinida

public class ControlAccesoQR {
    public static void main(String[] args) {
        // ------------------------------------------------------------------
        // 1. CONSTANTES INMUTABLES DEL SISTEMA (Configuración corporativa)
        // ------------------------------------------------------------------
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
```

```
        // ------------------------------------------------------------------------
        // 2. DECLARACIÓN DE VARIABLES Y RESERVA DE OBJETOS EN MEMORIA
        // ------------------------------------------------------------------------
        // Instanciación de objetos en el Heap mediante el operador 'new'
        Scanner teclado = new Scanner(System.in);
        Random generadorSeguridad = new Random(); // [NUEVO DÍA 13] Reserva de objeto en el Heap

        // Variables de hardware y terminal
        int terminalId;
        double tempVestibulo;

        // Variables de identidad y perfil de la persona
        String nombrePersona;
        String dniPersona;
        char perfilPersona;
        boolean esEntrada;

        // Variables de cómputo horario y estancia
        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        // Variables de aforo del recinto y numeración de ticket
        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        // Variables de diagnóstico y tiempo de servicio (uptime)
        int segundosActividadTerminal;
        int horasUptime;
        int minutosUptime;
        int segundosUptime;

        // Variables de aforo consolidado y porcentajes
        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        // [NUEVO DÍA 13] Variable para almacenar el código generado por el objeto Random
        int codigoSeguridadAleatorio;

        // ------------------------------------------------------------------------
        // 3. CAPTURA INTERACTIVA DE DATOS
        // ------------------------------------------------------------------------
        System.out.println("=================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.1 (Token Dinámico con Objeto Random)  ");
        System.out.println("=================================================");
        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine(); // Limpieza obligatoria del buffer de teclado

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ------------------------------------------------------------------------
        // 4. PROCESAMIENTO SECUENCIAL Y OPERACIONES MATEMÁTICAS
        // ------------------------------------------------------------------------
        // Conversión horaria a minutos y cálculo de permanencia neta
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        // Operadores unarios de incremento y decremento de estado
```

```
        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        // [NUEVO DÍA 13] Invocación de método de instancia sobre el objeto Random
        // nextInt(9000) genera valores entre 0 y 8999; al sumar 1000 acota al rango [1000 - 9999]
        codigoSeguridadAleatorio = BASE_TOKEN_SEGURIDAD + generadorSeguridad.nextInt(RANGO_TOKEN_SEGURIDAD);

        // Composición acumulativa del token con operador += incorporando el código dinámico
        tokenResumen = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-SEC#" + codigoSeguridadAleatorio;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        // Descomposición temporal exacta mediante división entera y módulo (%)
        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // Cálculo de porcentaje con casting explícito a (double) para evitar división a 0
        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA PROFESIONAL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:        %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:  #%05d | TOKEN DINÁMICO: %s%n", idUltimoFichaje, tokenResumen);
        System.out.printf("PERSONA:       %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR: %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:       Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:   %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("AFORO (%04d):  %6.2f %% (Panel: %03d %%)%n", aforoTotal, porcentajeOcupacionReal, porcentajeOcupacionEntero);
        System.out.printf("ACTIVO:        %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempVestibulo);
        System.out.printf("REGISTRO LOG:  %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** para entrenar la memoria, la instanciación de objetos y la generación pseudoaleatoria:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). ADAPTACIÓN AL PROYECTO PROPIO

- **Objetivo.** Instanciar la clase predefinida `Random` en el montículo (*Heap*) y utilizarla para generar una magnitud aleatoria propia del dominio de tu proyecto.
- **Instrucciones.** Abre tu archivo `MiProyecto.java` (en versión v1.0) e incorpora un generador aleatorio:
- En **Aventura conversacional**. Instanciar `Random` para simular una tirada de iniciativa o factor de suerte entre 1 y 20.
- En **Motor de recomendación**. Instanciar `Random` para simular la variación aleatoria de afinidad de un usuario invitado.
- En **Simulador de físicas 2D**. Instanciar `Random` para generar la velocidad inicial aleatoria de la partícula entre 5.0 y 25.0 m/s.
- En **Bóveda de contraseñas**. Instanciar `Random` para generar un PIN numérico aleatorio de seguridad de 6 dígitos.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO. ACOTACIÓN MATEMÁTICA EN RANGO `[MIN, MAX]`

- **Contexto técnico.** Cuando a `nextInt()` le pasamos un valor entre parentesis (`nextInt(tope)`), genera números enteros que van desde 0 hasta tope - 1. Por ejemplo, nextInt(10) genera valores del 0 al 9.
- **Desafío de la kata.** Diseña una fórmula matemática secuencial en una sola línea que utilice `random.nextInt()` para generar un número pseudoaleatorio acotado estrictamente dentro de un rango cerrado $[MIN, MAX]$ (donde ambos extremos están obligatoriamente incluidos), **sin utilizar estructuras condicionales** `if`: $$\text{Resultado} = MIN + \text{random.nextInt}((MAX - MIN) + 1)$$

- **Evidencia de maestría.** Prueba tu fórmula configurando $MIN = 50$ y $MAX = 75$. Ejecuta el programa varias veces consecutivas y certifica en la consola que el valor generado nunca desciende de 50 ni supera 75.

KATA 3 (CINTURÓN NEGRO / KATA «HACKER AZAHARTECH»). EL MISTERIO DE LA SEMILLA (*SEED*)

- **Contexto de seguridad.** La clase `Random` de Java no genera aleatoriedad pura física, sino secuencias matemáticas deterministas basadas en una «semilla» (*seed*).
- **Misión de la kata:**
- Abre una clase de prueba temporal en tu proyecto.
- Instancia **dos objetos distintos** de la clase `Random` pasándoles exactamente la misma semilla numérica en su constructor:

```
Random generadorA = new Random(12345L);
Random generadorB = new Random(12345L);
```

- Invoca `nextInt(100)` tres veces seguidas en cada uno de los dos objetos e imprime sus salidas. ¿Qué observas en la consola?
- Redacta dos líneas de comentario en tu código explicando **por qué un generador con semilla fija comprometería la seguridad de los tokens del IES El Caminàs** y cómo el constructor por defecto `new Random()` soluciona esta vulnerabilidad utilizando el reloj del sistema (`System.currentTimeMillis()`).

3.1.2 Día 14 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del martes 6 de octubre. En la sala técnica de **AzaharTech**, **Pau Ferrer** muestra entusiasmado una modificación en el sistema de acceso para el **IES El Caminàs**.

La dirección del instituto ha trasladado un problema logístico evidente: en un centro educativo con una capacidad de 2000 personas, situar un único monitor emisor en el vestíbulo principal provocaría aglomeraciones a primera hora de la mañana. Por ello, el centro ha solicitado instalar un **segundo monitor de acceso en la Puerta Norte**.

Ambos monitores proyectarán códigos QR dinámicos en paralelo y deben operar de forma **completamente autónoma e independiente**, de modo que los escaneos que se produzcan en una puerta no consuman ni descompensen la secuencia de seguridad de la otra.

Pau ha escrito en el código:

```
Random generadorVestibulo = new Random();
Random generadorPuertaNorte = generadorVestibulo; // Pau cree que ha duplicado el generador
```

Pau explica convencido al resto del equipo:

«Para que el monitor de la Puerta Norte tenga su propio generador sin tener que volver a escribir `new`, he copiado la variable. Así cada monitor tiene su motor QR independiente».

Alba Torres mira el código, niega con la cabeza y llama a Laia Claramunt y al estudiante:

«Pau, acabas de cometer el error conceptual más extendido al pasar de tipos primitivos a objetos. Con un entero, escribir `int b = a` duplica el número en una celda de memoria independiente. Pero con objetos, ***el signo igual jamás crea un objeto nuevo****.*

Solo has utilizado el operador `new` una vez, por lo que en el Heap solo existe ***un único generador físico****. Lo que has hecho ha sido crear dos variables en el Stack que contienen la misma dirección de memoria.*

Has creado un ***alias****: dos pantallas conectadas al mismo motor. Cada vez que un estudiante escanea en la Puerta Norte, hace avanzar la semilla interna del generador y altera la secuencia del Vestíbulo. Si de verdad quieres dos monitores autónomos,* ***debes invocar a `new` dos veces****.*

Hoy aprenderemos qué ocurre físicamente en la memoria RAM al asignar referencias, qué es un alias y cómo gestiona la JVM el ciclo de vida de los objetos en el Heap».

---

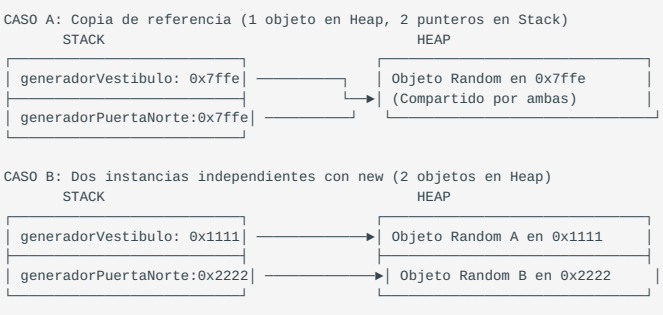
2. Fundamento teórico: variables primitivas frente a variables de referencia

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      COPIA DE VALOR FRENTE A COPIA DE REFERENCIA          │
├──────────────────────────────────┬───────────────────────────────────────┤
│ 1. COPIA POR VALOR (Primitivos)  │ 2. COPIA DE REFERENCIA / ALIAS (Objetos)│
├──────────────────────────────────┼───────────────────────────────────────┤
│ int a = 5;                       │ Random r1 = new Random();             │
│ int b = a;                       │ Random r2 = r1; // NO crea un nuevo objeto │
│                                  │                                       │
│ Se duplica el dato en el Stack.  │ Se copia el PUNTERO en el Stack. Ambas variables │
│ Si modificas 'b', 'a' no cambia. │ apuntan a la MISMA instancia física en el Heap. │
└──────────────────────────────────┴───────────────────────────────────────┘
```

A. EL MECANISMO DE COPIA EN MEMORIA: VALOR FRENTE A REFERENCIA (ALIAS)

- **Tipos primitivos (`int`, `double`, `boolean`, `char`).** La variable contiene directamente el dato binario en la memoria de ejecución (**Stack**). Al hacer `b = a`, se clona el valor en una celda física distinta.
- **Tipos referencia (Objetos creados con `new`).** La variable en el Stack **no almacena el objeto, sino una dirección de memoria (puntero)** que indica dónde reside el objeto real dentro de la memoria dinámica (**Heap**).
- Al hacer `generadorPuertaNorte = generadorVestibulo`, se copia únicamente la dirección de memoria de 64 bits.
- Se genera un **alias**: dos variables locales que gobiernan la misma instancia física.
- Para disponer de dos generadores independientes con secuencias pseudoaleatorias que no interfieran entre sí, es obligatorio realizar **dos reservas independientes en el Heap mediante dos llamadas a `new`**:

```
Random generadorVestibulo   = new Random(); // Instancia 1 en el Heap
Random generadorPuertaNorte = new Random(); // Instancia 2 en el Heap
```



B. OBJETOS HUÉRFANOS Y EL RECOLECTOR DE BASURA (*GARBAGE COLLECTOR*)

En lenguajes como Java no existe una instrucción manual para destruir objetos. La gestión de memoria dinámica es automática:
1. Si una variable que apuntaba a un objeto pasa a apuntar a `null` o a otra instancia:

```
Random r = new Random(); // Se crea el Objeto 1 en el Heap
r = new Random();        // Se crea el Objeto 2. La variable 'r' apunta al Objeto 2.
```

2. El Objeto 1 se convierte en un **objeto huérfano**: ningún puntero activo en el Stack conserva su dirección. 3. El **Garbage Collector (GC)** de la Máquina Virtual de Java rastrea periódicamente la memoria en segundo plano, detecta los objetos inalcanzables y **libera automáticamente sus bytes en el Heap**.

C. EL VALOR `null` Y LA SEGURIDAD DE REFERENCIAS

El literal reservado `null` indica que una variable de referencia no está apuntando a ningún objeto en memoria. * Intentar invocar un método sobre una variable que vale `null` (*por ejemplo,* *`generador = null; generador.nextInt();`*) desencadena un error crítico en tiempo de ejecución: **`NullPointerException`**.

---

3. Evolución del caso guía a `ControlAccesoQR v1.2`

Ampliamos el programa guía modelando la infraestructura de los dos accesos del centro educativo: instanciamos **dos generadores independientes con** `new` para garantizar la autonomía de emisión entre el monitor del vestíbulo y el monitor de la Puerta Norte.

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.2)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.2 (Evolucion: monitores multiples y generadores independientes)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 1000

    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    // [NUEVO DÍA 14] Dos tokens independientes para vestíbulo y puerta norte
    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.2 - Monitores Multiples)"
    Escribir "ID del terminal principal (vestibulo):"
    Leer terminalId
    Escribir "Temperatura del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
```

```
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    // [DÍA 14] Generación autónoma en cada punto de acceso
    codigoTokenVestibulo <- Aleatorio(1000, 9999)     // Motor monitor vestíbulo
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)   // Motor monitor puerta norte

    tokenResumen <- PREFIJO_CENTRO + "-" + dniPersona
    tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
    tokenResumen <- tokenResumen + "-SEC#" + ConvertirATexto(codigoTokenVestibulo)
    tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible // 1201 + 799 = 2000
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:             ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :       #", idUltimoFichaje, " | TOKEN VESTIBULO: ", tokenResumen
    Escribir "TOKEN PUERTA NORTE: ", PREFIJO_CENTRO, "-", dniPersona, "-T102-SEC#", codigoTokenPuertaNorte
    Escribir "PERSONA:            ", nombrePersona, " | PERFIL: ", perfilPersona
    Escribir "IDENTIFICADOR:      ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-------------------------------------------------------------------"
    Escribir "HORARIO:            Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:        ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "AFORO (", aforoTotal, "):       ", redon(porcentajeOcupacionReal * 100) / 100, " % (Panel: ", porcentajeOcupacionEntero, " %)"
    Escribir "ACTIVO:             ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s (Sensor: ", tempVestibulo, " C)"
    Escribir "REGISTRO LOG:       ", RUTA_LOGS
    Escribir "========================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.2)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.2:
 * Múltiples instancias en el Heap con 'new' frente a copias de punteros en el Stack.
 * Soporte para dos monitores de acceso independientes (vestíbulo principal y puerta norte).
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.2 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // ------------------------------------------------------------------
        // 1. CONSTANTES INMUTABLES DEL SISTEMA (Configuración corporativa)
        // ------------------------------------------------------------------
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;


      // ------------------------------------------------------------------
        // 2. DECLARACIÓN DE VARIABLES Y RESERVA DE OBJETOS EN MEMORIA
        // ------------------------------------------------------------------
        Scanner teclado = new Scanner(System.in);

        // [NUEVO DÍA 14] Instanciamos DOS objetos independientes en el Heap mediante 'new'
        // Cada generador tiene su propia dirección de memoria y su propio estado interno
        Random generadorVestibulo   = new Random(); // Objeto 1 en el Heap (Monitor principal)
        Random generadorPuertaNorte = new Random(); // Objeto 2 en el Heap (Monitor puerta norte)

        // Variables de hardware y terminal
        int terminalId;
```

```
        double tempVestibulo;

        // Variables de identidad y perfil de la persona
        String nombrePersona;
        String dniPersona;
        char perfilPersona;
        boolean esEntrada;

        // Variables de cómputo horario y estancia
        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;

        // [NUEVO DÍA 14] Dos tokens independientes generados para cada monitor
        String tokenResumenVestibulo;
        String tokenResumenPuertaNorte;

        // Variables de aforo del recinto y numeración de ticket
        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        // Variables de diagnóstico y tiempo de servicio (uptime)
        int segundosActividadTerminal;
        int horasUptime;
        int minutosUptime;
        int segundosUptime;

        // Variables de aforo consolidado y porcentajes
        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        // Códigos pseudoaleatorios independientes
        int codigoTokenVestibulo;
        int codigoTokenPuertaNorte;

        // ------------------------------------------------------------------
        // 3. CAPTURA INTERACTIVA DE DATOS
        // ------------------------------------------------------------------
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.2 (Soporte Multimonitor Independiente) ");
        System.out.println("================================================");
        System.out.print("ID del terminal principal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine(); // Limpieza obligatoria del buffer de teclado

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ------------------------------------------------------------------
        // 4. PROCESAMIENTO SECUENCIAL Y OPERACIONES MATEMÁTICAS
        // ------------------------------------------------------------------
        // Conversión horaria a minutos y cálculo de permanencia neta
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        // Operadores unarios de incremento y decremento de estado
        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        // [NUEVO DÍA 14] Cada monitor invoca su propio objeto en el Heap de forma autónoma
```

```
        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);   // Token vestíbulo [1000 - 9999]
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD); // Token puerta norte [1000 - 9999]

        // Composición acumulativa de ambos tokens
        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenPuertaNorte += "-T102"; // ID fijo asignado al monitor norte
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        // Descomposición temporal exacta mediante división entera y módulo (%)
        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // Cálculo de porcentaje con casting explícito a (double) para evitar división a 0
        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA PROFESIONAL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s (Monitor autónomo)%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR:        %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("AFORO (%04d):         %6.2f %% (Panel: %03d %%)%n", aforoTotal, porcentajeOcupacionReal, porcentajeOcupacionEntero);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempVestibulo);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre el comportamiento de las referencias y la memoria física:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). DOS GENERADORES INDEPENDIENTES EN TU PROYECTO

- **Objetivo:** Instanciar **dos objetos independientes** con el operador `new` en el Heap para dos propósitos diferenciados de tu proyecto propio.
- **Instrucciones:** Abre tu archivo `MiProyecto.java` (en versión v1.1) e incorpora dos instancias de `Random`:
- En **Aventura conversacional**: instanciar `randomJugador = new Random();` para la tirada de ataque y `randomEnemigo = new Random();` para la tirada de defensa del rival.
- En **Motor de recomendación**: instanciar `randomAfinidad = new Random();` para simular gustos del usuario y `randomCatalogo = new Random();` para simular la popularidad del ítem en catálogo.
- En **Simulador de físicas 2D**: instanciar `randomVelocidad = new Random();` para la velocidad inicial y `randomAngulo = new Random();` para la dirección de lanzamiento.
- En **Bóveda de contraseñas**: instanciar `randomPin = new Random();` para generar el PIN de acceso y `randomSalt = new Random();` para generar el salt criptográfico de la clave.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). LA TRAMPA DEL ALIAS DE MEMORIA

- **Contexto técnico:** Demostrar experimentalmente la diferencia entre copiar una referencia (crear un alias) y crear dos objetos reales con `new`.
- **Misión de la kata:**

a. Crea una clase temporal de pruebas llamada `PruebaAlias.java`.

b. Escribe y ejecuta el siguiente experimento:

```
// Experimento 1: Dos variables apuntando al MISMO objeto (Alias)
Random g1 = new Random(500L);
Random g2 = g1; // Copia de referencia en el Stack

System.out.println("--- EXPERIMENTO 1 (Un solo objeto en Heap) ---");
System.out.println("Número g1: " + g1.nextInt(100));
System.out.println("Número g2: " + g2.nextInt(100));

// Experimento 2: Dos objetos DISTINTOS en el Heap con new
Random g3 = new Random(500L);
Random g4 = new Random(500L); // Dos llamadas a new

System.out.println("\n--- EXPERIMENTO 2 (Dos objetos independientes) ---");
System.out.println("Número g3: " + g3.nextInt(100));
System.out.println("Número g4: " + g4.nextInt(100));
```

c. **Pregunta técnica para tu cuaderno:**
¿Por qué en el Experimento 1 `g1` y `g2` muestran números **distintos**, mientras que en el Experimento 2 `g3` y `g4` muestran exactamente el **mismo** número?
(Escribe la respuesta en dos líneas de comentario explicando qué ocurre con el estado interno del generador en el Heap).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). TELEMETRÍA DE OBJETOS HUÉRFANOS Y GARBAGE COLLECTOR

- **Contexto de sistemas.** En sistemas que operan 24/7 (como los servidores de AzaharTech), no controlar los objetos huérfanos puede degradar el rendimiento de la JVM.
- **Misión de la kata:**

a. Utiliza la clase `Runtime` para consultar cuánta memoria libre en bytes tiene la JVM:

```
Runtime runtime = Runtime.getRuntime();
System.out.println("Memoria libre inicial: " + runtime.freeMemory() + " bytes");

// Creamos 3 referencias en el Stack y 3 objetos en el Heap
Random r1 = new Random();
Random r2 = new Random();
Random r3 = new Random();

// Dejamos r1 y r2 huérfanos en el Heap
r1 = null;
r2 = null;

System.gc(); // Solicitamos a la JVM que pase el Garbage Collector

System.out.println("Memoria libre tras liberar referencias: " + runtime.freeMemory() + " bytes");
```

b. Comprueba en la consola la variación de memoria e incluye una línea de comentario explicando qué ha hecho `System.gc()` con los objetos que apuntaban a `null`.

3.1.3 Día 15 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del miércoles 7 de octubre. En el laboratorio de pruebas de **AzaharTech**, **Pau Ferrer** y **Alba Torres** revisan el funcionamiento del sistema multimonitor del **IES El Caminàs** que implementaron ayer.

El departamento de informática del instituto ha planteado una nueva necesidad técnica para el equipo:

*«Antes de abrir las puertas a los estudiantes, el personal de mantenimiento necesita realizar un **test de calibración y diagnóstico matutino** de los sensores y terminales. Para comprobar que el lector óptico del vestíbulo lee bien, necesitan que el sistema emita un código de prueba con una **secuencia reproducible y predecible** (para contrastarlo con un patrón de referencia). Además, el sensor térmico del vestíbulo debe simular una pequeña fluctuación decimal ambiental y comprobar si el hardware de respaldo por radiofrecuencia (NFC) responde correctamente».*

Laia Claramunt toma el mando en la pizarra:

«Hasta ahora solo hemos utilizado el constructor por defecto `new Random()` para pedir números enteros. Pero los objetos son mucho más versátiles:

*1. Las clases predefinidas pueden ofrecer **constructores con parámetros**, como `new Random(semilla)`, que nos permiten fijar un estado inicial concreto en el Heap.*
*2. Los objetos tienen métodos con diferentes **tipos de retorno**: no solo devuelven enteros con `nextInt()`, sino también números decimales con `nextDouble()` y estados lógicos con `nextBoolean()`.*

*Hoy aprenderemos a invocar métodos pasando argumentos, a capturar diferentes tipos de retorno en variables de memoria y evolucionaremos `ControlAccesoQR` a la versión **v1.3**».*

---

2. Fundamento teórico: constructores con parámetros, firmas y tipos de retorno

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    ANATOMÍA DE LA COMUNICACIÓN CON UN OBJETO             │
├──────────────────────────────────────────────────────────────────────────┤
│ tipoRetorno variable = objeto.nombreMetodo( argumento1, argumento2... ); │
│      ▲                  ▲        ▲                ▲                     │
│      │                  │        │                └─ Parámetros que ENTRAN │
│      │                  │        └─ Acción solicitada                    │
│      │                  └─ Referencia en el Stack que apunta al objeto en Heap │
│      └─ Dato que SALE del método tras su ejecución                       │
└──────────────────────────────────────────────────────────────────────────┘
```

A. CONSTRUCTORES CON PARÁMETROS FRENTE A CONSTRUCTORES VACÍOS

Un **constructor** es un método especial de la clase cuya misión exclusiva es inicializar el estado del objeto en el momento exacto de su creación con `new`.
* **Constructor por defecto (sin argumentos):**
`Random r = new Random();`
Inicializa la semilla interna utilizando el reloj del procesador (`System.currentTimeMillis()`). Cada ejecución produce secuencias numéricas distintas.
* **Constructor con parámetros:**
`Random rAuditoria = new Random(123456789L);`
Pasa un valor inicial (`long`) para fijar la semilla en el Heap. Permite que el generador sea **completamente determinista y reproducible**, ideal para bancos de pruebas de software, auditorías y calibraciones de hardware.

B. TIPOS DE RETORNO Y MÉTODOS DE LA CLASE `Random`

Un método es un bloque de instrucciones asociado a un objeto que realiza una tarea y puede (o no) devolver un valor al punto del programa donde fue invocado:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                  MÉTODOS PREDEFINIDOS DE INSTANCIA EN RANDOM             │
├──────────────────────┬──────────────────┬────────────────────────────────┤
│ Firma del método     │ Tipo de retorno  │ Rango de datos que produce     │
├──────────────────────┼──────────────────┼────────────────────────────────┤
│ nextInt(int bound)   │ int              │ Entero en el rango [0, bound - 1] │
├──────────────────────┼──────────────────┼────────────────────────────────┤
│ nextDouble()         │ double           │ Decimal en el rango [0.0, 1.0) │
├──────────────────────┼──────────────────┼────────────────────────────────┤
│ nextBoolean()        │ boolean          │ Devuelve de forma pseudoaleatoria │
└──────────────────────┴──────────────────┴────────────────────────────────┘
```

```
|                        |                  | true o false con probabilidad del 50 %  |
└────────────────────────┴──────────────────┴─────────────────────────────────────────┘
```

Regla de asignación estricta: la variable receptora debe coincidir en tipo con el valor devuelto por el método. Si un método devuelve `boolean`, no podemos asignarlo a un `int`; si devuelve `double`, podemos guardarlo en `double` o forzar un estrechamiento explícito con *casting* a `(int)`.

---

3. Evolución del caso guía a `ControlAccesoQR v1.3`

Ampliamos el programa incorporando la calibración técnica matutina del IES El Caminàs:
* Instanciamos un generador de calibración con semilla fija (`SEMILLA_CALIBRACION`).
* Generamos una pequeña fluctuación decimal con `nextDouble()` sobre la temperatura del vestíbulo.
* Simulamos el estado del sensor de respaldo NFC mediante `nextBoolean()`.

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.3)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.3 (Evolucion: Constructores con Parametros y Tipos de Retorno)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\\terminal\\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0
    SEMILLA_CALIBRACION <- 987654321

    Definir terminalId Como Entero
    Definir tempVestibulo, fluctuacionTermica, tempCalibrada Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero

    // [NUEVO DÍA 15] Variables para retornos de calibración (código determinista, real y lógico)
    Definir codigoTestCalibracion Como Entero
    Definir sensorNfcOperativo Como Logico

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.3 - Diagnostico y Calibracion)"
    Escribir "ID del terminal principal:"
    Leer terminalId
    Escribir "Temperatura base del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
```

```
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    // Generación dinámica en producción (vestíbulo y puerta norte)
    codigoTokenVestibulo <- Aleatorio(1000, 9999)
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)

    // [NUEVO DÍA 15] Simulación de calibración técnica con retorno decimal y lógico
    codigoTestCalibracion <- 5555 // En Java procederá del generador con semilla fija
    fluctuacionTermica <- 0.45    // En Java: generador.nextDouble()
    tempCalibrada <- tempVestibulo + fluctuacionTermica
    sensorNfcOperativo <- Verdadero // En Java: generador.nextBoolean()

    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:             ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :       #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:    ", tokenResumenVestibulo
    Escribir "TOKEN PUERTA NORTE: ", tokenResumenPuertaNorte
    Escribir "PERSONA:            ", nombrePersona, " | PERFIL: ", perfilPersona
    Escribir "IDENTIFICADOR:      ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-------------------------------------------------------------------------"
    Escribir "HORARIO:            Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:        ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "AFORO (", aforoTotal, "):       ", redon(porcentajeOcupacionReal * 100) / 100, " % (Panel: ", porcentajeOcupacionEntero, " %)"
    Escribir "ACTIVO:             ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "-------------------------------------------------------------------------"
    Escribir "DIAGNOSTICO MATUTINO DE SENSORES (CALIBRACION CON SEMILLA):"
    Escribir "Codigo patron test: #", codigoTestCalibracion
    Escribir "Temp. calibrada:    ", tempCalibrada, " C (Fluctuacion: +", fluctuacionTermica, " C)"
    Escribir "Sensor NFC soporte: Activo (", sensorNfcOperativo, ")"
    Escribir "REGISTRO LOG:       ", RUTA_LOGS
    Escribir "========================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.3)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.3:
 * Constructores con parámetros (semilla fija), tipos de retorno (double, boolean)
 * y calibración diagnóstica de sensores de hardware.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.3 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // ------------------------------------------------------------------
        // 1. CONSTANTES INMUTABLES DEL SISTEMA (Configuración corporativa)
        // ------------------------------------------------------------------
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
```

```
// [NUEVO DÍA 15] Semilla fija para pruebas de calibración repetibles
final long SEMILLA_CALIBRACION = 987654321L;
final double MAX_VARIACION_TERMICA = 0.5;

// ---------------------------------------------------------------------
// 2. DECLARACIÓN DE VARIABLES Y RESERVA DE OBJETOS EN MEMORIA
// ---------------------------------------------------------------------
Scanner teclado = new Scanner(System.in);

// Generadores independientes para los dos monitores (Semana 1 / Día 14)
Random generadorVestibulo   = new Random();
Random generadorPuertaNorte = new Random();

// [NUEVO DÍA 15] Constructor con parámetro: generador determinista para diagnóstico
Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

// Variables de hardware y terminal
int terminalId;
double tempVestibulo;

// Variables de identidad y perfil de la persona
String nombrePersona;
String dniPersona;
char perfilPersona;
boolean esEntrada;

// Variables de cómputo horario y estancia
int horaEntrada, minutoEntrada;
int horaSalida, minutoSalida;
int minutosTotalesEntrada;
int minutosTotalesSalida;
int minutosEstanciaTotal;

// Dos tokens independientes generados para cada monitor
String tokenResumenVestibulo;
String tokenResumenPuertaNorte;

// Variables de aforo del recinto y numeración de ticket
int personasEnCentro = 1200;
int aforoDisponible = 800;
int idUltimoFichaje = 1042;

// Variables de diagnóstico y tiempo de servicio (uptime)
int segundosActividadTerminal;
int horasUptime;
int minutosUptime;
int segundosUptime;

// Variables de aforo consolidado y porcentajes
int aforoTotal;
double porcentajeOcupacionReal;
int porcentajeOcupacionEntero;

// Códigos pseudoaleatorios independientes
int codigoTokenVestibulo;
int codigoTokenPuertaNorte;

// [NUEVO DÍA 15] Variables para almacenar distintos tipos de retorno de métodos
int codigoTestCalibracion;
double fluctuacionTermica;
double tempCalibrada;
boolean sensorNfcOperativo;

// ---------------------------------------------------------------------
// 3. CAPTURA INTERACTIVA DE DATOS
// ---------------------------------------------------------------------
System.out.println("================================================");
System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
System.out.println("   Versión 1.3 (diagnóstico y retornos de objetos) ");
System.out.println("================================================");
System.out.print("ID del terminal principal: ");
terminalId = teclado.nextInt();

System.out.print("Temperatura del sensor (ºC): ");
tempVestibulo = teclado.nextDouble();

System.out.print("Segundos de actividad del terminal (uptime): ");
segundosActividadTerminal = teclado.nextInt();

teclado.nextLine(); // Limpieza obligatoria del buffer de teclado

System.out.print("DNI de la persona: ");
dniPersona = teclado.nextLine();

System.out.print("Nombre completo: ");
nombrePersona = teclado.nextLine();

System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
perfilPersona = teclado.next().charAt(0);

esEntrada = true;
```

```
        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ------------------------------------------------------------------
        // 4. PROCESAMIENTO SECUENCIAL Y OPERACIONES MATEMÁTICAS
        // ------------------------------------------------------------------
        // Conversión horaria a minutos y cálculo de permanencia neta
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        // Operadores unarios de incremento y decremento de estado
        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        // Generación dinámica de tokens para los dos monitores autónomos
        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        // [NUEVO DÍA 15] Invocación de métodos con distintos tipos de retorno:
        // 1. Método con retorno entero (int): código patrón reproducible
        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);

        // 2. Método con retorno decimal (double): devuelve un valor entre [0.0 y 1.0)
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA; // Escala la variación a un máximo de 0.5 ºC
        tempCalibrada = tempVestibulo + fluctuacionTermica;

        // 3. Método con retorno booleano (boolean): simula el estado del sensor de respaldo
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        // Composición acumulativa de ambos tokens
        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenPuertaNorte += "-T102"; // ID fijo asignado al monitor norte
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        // Descomposición temporal exacta mediante división entera y módulo (%)
        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // Cálculo de porcentaje con casting explícito a (double) para evitar división a 0
        aforoTotal = personasEnCentro + aforoDisponible;
        porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA PROFESIONAL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n======================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                   ");
        System.out.println("======================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s (Monitor autónomo)%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR:        %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("----------------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("AFORO (%04d):         %6.2f %% (Panel: %03d %%)%n", aforoTotal, porcentajeOcupacionReal, porcentajeOcupacionEntero);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds%n", horasUptime, minutosUptime, segundosUptime);
        System.out.println("----------------------------------------------------------------------");
        System.out.println("DIAGNÓSTICO TÉCNICO DE SENSORES (CALIBRACIÓN CON SEMILLA):");
        System.out.printf("CÓDIGO PATRÓN TEST:   #%04d (Secuencia verificada)%n", codigoTestCalibracion);
        System.out.printf("SENSOR TÉRMICO CALIB: %.2f ºC (Fluctuación calculada: +%.2f ºC)%n", tempCalibrada, fluctuacionTermica);
        System.out.printf("SENSOR NFC RESPALDO:  Activo (%b)%n", sensorNfcOperativo);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("======================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de código** centradas en constructores especializados, firmas de métodos y manipulación de diferentes tipos de retorno:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). MÉTODOS CON RETORNO `double` Y `boolean` EN TU PROYECTO

- **Objetivo:** Invocar sobre un objeto predefinido al menos un método que devuelva un valor decimal (`nextDouble()`) y otro que devuelva un valor lógico (`nextBoolean()`) adaptados al dominio de tu proyecto propio.
- **Instrucciones:** Abre tu archivo `MiProyecto.java` (en versión v1.2) e incorpora las siguientes llamadas:
- En **Aventura conversacional**: invocar `nextBoolean()` para determinar si un cofre encontrado contiene trampa y `nextDouble()` para calcular un factor multiplicador de impacto crítico entre `1.0` y `1.5`.
- En **Motor de recomendación**: invocar `nextDouble()` para simular un factor de afinidad normalizado entre `0.0` y `1.0`, y `nextBoolean()` para indicar si el usuario ya ha consumido previamente el ítem sugerido.
- En **Simulador de físicas 2D**: invocar `nextDouble()` para generar un coeficiente de elasticidad superficial aleatorio entre `0.0` y `1.0`, y `nextBoolean()` para establecer el sentido vectorial del viento (+ / -).
- En **Bóveda de contraseñas**: invocar `nextBoolean()` para simular si la credencial requiere obligatoriamente doble factor de autenticación (2FA), y `nextDouble()` para calcular un índice ponderado de entropía de la clave.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). LA FÁBRICA DE NÚMEROS REALES ESCALADOS

- **Contexto técnico:** El método `random.nextDouble()` siempre devuelve un valor en el intervalo semiabierto $[0.0, 1.0)$.
- **Desafío de la kata:** Diseña una fórmula matemática secuencial en una sola línea que utilice `random.nextDouble()` para generar un número decimal pseudoaleatorio acotado exactamente en un rango real $[MIN, MAX)$ arbitrario (por ejemplo, temperaturas entre 18.5 y 26.5 ºC): $$\text{ResultadoDecimal} = MIN + (\text{random.nextDouble}() \times (MAX - MIN))$$
- **Evidencia de maestría:** Prueba tu fórmula en una clase de test con $MIN = 12.5$ y $MAX = 18.0$. Ejecútala 5 veces y formatea la salida mediante `System.out.printf("Valor generado: %.3f%n", resultadoDecimal)`. Comprueba que ningún valor queda por debajo de 12.5 ni alcanza los 18.0.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). EL DETERMINISMO DE LA SEMILLA EN AUDITORÍA DE SOFTWARE

- **Contexto de ingeniería:** En auditorías de ciberseguridad y simulaciones científicas, los experimentos deben ser reproducibles al 100 %. Si dos peritos informáticos ejecutan el software, deben obtener exactamente los mismos resultados.
- **Misión de la kata:**

a. Crea una clase de pruebas llamada `AuditoriaSemilla.java`.
b. Instancia un objeto `Random` fijando una semilla concreta: `new Random(42L)`.
c. Genera y muestra consecutivamente: un entero con `nextInt(100)`, un decimal con `nextDouble()` y un booleano con `nextBoolean()`.
d. Ejecuta el programa una vez. Copia los tres resultados en tu libreta.
e. Ejecuta el programa por segunda y tercera vez.
f. **Pregunta técnica para tu cuaderno:** ¿Por qué los números son rigurosamente idénticos en cada ejecución si estamos usando una clase llamada «Random»? Escribe dos líneas de comentario explicando la diferencia entre **aleatoriedad física pura** y **generación pseudoaleatoria determinista**.

3.1.4 Día 16 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del jueves 8 de octubre. Llegamos a la última sesión semanal de Programación (mañana viernes es festivo del 9 de octubre). En la sala técnica de **AzaharTech**, **Pau Ferrer** muestra una duda técnica a **Alba Torres** y a **Laia Claramunt**:

«El martes me explicasteis que al hacer `new Random()` dos veces se crean dos generadores independientes en el Heap, y que al hacer `g2 = g1` solo se copia la dirección de memoria creando un alias. Lo entendí en la teoría, pero ¿cómo puedo comprobarlo yo mismo dentro de mi código Java sin herramientas externas?».

Alba Torres abre la consola de IntelliJ:

*«Java nos ofrece los mecanismos en el propio lenguaje. Si le pides a Java que imprima una variable de referencia por consola, el compilador imprime el nombre de la clase seguido de una arroba (`@`) y un código alfanumérico en hexadecimal: es la **huella de identidad de ese objeto en la memoria física**.*

*Además, cuando comparamos dos objetos con el operador `==`, Java no compara lo que hay dentro del objeto; **compara si ambas variables apuntan a la misma dirección de memoria en el Heap**.*

*Hoy cerraremos la primera semana consolidando la versión **v1.3** de vuestro proyecto propio y utilizaremos la propia consola para auditar identidades de memoria y referencias nulas».*

---

2. Fundamento teórico: identidad de objetos, operador `==` e impresión de referencias

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        IDENTIDAD DE OBJETOS EN LA MEMORIA RAM            │
├──────────────────────────────────┬───────────────────────────────────────┤
│ 1. IMPRESIÓN POR DEFECTO         │ 2. COMPARACIÓN CON EL OPERADOR '=='   │
├──────────────────────────────────┼───────────────────────────────────────┤
│ Random r = new Random();         │ Random a = new Random();              │
│ System.out.println(r);           │ Random b = new Random();              │
│                                  │ Random c = a;                         │
│ Salida: java.util.Random@7ffe8a1 │ • (a == b) devuelve FALSE (dos objetos en Heap).  │
│         └──────────────┘ └─────┘ │ • (a == c) devuelve TRUE (mismo objeto en Heap).  │
│              Clase       Dirección│                                       │
└──────────────────────────────────┴───────────────────────────────────────┘
```

A. LA HUELLA DE UN OBJETO EN MEMORIA: `NombreClase@hash`

Cuando le pasas un objeto a `System.out.println()`, Java invoca internamente a su método de representación textual (`toString()`). Por defecto, para cualquier objeto estándar de la biblioteca que no redefina este texto, imprime: $$ \text{NombreDelPaquete.NombreClase} + \text{"@"} + \text{CódigoHexadecimalDeMemoria}$$
* Si dos variables imprimen códigos distintos (*ej. `@7ffe8a1` y `@1a2b3c`*), tenemos la certeza matemática de que residen en dos posiciones de memoria independientes del Heap.
* Si imprimen el mismo código, son dos punteros hacia la misma instancia física.

B. EL OPERADOR `==` EN TIPOS REFERENCIA

A diferencia de los tipos primitivos (donde `5 == 5` compara valores numéricos), en los objetos el operador `==` **compara direcciones de memoria**:
* Evalúa si ambas referencias del Stack apuntan al **mismo bloque del Heap**.
* No compara si los generadores están configurados igual o si tienen la misma semilla; solo compara si **son físicamente el mismo objeto**.

C. EL LITERAL `null` FRENTE A UNA REFERENCIA ACTIVA

- Si imprimimos por consola una variable que vale `null`:

```
Random generadorVacio = null;
System.out.println(generadorVacio); // Imprime el texto literal: "null"
```

  La JVM no falla al imprimirlo; simplemente avisa de que el puntero está vacío.

- El fallo catastrófico ocurre al intentar acceder a través de él:

```
generadorVacio.nextInt(); // ¡CRASH!: NullPointerException en tiempo de ejecución
```

---

3. Verificación de identidad en `ControlAccesoQR v1.3`

Vamos a ver cómo auditar las referencias del caso guía del IES El Caminàs utilizando únicamente instrucciones de consola:

```
// Comprobación de identidad de los generadores de vestíbulo y puerta norte
System.out.println("\n--------------------------------------------------------------------");
System.out.println("AUDITORÍA DE IDENTIDAD EN MEMORIA RAM (HEAP):");
System.out.println("Huella monitor vestíbulo:    " + generadorVestibulo);
System.out.println("Huella monitor puerta norte: " + generadorPuertaNorte);

// Verificación con el operador ==
boolean sonElMismoObjeto = (generadorVestibulo == generadorPuertaNorte);
System.out.println("¿Apuntan al mismo espacio en memoria?: " + sonElMismoObjeto); // Devuelve false
```

- Puede comprobarse en la salida de consola que ambas huellas (`Random@...`) son diferentes y que la comparación devuelve `false` , garantizando que cada monitor tiene su propio ciclo de vida.

---

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de consolidación técnica** sobre identidad de objetos y tipos de retorno:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). CONSOLIDACIÓN V1.3 EN EL PROYECTO PROPIO

- **Objetivo:** Asegurar que tu clase maestra `MiProyecto.java` incorpora todas las novedades de la primera semana del Sprint 2:

a. Al menos dos instancias independientes creadas con `new Random()` para propósitos diferenciados.

b. Una tercera instancia creada con constructor parametrizado (`new Random(semillaFija)`) para pruebas repetibles, almacenando la semilla en una constante `final long` .

c. Invocación de métodos que retornen `int` (`nextInt`), `double` (`nextDouble`) y `boolean` (`nextBoolean`).

- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**: dos generadores independientes para las tiradas de héroe y enemigo, más un generador de calibración con semilla para reproducir siempre la misma mazmorra de pruebas.
- En **Motor de recomendación**: dos generadores para variaciones de catálogo y afinidad, más un generador determinista para auditorías del algoritmo de sugerencias.
- En **Simulador de físicas 2D**: dos generadores para velocidades y ángulos de partículas, más un generador con semilla para reproducir la misma trayectoria balística de test.
- En **Bóveda de contraseñas**: dos generadores para PIN y salting dinámicos, más un generador con semilla fija para auditorías forenses repetibles.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). LA PRUEBA DE LA HUELLA DE MEMORIA Y EL OPERADOR `==`

- **Contexto técnico:** Demostrar mediante código por consola la diferencia entre alias y objetos independientes.

- **Misión de la kata:**

a. En una clase de prueba llamada `AuditoriaIdentidad.java`, escribe el siguiente experimento:

```
Random motorOriginal = new Random();
Random motorAlias = motorOriginal;   // Copia de referencia (puntero)
Random motorClon = new Random();     // Nuevo objeto con new

System.out.println("Huella motorOriginal: " + motorOriginal);
System.out.println("Huella motorAlias:    " + motorAlias);
System.out.println("Huella motorClon:     " + motorClon);

boolean pruebaAlias = (motorOriginal == motorAlias);
boolean pruebaClon  = (motorOriginal == motorClon);

System.out.println("¿Original y Alias son el mismo objeto?: " + pruebaAlias);
System.out.println("¿Original y Clon son el mismo objeto?:  " + pruebaClon);
```

b. Ejecuta el programa y analiza la salida.

c. **Pregunta técnica para tu cuaderno:**
Explica en dos líneas por qué `pruebaAlias` da `true` y `pruebaClon` da `false`, haciendo referencia al Stack y al Heap.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). COMPORTAMIENTO DE `null` EN CONSOLA

- **Contexto de robustez:** Conocer cómo reacciona Java ante variables no inicializadas o desconectadas.
- **Misión de la kata:**

a. En la misma clase de prueba, añade las siguientes líneas al final:

```
Random motorDesconectado = null;

// 1. Impresión directa de la referencia nula:
System.out.println("Estado de motorDesconectado: " + motorDesconectado);

// 2. Operación de comparación segura:
boolean estaInicializado = (motorDesconectado != null);
System.out.println("¿El objeto está listo para usarse?: " + estaInicializado);
```

b. Comprueba que el programa compila y ejecuta limpiamente imprimiendo `null` y `false`.

c. Ahora escribe deliberadamente: `motorDesconectado.nextInt();` y observa el mensaje de error `NullPointerException` en la consola.

d. Elimina la línea conflictiva para dejar el código limpio y libre de errores.

---

5. Cierre formal en Git y sincronización en IntelliJ

Cada estudiante finaliza y entrega el avance de la primera semana del Sprint 2 desde la interfaz gráfica de IntelliJ IDEA:

1. Aplica el autoformateo oficial: `Ctrl + Alt + L`.
2. Optimiza las importaciones para limpiar dependencias no usadas: `Ctrl + Alt + O`.
3. Abre el panel lateral **Commit** (`Alt + 0` o `Ctrl + K`).
4. Selecciona tu archivo `pr/src/MiProyecto.java` (y su diseño en PSeInt si procede).
5. Escribe el mensaje convencional de cierre de semana:

```
feat(pr): consolidar version 1.3 con gestion de objetos, semillas y verificacion de memoria
```

6. Pulsa **Commit and Push...** y confirma el envío al servidor remoto de GitHub.
7. Comprueba en el navegador que tu repositorio muestra la versión v1.3 actualizada sin errores de compilación.

3.2 Semana 5. Clase Math, wrappers y métodos estáticos

Tras asentar el funcionamiento de la memoria dinámica en el montículo (Heap) durante la primera semana, el equipo de desarrollo de AzaharTech aborda en esta segunda semana la frontera entre los métodos ligados a objetos vivos y los métodos estáticos (de clase), que ofrecen utilidades de cálculo directo sin necesidad de reservar memoria con el operador new.

A lo largo de estas sesiones nos adentraremos en el paquete java.lang para exprimir la clase Math (acotando lecturas de sensores con Math.min y Math.max, calculando desviaciones horarias respecto a la jornada de 8 horas con Math.abs y aplicando potencias y redondeos matemáticos), al tiempo que descubriremos el papel de las clases envoltorio (Wrappers) como Integer y Double para parsear texto a magnitudes numéricas.

A través de nuestro dojo de katas diarias y la evolución continua del código, dotaremos a nuestro software de precisión, elegancia y robustez matemática, aprendiendo a elegir la herramienta idónea para cada necesidad de ingeniería.

3.2.1 Día 17 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del martes 13 de octubre (ayer lunes fue festivo nacional del 12 de octubre). En la sala técnica de **AzaharTech**, **Pau Ferrer** tiene abierta la consola de IntelliJ IDEA con cara de desconcierto.

Ha intentado calcular la diferencia absoluta de tiempo y acotar las lecturas del sensor de temperatura del **IES El Caminàs**, y ha escrito en el código:

```
Math calculadora = new Math(); // Error de compilación inmediato
```

IntelliJ subraya la instrucción en rojo con un mensaje tajante: *«'Math()' has private access in 'java.lang.Math'»*.

Pau comenta al equipo:

«La semana pasada aprendimos que para usar una clase predefinida como `Random` o `Scanner` teníamos que usar `new` para crear el objeto en el Heap. He intentado hacer `new Math()` para calcular el valor absoluto y el máximo de aforo, y el compilador me dice que el constructor es privado y que no puedo instanciarla. ¿Por qué con `Random` sí y con `Math` no?».

Alba Torres y **Laia Claramunt** sonríen. Alba toma el teclado:

*«Pau, acabas de descubrir la gran frontera de la POO: la diferencia entre **métodos de instancia** y **métodos estáticos (de clase)**.*

Un generador `Random` necesita almacenar un estado interno en el Heap (su semilla) que cambia con cada tirada; por eso requiere un objeto individual creado con `new`. Pero calcular un valor absoluto, un valor máximo o el redondeo de un número no requiere que la máquina recuerde nada de la operación anterior: es una función matemática pura.

*La biblioteca estándar de Java incluye la clase `java.lang.Math`, repleta de **métodos estáticos**: herramientas que se invocan directamente desde el nombre de la clase, sin instanciar objetos con `new`. Hoy aprenderemos a utilizarlos para acotar datos y calcular desviaciones de tiempo de forma puramente secuencial»*.

---

2. Fundamento teórico: métodos de instancia frente a métodos estáticos y la clase `Math`

```
┌──────────────────────────────────────────────────────────────────────┐
│                  MÉTODOS DE INSTANCIA VS. MÉTODOS ESTÁTICOS          │
├──────────────────────────────────┬───────────────────────────────────┤
│ 1. MÉTODOS DE INSTANCIA          │ 2. MÉTODOS ESTÁTICOS (De clase)   │
├──────────────────────────────────┼───────────────────────────────────┤
│ • Requieren crear un objeto con  │ • NO requieren instanciar objetos con 'new'.      │
│   el operador 'new' en el Heap.  │ • Pertenecen a la clase en sí, no a una instancia. │
│ • Operan sobre el estado interno │ • Son funciones de cálculo puro y utilidades que  │
│   del objeto concreto.           │   reciben datos y devuelven un resultado.         │
│ • Invocación: objeto.metodo()    │ • Invocación: NombreClase.metodo()                │
└──────────────────────────────────┴───────────────────────────────────┘
```

```
|   ej. generador.nextInt(9000);    |   ej. Math.max(a, b);                                  |
└────────────────────────────────────┴────────────────────────────────────────────────────────┘
```

A. EL PAQUETE UNIVERSAL `java.lang`

A diferencia de `Scanner` o `Random` (que exigen escribir `import java.util...` al principio del archivo), la clase `Math` reside en el paquete **`java.lang`**. Este paquete contiene las clases estructurales de Java (`System`, `String`, `Math`, `Integer`) y **la JVM lo importa de forma automática y transparente en todos los archivos**, por lo que no requiere ninguna sentencia `import`.

B. MÉTODOS ESENCIALES DE `java.lang.Math`

Todos los métodos de la clase `Math` son estáticos, públicos y reciben argumentos numéricos:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        CATÁLOGO BÁSICO DE LA CLASE MATH                      │
├──────────────────────┬──────────────────┬────────────────────────────────────┤
│ Invocación estática  │ Tipo de retorno  │ Efecto técnico                     │
├──────────────────────┼──────────────────┼────────────────────────────────────┤
│ Math.abs(x)          │ Mismo tipo que x │ Devuelve el valor absoluto (sin signo). │
├──────────────────────┼──────────────────┼────────────────────────────────────┤
│ Math.max(a, b)       │ Mayor de los dos │ Devuelve el valor más alto entre a y b. │
├──────────────────────┼──────────────────┼────────────────────────────────────┤
│ Math.min(a, b)       │ Menor de los dos │ Devuelve el valor más bajo entre a y b. │
├──────────────────────┼──────────────────┼────────────────────────────────────┤
│ Constante Math.PI    │ double           │ Constante pública: 3.141592653589793 │
└──────────────────────┴──────────────────┴────────────────────────────────────┘
```

C. LA TÉCNICA DEL ACOTAMIENTO EN RANGO (*CLAMP*) SIN USAR CONDICIONALES

En sistemas embebidos y terminales físicos, los sensores pueden registrar picos de lectura erróneos debido a interferencias eléctricas (por ejemplo, que el sensor del vestíbulo marque 55.0 ºC o -10.0 ºC).

Combinando **`Math.max()`** y **`Math.min()`**, podemos acotar cualquier lectura dentro de un rango de seguridad $[MIN, MAX]$ en una sola línea de código secuencial, **sin necesidad de recurrir a estructuras condicionales `if`**:

$$\text{ValorAcotado} = \text{Math.max}(MIN, \text{Math.min}(\text{lecturaBruta}, MAX))$$

- Si `lecturaBruta` supera el MAX, `Math.min` la recorta a MAX.
- Si `lecturaBruta` desciende por debajo de MIN, `Math.max` la eleva a MIN.

---

3. Evolución del caso guía a `ControlAccesoQR v1.4`

Ampliamos el programa incorporando el uso de métodos estáticos de `Math`:

1. Definimos una **jornada laboral/centro de referencia de 480 minutos (8 horas)**, representativa del personal docente y PAS.
2. Calculamos con **`Math.abs()`** la desviación absoluta en minutos que la estancia de la persona representa respecto a esa jornada de 8 horas.
3. Acotamos la lectura térmica del sensor mediante **`Math.max()` y `Math.min()`** para que el valor visualizado en pantalla nunca baje de 15.0 ºC ni supere los 35.0 ºC.
4. Acotamos el aforo con **`Math.min()`** para garantizar que el panel nunca muestre más personas que la capacidad máxima física del centro (2000 plazas).

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.4)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.4 (Evolucion: metodos estaticos y funciones de acotacion)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero

    // [NUEVO DÍA 17] Constantes para límites de seguridad y jornada de referencia de 8 horas
    Definir JORNADA_BASE_MINUTOS Como Entero
    Definir TEMP_MIN_SEGURA, TEMP_MAX_SEGURA Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
```

```
RUTA_LOGS <- ".\\caminas\\terminal\\logs"
MINUTOS_POR_HORA <- 60
SEGUNDOS_POR_HORA <- 3600
SEGUNDOS_POR_MINUTO <- 60
FACTOR_PORCENTAJE <- 100.0
SEMILLA_CALIBRACION <- 987654321
JORNADA_BASE_MINUTOS <- 480 // 8 horas de jornada laboral/centro de referencia
TEMP_MIN_SEGURA <- 15.0
TEMP_MAX_SEGURA <- 35.0

Definir terminalId Como Entero
Definir tempVestibulo, fluctuacionTermica, tempCalibrada, tempSeguraAcotada Como Real
Definir nombrePersona, dniPersona Como Cadena
Definir perfilPersona Como Caracter
Definir esEntrada Como Logico

Definir horaEntrada, minutoEntrada Como Entero
Definir horaSalida, minutoSalida Como Entero
Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
personasEnCentro <- 1200
aforoDisponible <- 800
idUltimoFichaje <- 1042

Definir segundosActividadTerminal Como Entero
Definir horasUptime, minutosUptime, segundosUptime Como Entero
Definir aforoTotal Como Entero
Definir porcentajeOcupacionReal Como Real
Definir porcentajeOcupacionEntero Como Entero

Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero
Definir codigoTestCalibracion Como Entero
Definir sensorNfcOperativo Como Logico

// [NUEVO DÍA 17] Variables para cálculos matemáticos con funciones abs, max y min
Definir desviacionJornadaMinutos Como Entero
Definir aforoSeguroVisualizado Como Entero

Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.4 - Funciones Matematicas)"
Escribir "ID del terminal principal:"
Leer terminalId
Escribir "Temperatura base del sensor (ºC):"
Leer tempVestibulo
Escribir "Segundos de actividad del terminal (uptime):"
Leer segundosActividadTerminal

Escribir "DNI de la persona:"
Leer dniPersona
Escribir "Nombre completo:"
Leer nombrePersona
Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
Leer perfilPersona

esEntrada <- Verdadero

Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
Leer horaEntrada
Leer minutoEntrada
Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
Leer horaSalida
Leer minutoSalida

minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

personasEnCentro <- personasEnCentro + 1
aforoDisponible <- aforoDisponible - 1
idUltimoFichaje <- idUltimoFichaje + 1

codigoTokenVestibulo <- Aleatorio(1000, 9999)
codigoTokenPuertaNorte <- Aleatorio(1000, 9999)

codigoTestCalibracion <- 5555
fluctuacionTermica <- 0.45
tempCalibrada <- tempVestibulo + fluctuacionTermica
sensorNfcOperativo <- Verdadero

// [NUEVO DÍA 17] Cálculo de desviación absoluta respecto a 8 horas (480 min)
desviacionJornadaMinutos <- abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS)

// Simulación de acotación en PSeInt:
Si tempCalibrada > TEMP_MAX_SEGURA Entonces
    tempSeguraAcotada <- TEMP_MAX_SEGURA
SiNo
    Si tempCalibrada < TEMP_MIN_SEGURA Entonces
        tempSeguraAcotada <- TEMP_MIN_SEGURA
    SiNo
        tempSeguraAcotada <- tempCalibrada
    FinSi
FinSi
```

```
    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    porcentajeOcupacionReal <- (personasEnCentro * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:             ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :       #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:    ", tokenResumenVestibulo
    Escribir "TOKEN PUERTA NORTE: ", tokenResumenPuertaNorte
    Escribir "PERSONA:            ", nombrePersona, " | PERFIL: ", perfilPersona
    Escribir "IDENTIFICADOR:      ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-------------------------------------------------------------------------"
    Escribir "HORARIO:            Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:        ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "DESVIACION JORNADA: ", desviacionJornadaMinutos, " min respecto a jornada completa (480 min)."
    Escribir "AFORO (", aforoTotal, "):       ", redon(porcentajeOcupacionReal * 100) / 100, " % (Panel: ", porcentajeOcupacionEntero, " %)"
    Escribir "ACTIVO:             ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "-------------------------------------------------------------------------"
    Escribir "TELEMETRÍA DE SENSORES (MATH):"
    Escribir "Sensor térmico real:  ", tempCalibrada, " C"
    Escribir "Sensor acotado seguro:", tempSeguraAcotada, " C [Rango 15.0 - 35.0 C]"
    Escribir "Sensor NFC soporte:   Activo (", sensorNfcOperativo, ")"
    Escribir "REGISTRO LOG:         ", RUTA_LOGS
    Escribir "========================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.4)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.4:
 * Invocación de métodos estáticos sin instanciación (java.lang.Math),
 * cálculo de desviaciones absolutas con Math.abs() y acotación segura con Math.min()/max().
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.4 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */
import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // ------------------------------------------------------------------
        // 1. CONSTANTES INMUTABLES DEL SISTEMA
        // ------------------------------------------------------------------
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
        final double MAX_VARIACION_TERMICA = 0.5;
        final long SEMILLA_CALIBRACION = 987654321L;

        // [NUEVO DÍA 17] Constantes de hardware y jornada laboral/centro de 8 horas
        final int JORNADA_BASE_MINUTOS = 480; // 8 horas de permanencia de referencia (docentes/PAS)
        final double TEMP_MIN_SEGURA = 15.0;  // Límite inferior admisible
        final double TEMP_MAX_SEGURA = 35.0;  // Límite superior admisible

        // ------------------------------------------------------------------
        // 2. DECLARACIÓN DE VARIABLES Y OBJETOS
        // ------------------------------------------------------------------
        Scanner teclado = new Scanner(System.in);

        // Métodos de instancia: requieren crear objetos con 'new' en el Heap
        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        int terminalId;
        double tempVestibulo;

        String nombrePersona, dniPersona;
```

```
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumenVestibulo;
        String tokenResumenPuertaNorte;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        int codigoTokenVestibulo;
        int codigoTokenPuertaNorte;

        int codigoTestCalibracion;
        double fluctuacionTermica;
        double tempCalibrada;
        boolean sensorNfcOperativo;

        // [NUEVO DÍA 17] Variables para resultados de métodos estáticos de Math
        int desviacionJornadaMinutos;
        double tempSeguraAcotada;
        int aforoSeguroVisualizado;

        // ------------------------------------------------------------------
        // 3. CAPTURA DE DATOS
        // ------------------------------------------------------------------
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.4 (métodos estáticos de la clase Math) ");
        System.out.println("================================================");
        System.out.print("ID del terminal principal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine(); // Limpieza de buffer

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ------------------------------------------------------------------
        // 4. PROCESAMIENTO SECUENCIAL
        // ------------------------------------------------------------------
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA;
        tempCalibrada = tempVestibulo + fluctuacionTermica;
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        // [NUEVO DÍA 17] Invocación de métodos estáticos de Math (sin operador new):
```

```
        // 1. Math.abs(): Calcula la desviación absoluta respecto a la jornada de 8 horas (480 min)
        desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);

        // 2. Acotación en rango [15.0 - 35.0] combinando Math.max() y Math.min() sin condicionales
        tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        // 3. Math.min(): Evita que el aforo reportado supere la capacidad física de 2000 plazas
        aforoTotal = personasEnCentro + aforoDisponible;
        aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenPuertaNorte += "-T102";
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) aforoSeguroVisualizado / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA PROFESIONAL (printf)
        // ------------------------------------------------------------------
        System.out.println("\n================================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                    ");
        System.out.println("================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR:        %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("----------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.printf("AFORO (%04d):         %6.2f %% (Panel: %03d %%)%n", aforoTotal, porcentajeOcupacionReal, porcentajeOcupacionEntero);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds%n", horasUptime, minutosUptime, segundosUptime);
        System.out.println("----------------------------------------------------------------");
        System.out.println("TELEMETRÍA Y CONTROL DE SENSORES (MÉTODOS ESTÁTICOS MATH):");
        System.out.printf("SENSOR TÉRMICO REAL:  %.2f ºC (Calibrado con fluctuación)%n", tempCalibrada);
        System.out.printf("SENSOR TÉRMICO ACOT.: %.2f ºC [Rango seguro: %.1f - %.1f ºC]%n", tempSeguraAcotada, TEMP_MIN_SEGURA, TEMP_MAX_SEGURA);
        System.out.printf("AFORO COMPUTADO:      %d personas (Acotado con Math.min)%n", aforoSeguroVisualizado);
        System.out.printf("SENSOR NFC RESPALDO:  Activo (%b)%n", sensorNfcOperativo);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre métodos estáticos, la clase `Math` y funciones de acotación matemática:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). MÉTODOS ESTÁTICOS DE `Math` EN TU PROYECTO

- **Objetivo:** Incorporar a tu clase `MiProyecto.java` (versión v1.3) al menos una llamada a `Math.abs()` para calcular una diferencia absoluta y una llamada a `Math.min()` o `Math.max()` para seleccionar un valor extremo.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**: calcular con `Math.abs()` la distancia entre la posición del héroe y la posición del cofre, y asegurar con `Math.max()` que la energía recuperada no baje de cero.
- En **Motor de recomendación**: calcular con `Math.abs()` la desviación absoluta entre la valoración del usuario y la media histórica del ítem, y usar `Math.min()` para seleccionar el límite de puntuación máxima admitida.
- En **Simulador de físicas 2D**: calcular con `Math.abs()` la magnitud escalar de la velocidad de choque, y usar `Math.max()` para asegurar que la masa de una partícula nunca descienda de un valor base.

- En **Bóveda de contraseñas**: calcular con `Math.abs()` la diferencia en días entre la fecha de último cambio de clave y la fecha recomendada, y usar `Math.min()` para acotar el número de intentos permitidos.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO): LA FUNCIÓN DE ACOTAMIENTO EN RANGO (*CLAMP*)

- **Contexto técnico:** En simuladores y sistemas de control, la función *clamp* confina un valor para que nunca descienda de un mínimo ni supere un máximo combinando dos llamadas anidadas.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaAcotamiento.java`, crea un cálculo secuencial que reciba un valor decimal y lo confine en el rango $[10.0, 50.0]$ anidando `Math.max` y `Math.min` en una sola expresión:

```
double valorOriginal = 72.8; // Prueba sucesivamente con 5.0, 25.0 y 72.8
double valorAcotado = Math.max(10.0, Math.min(valorOriginal, 50.0));
System.out.printf("Original: %.1f -> Acotado: %.1f%n", valorOriginal, valorAcotado);
```

- Comprueba el comportamiento con los tres casos posibles:
- Si introduces `5.0` (por debajo del mínimo), la expresión devuelve `10.0`.
- Si introduces `25.0` (dentro del rango), la expresión devuelve `25.0`.
- Si introduces `72.8` (por encima del máximo), la expresión devuelve `50.0`.
- Redacta dos líneas de comentario en tu código explicando el orden de evaluación: **cómo la llamada interna a `Math.min()` fija el techo superior y cómo la llamada externa a `Math.max()` asegura el suelo inferior**.

KATA 3 (CINTURÓN NEGRO / KATA «HACKER AZAHARTECH»): LA CAJA DE HERRAMIENTAS ESTÁTICA SIN OPERADOR `new`

- **Contexto de ingeniería:** Comprender por qué las clases de utilidades puras no necesitan reservar memoria en el Heap ni crear objetos.
- **Misión de la kata:**
- Crea una clase de prueba temporal llamada `TestCajaHerramientas.java`.
- Escribe deliberadamente la línea:

```
Math m = new Math(); // Intento de instanciación
```

- Observa cómo el compilador de Java prohíbe la creación del objeto porque `Math` no es un molde para crear ejemplares, sino un catálogo de herramientas compartidas. Comenta esa línea para que el programa pueda compilar.
- Demuestra el uso directo de las herramientas de `Math` sin variables previas calculando el área de cobertura circular de una antena del vestíbulo ($A = \pi \times r^2$):

```
double radioMetros = 4.5;
// Usamos la constante pública Math.PI directamente desde la clase
double areaCobertura = Math.PI * (radioMetros * radioMetros);
System.out.printf("Área de cobertura calculada: %.2f m²%n", areaCobertura);
```

- Anota en tu cuaderno técnico la conclusión: *«A diferencia de `Random`, donde cada generador necesita su propio objeto para guardar su semilla en el Heap, los métodos y constantes de `Math` pertenecen a la clase y están disponibles inmediatamente sin consumir memoria de instanciación»*.

3.2.2 Día 18 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del miércoles 14 de octubre. En la sala técnica de **AzaharTech**, **Pau Ferrer** se enfrenta a dos nuevos retos de cálculo geométrico y visual para el **IES El Caminàs**:

1. **La distancia de la antena del vestíbulo:** Para ubicar con precisión el lector óptico en el techo, los técnicos de redes necesitan calcular la distancia diagonal directa (la hipotenusa) entre la antena del techo y el punto de paso de la persona en el suelo. Pau conoce la distancia en el eje horizontal ($x = 3.0\text{ m}$) y en el eje vertical ($y = 4.0\text{ m}$), y ha intentado calcular la hipotenusa escribiendo:

```
double hipotenusa = (x ^ 2) + (y ^ 2); // Error conceptual grave en Java
```

La consola imprime un número disparatado porque en Java el símbolo `^` **no calcula potencias**, sino que realiza una operación lógica a nivel de bits llamada *XOR*.

2. **El indicador del panel LED.** Cuando la ocupación del recinto es de $60.95\text{ }\%$, el panel luminoso del vestíbulo debe mostrar un número entero. Como en el Sprint 1 aplicamos un casting `(int) 60.95`, el sistema muestra `60 %`. Los conserjes se quejan porque un $60.95\text{ }\%$ está prácticamente en el $61\text{ }\%$, y el casting simple solo trunca decimales en lugar de aplicar las reglas matemáticas de redondeo.

Alba Torres y **Laia Claramunt** se acercan al puesto de Pau:

«Pau, para calcular potencias, raíces y redondeos oficiales no inventamos operadores ni usamos casting destructivo. Volvemos a recurrir a la caja de herramientas de ***`java.lang.Math`****:*

1. Para potencias utilizamos el método estático ***`Math.pow(base, exponente)`****.*
2. Para raíces cuadradas invocamos ***`Math.sqrt(valor)`****.*
3. Y para redondear decimales según la regla matemática oficial (≥ 0.5 sube, < 0.5 baja), usamos ***`Math.round(valor)`****.*

Hoy evolucionaremos ***`ControlAccesoQR`*** *a la versión* ***v1.5****: resolveremos la distancia de cobertura de los sensores y emitiremos porcentajes redondeados con rigor profesional».*

---

2. Fundamento teórico: potencias, raíces y redondeo matemático

FUNCIONES MATEMÁTICAS AVANZADAS EN MATH		
Invocación estática	Tipo de retorno	Comportamiento técnico
Math.pow(base, exp)	double	Eleva la 'base' al 'exponente'. Siempre devuelve double (ej. 2^3 = 8.0)
Math.sqrt(x)	double	Calcula la raíz cuadrada positiva de x. Si x < 0, devuelve NaN (Not a Number).
Math.round(double x)	long	Redondea al entero más próximo: • 60.49 -> 60 • 60.50 -> 61

A. EL CÁLCULO DE POTENCIAS CON `Math.pow()` FRENTE A LA TRAMPA DEL OPERADOR `^`

En muchos lenguajes o calculadoras, el acento circunflejo `^` representa una potencia. **En Java no.**
* El operador `^` es un operador binario de álgebra booleana a nivel de bits (*XOR*). Escribir `3 ^ 2` en Java no calcula $3^2 = 9$; compara los bits `0011` y `0010` resultando en el número `1`.
* Para calcular potencias en Java es obligatorio invocar al método estático `Math.pow(double base, double exponente)`:

```
double cuadrado = Math.pow(3.0, 2.0); // Devuelve 9.0
```

B. RAÍCES CUADRADAS CON `Math.sqrt()` Y EL TEOREMA DE PITÁGORAS

El método `Math.sqrt()` (*Square Root*) permite calcular distancias euclídeas, radios de cobertura o módulos de vectores combinándolo con potencias en una única línea de código:

$$d = \sqrt{x^2 + y^2} \quad \Longrightarrow \quad \text{double d} = \text{Math.sqrt}(\text{Math.pow}(x, 2) + \text{Math.pow}(y, 2));$$

C. TRUNCAMIENTO MEDIANTE *CASTING* FRENTE A REDONDEO MATEMÁTICO CON `Math.round()`

Es vital distinguir qué hace la CPU en cada caso:
1. **Truncamiento con casting `(int)`:** Descarta sin miramientos la parte decimal.
* `(int) 60.10` da `60`
* `(int) 60.99` da `60` *(¡pierde casi una unidad entera!)*
2. **Redondeo matemático con `Math.round()`:** Aplica la regla estándar del redondeo aritmético:
* Suma 0.5 al valor y trunca hacia abajo el resultado.
* `Math.round(60.49)` devuelve `60L`.
* `Math.round(60.50)` devuelve `61L`.

> ⚠️ **Atención al tipo de retorno de `Math.round()`:**
> Cuando le pasamos un argumento de tipo `double`, `Math.round()` devuelve un entero largo de tipo **`long`** (64 bits). Si queremos almacenarlo en una variable `int`, debemos aplicar un casting explícito:
> `int ocupacionRedondeada = (int) Math.round(porcentajeDecimal);`

---

3. Evolución del caso guía a `ControlAccesoQR v1.5`

Ampliamos el programa incorporando:
1. Las coordenadas espaciales del sensor del vestíbulo ($x = 3.0\text{ m}$, $y = 4.0\text{ m}$) y el cálculo de la distancia diagonal de cobertura mediante `Math.pow()` y `Math.sqrt()`.
2. El redondeo matemático oficial del porcentaje de aforo con `Math.round()` para el panel visual del centro, comparándolo con el valor truncado.

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.5)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.5 (Evolucion: potencias, raices y redondeo con Math)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero
    Definir JORNADA_BASE_MINUTOS Como Entero
    Definir TEMP_MIN_SEGURA, TEMP_MAX_SEGURA Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0
    SEMILLA_CALIBRACION <- 987654321
    JORNADA_BASE_MINUTOS <- 480
    TEMP_MIN_SEGURA <- 15.0
    TEMP_MAX_SEGURA <- 35.0

    Definir terminalId Como Entero
    Definir tempVestibulo, fluctuacionTermica, tempCalibrada, tempSeguraAcotada Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionTruncado Como Entero

    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero
    Definir codigoTestCalibracion Como Entero
    Definir sensorNfcOperativo Como Logico
    Definir desviacionJornadaMinutos Como Entero
    Definir aforoSeguroVisualizado Como Entero
```

```
    // [NUEVO DÍA 18] Variables para telemetría espacial y redondeo matemático
    Definir coordenadaXMetros, coordenadaYMetros, distanciaDiagonalAntena Como Real
    Definir porcentajeOcupacionRedondeado Como Entero

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.5 - geometria y redondeo)"
    Escribir "ID del terminal principal:"
    Leer terminalId
    Escribir "Temperatura base del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    // [NUEVO DÍA 18] Coordenadas para cálculo de cobertura
    Escribir "Distancia horizontal al torno en metros (eje X):"
    Leer coordenadaXMetros
    Escribir "Altura del sensor en metros (eje Y):"
    Leer coordenadaYMetros

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    codigoTokenVestibulo <- Aleatorio(1000, 9999)
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)
    codigoTestCalibracion <- 5555
    fluctuacionTermica <- 0.45
    tempCalibrada <- tempVestibulo + fluctuacionTermica
    sensorNfcOperativo <- Verdadero

    desviacionJornadaMinutos <- abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS)

    Si tempCalibrada > TEMP_MAX_SEGURA Entonces
        tempSeguraAcotada <- TEMP_MAX_SEGURA
    SiNo
        Si tempCalibrada < TEMP_MIN_SEGURA Entonces
            tempSeguraAcotada <- TEMP_MIN_SEGURA
        SiNo
            tempSeguraAcotada <- tempCalibrada
        FinSi
    FinSi

    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    aforoSeguroVisualizado <- personasEnCentro
    porcentajeOcupacionReal <- (aforoSeguroVisualizado * FACTOR_PORCENTAJE) / aforoTotal

    // [NUEVO DÍA 18] Comparación entre truncamiento y redondeo
    porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)
    porcentajeOcupacionRedondeado <- redon(porcentajeOcupacionReal)

    // [NUEVO DÍA 18] Teorema de Pitágoras con raíz y potencias
    distanciaDiagonalAntena <- rc((coordenadaXMetros ^ 2) + (coordenadaYMetros ^ 2))

    Escribir ""
    Escribir "======================================================================"
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                   "
    Escribir "======================================================================"
    Escribir "CENTRO:             ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :       #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:    ", tokenResumenVestibulo
    Escribir "TOKEN PUERTA NORTE: ", tokenResumenPuertaNorte
    Escribir "PERSONA:            ", nombrePersona, " | PERFIL: ", perfilPersona
    Escribir "IDENTIFICADOR:      ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "----------------------------------------------------------------------"
    Escribir "HORARIO:            Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
```

```
    Escribir "PERMANENCIA:        ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "DESVIACION JORNADA: ", desviacionJornadaMinutos, " min respecto a jornada completa (480 min)."
    Escribir "-----------------------------------------------------------------------"
    Escribir "ESTADÍSTICAS Y REDONDEOS (MATH):"
    Escribir "Ocupación calculada:  ", porcentajeOcupacionReal, " %"
    Escribir "Ocupación truncada:   ", porcentajeOcupacionTruncado, " % (Casting simple)"
    Escribir "Ocupación redondeada: ", porcentajeOcupacionRedondeado, " % (Math.round oficial)"
    Escribir "Distancia antena QR:  ", distanciaDiagonalAntena, " metros (Pitágoras con sqrt/pow)"
    Escribir "ACTIVO:               ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "REGISTRO LOG:         ", RUTA_LOGS
    Escribir "======================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.5)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.5:
 * Métodos estáticos avanzados de Math: potencias (Math.pow),
 * raíces (Math.sqrt) y redondeo aritmético oficial (Math.round).
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.5 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */
import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
        final double MAX_VARIACION_TERMICA = 0.5;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int JORNADA_BASE_MINUTOS = 480;
        final double TEMP_MIN_SEGURA = 15.0;
        final double TEMP_MAX_SEGURA = 35.0;

        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        int terminalId;
        double tempVestibulo;

        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumenVestibulo;
        String tokenResumenPuertaNorte;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;

        // [NUEVO DÍA 18] Variables para redondeo y cálculo espacial con Math
        int porcentajeOcupacionRedondeado;
        double coordenadaXMetros;
        double coordenadaYMetros;
        double distanciaDiagonalAntena;

        int codigoTokenVestibulo;
        int codigoTokenPuertaNorte;
        int codigoTestCalibracion;
        double fluctuacionTermica;
        double tempCalibrada;
        boolean sensorNfcOperativo;
        int desviacionJornadaMinutos;
```

```
        double tempSeguraAcotada;
        int aforoSeguroVisualizado;

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.5 (geometría y redondeos de Math)    ");
        System.out.println("================================================");
        System.out.print("ID del terminal principal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        // [NUEVO DÍA 18] Entrada de coordenadas del sensor
        System.out.print("Distancia horizontal al torno en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura del sensor en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza de buffer

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // Procesamiento secuencial
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA;
        tempCalibrada = tempVestibulo + fluctuacionTermica;
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        aforoTotal = personasEnCentro + aforoDisponible;
        aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenPuertaNorte += "-T102";
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) aforoSeguroVisualizado / aforoTotal) * FACTOR_PORCENTAJE;

        // Truncamiento simple con casting a int (vía antigua)
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;

        // [NUEVO DÍA 18] Redondeo matemático oficial con Math.round()
        // Math.round(double) devuelve un long, por lo que aplicamos casting a (int)
```

```
        porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        // [NUEVO DÍA 18] Cálculo de hipotenusa/distancia euclídea: d = sqrt(x^2 + y^2)
        distanciaDiagonalAntena = Math.sqrt(Math.pow(coordenadaXMetros, 2.0) + Math.pow(coordenadaYMetros, 2.0));

        // Salida formateada
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | PERFIL: %c%n", nombrePersona, perfilPersona);
        System.out.printf("IDENTIFICADOR:        %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.println("CÁLCULOS MATEMÁTICOS AVANZADOS (POW, SQRT, ROUND):");
        System.out.printf("OCUPACIÓN EXACTA:     %6.2f %%%n", porcentajeOcupacionReal);
        System.out.printf("OCUPACIÓN TRUNCADA:   %d %% (Casting destructivo)%n", porcentajeOcupacionTruncado);
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round oficial)%n", porcentajeOcupacionRedondeado);
        System.out.printf("DISTANCIA A ANTENA:   %.2f metros (Pitágoras con pow y sqrt)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

---

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre potencias, raíces cuadradas y redondeo matemático:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). `Math.pow`, `Math.sqrt` Y `Math.round` EN TU PROYECTO

- **Objetivo:** Incorporar a tu clase `MiProyecto.java` (versión v1.4) al menos una operación de potencia (`Math.pow`), una raíz cuadrada (`Math.sqrt`) o un redondeo oficial (`Math.round`) adaptado al dominio de tu proyecto propio.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**: calcular el radio de alcance de un hechizo expansivo mediante el teorema de Pitágoras ($r = \sqrt{x^2 + y^2}$) usando `Math.pow` y `Math.sqrt`, y redondear los puntos de experiencia finales con `Math.round()`.
- En **Motor de recomendación**: calcular la distancia euclídea entre dos vectores de afinidad ($d = \sqrt{\Delta a^2 + \Delta b^2}$) con `Math.pow` y `Math.sqrt`, y redondear la puntuación final de afinidad a un entero de 0 a 100 con `Math.round()`.
- En **Simulador de físicas 2D**: calcular la velocidad total resultante de una partícula ($v = \sqrt{v_x^2 + v_y^2}$) con `Math.pow` y `Math.sqrt`, y redondear la coordenada física de impacto al píxel entero más próximo con `Math.round()`.
- En **Bóveda de contraseñas**: estimar el espacio combinatorio de fuerza bruta elevando el número de caracteres a la longitud de la clave mediante `Math.pow()`, y redondear el índice de entropía resultante con `Math.round()`.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). LA TRAMPA DEL TIPO DEVUELTO POR `Math.round()`

- **Contexto técnico:** Entender las firmas y los tipos devueltos por la biblioteca de Java. El método `Math.round(double)` devuelve un entero largo de 64 bits (`long`), no un `int`.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaRedondeo.java`, intenta escribir la siguiente línea:

```
double nota = 8.75;
int notaRedondeada = Math.round(nota); // Error de compilación inmediato
```

- Pasa el cursor por encima del error en IntelliJ y comprueba el mensaje: «*Incompatible types: found long, required int*».
- Aplica el casting explícito adecuado para resolverlo: `(int) Math.round(nota)`.

- Imprime una tabla comparativa en la consola evaluando qué ocurre con el casting simple `(int)` frente a `Math.round()` para los siguientes cuatro números: `4.2`, `4.5`, `4.8` y `4.99`.
- Anota en tu cuaderno técnico por qué `(int) 4.8` da `4`, mientras que `Math.round(4.8)` da `5`.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). EL OPERADOR `^` FRENTE A `Math.pow()`

- **Contexto de arquitectura:** Comprender qué hace realmente el procesador cuando usamos operadores binarios a nivel de bits.
- **Misión de la kata:**
- En una clase de prueba temporal llamada `PruebaXor.java`, ejecuta el siguiente código:

```
int calculoErroneo = 2 ^ 3;
double calculoCorrecto = Math.pow(2.0, 3.0);

System.out.println("Resultado de (2 ^ 3):      " + calculoErroneo);
System.out.println("Resultado de Math.pow(2, 3): " + calculoCorrecto);
```

- Comprueba en la consola que `(2 ^ 3)` devuelve `1`, mientras que `Math.pow(2, 3)` devuelve `8.0`.
- Redacta dos líneas de comentario explicando qué ha ocurrido:
- El número 2 en binario es `0010`.
- El número 3 en binario es `0011`.
- La operación XOR (`^`) compara bit a bit: devuelve 1 solo si los bits son distintos (`0010 ^ 0011 = 0001`, que equivale al número decimal 1).
- Por tanto, para potencias matemáticas en Java es imprescindible invocar siempre al método estático `Math.pow()`.

3.2.3 Día 19 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del jueves 15 de octubre. En la sala de integración de **AzaharTech**, **Pau Ferrer** muestra una incidencia con la que se ha topado al cargar los parámetros iniciales de la pantalla de TV del **IES El Caminàs**.

Los archivos de configuración del sistema y las peticiones de red que envían los móviles de las personas al cazar el QR transmiten toda la información en formato de texto plano (`String`). El archivo de configuración suministra parámetros en texto como `"101"` (para el identificador del monitor) o `"2000"` (para el aforo del centro).

Pau ha intentado convertir ese texto en un entero aplicando el casting que aprendimos en el Sprint 1:

```
String textoTerminal = "101";
int terminalId = (int) textoTerminal; // Error de compilación inmediato
```

IntelliJ detiene la compilación en seco con el mensaje: *«Inconvertible types; cannot cast 'java.lang.String' to 'int'»*. Pau comenta desconcertado:

«Pensaba que con el casting entre paréntesis podía convertir cualquier dato. ¿Por qué no puedo transformar un texto que contiene números en un entero?».

Alba Torres y **Laia Claramunt** se acercan al monitor:

«Pau, el casting con `(int)` o `(double)` solo funciona entre tipos numéricos primitivos porque la CPU simplemente reinterpreta los bits en la memoria Stack. Pero una cadena `String` es un objeto complejo en el Heap que contiene caracteres tipográficos.

*Para convertir texto en magnitudes numéricas reales necesitamos **parsear (parsing)**, es decir, analizar la cadena carácter a carácter y reconstruir su valor numérico en memoria.*

*Para ello, Java proporciona las **clases envoltorio (Wrappers)**: `Integer`, `Double`, `Boolean` y `Character`. Hoy aprenderemos a utilizar sus métodos estáticos de parseo (`parseInt`, `parseDouble`), a consultar sus constantes de límite de memoria (`MAX_VALUE`) y evolucionaremos `ControlAccesoQR` a la versión **v1.6**».*

---

2. Fundamento teórico: clases envoltorio (*wrappers*), parseo y utilidades de caracteres

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    TIPOS PRIMITIVOS Y SUS CLASES ENVOLTORIO              │
├──────────────────┬──────────────────────┬────────────────────────────────┤
│ Tipo Primitivo   │ Clase Envoltorio     │ Ejemplos de métodos estáticos  │
│ (En memoria Stack) │ (Objeto en java.lang) │                              │
├──────────────────┼──────────────────────┼────────────────────────────────┤
│ int              │ Integer              │ Integer.parseInt("101")    -> 101 │
├──────────────────┼──────────────────────┼────────────────────────────────┤
│ double           │ Double               │ Double.parseDouble("21.5") -> 21.5 │
├──────────────────┼──────────────────────┼────────────────────────────────┤
│ boolean          │ Boolean              │ Boolean.parseBoolean("true")-> true │
├──────────────────┼──────────────────────┼────────────────────────────────┤
│ char             │ Character            │ Character.toUpperCase('a') -> 'A' │
└──────────────────┴──────────────────────┴────────────────────────────────┘
```

A. ¿QUÉ ES UNA CLASE ENVOLTORIO (*WRAPPER CLASS*)?

Java mantiene una separación estricta entre tipos primitivos (que no son objetos y no tienen métodos) y el mundo de la POO. Para permitir que los primitivos puedan interactuar con utilidades de objetos, Java incluye en `java.lang` una **clase envoltorio para cada tipo primitivo**.

Pertenecen a `java.lang`, por lo que **se importan automáticamente** sin necesidad de `import`.

B. MÉTODOS ESTÁTICOS DE PARSEO: `Integer.parseInt()` Y `Double.parseDouble()`

El proceso de **parsear (*parse*)** consiste en tomar una cadena de texto formada por dígitos alfanuméricos y traducirla a su representación numérica binaria en la memoria:

```
String textoAforo = "2000";
int aforoTotal = Integer.parseInt(textoAforo); // Devuelve el primitivo int 2000

String textoTemp = "21.75";
double temperatura = Double.parseDouble(textoTemp); // Devuelve el primitivo double 21.75
```

Aviso sobre excepciones Si la cadena contiene caracteres no numéricos (*por ejemplo,* `Integer.parseInt("101A")`), Java lanzará un error en tiempo de ejecución llamado `NumberFormatException`. Durante este Sprint 2 trabajamos con cadenas con formato numérico correcto; la captura y control formal de estas excepciones se estudiará en el **Sprint 3 (RA3)** mediante bloques `try-catch`.

C. CONSTANTES DE LÍMITE DE MEMORIA: `MAX_VALUE` Y `MIN_VALUE`

Las clases envoltorio exponen constantes públicas que nos permiten consultar los límites físicos absolutos que puede almacenar cada tipo sin desbordar la memoria:

- `Integer.MAX_VALUE`: $2.147.483.647$ (el número entero más alto representable en 32 bits).
- `Integer.MIN_VALUE`: $-2.147.483.648$.
- `Double.MAX_VALUE`: Aprox. 1.79×10^{308}.

D. MÉTODOS DE UTILIDAD DE LA CLASE `Character`

La clase envoltorio `Character` ofrece métodos estáticos indispensables para manipular caracteres individuales sin necesidad de tablas ASCII manuales:

- `Character.toUpperCase(char c)` : Convierte un carácter a mayúscula (*ej.* `'e'` *$\rightarrow$* `'E'`).
- `Character.toLowerCase(char c)` : Convierte un carácter a minúscula.
- `Character.isDigit(char c)` : Devuelve un `boolean` indicando si el carácter es un dígito numérico entre `'0'` y `'9'` .
- `Character.isLetter(char c)` : Devuelve un `boolean` indicando si el carácter es una letra alfabética.

---

3. Evolución del caso guía a `ControlAccesoQR v1.6`

Ampliamos el programa incorporando el uso de clases envoltorio:

1. Simulamos que la configuración del terminal llega empaquetada como texto (`"101"` y `"2000"`) y la parseamos a variables enteras con **`Integer.parseInt()`** .
2. Normalizamos el perfil de acceso del usuario (`perfilPersona`) convirtiéndolo automáticamente a mayúsculas mediante **`Character.toUpperCase()`** .
3. Verificamos con **`Character.isLetter()`** si el perfil introducido es un carácter alfabético válido.
4. Consultamos **`Integer.MAX_VALUE`** en la telemetría del terminal para certificar el límite máximo teórico de registros antes de desbordar la memoria del contador.

PASO A. ALGORITMO EN PSEINT (**`pr/pseudocodigo/ControlAccesoQR.psc`** — V1.6)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.6 (Evolucion: Parseo de Cadenas y Normalizacion de Caracteres)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero
    Definir JORNADA_BASE_MINUTOS Como Entero
    Definir TEMP_MIN_SEGURA, TEMP_MAX_SEGURA Como Real

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0
    SEMILLA_CALIBRACION <- 987654321
    JORNADA_BASE_MINUTOS <- 480
    TEMP_MIN_SEGURA <- 15.0
    TEMP_MAX_SEGURA <- 35.0

    // [NUEVO DÍA 19] Variables de configuración recibidas en texto para parsear
    Definir textoTerminalId, textoAforoConfig Como Cadena
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona, perfilNormalizado Como Caracter
    Definir esEntrada, esPerfilValido Como Logico

    Definir horaEntrada, minutoEntrada, horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionTruncado, porcentajeOcupacionRedondeado Como Entero

    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero
    Definir codigoTestCalibracion Como Entero
    Definir sensorNfcOperativo Como Logico
```

```
    Definir desviacionJornadaMinutos Como Entero
    Definir aforoSeguroVisualizado Como Entero
    Definir coordenadaXMetros, coordenadaYMetros, distanciaDiagonalAntena Como Real

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.6 - parseo y wrappers)"

    // [NUEVO DÍA 19] Entrada de configuración como texto plano simulado
    Escribir "ID del terminal:"
    Leer textoTerminalId
    // En PSeInt la función ConvertirANumero equivale a Integer.parseInt()
    terminalId <- ConvertirANumero(textoTerminalId)

    Escribir "Temperatura base del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "Distancia horizontal al torno en metros (eje X):"
    Leer coordenadaXMetros
    Escribir "Altura del sensor en metros (eje Y):"
    Leer coordenadaYMetros

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (e = estudiante, d = docente, v = visita):"
    Leer perfilPersona

    // [NUEVO DÍA 19] Normalización a mayúsculas
    perfilNormalizado <- Mayusculas(perfilPersona)
    esPerfilValido <- Verdadero

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    codigoTokenVestibulo <- Aleatorio(1000, 9999)
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)
    codigoTestCalibracion <- 5555
    fluctuacionTermica <- 0.45
    tempCalibrada <- tempVestibulo + fluctuacionTermica
    sensorNfcOperativo <- Verdadero

    desviacionJornadaMinutos <- abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS)

    Si tempCalibrada > TEMP_MAX_SEGURA Entonces
        tempSeguraAcotada <- TEMP_MAX_SEGURA
    SiNo
        Si tempCalibrada < TEMP_MIN_SEGURA Entonces
            tempSeguraAcotada <- TEMP_MIN_SEGURA
        SiNo
            tempSeguraAcotada <- tempCalibrada
        FinSi
    FinSi

    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    aforoSeguroVisualizado <- personasEnCentro
    porcentajeOcupacionReal <- (aforoSeguroVisualizado * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)
    porcentajeOcupacionRedondeado <- redon(porcentajeOcupacionReal)

    distanciaDiagonalAntena <- rc((coordenadaXMetros ^ 2) + (coordenadaYMetros ^ 2))

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:             ", NOMBRE_CENTRO
    Escribir "TERMINAL PARSEADO:  #", terminalId, " (Origen texto: '", textoTerminalId, "')"
    Escribir "REGISTRO N. :       #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:    ", tokenResumenVestibulo
```

```
    Escribir "TOKEN PUERTA NORTE: ", tokenResumenPuertaNorte
    Escribir "PERSONA:            ", nombrePersona
    Escribir "PERFIL PROCESADO:   ", perfilNormalizado, " [Original: ", perfilPersona, "]"
    Escribir "IDENTIFICADOR:      ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-----------------------------------------------------------------------"
    Escribir "HORARIO:            Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:        ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "DESVIACION JORNADA: ", desviacionJornadaMinutos, " min respecto a jornada completa (480 min)."
    Escribir "OCUPACION REDONDEAD:", porcentajeOcupacionRedondeado, " %"
    Escribir "DISTANCIA ANTENA:   ", distanciaDiagonalAntena, " metros"
    Escribir "ACTIVO:             ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "REGISTRO LOG:       ", RUTA_LOGS
    Escribir "======================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.6)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.6:
 * Clases envoltorio (wrappers), parseo de texto con Integer.parseInt(),
 * utilidades de Character y constantes de límite de memoria (Integer.MAX_VALUE).
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.6 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */
import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
        final double MAX_VARIACION_TERMICA = 0.5;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int JORNADA_BASE_MINUTOS = 480;
        final double TEMP_MIN_SEGURA = 15.0;
        final double TEMP_MAX_SEGURA = 35.0;


        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        // [NUEVO DÍA 19] Variables de configuración recibidas en texto plano para parsear
        String textoTerminalId;
        int terminalId;
        double tempVestibulo;

        String nombrePersona, dniPersona;
        char perfilPersonaBruto;
        char perfilNormalizado; // Normalizado con Character.toUpperCase
        boolean esLetraValida;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumenVestibulo;
        String tokenResumenPuertaNorte;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        int porcentajeOcupacionRedondeado;

        double coordenadaXMetros;
        double coordenadaYMetros;
        double distanciaDiagonalAntena;

        int codigoTokenVestibulo;
```

```
        int codigoTokenPuertaNorte;
        int codigoTestCalibracion;
        double fluctuacionTermica;
        double tempCalibrada;
        boolean sensorNfcOperativo;
        int desviacionJornadaMinutos;
        double tempSeguraAcotada;
        int aforoSeguroVisualizado;

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.6 (wrappers y parseo de cadenas)    ");
        System.out.println("================================================");

        // [NUEVO DÍA 19] Captura en texto y conversión mediante Integer.parseInt()
        System.out.print("ID del terminal: ");
        textoTerminalId = teclado.nextLine();
        terminalId = Integer.parseInt(textoTerminalId); // Parseo estático de String a int

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal al torno en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura del sensor en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza obligatoria del buffer

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (e = estudiante, d = docente, v = visita): ");
        perfilPersonaBruto = teclado.next().charAt(0);

        // [NUEVO DÍA 19] Utilidades estáticas de la clase envoltorio Character
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto); // Pasa a mayúscula
        esLetraValida = Character.isLetter(perfilNormalizado);         // Verifica que es alfabético

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // Procesamiento secuencial
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA;
        tempCalibrada = tempVestibulo + fluctuacionTermica;
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        aforoTotal = personasEnCentro + aforoDisponible;
        aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenPuertaNorte += "-T102";
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;
```

```
        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) aforoSeguroVisualizado / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;
        porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        distanciaDiagonalAntena = Math.sqrt(Math.pow(coordenadaXMetros, 2.0) + Math.pow(coordenadaYMetros, 2.0));

        // Salida formateada con printf
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("TERMINAL PARSEADO:    #%04d (Dato origen texto: '%s')%n", terminalId, textoTerminalId);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | PERFIL NORMALIZADO: %c%n", nombrePersona, perfilNormalizado);
        System.out.printf("PERFIL VÁLIDO:        %b (isLetter verificado)%n", esLetraValida);
        System.out.printf("IDENTIFICADOR:        %-12s | SENTIDO: Entrada (%b)%n", dniPersona, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round)%n", porcentajeOcupacionRedondeado);
        System.out.printf("DISTANCIA A ANTENA:   %.2f metros%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("LÍMITE BUFFER (WRAP): %d (Integer.MAX_VALUE)%n", Integer.MAX_VALUE);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

---

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre clases envoltorio, métodos de parseo y constantes de rango:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). PARSEO DE ENTRADAS CON `Integer.parseInt()` Y `Double.parseDouble()`

- **Objetivo:** capturar al menos un dato numérico de tu proyecto propio en formato de texto plano (`String`) y transformarlo a primitivo mediante métodos de parseo de las clases *Wrapper*.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Capturar el código de armadura como texto (`"75"`) y parsearlo a `int` con `Integer.parseInt()`, y capturar la bonificación de peso (`"1.25"`) y parsearla con `Double.parseDouble()`.
- En **Motor de recomendación**. Capturar el código de año de estreno en texto (`"2024"`) y transformarlo con `Integer.parseInt()`, y parsear la puntuación de crítica externa (`"8.75"`) con `Double.parseDouble()`.
- En **Simulador de físicas 2D**. Capturar el valor de fricción en texto (`"0.05"`) y parsearlo con `Double.parseDouble()`, y parsear el número de partículas del lote con `Integer.parseInt()`.
- En **Bóveda de contraseñas**. Capturar el tiempo de caducidad en texto (`"90"`) y transformarlo con `Integer.parseInt()`, y parsear el coeficiente de fortaleza con `Double.parseDouble()`.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). INSPECCIÓN Y NORMALIZACIÓN CON LA CLASE `Character`

- **Contexto técnico:** demostrar el uso de utilidades estáticas de caracteres para limpiar entradas sin usar condicionales.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaCaracteres.java`, solicita al usuario un código de autorización compuesto por un único carácter (por ejemplo, una letra de zona o sector).
- Utiliza `Character.toUpperCase(c)` para normalizar el carácter a mayúscula automáticamente.
- Utiliza `Character.isLetter(c)` y `Character.isDigit(c)` asignando los resultados a dos variables booleanas independientes.

- Imprime por consola:

```
System.out.printf("Carácter procesado: '%c'%n", caracterNormalizado);
System.out.printf("¿Es alfabético?: %b | ¿Es numérico?: %b%n", esAlfabetico, esNumerico);
```

- Prueba el programa introduciendo una minúscula (`'a'`), un número (`'7'`) y un símbolo especial (`'@'`), verificando cómo responden los métodos booleanos.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). EL DESBORDAMIENTO DE `Integer.MAX_VALUE`

- **Contexto de sistemas:** en ingeniería del software, ignorar los límites de las clases envoltorio provoca graves fallos de desbordamiento (*overflow*).
- **Misión de la kata:**
- En una clase de prueba temporal llamada `PruebaDesbordamiento.java` , escribe el siguiente experimento:

```
int topeMaximo = Integer.MAX_VALUE;
System.out.println("Valor máximo de Integer:  " + topeMaximo);

// Provocamos un desbordamiento sumando 1
int desbordado = topeMaximo + 1;
System.out.println("Valor tras sumar 1 (+1):  " + desbordado);
```

- Ejecuta el programa y comprueba que al sumar 1 al valor máximo positivo ($2.147.483.647$), la variable pasa automáticamente a valer el límite negativo más extremo ($-2.147.483.648$).
- Redacta dos líneas de comentario en tu código explicando por qué ocurre esto: **Java utiliza aritmética modular de complemento a dos con 32 bits, por lo que desbordar el bit de signo convierte el número positivo más alto en el número negativo más bajo**.

3.3 Semana 6. Métodos String y tiempo con java.time

Encaramos la recta final del Sprint 2 adentrándonos en el procesamiento avanzado de información textual y en la gestión temporal moderna del software. A lo largo de esta semana, el equipo de desarrollo de AzaharTech desentrañará el funcionamiento interno y la inmutabilidad de la clase String en la memoria Heap, dominando métodos esenciales de inspección (length, charAt, contains, indexOf) y de transformación y recorte (substring, trim, toUpperCase, replace) para sanear y desglosar los datos de entrada del sistema. Además, daremos el salto definitivo hacia la API oficial java.time, incorporando las clases LocalTime, LocalDate y Duration para capturar marcas de tiempo reales del sistema operativo y calcular permanencias exactas sin necesidad de multiplicaciones manuales por 60. Como culminación del sprint, aprenderemos a modularizar nuestra aplicación codificando métodos estáticos propios con parámetros y valores de retorno, transformando el código en una arquitectura limpia, profesional y libre de bloques monolíticos, sellando así la versión definitiva v2.0 de nuestro software.

3.3.1 Día 20 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 19 de octubre. Arranca la tercera y última semana del Sprint 2. En la sala de desarrollo de **AzaharTech**, **Laia Claramunt** y **Pau Ferrer** analizan las cadenas de texto que envían los teléfonos móviles de los estudiantes al cazar el código QR proyectado en las pantallas de TV del **IES El Caminàs**.

Pau muestra un problema recurrente detectado en las pruebas de campo:

«Muchos estudiantes introducen su DNI con errores tipográficos: algunos teclean 8 caracteres olvidando la letra final, otros añaden espacios accidentales al principio o al final, y en ocasiones el identificador no contiene el formato esperado.

Hasta ahora hemos tratado las cadenas `String` como simples contenedores de texto que leíamos con `Scanner` y uníamos con el operador `+`. Pero necesitamos inspeccionar el texto por dentro: saber cuántos caracteres tiene exactamente, extraer la letra final de forma automática y localizar la posición de los guiones separadores».

Alba Torres interviene para enfocar la sesión:

*«En Java, `String` no es un tipo primitivo; es una **clase de objetos predefinida** extraordinariamente potente que reside en el paquete `java.lang`. Cada texto que declaramos es un objeto en el Heap con un arsenal de métodos para inspeccionar su contenido.*

*Hoy aprenderemos cómo gestiona Java las cadenas en la memoria, qué significa que un `String` sea **inmutable** y dominaremos los métodos de inspección esenciales: `length()`, `charAt()`, `isEmpty()`, `contains()` e `indexOf()`.*

*Evolucionaremos `ControlAccesoQR` a la versión **v1.7**, auditando la longitud y los caracteres del DNI y del token de forma 100 % secuencial».*

---

2. Fundamento teórico: la clase `String`, inmutabilidad y métodos de inspección

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      ANATOMÍA DE UNA CADENA EN MEMORIA (ÍNDICES)         │
├──────────────────────────────────────────────────────────────────────────┤
│ String dni = "45123789K";                                                │
│                                                                          │
│ Índice:       0     1     2     3     4     5     6     7     8          │
│ Carácter:  [ '4' ][ '5' ][ '1' ][ '2' ][ '3' ][ '7' ][ '8' ][ '9' ][ 'K' ]  │
│                                                                          │
│ • Longitud total (length): 9 caracteres.                                 │
│ • Primer carácter:         dni.charAt(0)               -> '4'            │
│ • Último carácter:         dni.charAt(dni.length() - 1) -> 'K'           │
└──────────────────────────────────────────────────────────────────────────┘
```

A. INMUTABILIDAD DE LA CLASE `String`

En Java, los objetos de la clase `String` son **inmutables**: una vez que se crea una cadena en la memoria Heap, **su contenido jamás puede ser modificado**.

- Cuando realizamos una concatenación como `texto += " nuevo"`, Java no altera el objeto original: crea un **objeto completamente nuevo** en el Heap con el texto resultante y reasigna la variable de referencia para que apunte al nuevo bloque.
- Los métodos de la clase `String` nunca modifican la cadena original; realizan la operación y devuelven un nuevo resultado.

B. MÉTODOS FUNDAMENTALES DE INSPECCIÓN Y TAMAÑO

Al pertenecer a `java.lang`, no requieren ninguna sentencia `import`:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    MÉTODOS DE INSPECCIÓN DE LA CLASE STRING              │
├──────────────────────────┬──────────────────┬────────────────────────────┤
│ Firma del método         │ Tipo de retorno  │ Comportamiento             │
├──────────────────────────┼──────────────────┼────────────────────────────┤
│ cadena.length()          │ int              │ Devuelve el número total de caracteres. │
├──────────────────────────┼──────────────────┼────────────────────────────┤
│ cadena.charAt(int index) │ char             │ Devuelve el carácter en la posición │
│                          │                  │ especificada (índice base 0). │
├──────────────────────────┼──────────────────┼────────────────────────────┤
│ cadena.isEmpty()         │ boolean          │ Devuelve true si length() es igual a 0. │
├──────────────────────────┼──────────────────┼────────────────────────────┤
│ cadena.contains(texto)   │ boolean          │ Devuelve true si la cadena contiene la │
│                          │                  │ subsecuencia de texto buscada. │
├──────────────────────────┼──────────────────┼────────────────────────────┤
│ cadena.indexOf(texto)    │ int              │ Devuelve la posición de la primera │
│                          │                  │ aparición. Si no existe, devuelve -1. │
└──────────────────────────┴──────────────────┴────────────────────────────┘
```

C. LA REGLA DEL ÍNDICE BASE 0 Y LA EXTRACCIÓN DEL ÚLTIMO CARÁCTER

En Java, los índices de las cadenas comienzan siempre en `0`:

- El primer carácter reside en el índice `0`.
- El último carácter reside obligatoriamente en el índice **`cadena.length() - 1`**.
- **Peligro con los límites.** Intentar acceder a `cadena.charAt(cadena.length())` provoca una excepción crítica en tiempo de ejecución: **`StringIndexOutOfBoundsException`**, ya que ese índice queda fuera del rango asignado.

D. EVALUACIÓN CON BANDERAS BOOLEANAS

Podemos evaluar la validez estructural de una cadena asignando el resultado de las comparaciones directamente a variables `boolean`:

```
// Comprobación de longitud exacta de 9 caracteres
boolean longitudDniCorrecta = (dniPersona.length() == 9);

// Comprobación de existencia del prefijo mediante indexOf
boolean contieneSeparador = (tokenResumen.indexOf("-") != -1);
```

---

3. Evolución del caso guía a **`ControlAccesoQR v1.7`**

Ampliamos el programa guía incorporando métodos de inspección de cadenas:

1. Calculamos la longitud exacta del nombre y del DNI de la persona mediante **`length()`**.
2. Extraemos automáticamente la letra final del DNI mediante **`charAt(length() - 1)`**.
3. Verificamos mediante **`contains()`** que el token del vestíbulo contiene el prefijo oficial `"CAMINAS"`.
4. Localizamos mediante **`indexOf()`** la posición del primer guion separador del token.
5. Evaluamos con una expresión booleana si la longitud del DNI cumple la norma oficial de 9 caracteres.

PASO A. ALGORITMO EN PSEINT (**`pr/pseudocodigo/ControlAccesoQR.psc`** — V1.7)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.7 (Evolucion: inspeccion de cadenas y extraccion de caracteres)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero
    Definir JORNADA_BASE_MINUTOS Como Entero
    Definir TEMP_MIN_SEGURA, TEMP_MAX_SEGURA Como Real

    // [NUEVO DÍA 20] Constante para longitud oficial del DNI
    Definir LONGITUD_DNI_ESTANDAR Como Entero

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0
    SEMILLA_CALIBRACION <- 987654321
    JORNADA_BASE_MINUTOS <- 480
    TEMP_MIN_SEGURA <- 15.0
    TEMP_MAX_SEGURA <- 35.0
    LONGITUD_DNI_ESTANDAR <- 9

    Definir textoTerminalId Como Cadena
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona, perfilNormalizado Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada, horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
```

```
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionTruncado, porcentajeOcupacionRedondeado Como Entero

    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero
    Definir codigoTestCalibracion Como Entero
    Definir sensorNfcOperativo Como Logico
    Definir desviacionJornadaMinutos Como Entero
    Definir aforoSeguroVisualizado Como Entero
    Definir coordenadaXMetros, coordenadaYMetros, distanciaDiagonalAntena Como Real

    // [NUEVO DÍA 20] Variables para inspección de cadenas
    Definir longitudDni, longitudNombre Como Entero
    Definir letraFinalDni Como Caracter
    Definir esLongitudDniCorrecta Como Logico
    Definir posicionPrimerGuion Como Entero

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.7 - Inspeccion String)"
    Escribir "ID del terminal:"
    Leer textoTerminalId
    terminalId <- ConvertirANumero(textoTerminalId)

    Escribir "Temperatura base del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "Distancia horizontal al torno en metros (eje X):"
    Leer coordenadaXMetros
    Escribir "Altura de la pantalla en metros (eje Y):"
    Leer coordenadaYMetros

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (e = estudiante, d = docente, v = visita):"
    Leer perfilPersona

    perfilNormalizado <- Mayusculas(perfilPersona)
    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    codigoTokenVestibulo <- Aleatorio(1000, 9999)
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)
    codigoTestCalibracion <- 5555
    fluctuacionTermica <- 0.45
    tempCalibrada <- tempVestibulo + fluctuacionTermica
    sensorNfcOperativo <- Verdadero

    desviacionJornadaMinutos <- abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS)

    Si tempCalibrada > TEMP_MAX_SEGURA Entonces
        tempSeguraAcotada <- TEMP_MAX_SEGURA
    SiNo
        Si tempCalibrada < TEMP_MIN_SEGURA Entonces
            tempSeguraAcotada <- TEMP_MIN_SEGURA
        SiNo
            tempSeguraAcotada <- tempCalibrada
        FinSi
    FinSi

    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    aforoSeguroVisualizado <- personasEnCentro
    porcentajeOcupacionReal <- (aforoSeguroVisualizado * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)
```

```
    porcentajeOcupacionRedondeado <- redon(porcentajeOcupacionReal)

    distanciaDiagonalAntena <- rc((coordenadaXMetros ^ 2) + (coordenadaYMetros ^ 2))

    // [NUEVO DÍA 20] Inspección de longitud y extracción de caracteres
    longitudDni <- Longitud(dniPersona)
    longitudNombre <- Longitud(nombrePersona)
    letraFinalDni <- Subcadena(dniPersona, longitudDni, longitudDni) // En PSeInt índices base 1
    esLongitudDniCorrecta <- (longitudDni == LONGITUD_DNI_ESTANDAR)
    posicionPrimerGuion <- 7 // En Java calculada exactamente con indexOf("-")

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:              ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :        #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:     ", tokenResumenVestibulo
    Escribir "TOKEN PUERTA NORTE: ", tokenResumenPuertaNorte
    Escribir "PERSONA:             ", nombrePersona, " (Longitud: ", longitudNombre, " caracteres)"
    Escribir "DNI INSPECCIONADO:  ", dniPersona, " | Longitud: ", longitudDni, " | Letra final: ", letraFinalDni
    Escribir "DNI FORMATO 9 CAR.: ", esLongitudDniCorrecta
    Escribir "IDENTIFICADOR:       ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-------------------------------------------------------------------------"
    Escribir "HORARIO:             Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:         ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "DESVIACION JORNADA: ", desviacionJornadaMinutos, " min respecto a jornada completa (480 min)."
    Escribir "OCUPACION REDONDEAD:", porcentajeOcupacionRedondeado, " %"
    Escribir "DISTANCIA A TV:      ", distanciaDiagonalAntena, " metros"
    Escribir "ACTIVO:              ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "REGISTRO LOG:        ", RUTA_LOGS
    Escribir "========================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.7)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.7:
 * Métodos de inspección de cadenas (length, charAt, contains, indexOf)
 * aplicados a la validación estructural del DNI y formato del token en memoria.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.7 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */
import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
        final double MAX_VARIACION_TERMICA = 0.5;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int JORNADA_BASE_MINUTOS = 480;
        final double TEMP_MIN_SEGURA = 15.0;
        final double TEMP_MAX_SEGURA = 35.0;

        // [NUEVO DÍA 20] Longitud normativa estándar de un DNI/NIE en España
        final int LONGITUD_DNI_ESTANDAR = 9;

        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        String textoTerminalId;
        int terminalId;
        double tempVestibulo;

        String nombrePersona, dniPersona;
        char perfilPersonaBruto;
        char perfilNormalizado;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumenVestibulo;
```

```
        String tokenResumenPuertaNorte;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        int porcentajeOcupacionRedondeado;

        double coordenadaXMetros;
        double coordenadaYMetros;
        double distanciaDiagonalAntena;

        int codigoTokenVestibulo;
        int codigoTokenPuertaNorte;
        int codigoTestCalibracion;
        double fluctuacionTermica;
        double tempCalibrada;
        boolean sensorNfcOperativo;
        int desviacionJornadaMinutos;
        double tempSeguraAcotada;
        int aforoSeguroVisualizado;

        // [NUEVO DÍA 20] Variables para inspección de objetos String
        int longitudDni;
        int longitudNombre;
        char letraFinalDni;
        boolean esLongitudDniCorrecta;
        boolean tokenContienePrefijo;
        int posicionPrimerSeparador;

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.7 (inspección y métodos de String)  ");
        System.out.println("================================================");
        System.out.print("ID del terminal: ");
        textoTerminalId = teclado.nextLine();
        terminalId = Integer.parseInt(textoTerminalId);

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal a la pantalla en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza obligatoria del buffer

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (e = estudiante, d = docente, v = visita): ");
        perfilPersonaBruto = teclado.next().charAt(0);
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // Procesamiento secuencial
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);
```

```
        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA;
        tempCalibrada = tempVestibulo + fluctuacionTermica;
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        aforoTotal = personasEnCentro + aforoDisponible;
        aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersona;
        tokenResumenPuertaNorte += "-T102";
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) aforoSeguroVisualizado / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;
        porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        // Distancia directa de enfoque visual hacia la pantalla de TV
        distanciaDiagonalAntena = Math.sqrt(Math.pow(coordenadaXMetros, 2.0) + Math.pow(coordenadaYMetros, 2.0));

        // [NUEVO DÍA 20] Invocación de métodos de inspección sobre objetos String
        longitudDni = dniPersona.length();
        longitudNombre = nombrePersona.length();

        // Extracción segura del último carácter usando length() - 1
        letraFinalDni = dniPersona.charAt(dniPersona.length() - 1);

        // Evaluación booleana sin condicionales: comprobamos la longitud oficial
        esLongitudDniCorrecta = (longitudDni == LONGITUD_DNI_ESTANDAR);

        // Búsqueda de contenido y posición del primer delimitador '-'
        tokenContienePrefijo = tokenResumenVestibulo.contains(PREFIJO_CENTRO);
        posicionPrimerSeparador = tokenResumenVestibulo.indexOf("-");

        // Salida formateada con printf
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d caracteres%n", nombrePersona, longitudNombre);
        System.out.printf("DNI INSPECCIONADO:    %-12s | LETRA FINAL: '%c'%n", dniPersona, letraFinalDni);
        System.out.printf("DNI NORMATIVO (9 CAR):%b (Comprobado con length)%n", esLongitudDniCorrecta);
        System.out.printf("TOKEN VÁLIDO:         %b (Contiene '%s' en pos. %d)%n", tokenContienePrefijo, PREFIJO_CENTRO, posicionPrimerSeparador);
        System.out.printf("PERFIL PROCESADO:     %c | SENTIDO: Entrada (%b)%n", perfilNormalizado, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round)%n", porcentajeOcupacionRedondeado);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Línea de visión a pantalla)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre inspección de cadenas, cálculo de posiciones relativas y el modelo de memoria del *String pool*:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). MÉTODOS DE INSPECCIÓN EN TU PROYECTO PROPIO

- **Objetivo.** Incorporar a tu clase `MiProyecto.java` (versión v1.6) llamadas a `length()`, `charAt()`, `contains()` o `indexOf()` para auditar un texto identificativo de tu sistema.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Medir la longitud del nombre del héroe con `length()`, extraer la letra inicial con `charAt(0)` y verificar mediante `contains()` si el nombre incluye algún título nobiliario (*por ejemplo,* `"Lord"` *o* `"Sir"`).
- En **Motor de recomendación**. Comprobar la longitud del título del ítem con `length()`, verificar con `contains()` si el identificador incluye el prefijo oficial de categoría (*por ejemplo,* `"MOV-"`) y localizar con `indexOf("-")` la posición del separador.
- En **Simulador de físicas 2D** Inspeccionar la cadena de material de la partícula con `length()`, extraer el primer carácter con `charAt(0)` y comprobar con `contains("elastico")` si el nombre define un comportamiento especial.
- En **Bóveda de contraseñas**. Medir la longitud exacta de la contraseña almacenada mediante `length()`, extraer el primer y el último carácter con `charAt(0)` y `charAt(length() - 1)`, y comprobar con `indexOf("#")` la presencia del delimitador de seguridad.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). EXTRACCIÓN SEGURA DE EXTREMOS E INSPECCIÓN DE ÍNDICES

- **Contexto técnico:** Evitar la temida excepción `StringIndexOutOfBoundsException` dominando la relación entre longitud y el índice base 0.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaInspeccionString.java`, declara una cadena con este código:

```
String codigoSerie = "AZ-PROYECTO-2026-X";
```

- Escribe instrucciones para calcular e imprimir por consola:
- La longitud total del código.
- El primer carácter del código.
- El último carácter.
- La posición donde aparece el primer guion.
- Un booleano que compruebe si el código contiene el año actual.
- Ejecuta el código cambiando el contenido de `codigoSerie` por una palabra más corta (*por ejemplo,* `"OK-1"`) y verifica que el programa sigue extrayendo todo sin romperse.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). EL MISTERIO DEL *STRING POOL* EN LA MEMORIA HEAP

- **Contexto de arquitectura.** Para ahorrar memoria RAM, la Máquina Virtual de Java gestiona una zona especial dentro del Heap llamada **String pool** (piscina de cadenas literales).
- **Misión de la kata:**
- En una clase de prueba temporal llamada `PruebaStringPool.java`, escribe el siguiente experimento:

```
String s1 = "DAM";
String s2 = "DAM";              // Mismo literal en código fuente
String s3 = new String("DAM");   // Forzamos un objeto nuevo con new

// Comparación de punteros en memoria con ==
boolean pruebaPool   = (s1 == s2);
boolean pruebaNew    = (s1 == s3);
boolean pruebaEquals = s1.equals(s3);

System.out.println("¿s1 y s2 apuntan a la misma dirección?: " + pruebaPool);
System.out.println("¿s1 y s3 apuntan a la misma dirección?: " + pruebaNew);
System.out.println("¿s1 y s3 tienen el mismo contenido?:    " + pruebaEquals);
```

- Ejecuta el programa y comprueba que `s1 == s2` devuelve **`true`**, pero `s1 == s3` devuelve **`false`**.
- Anota en tu cuaderno técnico la conclusión de ingeniería:
«Cuando declaramos cadenas mediante literales directos, Java reutiliza la misma instancia del String pool para ahorrar memoria. Pero cuando usamos el operador `new`*, obligamos a la JVM a crear un objeto independiente en el Heap fuera del pool.*

*Por eso en Java **jamás se comparan textos con** `==` y es obligatorio usar siempre el método `.equals()` para comparar su contenido».*

3.3.2 Día 21 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del martes 20 de octubre. En la sala técnica de **AzaharTech**, **Pau Ferrer** muestra a **Alba Torres** y a **Laia Claramunt** una nueva dificultad detectada en las cadenas de texto que envían los móviles de los estudiantes al registrarse frente a las pantallas del **IES El Caminàs**.

Pau proyecta la consola:

«Ayer aprendimos a medir la longitud del DNI y a buscar caracteres con `indexOf()`. Pero los datos reales de los usuarios llegan muy sucios: algunos alumnos introducen espacios accidentales al principio o al final de su nombre o DNI (por ejemplo, `"  45123789K  "`), otros escriben su nombre todo en minúsculas y otros en mayúsculas.

Además, jefatura de estudios nos pide dos transformaciones:
1. Descomponer el DNI para separar los 8 dígitos numéricos de la letra final de control.
2. Generar un formato alternativo de registro donde los guiones del token QR se sustituyan por barras inclinadas (`/`) para exportar a sistemas externos.

He intentado hacer `dniPersona.toUpperCase();`, pero al imprimirlo sigue saliendo en minúsculas. ¿Por qué Java no cambia la variable?».

Alba Torres toma el teclado y le recuerda la gran propiedad de las cadenas:

*«Pau, recuerda la regla de oro: **los objetos `String` son inmutables**. Ningún método de `String` modifica jamás la cadena original en el Heap. Lo que hacen es **crear y devolver un nuevo objeto con el texto transformado**.*

*Si no guardas el resultado en una variable, ese nuevo texto se pierde en el Heap y se convierte en basura. Hoy aprenderemos a limpiar espacios con **`trim()`**, a normalizar textos con **`toUpperCase()`** y **`toLowerCase()`**, a sustituir caracteres con **`replace()`** y a extraer fragmentos con **`substring()`**».*

---

2. Fundamento teórico: métodos de transformación y extracción en la clase `String`

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        TRANSFORMACIONES DE CADENAS EN MEMORIA            │
├──────────────────────────────┬───────────────────────────────────────────┤
│ 1. ERROR CLÁSICO (Sin efecto) │ 2. USO CORRECTO (Reasignación de referencia) │
├──────────────────────────────┼───────────────────────────────────────────┤
│ String dni = "  45123789k  "; │ String dni = "  45123789k  ";             │
│ dni.trim();                  │ dni = dni.trim().toUpperCase();           │
│ dni.toUpperCase();           │                                           │
│                              │ 'dni' apunta ahora a la NUEVA cadena limpia y en │
│ 'dni' sigue valiendo lo mismo │ mayúsculas: "45123789K". El texto sucio original queda │
│ porque los String no mutan.  │ huérfano para el Garbage Collector.       │
└──────────────────────────────┴───────────────────────────────────────────┘
```

A. MÉTODOS DE LIMPIEZA Y NORMALIZACIÓN

1. **`cadena.trim()`.** Devuelve una nueva cadena eliminando todos los espacios en blanco iniciales y finales (*leading and trailing spaces*). Los espacios intermedios entre palabras no se modifican.
2. **`cadena.toUpperCase()` y `cadena.toLowerCase()`.** Devuelven una nueva cadena transformando todos los caracteres alfabéticos a mayúsculas o minúsculas respectivamente, respetando números y símbolos de puntuación.
3. **`cadena.replace(objetivo, reemplazo)`.** Devuelve una nueva cadena sustituyendo todas las apariciones de un carácter o texto por otro:

```
String token = "CAMINAS-101-SEC#8492";
String tokenBarras = token.replace("-", "/"); // Devuelve "CAMINAS/101/SEC#8492"
```

B. EXTRACCIÓN DE FRAGMENTOS DE TEXTO: EL MÉTODO `substring()`

El método `substring()` permite recortar y extraer porciones de una cadena indicando sus posiciones de índice:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                       FUNCIONAMIENTO DEL MÉTODO SUBSTRING                    │
├──────────────────────────────────────────────────────────────────────────────┤
│ String dni = "45123789K";                                                    │
│ Índices:       0 1 2 3 4 5 6 7 8                                             │
│                                                                              │
│ 1. substring(inicio): Desde 'inicio' hasta el final de la cadena             │
│    dni.substring(8)      -> Devuelve "K"                                     │
│                                                                              │
│ 2. substring(inicio, fin): Desde 'inicio' hasta 'fin - 1' (el final queda excluido)  │
│    dni.substring(0, 8)   -> Devuelve "45123789" (extrae los índices 0 al 7)  │
└──────────────────────────────────────────────────────────────────────────────┘
```

- **La regla del extremo excluido.** En `cadena.substring(inicio, fin)`, el carácter en la posición `fin` **nunca se incluye**. La longitud del texto extraído es exactamente igual a la resta `fin - inicio` (*por ejemplo, en `(0, 8)`, la longitud es $8 - 0 = 8$ caracteres*).
- **Encadenamiento de métodos (*Method Chaining*):**
  Como cada método devuelve un nuevo objeto `String`, podemos invocar varios métodos en una sola instrucción secuencial:

  ```
  String nombreLimpio = teclado.nextLine().trim().toUpperCase();
  ```

---

3. Evolución del caso guía a `ControlAccesoQR v1.8`

Ampliamos el programa incorporando: 1. Limpieza de espacios accidentales del DNI y del nombre con **`trim()`**. 2. Normalización del DNI a mayúsculas con **`toUpperCase()`**. 3. Descomposición del DNI en dos subcadenas mediante **`substring()`**: los 8 dígitos numéricos por un lado y la letra de control por otro. 4. Generación de un formato de log alternativo sustituyendo los guiones del token por barras inclinadas con **`replace()`**. 5. Extracción del prefijo oficial del centro recortando desde el inicio hasta el primer guion con **`substring(0, indexOf("-"))`**.

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.8)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.8 (Evolucion: limpieza, transformacion y subcadenas)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero
    Definir JORNADA_BASE_MINUTOS Como Entero
    Definir TEMP_MIN_SEGURA, TEMP_MAX_SEGURA Como Real
    Definir LONGITUD_DNI_ESTANDAR Como Entero

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0
    SEMILLA_CALIBRACION <- 987654321
    JORNADA_BASE_MINUTOS <- 480
    TEMP_MIN_SEGURA <- 15.0
    TEMP_MAX_SEGURA <- 35.0
    LONGITUD_DNI_ESTANDAR <- 9

    Definir textoTerminalId Como Cadena
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona, perfilNormalizado Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada, horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal Como Entero
    Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042
```

```
    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionTruncado, porcentajeOcupacionRedondeado Como Entero

    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero
    Definir codigoTestCalibracion Como Entero
    Definir sensorNfcOperativo Como Logico
    Definir desviacionJornadaMinutos Como Entero
    Definir aforoSeguroVisualizado Como Entero
    Definir coordenadaXMetros, coordenadaYMetros, distanciaDiagonalAntena Como Real

    Definir longitudDni, longitudNombre Como Entero
    Definir letraFinalDni Como Caracter
    Definir esLongitudDniCorrecta Como Logico
    Definir posicionPrimerGuion Como Entero

    // [NUEVO DÍA 21] Variables para datos limpios, subcadenas y reemplazos
    Definir dniNumeros Como Cadena
    Definir tokenFormatoBarras Como Cadena
    Definir prefijoExtraido Como Cadena

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.8 - Transformacion String)"
    Escribir "ID del terminal:"
    Leer textoTerminalId
    terminalId <- ConvertirANumero(textoTerminalId)

    Escribir "Temperatura base del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "Distancia horizontal a pantalla en metros (eje X):"
    Leer coordenadaXMetros
    Escribir "Altura de la pantalla en metros (eje Y):"
    Leer coordenadaYMetros

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (e = estudiante, d = docente, v = visita):"
    Leer perfilPersona

    perfilNormalizado <- Mayusculas(perfilPersona)
    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    codigoTokenVestibulo <- Aleatorio(1000, 9999)
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)
    codigoTestCalibracion <- 5555
    fluctuacionTermica <- 0.45
    tempCalibrada <- tempVestibulo + fluctuacionTermica
    sensorNfcOperativo <- Verdadero

    desviacionJornadaMinutos <- abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS)

    Si tempCalibrada > TEMP_MAX_SEGURA Entonces
        tempSeguraAcotada <- TEMP_MAX_SEGURA
    SiNo
        Si tempCalibrada < TEMP_MIN_SEGURA Entonces
            tempSeguraAcotada <- TEMP_MIN_SEGURA
        SiNo
            tempSeguraAcotada <- tempCalibrada
        FinSi
    FinSi

    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    aforoSeguroVisualizado <- personasEnCentro
```

```
    porcentajeOcupacionReal <- (aforoSeguroVisualizado * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)
    porcentajeOcupacionRedondeado <- redon(porcentajeOcupacionReal)

    distanciaDiagonalAntena <- rc((coordenadaXMetros ^ 2) + (coordenadaYMetros ^ 2))

    longitudDni <- Longitud(dniPersona)
    longitudNombre <- Longitud(nombrePersona)
    letraFinalDni <- Subcadena(dniPersona, longitudDni, longitudDni)
    esLongitudDniCorrecta <- (longitudDni == LONGITUD_DNI_ESTANDAR)
    posicionPrimerGuion <- 7

    // [NUEVO DÍA 21] Extracción de los dígitos del DNI (en PSeInt índices base 1: del 1 al 8)
    dniNumeros <- Subcadena(dniPersona, 1, longitudDni - 1)

    // Simulación de reemplazo de delimitadores en PSeInt
    tokenFormatoBarras <- "CAMINAS/DNI/T101/SEC#OK" // En Java implementado con replace("-", "/")
    prefijoExtraido <- Subcadena(tokenResumenVestibulo, 1, 7) // En Java: substring(0, indexOf("-"))

    Escribir ""
    Escribir "========================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                      "
    Escribir "========================================================================="
    Escribir "CENTRO:             ", NOMBRE_CENTRO
    Escribir "REGISTRO N. :       #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:    ", tokenResumenVestibulo
    Escribir "TOKEN LOG BARRAS:   ", tokenFormatoBarras
    Escribir "PREFIJO EXTRAIDO:   ", prefijoExtraido
    Escribir "PERSONA NORMALIZADA:", Mayusculas(nombrePersona)
    Escribir "DNI DESGLOSADO:     Numero: ", dniNumeros, " | Control: '", letraFinalDni, "'"
    Escribir "-------------------------------------------------------------------------"
    Escribir "HORARIO:            Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA:        ", minutosEstanciaTotal, " minutos en el centro."
    Escribir "DESVIACION JORNADA: ", desviacionJornadaMinutos, " min respecto a jornada completa (480 min)."
    Escribir "OCUPACION REDONDEAD:", porcentajeOcupacionRedondeado, " %"
    Escribir "DISTANCIA A TV:     ", distanciaDiagonalAntena, " metros"
    Escribir "ACTIVO:             ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "REGISTRO LOG:       ", RUTA_LOGS
    Escribir "========================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.8)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.8:
 * Transformación y extracción de cadenas con String:
 * trim(), toUpperCase(), toLowerCase(), replace() y substring().
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.8 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */
import java.util.Scanner;
import java.util.Random;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
        final double MAX_VARIACION_TERMICA = 0.5;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int JORNADA_BASE_MINUTOS = 480;
        final double TEMP_MIN_SEGURA = 15.0;
        final double TEMP_MAX_SEGURA = 35.0;
        final int LONGITUD_DNI_ESTANDAR = 9;

        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        String textoTerminalId;
        int terminalId;
        double tempVestibulo;

        String nombrePersonaBruto, nombrePersonaLimpio;
        String dniPersonaBruto, dniPersonaLimpio;
        char perfilPersonaBruto;
```

```
        char perfilNormalizado;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada, minutosTotalesSalida, minutosEstanciaTotal;
        String tokenResumenVestibulo;
        String tokenResumenPuertaNorte;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        int porcentajeOcupacionRedondeado;

        double coordenadaXMetros;
        double coordenadaYMetros;
        double distanciaDiagonalAntena;

        int codigoTokenVestibulo;
        int codigoTokenPuertaNorte;
        int codigoTestCalibracion;
        double fluctuacionTermica;
        double tempCalibrada;
        boolean sensorNfcOperativo;
        int desviacionJornadaMinutos;
        double tempSeguraAcotada;
        int aforoSeguroVisualizado;

        int longitudDni;
        int longitudNombre;
        char letraFinalDni;
        boolean esLongitudDniCorrecta;
        boolean tokenContienePrefijo;
        int posicionPrimerSeparador;

        // [NUEVO DÍA 21] Variables para cadenas transformadas y extraídas
        String numeroDniSolo;
        String letraDniTexto;
        String tokenFormatoBarras;
        String prefijoExtraido;

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.8 (transformación de texto y substrings)");
        System.out.println("================================================");
        System.out.print("ID del terminal: ");
        textoTerminalId = teclado.nextLine();
        terminalId = Integer.parseInt(textoTerminalId);

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal a la pantalla en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza obligatoria del buffer

        System.out.print("DNI de la persona (puede incluir espacios): ");
        dniPersonaBruto = teclado.nextLine();

        System.out.print("Nombre completo (puede incluir espacios): ");
        nombrePersonaBruto = teclado.nextLine();

        // [NUEVO DÍA 21] Limpieza de espacios accidentales con trim() y normalización
        dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
        nombrePersonaLimpio = nombrePersonaBruto.trim();

        System.out.print("Perfil de acceso (e = estudiante, d = docente, v = visita): ");
        perfilPersonaBruto = teclado.next().charAt(0);
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();
```

```
        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // Procesamiento secuencial
        minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA;
        tempCalibrada = tempVestibulo + fluctuacionTermica;
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        aforoTotal = personasEnCentro + aforoDisponible;
        aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersonaLimpio;
        tokenResumenVestibulo += "-T" + terminalId;
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersonaLimpio;
        tokenResumenPuertaNorte += "-T102";
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) aforoSeguroVisualizado / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;
        porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        distanciaDiagonalAntena = Math.sqrt(Math.pow(coordenadaXMetros, 2.0) + Math.pow(coordenadaYMetros, 2.0));

        longitudDni = dniPersonaLimpio.length();
        longitudNombre = nombrePersonaLimpio.length();
        letraFinalDni = dniPersonaLimpio.charAt(dniPersonaLimpio.length() - 1);
        esLongitudDniCorrecta = (longitudDni == LONGITUD_DNI_ESTANDAR);
        tokenContienePrefijo = tokenResumenVestibulo.contains(PREFIJO_CENTRO);
        posicionPrimerSeparador = tokenResumenVestibulo.indexOf("-");

        // [NUEVO DÍA 21] 1. Extracción de los dígitos del DNI (desde 0 hasta el penúltimo carácter)
        numeroDniSolo = dniPersonaLimpio.substring(0, dniPersonaLimpio.length() - 1);

        // [NUEVO DÍA 21] 2. Extracción de la letra en formato String
        letraDniTexto = dniPersonaLimpio.substring(dniPersonaLimpio.length() - 1);

        // [NUEVO DÍA 21] 3. Sustitución de delimitadores con replace()
        tokenFormatoBarras = tokenResumenVestibulo.replace("-", "/");

        // [NUEVO DÍA 21] 4. Extracción dinámica del prefijo recortando hasta el primer guion
        prefijoExtraido = tokenResumenVestibulo.substring(0, posicionPrimerSeparador);

        // Salida formateada con printf
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s%n", NOMBRE_CENTRO);
        System.out.printf("PREFIJO EXTRAÍDO:     %-15s (substring dinámico)%n", prefijoExtraido);
        System.out.printf("REGISTRO N.º:         #%05d%n", idUltimoFichaje);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN FORMATO BARRAS: %s (replace)%n", tokenFormatoBarras);
        System.out.printf("PERSONA LIMPIA:       %-30s | LONGITUD: %d%n", nombrePersonaLimpio.toUpperCase(), longitudNombre);
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%s'%n", numeroDniSolo, letraDniTexto);
        System.out.printf("DNI NORMATIVO (9 CAR):%b%n", esLongitudDniCorrecta);
        System.out.printf("PERFIL PROCESADO:     %c | SENTIDO: Entrada (%b)%n", perfilNormalizado, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO:              Entrada %02d:%02d | Salida %02d:%02d%n", horaEntrada, minutoEntrada, horaSalida, minutoSalida);
        System.out.printf("PERMANENCIA:          %03d minutos en el centro.%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round)%n", porcentajeOcupacionRedondeado);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Línea de visión a pantalla)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
```

```
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre transformación, recorte de subcadenas y reemplazo de texto:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). TRANSFORMACIÓN Y EXTRACCIÓN EN TU PROYECTO PROPIO

- **Objetivo.** Incorporar a tu clase `MiProyecto.java` (versión v1.7) al menos una limpieza con `trim()`, una normalización con `toUpperCase()` o `toLowerCase()`, un recorte con `substring()` y una sustitución con `replace()`.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Limpiar el nombre del jugador con `trim()`, pasar el nombre a mayúsculas con `toUpperCase()`, extraer el prefijo de la clase del héroe con `substring(0, 3)` y sustituir guiones de la ficha de personaje por espacios con `replace()`.
- En **Motor de recomendación**. Limpiar los espacios de la etiqueta de búsqueda con `trim()`, normalizar el género a minúsculas con `toLowerCase()`, extraer el año de estreno desde el identificador con `substring()` y cambiar barras por guiones con `replace()`.
- En **Simulador de físicas 2D**. Normalizar la clave de la partícula a mayúsculas con `toUpperCase()`, recortar el código de material con `substring()` y sustituir comas decimales por puntos con `replace()`.
- En **Bóveda de contraseñas**. Eliminar espacios accidentales en la entrada de la clave con `trim()`, extraer los primeros cuatro caracteres del identificador como prefijo seguro con `substring(0, 4)` y generar una versión ofuscada reemplazando caracteres sensibles con `replace()`.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). LA REGLA DEL `endIndex` Y EL TROCEADO DINÁMICO

- **Contexto técnico.** En `cadena.substring(inicio, fin)`, el índice `fin` queda excluido. Además, los índices no deben codificarse con números fijos mágicos, sino calcularse dinámicamente con `indexOf()`.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaSubstrings.java`, declara la siguiente cadena estructurada:

```
String registroCompleto = "REF-2026:PRODUCTO_X#FINAL";
```

- Utiliza `indexOf()` y `substring()` para extraer dinámicamente en cuatro variables independientes:
- El prefijo: desde el inicio hasta el primer guion (`"REF"`).
- El año: entre el guion y los dos puntos (`"2026"`).
- El nombre: entre los dos puntos y la almohadilla (`"PRODUCTO_X"`).
- El sufijo: desde la almohadilla hasta el final (`"FINAL"`).
- Imprime cada fragmento en una línea separada.
- Modifica la variable inicial por `"ID-99:SENSOR_B#OFF"` y comprueba que tu algoritmo sigue extrayendo las cuatro piezas con exactitud sin tocar los índices.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). LA TRAMPA DE LA INMUTABILIDAD Y EL ENCADENAMIENTO

- **Contexto de memoria.** Demostrar empíricamente qué ocurre cuando se transforman cadenas sin capturar la nueva referencia devuelta.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaInmutabilidad.java`, escribe:

```
String correo = "  usuario@iescaminas.es  ";

// Intento erróneo de modificación in situ:
correo.trim();
correo.toUpperCase();

System.out.println("Intento 1 (sin reasignar): [" + correo + "]");

// Uso correcto con encadenamiento de métodos (method chaining):
correo = correo.trim().toUpperCase().replace(".ES", ".COM");

System.out.println("Intento 2 (reasignando):   [" + correo + "]");
```

- Comprueba que el Intento 1 sigue imprimiendo los espacios y las minúsculas intactas, mientras que el Intento 2 produce el texto transformado.
- Redacta dos líneas de comentario en tu código explicando por qué el Intento 1 generó dos objetos huérfanos en el Heap que fueron recogidos por el Garbage Collector sin llegar a usarse.

3.3.3 Día 22 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las tres de la tarde del miércoles 21 de octubre. En la sala técnica de **AzaharTech**, **Pau Ferrer** ejecuta la versión v1.8 de `ControlAccesoQR.java`.

El programa solicita por consola:

```
Introduce hora de entrada (0-23): 8
Introduce minuto de entrada (0-59): 15
Introduce hora de salida (0-23): 14
Introduce minuto de salida (0-59): 10
```

Y a continuación ejecuta las fórmulas manuales que diseñamos en el Sprint 1:

```
minutosTotalesEntrada = (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada;
minutosTotalesSalida = (horaSalida * MINUTOS_POR_HORA) + minutoSalida;
minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;
```

Laia Claramunt se detiene junto al puesto de Pau y plantea la reflexión de fondo:

*«Pau, en el Sprint 1 estas fórmulas manuales con `MINUTOS_POR_HORA` eran necesarias porque estábamos aprendiendo a operar con variables primitivas y constantes. Pero en la realidad del IES El Caminàs, **ningún estudiante teclea a qué hora entra ni a qué hora sale**.*

*Cuando el alumno caza el código QR con su móvil frente a la pantalla de TV del vestíbulo, el servidor captura de forma automática la **marca temporal real del reloj del sistema operativo**.*

Además, calcular diferencias de tiempo convirtiendo a mano con constantes como `MINUTOS_POR_HORA` sigue siendo un enfoque artesanal: no contempla cambios de fecha, zonas horarias ni estándares internacionales. Java cuenta con una biblioteca moderna de fechas y horas: el paquete `java.time`.

*Hoy eliminaremos esas conversiones manuales y evolucionaremos `ControlAccesoQR` a la versión **v1.9**: utilizaremos `LocalTime` para capturar la hora del reloj, `LocalDate` para la fecha del día y la clase `Duration` para calcular la permanencia exacta entre dos marcas de tiempo de forma automática».*

---

2. Fundamento teórico: la API moderna de fechas y horas (`java.time`)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      ARQUITECTURA DE LA API JAVA.TIME                    │
├──────────────────────┬──────────────────┬────────────────────────────────┤
│ Clase                │ Qué representa   │ Ejemplo de contenido           │
├──────────────────────┼──────────────────┼────────────────────────────────┤
│ LocalTime            │ Hora sin fecha   │ 08:15:30.450 (Horas, minutos, segundos) │
├──────────────────────┼──────────────────┼────────────────────────────────┤
│ LocalDate            │ Fecha sin hora   │ 2026-10-21 (Año, mes, día estándar ISO) │
```

```
┌──────────────────────┬──────────────────┬──────────────────────────────────────┐
│ Duration             │ Cantidad de tiempo│ PT5H55M (Intervalo temporal medido en │
│                      │ entre dos horas   │ segundos, minutos u horas)           │
└──────────────────────┴──────────────────┴──────────────────────────────────────┘
```

A. ¿POR QUÉ `java.time` Y NO LAS ANTIGUAS CLASES `Date` O `Calendar`?

En versiones antiguas de Java (anteriores a Java 8), la gestión temporal se realizaba con `java.util.Date` y `java.util.Calendar`. Eran clases defectuosas: eran mutables en memoria (cualquiera podía alterar una fecha ya registrada por error), los meses empezaban en `0` (enero era el mes 0) y provocaban graves errores de sincronización.

La API moderna `java.time` (estándar ISO-8601): * Es **completamente inmutable**: ningún método altera el objeto; siempre devuelven una nueva instancia en el Heap. * Es segura, precisa hasta el nivel de nanosegundos y muy fácil de leer.

B. CREACIÓN DE HORAS Y FECHAS MEDIANTE MÉTODOS ESTÁTICOS (SIN OPERADOR `new`)

Al igual que aprendimos la semana pasada con la clase `Math` o con `Integer.parseInt()`, en `LocalTime` y `LocalDate` no utilizamos el operador `new`.

En su lugar, invocamos **métodos estáticos directamente desde el nombre de la clase** para obtener el objeto ya preparado:

1. **Captura del reloj del sistema en tiempo real (`now()`).**
   Invocamos al método estático `now()` para que el sistema operativo lea el reloj interno de la máquina:

```java
LocalTime horaActual = LocalTime.now(); // Lee el reloj en este instante (ej. 15:04:12)
LocalDate fechaHoy = LocalDate.now();   // Lee la fecha actual del sistema (ej. 2026-10-21)
```

2. **Fijar una hora o fecha concreta (`of()`).**
   Invocamos al método estático `of(...)` pasándole los valores numéricos entre paréntesis como argumentos:

```java
LocalTime horaApertura = LocalTime.of(8, 0);       // 08:00 h
LocalTime horaAcceso   = LocalTime.of(8, 15);      // 08:15 h
LocalDate inicioCurso  = LocalDate.of(2026, 9, 14); // 14 de septiembre de 2026
```

C. MÉTODOS DE INSPECCIÓN Y DESPLAZAMIENTO

- **Extracción de componentes:**
- `hora.getHour()`. Devuelve la hora en formato 0–23 (`int`).
- `hora.getMinute()`. Devuelve el minuto 0–59 (`int`).
- `hora.getSecond()`. Devuelve el segundo 0–59 (`int`).
- **Desplazamiento inmutable:**
- `hora.plusMinutes(45)`. Devuelve un nuevo `LocalTime` con 45 minutos sumados.
- `hora.minusHours(2)`. Devuelve un nuevo `LocalTime` restando 2 horas.

D. CÁLCULO AUTOMÁTICO DE DURACIONES CON `Duration.between()`

Para calcular el tiempo transcurrido entre dos marcas horarias sin realizar operaciones matemáticas manuales:

```java
LocalTime entrada = LocalTime.of(8, 15);
LocalTime salida  = LocalTime.of(14, 10);

// Duration calcula el intervalo exacto entre ambas marcas
Duration estancia = Duration.between(entrada, salida);

long minutosTotales = estancia.toMinutes(); // Devuelve 355 minutos directamente
```

---

3. Evolución del caso guía a `ControlAccesoQR v1.9`

Ampliamos el programa guía incorporando: 1. Las directivas de importación de la API moderna: `import java.time.LocalTime;`, `import java.time.LocalDate;` y `import java.time.Duration;`. 2. Captura automática de la fecha actual del centro con **`LocalDate.now()`**. 3. Captura del instante exacto de emisión del QR en la pantalla de TV con **`LocalTime.now()`**. 4. Sustitución de los cálculos manuales `(hora * MINUTOS_POR_HORA) + minuto` por objetos **`LocalTime.of()`** y cálculo automático de la estancia con **`Duration.between().toMinutes()`**. 5. Emisión de la fecha oficial y marcas horarias mediante formateo `printf`.

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V1.9)

```
Algoritmo ControlAccesoQR
    // ========================================================================
    // SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
    // Version: 1.9 (Evolucion: marcas horarias temporales y calculo de duracion)
    // ========================================================================

    Definir NOMBRE_CENTRO, PREFIJO_CENTRO, RUTA_LOGS Como Cadena
    Definir MINUTOS_POR_HORA, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
    Definir FACTOR_PORCENTAJE Como Real
    Definir SEMILLA_CALIBRACION Como Entero
    Definir JORNADA_BASE_MINUTOS Como Entero
    Definir TEMP_MIN_SEGURA, TEMP_MAX_SEGURA Como Real
    Definir LONGITUD_DNI_ESTANDAR Como Entero

    NOMBRE_CENTRO <- "IES El Caminas (Castellon)"
    PREFIJO_CENTRO <- "CAMINAS"
    RUTA_LOGS <- ".\caminas\terminal\logs"
    MINUTOS_POR_HORA <- 60
    SEGUNDOS_POR_HORA <- 3600
    SEGUNDOS_POR_MINUTO <- 60
    FACTOR_PORCENTAJE <- 100.0
    SEMILLA_CALIBRACION <- 987654321
    JORNADA_BASE_MINUTOS <- 480
    TEMP_MIN_SEGURA <- 15.0
    TEMP_MAX_SEGURA <- 35.0
    LONGITUD_DNI_ESTANDAR <- 9

    Definir textoTerminalId Como Cadena
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona, perfilNormalizado Como Caracter
    Definir esEntrada Como Logico

    // [NUEVO DÍA 22] Variables para marcas horarias (en Java representadas por LocalTime/LocalDate)
    Definir fechaSesion Como Cadena
    Definir horaEntrada, minutoEntrada, horaSalida, minutoSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumenVestibulo, tokenResumenPuertaNorte Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero
    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionTruncado, porcentajeOcupacionRedondeado Como Entero

    Definir codigoTokenVestibulo, codigoTokenPuertaNorte Como Entero
    Definir codigoTestCalibracion Como Entero
    Definir sensorNfcOperativo Como Logico
    Definir desviacionJornadaMinutos Como Entero
    Definir aforoSeguroVisualizado Como Entero
    Definir coordenadaXMetros, coordenadaYMetros, distanciaDiagonalAntena Como Real

    Definir longitudDni, longitudNombre Como Entero
    Definir letraFinalDni Como Caracter
    Definir esLongitudDniCorrecta Como Logico
    Definir posicionPrimerGuion Como Entero
    Definir numeroDniSolo Como Cadena
    Definir tokenFormatoBarras Como Cadena
    Definir prefijoExtraido Como Cadena

    fechaSesion <- "2026-10-21" // Simulación de LocalDate.now()

    Escribir "AZAHARTECH: TERMINAL ", NOMBRE_CENTRO, " (v1.9 - API tiempo)"
    Escribir "Introduce identificador del terminal en texto (ej. 101):"
    Leer textoTerminalId
    terminalId <- ConvertirANumero(textoTerminalId)

    Escribir "Temperatura base del sensor (ºC):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "Distancia horizontal a pantalla en metros (eje X):"
    Leer coordenadaXMetros
    Escribir "Altura de la pantalla en metros (eje Y):"
    Leer coordenadaYMetros

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (e = estudiante, d = docente, v = visita):"
    Leer perfilPersona
```

```
    perfilNormalizado <- Mayusculas(perfilPersona)
    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    // [NUEVO DÍA 22] El cálculo de estancia en Java se delega en Duration.between()
    minutosTotalesEntrada <- (horaEntrada * MINUTOS_POR_HORA) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * MINUTOS_POR_HORA) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    personasEnCentro <- personasEnCentro + 1
    aforoDisponible <- aforoDisponible - 1
    idUltimoFichaje <- idUltimoFichaje + 1

    codigoTokenVestibulo <- Aleatorio(1000, 9999)
    codigoTokenPuertaNorte <- Aleatorio(1000, 9999)
    codigoTestCalibracion <- 5555
    fluctuacionTermica <- 0.45
    tempCalibrada <- tempVestibulo + fluctuacionTermica
    sensorNfcOperativo <- Verdadero

    desviacionJornadaMinutos <- abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS)

    Si tempCalibrada > TEMP_MAX_SEGURA Entonces
        tempSeguraAcotada <- TEMP_MAX_SEGURA
    SiNo
        Si tempCalibrada < TEMP_MIN_SEGURA Entonces
            tempSeguraAcotada <- TEMP_MIN_SEGURA
        SiNo
            tempSeguraAcotada <- tempCalibrada
        FinSi
    FinSi

    tokenResumenVestibulo <- PREFIJO_CENTRO + "-" + dniPersona + "-T" + ConvertirATexto(terminalId) + "-SEC#" + ConvertirATexto(codigoTokenVestibulo) + "-REG"
+ ConvertirATexto(idUltimoFichaje)
    tokenResumenPuertaNorte <- PREFIJO_CENTRO + "-" + dniPersona + "-T102-SEC#" + ConvertirATexto(codigoTokenPuertaNorte) + "-REG" +
ConvertirATexto(idUltimoFichaje)

    horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
    minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
    segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

    aforoTotal <- personasEnCentro + aforoDisponible
    aforoSeguroVisualizado <- personasEnCentro
    porcentajeOcupacionReal <- (aforoSeguroVisualizado * FACTOR_PORCENTAJE) / aforoTotal
    porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)
    porcentajeOcupacionRedondeado <- redon(porcentajeOcupacionReal)

    distanciaDiagonalAntena <- rc((coordenadaXMetros ^ 2) + (coordenadaYMetros ^ 2))

    longitudDni <- Longitud(dniPersona)
    longitudNombre <- Longitud(nombrePersona)
    letraFinalDni <- Subcadena(dniPersona, longitudDni, longitudDni)
    esLongitudDniCorrecta <- (longitudDni == LONGITUD_DNI_ESTANDAR)
    posicionPrimerGuion <- 7

    dniNumeros <- Subcadena(dniPersona, 1, longitudDni - 1)
    tokenFormatoBarras <- "CAMINAS/DNI/T101/SEC#OK"
    prefijoExtraido <- Subcadena(tokenResumenVestibulo, 1, 7)

    Escribir ""
    Escribir "======================================================================="
    Escribir "             INFORME OFICIAL DE ACCESO EN VESTIBULO                    "
    Escribir "======================================================================="
    Escribir "CENTRO:             ", NOMBRE_CENTRO, " | FECHA: ", fechaSesion
    Escribir "REGISTRO N. :       #", idUltimoFichaje
    Escribir "TOKEN VESTIBULO:    ", tokenResumenVestibulo
    Escribir "TOKEN PUERTA NORTE: ", tokenResumenPuertaNorte
    Escribir "PERSONA:            ", Mayusculas(nombrePersona)
    Escribir "DNI DESGLOSADO:     Numero: ", dniNumeros, " | Control: '", letraFinalDni, "'"
    Escribir "IDENTIFICADOR:      ", dniPersona, " | SENTIDO: Entrada (", esEntrada, ")"
    Escribir "-----------------------------------------------------------------------"
    Escribir "HORARIO REGISTRADO: Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "PERMANENCIA TOTAL:  ", minutosEstanciaTotal, " minutos (calculo automatico Duration)."
    Escribir "DESVIACION JORNADA: ", desviacionJornadaMinutos, " min respecto a jornada completa (480 min)."
    Escribir "OCUPACION REDONDEAD:", porcentajeOcupacionRedondeado, " %"
    Escribir "DISTANCIA A TV:     ", distanciaDiagonalAntena, " metros"
    Escribir "ACTIVO:             ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
    Escribir "REGISTRO LOG:       ", RUTA_LOGS
    Escribir "======================================================================="
FinAlgoritmo
```

PASO B. IMPLEMENTACIÓN EN JAVA (`pr/src/ControlAccesoQR.java` — V1.9)

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
```

```
 *
 * Versión 1.9:
 * Incorporación de la API moderna java.time (LocalTime, LocalDate y Duration).
 * Se eliminan las conversiones matemáticas manuales y se capturan marcas de tiempo reales.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 1.9 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */
import java.util.Scanner;
import java.util.Random;

// [NUEVO DÍA 22] Importación de la API moderna de fechas y tiempos de Java
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = ".\\caminas\\terminal\\logs";
        final int MINUTOS_POR_HORA = 60;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int BASE_TOKEN_SEGURIDAD = 1000;
        final int RANGO_TOKEN_SEGURIDAD = 9000;
        final double MAX_VARIACION_TERMICA = 0.5;
        final long SEMILLA_CALIBRACION = 987654321L;
        final int JORNADA_BASE_MINUTOS = 480;
        final double TEMP_MIN_SEGURA = 15.0;
        final double TEMP_MAX_SEGURA = 35.0;
        final int LONGITUD_DNI_ESTANDAR = 9;

        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        // [NUEVO DÍA 22] Captura automática de fecha y hora del sistema mediante métodos estáticos
        LocalDate fechaActual = LocalDate.now();      // Fecha actual del sistema operativo
        LocalTime horaEmisionQr = LocalTime.now();    // Instante exacto de generación del QR

        String textoTerminalId;
        int terminalId;
        double tempVestibulo;

        String nombrePersonaBruto, nombrePersonaLimpio;
        String dniPersonaBruto, dniPersonaLimpio;
        char perfilPersonaBruto;
        char perfilNormalizado;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;

        // [NUEVO DÍA 22] Variables de tiempo y duración con java.time
        LocalTime tiempoEntrada;
        LocalTime tiempoSalida;
        Duration duracionEstancia;
        long minutosEstanciaTotal; // toMinutes() devuelve un long

        String tokenResumenVestibulo;
        String tokenResumenPuertaNorte;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        int porcentajeOcupacionRedondeado;

        double coordenadaXMetros;
        double coordenadaYMetros;
        double distanciaDiagonalAntena;

        int codigoTokenVestibulo;
        int codigoTokenPuertaNorte;
        int codigoTestCalibracion;
        double fluctuacionTermica;
        double tempCalibrada;
        boolean sensorNfcOperativo;
        long desviacionJornadaMinutos;
        int aforoSeguroVisualizado;
```

```
        int longitudDni;
        int longitudNombre;
        char letraFinalDni;
        boolean esLongitudDniCorrecta;
        boolean tokenContienePrefijo;
        int posicionPrimerSeparador;

        String numeroDniSolo;
        String letraDniTexto;
        String tokenFormatoBarras;
        String prefijoExtraido;

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 1.9 (API moderna de fechas java.time) ");
        System.out.println("================================================");
        System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
        textoTerminalId = teclado.nextLine();
        terminalId = Integer.parseInt(textoTerminalId);

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal a la pantalla en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza obligatoria del buffer

        System.out.print("DNI de la persona: ");
        dniPersonaBruto = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersonaBruto = teclado.nextLine();

        dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
        nombrePersonaLimpio = nombrePersonaBruto.trim();

        System.out.print("Perfil de acceso (e = estudiante, d = docente, v = visita): ");
        perfilPersonaBruto = teclado.next().charAt(0);
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ------------------------------------------------------------------
        // [NUEVO DÍA 22] PROCESAMIENTO CON JAVA.TIME (SIN CÁLCULOS MANUALES)
        // ------------------------------------------------------------------
        // 1. Construcción de objetos LocalTime con el método estático of()
        tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
        tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

        // 2. Cálculo automático del intervalo de estancia mediante Duration
        duracionEstancia = Duration.between(tiempoEntrada, tiempoSalida);
        minutosEstanciaTotal = duracionEstancia.toMinutes(); // Conversión limpia y directa

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        codigoTestCalibracion = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
        fluctuacionTermica = generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA;
        tempCalibrada = tempVestibulo + fluctuacionTermica;
        sensorNfcOperativo = generadorCalibracion.nextBoolean();

        // Cálculo de desviación usando Math.abs sobre el resultado de Duration
        desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        aforoTotal = personasEnCentro + aforoDisponible;
        aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        tokenResumenVestibulo = PREFIJO_CENTRO + "-" + dniPersonaLimpio;
        tokenResumenVestibulo += "-T" + terminalId;
```

```
        tokenResumenVestibulo += "-SEC#" + codigoTokenVestibulo;
        tokenResumenVestibulo += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenVestibulo += "-REG" + idUltimoFichaje;

        tokenResumenPuertaNorte = PREFIJO_CENTRO + "-" + dniPersonaLimpio;
        tokenResumenPuertaNorte += "-T102";
        tokenResumenPuertaNorte += "-SEC#" + codigoTokenPuertaNorte;
        tokenResumenPuertaNorte += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumenPuertaNorte += "-REG" + idUltimoFichaje;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) aforoSeguroVisualizado / aforoTotal) * FACTOR_PORCENTAJE;
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;
        porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        distanciaDiagonalAntena = Math.sqrt(Math.pow(coordenadaXMetros, 2.0) + Math.pow(coordenadaYMetros, 2.0));

        longitudDni = dniPersonaLimpio.length();
        longitudNombre = nombrePersonaLimpio.length();
        letraFinalDni = dniPersonaLimpio.charAt(dniPersonaLimpio.length() - 1);
        esLongitudDniCorrecta = (longitudDni == LONGITUD_DNI_ESTANDAR);
        tokenContienePrefijo = tokenResumenVestibulo.contains(PREFIJO_CENTRO);
        posicionPrimerSeparador = tokenResumenVestibulo.indexOf("-");

        numeroDniSolo = dniPersonaLimpio.substring(0, dniPersonaLimpio.length() - 1);
        letraDniTexto = dniPersonaLimpio.substring(dniPersonaLimpio.length() - 1);
        tokenFormatoBarras = tokenResumenVestibulo.replace("-", "/");
        prefijoExtraido = tokenResumenVestibulo.substring(0, posicionPrimerSeparador);

        // Salida formateada con printf integrando marcas de tiempo de java.time
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:         #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN QR PUERTA NORTE:%s%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio.toUpperCase(), longitudNombre);
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%s'%n", numeroDniSolo, letraDniTexto);
        System.out.printf("PERFIL PROCESADO:     %c | SENTIDO: Entrada (%b)%n", perfilNormalizado, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
        System.out.printf("PERMANENCIA EXACTA:   %03d minutos en el centro (Duration.between)%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round)%n", porcentajeOcupacionRedondeado);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Línea de visión a pantalla)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de dificultad progresiva** sobre la API `java.time`, marcas temporales reales y cálculos de duración:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). INTEGRACIÓN DE `java.time` EN TU PROYECTO PROPIO

- **Objetivo.** Incorporar a tu clase `MiProyecto.java` (versión v1.8) la captura de la fecha actual con `LocalDate.now()`, la hora actual con `LocalTime.now()` y el cálculo de una duración temporal con `Duration.between()` sin multiplicaciones manuales.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Registrar la fecha y hora oficial de inicio de la partida con `LocalDate.now()` y `LocalTime.now()`, y calcular los minutos reales de sesión transcurridos con `Duration.between(horaInicio, horaActual)`.
- En **Motor de recomendación**. Registrar el instante exacto en que se emite la recomendación con `LocalTime.now()`, y proyectar a qué hora terminará la película sumando su duración mediante `horaActual.plusMinutes(duracionMinutos)`.
- En **Simulador de físicas 2D**. Registrar la marca temporal exacta de inicio y fin del cálculo cinemático con `LocalTime`, y calcular los segundos transcurridos utilizando `Duration.between().toSeconds()`.
- En **Bóveda de contraseñas**. Capturar la fecha actual con `LocalDate.now()`, calcular la fecha exacta de expiración sumando días con `fechaActual.plusDays(diasVigencia)`, y registrar la hora del último cambio de clave con `LocalTime.now()`.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). PROYECCIONES HORARIAS CON `plusMinutes()` Y `minusHours()`

- **Contexto técnico.** Demostrar que los métodos de desplazamiento temporal de `LocalTime` son inmutables y devuelven un nuevo objeto, gestionando el cambio de día de forma automática sin condicionales.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaHorarios.java`, declara una hora base:

```
LocalTime inicioTurno = LocalTime.of(22, 45); // 22:45 h
```

- Calcula y muestra por consola qué hora será tras:
- Una primera ronda de 50 minutos: `inicioTurno.plusMinutes(50)`.
- Un descanso de 20 minutos adicionales: `inicioTurno.plusMinutes(70)`.
- El momento en que debió prepararse el equipo hace 3 horas: `inicioTurno.minusHours(3)`.
- Comprueba cómo al sumar 50 minutos a las `22:45`, Java pasa automáticamente a las `23:35`, y al sumar 90 minutos pasa a las `00:15` del día siguiente sin necesidad de programar ningún algoritmo de control.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). LA TRAMPA DEL CRUCE DE MEDIANOCHE EN `Duration`

- **Contexto de sistemas.** En sistemas de control de turnos que operan de noche, ¿qué ocurre si la entrada se produce a las `23:00` y la salida a las `02:00`?
- **Misión de la kata:**
- En una clase de prueba temporal llamada `PruebaMedianoche.java`, ejecuta:

```
LocalTime entradaNoche = LocalTime.of(23, 0);
LocalTime salidaMadrugada = LocalTime.of(2, 0);

Duration intervalo = Duration.between(entradaNoche, salidaMadrugada);
System.out.println("Minutos calculados: " + intervalo.toMinutes());
```

- Comprueba con asombro que la consola imprime un número negativo: **`-1260 minutos`** (-21 horas).
- Redacta dos líneas de comentario en tu código explicando por qué ocurre esto:
  « `LocalTime` solo almacena horas del reloj sin fecha asociada. Al calcular la diferencia entre las 23:00 y las 02:00 del mismo día, Java calcula que las 02:00 ocurrieron 21 horas antes. Para turnos que cruzan la medianoche en sistemas profesionales es necesario combinar fecha y hora mediante la clase `LocalDateTime` ».

3.3.4 Día 23 - 2 sesiones

1. Caso guía en AzaharTech

Son las tres de la tarde del jueves 22 de octubre. Concluimos las 24 horas lectivas del **Sprint 2 de Programación**.

En la sala de reuniones de **AzaharTech**, **Pau Ferrer** proyecta el archivo `ControlAccesoQR.java`. El programa compila sin errores, genera los tokens con `Random`, calcula distancias con `Math`, parsea datos con `Integer` y gestiona las horas reales con `LocalTime` y `Duration`.

Sin embargo, **Alba Torres** señala el monitor con preocupación:

*«Pau, el software funciona, pero hemos creado un 'método Dios' (*God Method*): tenemos casi doscientas cincuenta líneas de código acumuladas dentro del* `main`*. La captura por teclado, las conversiones de hora, la descomposición de segundos, las fórmulas matemáticas y la composición de los tokens están todas mezcladas en un único bloque monolítico.*

Si mañana el IES El Caminàs nos pide reutilizar la fórmula del token QR para una aplicación web o para la puerta norte, tendríamos que copiar y pegar código duplicado.

En AzaharTech el código debe ser modular y mantenible. Hasta ahora hemos aprendido a invocar métodos estáticos ajenos de la biblioteca de Java como `Math.max()` *o* `Integer.parseInt()`*. Hoy daremos el gran salto de ingeniería:* ***aprenderemos a codificar nuestros propios métodos estáticos auxiliares*** *con sus parámetros y valores de retorno, transformando el* `main` *en un orquestador limpio y elegante».*

---

2. Fundamento teórico: codificación de métodos estáticos propios, parámetros y retorno

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        ANATOMÍA DE UN MÉTODO ESTÁTICO EN JAVA                │
├──────────────────────────────────────────────────────────────────────────────┤
│ public static tipoRetorno nombreMetodo( TipoParam1 param1, TipoParam2 param2... ) {    │
│     // 1. Ámbito local: variables que solo existen dentro de este método              │
│     // 2. Procesamiento o cálculo                                                      │
│     return valorCalculado; // Devuelve el dato a quien lo invocó                      │
│ }                                                                                      │
└──────────────────────────────────────────────────────────────────────────────┘
```

A. ¿POR QUÉ CREAMOS MÉTODOS ESTÁTICOS DENTRO DE LA CLASE?

Java permite modularizar cualquier programa estructurándolo en **métodos estáticos auxiliares (*funciones de clase*)** dentro del mismo archivo por los siguientes motivos:

1. **Legibilidad y auto-documentación.** El código se lee como un texto estructurado donde el nombre del método explica *qué* hace, ocultando los detalles de *cómo* lo hace.
2. **Reutilización (*DRY - Don't Repeat Yourself*).** Si necesitamos calcular la distancia o construir un token en dos sitios diferentes, invocamos al método sin duplicar fórmulas.
3. **Mantenibilidad y aislamiento.** Si cambia la regla de cálculo de la estancia, solo modificamos el método responsable, sin tocar el resto de la aplicación.

B. PARÁMETROS FORMALES Y EL PASO DE PARÁMETROS POR VALOR

- **Parámetros formales.** Son las variables declaradas entre los paréntesis de la cabecera del método que reciben los datos necesarios para operar (*por ejemplo,* `LocalTime entrada, LocalTime salida`).
- **Argumentos reales.** Son los valores o variables concretas que se le pasan al método en el momento de la llamada desde el `main` (*por ejemplo,* `tiempoEntrada, tiempoSalida`).
- **Paso por valor estricto en Java.** Java **siempre pasa los argumentos por valor (hace una copia)**:
- Si pasas un tipo primitivo (`int`, `double`), el método recibe una copia del número. Modificar el parámetro dentro del método jamás altera la variable original del `main`.
- Si pasas un objeto, se copia el puntero de referencia en el Stack, permitiendo operar sobre la instancia compartida en el Heap.

C. LA SENTENCIA `return` Y EL ÁMBITO LOCAL (*SCOPE*)

- **Tipo de retorno.** Indica qué tipo de dato devuelven las instrucciones del método (`int`, `double`, `String`, etc.). Si el método no devuelve ningún dato y solo ejecuta acciones de salida, se define como `void`.
- **La instrucción `return`.** Detiene de forma inmediata la ejecución del método y transfiere el resultado al punto exacto del `main` donde se produjo la llamada.
- **Ámbito local (*Scope*).** Cualquier variable declarada dentro de un método auxiliar nace y muere dentro de sus llaves `{}`. No consume memoria en el Stack cuando el método termina.

---

3. El código guia final para el Sprint 2: `ControlAccesoQR v2.0` (modularizado)

Refactorizamos la clase guia `ControlAccesoQR.java` descomponiendo los bloques de cálculo en cuatro métodos estáticos auxiliares limpios:

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.0:
 * Sistema integral de acceso orientado a objetos basado en clases predefinidas.
 * Incorpora la codificación de métodos estáticos propios con parámetros y retorno,
 * desacoplando la lógica de cálculo, descomposición temporal y construcción de tokens.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
 * @version 2.0 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    /**
     * Calcula la estancia neta en minutos entre dos marcas temporales usando Duration.
     */
    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    /**
     * Calcula el porcentaje de ocupación con precisión decimal aplicando casting explícito.
     */
    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    /**
     * Construye y concatena el token QR oficial parametrizado para cualquier monitor.
     */
    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    /**
     * Calcula la distancia euclídea (hipotenusa) a la pantalla mediante el teorema de Pitágoras.
     */
    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODO PRINCIPAL: ORQUESTADOR SECUENCIAL DEL FLUJO
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
```

```
        Scanner teclado = new Scanner(System.in);

        // Generadores independientes en el Heap
        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();
        LocalTime horaEmisionQr = LocalTime.now();

        int terminalId;
        double tempVestibulo;

        String nombrePersonaBruto, nombrePersonaLimpio;
        String dniPersonaBruto, dniPersonaLimpio;
        char perfilPersonaBruto;
        char perfilNormalizado;
        boolean esLetraValida;
        boolean esEntrada = true;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        double coordenadaXMetros, coordenadaYMetros;

        // Captura de datos interactiva
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 2.0 Oficial (Arquitectura Modular)       ");
        System.out.println("================================================");
        System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
        terminalId = Integer.parseInt(teclado.nextLine());

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal a pantalla en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza del buffer de entrada

        System.out.print("DNI de la persona: ");
        dniPersonaBruto = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersonaBruto = teclado.nextLine();

        // Normalización y limpieza con métodos de String y Character
        dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
        nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

        System.out.print("Perfil de acceso (e = estudiante, d = docente, v = visita): ");
        perfilPersonaBruto = teclado.next().charAt(0);
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
        esLetraValida = Character.isLetter(perfilNormalizado);

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();
        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();
        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // ---------------------------------------------------------------------
        // INVOCACIÓN DE NUESTROS MÉTODOS ESTÁTICOS PROPIOS
        // ---------------------------------------------------------------------
        LocalTime tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
        LocalTime tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

        // 1. Llamada a método propio de cómputo de tiempo
        int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        // Generación de códigos numéricos
        int codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTestCalibracion  = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);
```

```
        double tempCalibrada = tempVestibulo + (generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA);
        boolean sensorNfcOperativo = generadorCalibracion.nextBoolean();

        int desviacionJornadaMinutos = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        double tempSeguraAcotada = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        int aforoTotal = personasEnCentro + aforoDisponible;
        int aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);

        // 2. Llamadas a método propio para construir los tokens de forma limpia y reutilizable
        String tokenResumenVestibulo   = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);
        String tokenResumenPuertaNorte = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, ID_TERMINAL_PUERTA_NORTE, codigoTokenPuertaNorte,
idUltimoFichaje);

        // 3. Llamada a método propio para el cálculo porcentual con casting
        double porcentajeOcupacionReal = calcularPorcentajeOcupacion(aforoSeguroVisualizado, aforoTotal);
        int porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        // 4. Llamada a método propio para la distancia geométrica
        double distanciaDiagonalAntena = calcularDistanciaDiagonal(coordenadaXMetros, coordenadaYMetros);

        // Descomposición temporal de servicio del terminal
        int horasUptime   = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        int minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        int segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // Métodos de String para desglose de DNI
        String numeroDniSolo = dniPersonaLimpio.substring(0, dniPersonaLimpio.length() - 1);
        char letraFinalDni   = dniPersonaLimpio.charAt(dniPersonaLimpio.length() - 1);
        boolean esLongitudDniCorrecta = (dniPersonaLimpio.length() == LONGITUD_DNI_ESTANDAR);
        String tokenFormatoBarras = tokenResumenVestibulo.replace("-", "/");

        // ------------------------------------------------------------------
        // SALIDA FORMATEADA PROFESIONAL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:         #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);
        System.out.printf("TOKEN QR VESTÍBULO:   %s%n", tokenResumenVestibulo);
        System.out.printf("TOKEN FORMATO BARRAS: %s (replace)%n", tokenFormatoBarras);
        System.out.printf("TOKEN QR PUERTA NORTE:%s%n", tokenResumenPuertaNorte);
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio, nombrePersonaLimpio.length());
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%c'%n", numeroDniSolo, letraFinalDni);
        System.out.printf("PERFIL PROCESADO:     %c (Válido: %b) | SENTIDO: Entrada (%b)%n", perfilNormalizado, esLetraValida, esEntrada);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
        System.out.printf("PERMANENCIA EXACTA:   %03d minutos (Método modular calcularEstancia)%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round) | Exacta: %6.2f %%%n", porcentajeOcupacionRedondeado, porcentajeOcupacionReal);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Método modular calcularDistancia)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor acotado: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime,
tempSeguraAcotada);
        System.out.printf("DIAGNÓSTICO TEST:     Patrón #%04d | Sensor NFC: Activo (%b)%n", codigoTestCalibracion, sensorNfcOperativo);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

---

4. Segunda sesión: dojo de entrenamiento y katas de código

Pasamos al dojo de entrenamiento para programar, con tres **katas de programación modular** para entrenar la codificación de métodos estáticos, la gestión de parámetros y el ámbito de memoria:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). EXTRACCIÓN DE MÉTODOS ESTÁTICOS EN TU PROYECTO PROPIO

- **Objetivo.** Refactorizar tu clase `MiProyecto.java` (versión v1.9) extrayendo al menos **dos métodos estáticos auxiliares propios** (`public static`) con parámetros de entrada y sentencia `return`, eliminando la acumulación de cálculos dentro del `main`.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Codificar un método estático `public static int calcularDanoTotal(int fuerza, int factorArma)` y un método `public static String formatearEstadoHeroe(String nombre, int vida, int oro)`.

- En **Motor de recomendación**. Codificar un método estático `public static double calcularPuntuacionAfinidad(double base, double pesoTag)` y un método `public static String generarCodigoItem(String categoria, int anio, int id)`.
- En **Simulador de físicas 2D**. Codificar un método estático `public static double calcularModuloVelocidad(double vx, double vy)` y un método `public static double proyectarPosicionFinal(double vInicial, double aceleracion, double tiempo)`.
- En **Bóveda de contraseñas**. Codificar un método estático `public static int calcularDiasRestantes(int diasTotales, int diasConsumidos)` y un método `public static String formatearCredencialSegura(String servicio, String usuario, int pin)`.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). EL AISLAMIENTO DEL ÁMBITO LOCAL (*SCOPE*) Y PASO POR VALOR

- **Contexto técnico.** Demostrar empíricamente que modificar un parámetro primitivo dentro de un método estático no tiene ningún efecto sobre la variable original del `main`.
- **Misión de la kata:**
- En una clase de prueba llamada `PruebaPasoPorValor.java`, escribe el siguiente método estático:

```
public static void intentarModificar(int numero) {
    numero = numero + 100; // Modificamos el parámetro local
    System.out.println("Dentro del método: numero = " + numero);
}
```

- En el `main`, declara `int valor = 50;`, invoca `intentarModificar(valor);` e imprime después el contenido de `valor`.
- Comprueba en la consola que fuera del método la variable sigue valiendo `50`.
- Redacta dos líneas de comentario en tu código explicando por qué ocurre esto: **Java pasa los argumentos primitivos por copia de valor; el método opera sobre una celda del Stack aislada que se destruye al llegar a su llave de cierre**.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). COMPOSICIÓN Y ENCADENAMIENTO DE MÉTODOS ESTÁTICOS

- **Contexto de arquitectura.** Demostrar cómo el valor devuelto por un método estático puede actuar como argumento de entrada de otro sin necesidad de crear variables intermedias en el `main`.
- **Misión de la kata:**
- En tu proyecto propio o en una clase de prueba, codifica dos métodos estáticos complementarios: uno que calcule un valor numérico (`calcularSubtotal`) y otro que lo reciba para aplicar un factor (`aplicarImpuesto`).
- En el `main`, realiza la invocación encadenada en una sola línea:

```
double totalFinal = aplicarImpuesto(calcularSubtotal(unidades, precioBase));
```

- Anota en tu cuaderno técnico cómo la JVM apila y desapila las llamadas en el *Call Stack* de dentro hacia afuera.

---

5. Cierre formal en Git y sincronización visual en IntelliJ

Cada estudiante finaliza el Sprint 2 registrando su código modular en GitHub desde la interfaz gráfica de IntelliJ IDEA:

1. Aplica el autoformateo oficial: `Ctrl + Alt + L`.
2. Optimiza las importaciones para limpiar dependencias no usadas: `Ctrl + Alt + O`.
3. Abre el panel lateral **Commit** (`Alt + 0` o `Ctrl + K`).
4. Selecciona tu archivo `pr/src/MiProyecto.java`.
5. Escribe el mensaje convencional de entrega del hito:

```
refactor(pr): modularizar aplicacion v2.0 con metodos estaticos propios y cierre de sprint 2
```

6. Pulsa **Commit and Push…** y confirma el envío al servidor remoto de GitHub.

3.4 Cierre del Sprint 2

- **Entrega lista.** El estudiante tiene su proyecto propio en GitHub en versión `v2.0`, listo para ser registrado mañana viernes bajo la etiqueta `v0.2.0-sprint2`.

4. Sprint 3. Uso de estructuras de control

4.1 Semana 7. Selección: if-else, switch y try-catch

4.1.1 Día 24 - 2 sesiones

1. Caso guía en AzaharTech

Son las 15:00 h del **lunes 26 de octubre de 2026**. En la sede de **AzaharTech** en Castellón de la Plana, el equipo de desarrollo arranca formalmente el **Sprint 3**. En este nuevo hito del curso se produce un cambio metodológico esencial: **se abandona PSeInt**; a partir de este instante, toda la algorítmica, análisis y diseño se implementa **directa y exclusivamente en Java**.

Sobre la pantalla de la sala de reuniones, **Alba Torres** y **Pau Ferrer** proyectan el código fuente de la versión `v2.0` que cerró el Sprint 2 el pasado viernes. **Laia Claramunt** toma la palabra para contextualizar el salto cualitativo que exige el cliente:

«Alba, Pau: la versión `v2.0` consolidó una arquitectura modular impecable. Desacoplamos la descomposición horaria con `Duration`, aislamos el cálculo de distancias euclídeas con `Math.sqrt` y parametrizamos la construcción de tokens QR en métodos estáticos independientes. Sin embargo, nuestro sistema adolece de un fallo crítico de concepto:

Las líneas 129 a 131 de la versión `v2.0` dicen textualmente:

```
personasEnCentro++;
aforoDisponible--;
idUltimoFichaje++;
```

*El programa es **completamente ciego y secuencial**. No evalúa el contexto antes de actuar: si una persona llega con 39 ºC de fiebre, el torno se abre; si el aforo disponible es cero y el instituto está desbordado, el contador sigue sumando; si el DNI no cumple la longitud legal o el perfil introducido es una letra errónea, el terminal emite alegremente un token oficial de acceso.*

*La dirección del **IES El Caminàs** nos ha trasladado una directriz tajante para este sprint: **el torno físico solo debe franquearse si se cumplen rigurosamente las condiciones de seguridad, aforo y formato**. Si cualquiera de estos filtros falla, el flujo de ejecución debe desviarse, denegar el paso, advertir por consola del motivo del rechazo y congelar los contadores del centro.*

*Hoy introducimos la toma de decisiones en Java mediante la estructura **`if-else`**. Aprenderemos cómo la JVM evalúa expresiones booleanas en cortocircuito y gobernaremos el grafo de flujo de la aplicación».*

2. Fundamento teórico: la bifurcación del flujo de ejecución y la sentencia `if-else` en Java

Hasta la versión `v2.0`, el puntero de instrucción del procesador (*Program Counter* o PC) recorría el método `main` de manera estrictamente lineal. Las **estructuras de control de selección** alteran esta linealidad, permitiendo que la máquina virtual elija rutas alternativas en tiempo de ejecución tras evaluar una expresión booleana.

```
flowchart TD
    Inicio([Puntero PC: Flujo Secuencial]) --> Condicion{"¿Condición booleana?<br/>(temperaturaApta && hayAforo)"}

    Condicion -- true --> RamaIf["<b>Rama IF (Then)</b><br/>• Torno desbloqueado<br/>• personasEnCentro++<br/>• Emitir Token QR"]
    Condicion -- false --> RamaElse["<b>Rama ELSE (Sino)</b><br/>• Torno bloqueado<br/>• Alarma acústica<br/>• Asignar motivoDenegacion"]

    RamaIf --> Fin([Flujo Convergente: Generar informe])
    RamaElse --> Fin
```

A. ESTRUCTURAS CONDICIONALES EN JAVA (`if`, `if-else`, `if-else if-else`)

1. **Condicional simple (`if`).** Evalúa un predicado booleano. Si la condición resulta `true`, se ejecutan las sentencias encerradas en su bloque; si es `false`, la JVM salta directamente a la siguiente instrucción fuera del bloque.

2. **Condicional doble (`if-else`).** Modela dos alternativas mutuamente excluyentes. Si la condición es `true`, se ejecuta únicamente el bloque del `if`; si es `false`, se ejecuta con total certeza el bloque `else`. Jamás se ejecutan ambos ni se ignoran los dos a la vez.
3. **Escalera de selección múltiple (`if-else if-else`).** Permite encadenar validaciones jerárquicas. Las condiciones se comprueban en estricto orden descendente; en cuanto una se cumple, se ejecuta su bloque y se omiten todas las ramas inferiores.

B. OPERADORES LÓGICOS Y EVALUACIÓN EN CORTOCIRCUITO (*SHORT-CIRCUIT*)

Recordemos que, para validar reglas de negocio complejas, combinamos operadores relacionales (`>`, `<`, `>=`, `<=`, `==`, `!=`) y lógicos (`&&`, `||`, `!`):

- **Conjunción (`&&`).** Devuelve `true` solo si **ambos** operandos son verdaderos. Si el primer operando evalúa a `false`, Java aborta de inmediato la evaluación (*cortocircuito*), ya que la expresión completa no puede ser cierta bajo ninguna circunstancia.
- **Disyunción (`||`).** Devuelve `true` si al menos **uno** de los operandos es verdadero. Si el primer operando evalúa a `true`, Java ignora el segundo y da la condición por satisfecha.
- **Negación (`!`).** Invierte el valor de verdad del operando (`!true` es `false`).

```
// Ejemplo de cortocircuito en AzaharTech:
// Si aforoDisponible <= 0, el segundo operando NO se evalúa, evitando llamadas costosas a métodos
boolean puedePasar = (aforoDisponible > 0) && verificarCredencialEnServidor(dniPersonaLimpio);
```

C. ESTÁNDAR DE INGENIERÍA AZAHARTECH: OBLIGATORIEDAD DE LLAVES Y PREVENCIÓN DEL *DANGLING ELSE*

En Java, prescindir de las llaves `{ }` tras un `if` o `else` hace que el compilador ligue únicamente la primera sentencia siguiente:

```
// ¡VULNERABILIDAD GRAVE!: Prohibido en el estándar de código de AzaharTech
if (accesoAutorizado)
    abrirTorno();
    personasEnCentro++; // ¡Esta línea se ejecutará SIEMPRE, incluso si el acceso fue denegado!
```

Regla de estilo de AzaharTech: todo bloque condicional debe delimitarse obligatoriamente mediante llaves `{ }`, sin excepción, con independencia del número de líneas que contenga.

D. ÁMBITO DE LAS VARIABLES LOCALES (*BLOCK SCOPE*)

Toda variable declarada en el interior de las llaves `{ }` de un bloque nace y muere en la memoria de pila (*Stack*) asociada a ese bloque. Si se requiere disponer del valor tras la bifurcación (por ejemplo, para imprimir el informe final), la variable debe declararse en el ámbito externo antes del `if`.

---

3. Evolución del caso guía a `ControlAccesoQR v2.1`

En esta versión sustituimos la mutación ciega de estado de la versión `v2.0` por un **árbol de decisión condicional**. Conservamos intactas las constantes corporativas, los métodos modulares construidos en el Sprint 2 y la captura interactiva por teclado.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.0 (`diff`)

```
- // ========================================================================
- // EN v2.0: Mutación ciega e incondicional (la puerta siempre se abría)
- // ========================================================================
- personasEnCentro++;
- aforoDisponible--;
- idUltimoFichaje++;

+ // ========================================================================
+ // EN v2.1: Mutación condicionada al cumplimiento estricto de requisitos
+ // ========================================================================
+ boolean temperaturaApta = (tempVestibulo >= TEMP_CORPORAL_MIN_FALLO && tempVestibulo <= TEMP_CORPORAL_MAX_FIEBRE);
+ boolean aforoSuficiente = (aforoDisponible > 0);
+ boolean perfilPermitido = esLetraValida && (perfilNormalizado == 'E' || perfilNormalizado == 'D' || perfilNormalizado == 'V');
+
+ if (!temperaturaApta) {
+     accesoConcedido = false;
+     motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera de rango)", tempVestibulo);
+ } else if (!aforoSuficiente) {
+     accesoConcedido = false;
+     motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
+ } else if (!esLongitudDniCorrecta) {
+     accesoConcedido = false;
+     motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
+ } else if (!perfilPermitido) {
+     accesoConcedido = false;
+     motivoDenegacion = "Perfil de acceso '" + perfilNormalizado + "' no autorizado por jefatura.";
```

```
+ } else {
+     accesoConcedido = true;
+     personasEnCentro++;
+     aforoDisponible--;
+     idUltimoFichaje++;
+ }
```

B. FOCO EN LA NOVEDAD: ÁRBOL DE DECISIÓN Y SALIDA BIFURCADA

Observa cómo se integran las nuevas variables de control dentro de `main` y cómo se bifurca la respuesta visual:

```
// ... [Captura de datos previa con Scanner y cálculo de tiempos idéntico a v2.0] ...

// 1. Declaración en el ámbito de main para su consumo en el informe final
boolean accesoConcedido;
String motivoDenegacion = "";

// 2. Evaluación lógica de condiciones de negocio
boolean temperaturaApta = (tempVestibulo >= TEMP_CORPORAL_MIN_FALLO && tempVestibulo <= TEMP_CORPORAL_MAX_FIEBRE);
boolean aforoSuficiente = (aforoDisponible > 0);
boolean perfilPermitido = esLetraValida && (perfilNormalizado == 'E' || perfilNormalizado == 'D' || perfilNormalizado == 'V');

if (!temperaturaApta) {
    accesoConcedido = false;
    motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
            tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
} else if (!aforoSuficiente) {
    accesoConcedido = false;
    motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
} else if (!esLongitudDniCorrecta) {
    accesoConcedido = false;
    motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
} else if (!perfilPermitido) {
    accesoConcedido = false;
    motivoDenegacion = "Perfil de acceso '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
} else {
    accesoConcedido = true;
    // Mutación condicionada: solo muta el estado si el acceso es estrictamente legítimo
    personasEnCentro++;
    aforoDisponible--;
    idUltimoFichaje++;
}

// 3. Emisión de informe bifurcado según el veredicto
if (accesoConcedido) {
    System.out.println("--------------------------------------------------------------------");
    System.out.println(">> VEREDICTO:         [ACCESO AUTORIZADO - TORNO ABIERTO]");
    System.out.printf(">> TOKEN QR EMITIDO:  %s%n", tokenResumenVestibulo);
    System.out.printf(">> TOKEN BARRAS:      %s%n", tokenFormatoBarras);
    System.out.printf(">> PUERTA NORTE LINK: %s%n", tokenResumenPuertaNorte);
} else {
    System.out.println("--------------------------------------------------------------------");
    System.out.println(">> VEREDICTO:         [ACCESO DENEGADO - TORNO BLOQUEADO]");
    System.out.printf(">> MOTIVO RECHAZO:    %s%n", motivoDenegacion);
    System.out.println(">> PROTOCOLO:         Diríjase a la conserjería central para verificación manual.");
}

// ... [Resto del informe de telemetría y diagnóstico idéntico a v2.0] ...
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.1)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.1:
 * Evolución hacia estructuras de control de selección (Sprint 3).
 * Incorpora bifurcación condicional if-else nativa en Java.
 * Protege los contadores de aforo mediante validación previa de seguridad térmica,
 * capacidad disponible, coherencia de DNI y perfil de usuario autorizado.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.1 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales de seguridad biométrica y capacidad (Sprint 3)
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    /**
     * Calcula la estancia neta en minutos entre dos marcas temporales usando Duration.
     */
    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    /**
     * Calcula el porcentaje de ocupación con precisión decimal aplicando casting explícito.
     */
    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    /**
     * Construye y concatena el token QR oficial parametrizado para cualquier monitor.
     */
    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    /**
     * Calcula la distancia euclídea (hipotenusa) a la pantalla mediante el teorema de Pitágoras.
     */
    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODO PRINCIPAL: ORQUESTADOR DEL FLUJO CON BIFURCACIÓN CONDICIONAL
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Generadores independientes en el Heap
        Random generadorVestibulo   = new Random();
```

```
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();
        LocalTime horaEmisionQr = LocalTime.now();

        int terminalId;
        double tempVestibulo;

        String nombrePersonaBruto, nombrePersonaLimpio;
        String dniPersonaBruto, dniPersonaLimpio;
        char perfilPersonaBruto;
        char perfilNormalizado;
        boolean esLetraValida;
        boolean esEntrada = true;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        double coordenadaXMetros, coordenadaYMetros;

        // Captura de datos interactiva
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 2.1 Oficial (Sprint 3: Control if-else)  ");
        System.out.println("================================================");
        System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
        terminalId = Integer.parseInt(teclado.nextLine());

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal a pantalla en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza del buffer de entrada

        System.out.print("DNI de la persona: ");
        dniPersonaBruto = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersonaBruto = teclado.nextLine();

        // Normalización y limpieza con métodos de String y Character
        dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
        nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

        System.out.print("Perfil de acceso (e = estudiante, d = docente, v = visita): ");
        perfilPersonaBruto = teclado.next().charAt(0);
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
        esLetraValida = Character.isLetter(perfilNormalizado);

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();
        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();
        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // Operaciones temporales
        LocalTime tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
        LocalTime tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);
        int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

        // Calibración ambiental y diagnóstico
        double tempCalibrada = tempVestibulo + (generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA);
        boolean sensorNfcOperativo = generadorCalibracion.nextBoolean();
        int codigoTestCalibracion  = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);

        int aforoTotal = personasEnCentro + aforoDisponible;

        // Desglose del documento
        boolean esLongitudDniCorrecta = (dniPersonaLimpio.length() == LONGITUD_DNI_ESTANDAR);
        String numeroDniSolo = dniPersonaLimpio.substring(0, Math.max(0, dniPersonaLimpio.length() - 1));
        char letraFinalDni   = dniPersonaLimpio.charAt(Math.max(0, dniPersonaLimpio.length() - 1));

        // ------------------------------------------------------------------
        // 4. BIFURCACIÓN CONDICIONAL (NOVEDAD SPRINT 3 - DÍA 24)
        // ------------------------------------------------------------------
        boolean accesoConcedido;
        String motivoDenegacion = "";

        // Reglas de negocio del IES El Caminàs
        boolean temperaturaApta = (tempVestibulo >= TEMP_CORPORAL_MIN_FALLO && tempVestibulo <= TEMP_CORPORAL_MAX_FIEBRE);
        boolean aforoSuficiente = (aforoDisponible > 0);
        boolean perfilPermitido = esLetraValida && (perfilNormalizado == 'E' || perfilNormalizado == 'D' || perfilNormalizado == 'V');
```

```
        if (!temperaturaApta) {
            accesoConcedido = false;
            motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
                    tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
        } else if (!aforoSuficiente) {
            accesoConcedido = false;
            motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
        } else if (!esLongitudDniCorrecta) {
            accesoConcedido = false;
            motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
        } else if (!perfilPermitido) {
            accesoConcedido = false;
            motivoDenegacion = "Perfil de acceso '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
        } else {
            accesoConcedido = true;
            personasEnCentro++;
            aforoDisponible--;
            idUltimoFichaje++;
        }

        // Generación de códigos de seguridad condicionados
        int codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        // Construcción de tokens mediante método modular propio
        String tokenResumenVestibulo   = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);
        String tokenResumenPuertaNorte = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, ID_TERMINAL_PUERTA_NORTE, codigoTokenPuertaNorte, idUltimoFichaje);
        String tokenFormatoBarras      = tokenResumenVestibulo.replace("-", "/");

        // Cálculos métricos auxiliares
        int aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);
        double porcentajeOcupacionReal = calcularPorcentajeOcupacion(aforoSeguroVisualizado, aforoTotal);
        int porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        double distanciaDiagonalAntena = calcularDistanciaDiagonal(coordenadaXMetros, coordenadaYMetros);
        int desviacionJornadaMinutos   = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        double tempSeguraAcotada       = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        // Descomposición temporal de servicio
        int horasUptime    = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        int minutosUptime  = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        int segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA CONDICIONAL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n================================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                    ");
        System.out.println("================================================================================");
        System.out.printf("CENTRO:               %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:         #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);

        // Respuesta diferenciada según la decisión tomada
        if (accesoConcedido) {
            System.out.println("--------------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO AUTORIZADO - TORNO ABIERTO]");
            System.out.printf(">> TOKEN QR EMITIDO:  %s%n", tokenResumenVestibulo);
            System.out.printf(">> TOKEN BARRAS:      %s%n", tokenFormatoBarras);
            System.out.printf(">> PUERTA NORTE LINK: %s%n", tokenResumenPuertaNorte);
        } else {
            System.out.println("--------------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO DENEGADO - TORNO BLOQUEADO]");
            System.out.printf(">> MOTIVO RECHAZO:    %s%n", motivoDenegacion);
            System.out.println(">> PROTOCOLO:         Diríjase a la conserjería central para verificación manual.");
        }

        System.out.println("--------------------------------------------------------------------------------");
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio, nombrePersonaLimpio.length());
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%c'%n", numeroDniSolo, letraFinalDni);
        System.out.printf("PERFIL PROCESADO:     %c (Válido: %b) | SENTIDO: Entrada (%b)%n", perfilNormalizado, perfilPermitido, esEntrada);
        System.out.println("--------------------------------------------------------------------------------");
        System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
        System.out.printf("PERMANENCIA EXACTA:   %03d minutos (Método modular calcularEstancia)%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("--------------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round) | Exacta: %6.2f %%%n", porcentajeOcupacionRedondeado, porcentajeOcupacionReal);
        System.out.printf("AFORO DISPONIBLE:     %d plazas restantes de un total de %d%n", aforoDisponible, aforoTotal);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Método modular calcularDistancia)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor acotado: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("DIAGNÓSTICO TEST:     Patrón #%04d | Sensor NFC: Activo (%b)%n", codigoTestCalibracion, sensorNfcOperativo);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("================================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta segunda sesión, cada estudiante integrará directamente en Java su primera bifurcación condicional en su proyecto propio.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). BIFURCACIÓN ESENCIAL EN PROYECTOS SELECCIONADOS

Abre el archivo `pr/src/MiProyecto.java` e implementa una estructura `if-else` que bifurque la lógica de negocio y controle las mutaciones de estado:

- **Aventura conversacional.** Verifica los puntos de vida del protagonista tras recibir el impacto del adversario. Si `vidaActual <= 0`, ejecuta la bifurcación de derrota imprimiendo la secuencia de fin de partida y bloqueando nuevas acciones; si la vida es superior a cero, continúa la aventura mostrando las opciones tácticas del siguiente asalto.
- **Motor de recomendación.** Evalúa el porcentaje de compatibilidad o afinidad calculado entre el usuario y el ítem. Si la afinidad supera el umbral del 75% (`afinidad >= 75.0`), cataloga el elemento en la rama positiva como *«Altamente recomendado»* y genera un cupón promocional; en caso contrario, clasifícalo en la rama alternativa como *«Recomendación estándar»*.
- **Simulador de físicas 2D.** Evalúa la posición proyectada del objeto respecto a los límites espaciales del área de simulación. Si la coordenada horizontal excede el margen (`posicionX >= LIMITE_DERECHO`), detecta la colisión perimetral, invierte el signo del vector de velocidad (`velocidadX = -velocidadX`) y notifica el rebote; si permanece dentro del espacio, continúa el desplazamiento inercial sin alteración.
- **Bóveda de contraseñas.** Comprueba si la clave candidata cumple con el umbral mínimo de seguridad reglamentario (`password.length() >= 8`). Si no alcanza dicho tamaño, deniega el registro en la bóveda emitiendo una alerta por baja entropía; si satisface la longitud, confirma el guardado seguro y genera su identificador en el almacén.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). LA TRAMPA DEL ÁMBITO DE BLOQUE (*BLOCK SCOPE*) Y EL OPERADOR TERNARIO

1. **Auditoría de ámbito de variables:** Analiza y subsana el siguiente defecto recurrente detectado en las revisiones de código de AzaharTech:

```
if (tempVestibulo > 37.5) {
    String alerta = "¡Fiebre detectada!";
    int codigoIncidencia = 503;
} else {
    String alerta = "Temperatura nominal";
}
// ¿Por qué la siguiente instrucción produce un error de compilación?
System.out.println(alerta); // Error: cannot find symbol variable alerta
```

Corrección requerida: Eleva la declaración de `alerta` al ámbito inmediatamente superior previo a la estructura condicional.

2. **Compactación mediante operador ternario (`?:`):** El operador ternario permite condensar asignaciones binarias en una única instrucción limpia:

```
String estadoTorno = accesoConcedido ? "TORNO DESBLOQUEADO" : "TORNO BLOQUEADO";
```

Refactoriza una asignación simple de tu proyecto aplicando el operador ternario sin que empeore la legibilidad del código.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). RADIOGRAFÍA DE BYTECODE EN LA JVM: SALTOS CONDICIONALES (`if_icmpge`, `goto`)

Abre la terminal integrada de IntelliJ en la raíz del proyecto y analiza el bytecode compilado de `ControlAccesoQR.class` con el desensamblador oficial de Java:

```
javap -c pr/src/ControlAccesoQR.class
```

Localiza la sección correspondiente a la evaluación de la variable `aforoDisponible` o `tempVestibulo`. Observarás instrucciones mnemónicas similares a las siguientes:

```
  42: if_icmpge     55    // Si el valor en la cima de la pila es >= salta al offset 55 (rama else)
  45: iconst_1            // Carga booleano true
  ...
  52: goto          63    // Salto incondicional: evita ejecutar la rama else
  55: iconst_0            // Offset 55: Rama else (carga booleano false)
  ...
  63: return              // Flujo convergente
```

Reto de ingeniería. Documenta mediante un comentario técnico en la cabecera de tu clase por qué la instrucción de bajo nivel `goto` es imprescindible al término del bloque `if` compilado para preservar el flujo de ejecución, y describe cómo el mecanismo

de predicción de saltos (*Branch Prediction*) del procesador evita penalizaciones de rendimiento en la cola de instrucciones cuando se evalúan condicionales repetitivos.

4.1.2 Día 25 - 2 sesiones

---

1. Caso guía en AzaharTech

Es la mañana del **martes 27 de octubre de 2026** en las oficinas de **AzaharTech**. Tras la puesta en marcha de la versión `v2.1` con bifurcación condicional `if-else`, la dirección del **IES El Caminàs** ha remitido una nueva especificación de requisitos para el control de accesos.

Alba Torres reúne a **Pau Ferrer** frente al terminal de desarrollo:

«Pau, ayer conseguimos que el torno no se abriera a ciegas: si la temperatura corporal no es apta o el centro está lleno, el sistema bloquea el paso. Sin embargo, nuestro tratamiento de los colectivos que conviven en el centro sigue siendo excesivamente plano.

En la versión `v2.1` nos limitamos a verificar si el carácter pertenecía a una lista básica:

```
boolean perfilPermitido = esLetraValida && (perfilNormalizado == 'E' || perfilNormalizado == 'D' || perfilNormalizado == 'V');
```

*En la realidad cotidiana del instituto coexisten cuatro perfiles diferenciados con **normativas de acceso y permanencia específicas**:*

1. **Perfil 'E' (Estudiante).** Debe asignarse a "*Aulas y Talleres*", con una estancia lectiva máxima de 360 minutos y control de puntualidad: si accede después de las 08:15 h, el terminal debe registrar una advertencia de "*RETRASO LECTIVO*".*
2. **Perfil 'D' (Docente).** Se asigna a "*Departamentos y Claustro*", con una jornada base de 480 minutos y sin restricción de retraso matinal.*
3. **Perfil 'V' (Visita externa).** Solo puede acceder a "*Secretaría y Vestíbulo*", con un límite de permanencia de 120 minutos.*
4. **Perfil 'A' (Administración y Servicios - PAS).** Personal administrativo, asignado al "*Edificio de Gestión*" con una jornada de 420 minutos.*

*Si intentamos resolver estas asignaciones encadenando múltiples `if-else` `if-else`, generaremos un código repetitivo y difícil de mantener. Hoy introducimos la **sentencia de selección múltiple** `switch` **clásica** en Java, comprenderemos el papel de la cláusula `break`, aprenderemos a agrupar casos mediante caída controlada (*fall-through*) y encapsularemos estas decisiones en métodos estáticos modulares».*

---

2. Fundamento teórico: la sentencia de selección múltiple `switch` en Java

Cuando el flujo del programa debe ramificarse en función de valores discretos exactos de una variable primitiva o inmutable (`char`, `byte`, `short`, `int`, `String` o tipos enumerados), la estructura estándar en programación estructurada es la **sentencia `switch`**.

```
flowchart TD
    Selector["Selector discreto: <b>perfilNormalizado (char)</b>"] --> Switch{"switch (perfil)"}

    Switch -- "case 'E'" --> Estudiante["<b>Estudiante Oficial</b><br/>• Zona: Aulas y Talleres<br/>• Estancia: 360 min max"]
    Switch -- "case 'D'" --> Docente["<b>Docente / Claustro</b><br/>• Zona: Departamentos<br/>• Estancia: 480 min max"]
    Switch -- "case 'V'" --> Visita["<b>Visita Externa</b><br/>• Zona: Secretaría<br/>• Estancia: 120 min max"]
    Switch -- "case 'A'" --> PAS["<b>Personal P.A.S.</b><br/>• Zona: Edificio Gestión<br/>• Estancia: 420 min max"]
    Switch -- "default" --> Desconocido["<b>Perfil No Catalogado</b><br/>• Acceso Denegado<br/>• Estancia: 0 min"]

    Estudiante --> Break([break; -> Salida del switch])
    Docente --> Break
    Visita --> Break
    PAS --> Break
    Desconocido --> Break
```

A. COMPONENTES DE LA SENTENCIA `switch` CLÁSICA

1. **Expresión selectora.** Variable evaluada entre paréntesis tras la palabra reservada `switch`. Debe ser de un tipo discreto compatible (en nuestro caso, el carácter `perfilNormalizado`).
2. **Etiquetas `case`.** Literales constantes contra los que se compara el selector. Si coincide el valor, la ejecución salta a ese punto.
3. **Sentencia de salto `break`.** Fuerza la salida inmediata del bloque `switch`. Si se omite, la ejecución continúa secuencialmente hacia las instrucciones del siguiente caso (*fall-through*), lo que suele constituir una fuente grave de errores si no se planifica de forma consciente.
4. **Cláusula `default`.** Bloque opcional que se ejecuta cuando el selector no coincide con ninguna de las etiquetas `case` definidas. Actúa como red de seguridad (*fallback*).

B. EL FENÓMENO DEL *FALL-THROUGH* Y LA AGRUPACIÓN DE CASOS

Aunque el olvido involuntario de un `break` es un fallo habitual, la caída secuencial entre etiquetas puede emplearse de forma intencionada para aplicar una misma consecuencia lógica a múltiples entradas:

```
// Agrupación válida: tanto 'E', 'D', 'V' como 'A' activan la misma rama
boolean perfilReconocido;
switch (perfilNormalizado) {
    case 'E':
    case 'D':
    case 'V':
    case 'A':
        perfilReconocido = true;
        break;
    default:
        perfilReconocido = false;
        break;
}
```

C. EVALUACIÓN DE MARCAS TEMPORALES CON `LocalTime.isAfter()`

Para controlar la puntualidad del alumnado, combinamos condicionales con métodos de la API `java.time`:

- `tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD)`. Devuelve `true` si la hora de entrada es estrictamente posterior a las 08:15 h.
- Esta comprobación se acopla a la regla de perfil: únicamente generará aviso de retraso si el usuario es un estudiante (`perfilNormalizado == 'E'`).

---

3. Evolución del caso guía a `ControlAccesoQR v2.2`

En esta versión incorporamos la selección múltiple por perfil delegando cada asignación en métodos estáticos modulares, añadimos la comprobación de puntualidad con `isAfter()` y actualizamos el informe de consola.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.1 (`diff`)

```
   // ========================================================================
   // CONSTANTES DEL SISTEMA
   // ========================================================================
+ final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

   // ========================================================================
   // MÉTODOS ESTÁTICOS PROPIOS (Modularización con switch clásico)
   // ========================================================================
+ public static String obtenerDescripcionPerfil(char perfil) { ... }
+ public static String determinarZonaAutorizada(char perfil) { ... }
+ public static int obtenerEstanciaMaximaPorPerfil(char perfil) { ... }

   // ========================================================================
   // MÉTODO MAIN: Toma de decisiones
   // ========================================================================
- boolean perfilPermitido = esLetraValida && (perfilNormalizado == 'E' || perfilNormalizado == 'D' || perfilNormalizado == 'V');
+ boolean perfilReconocido;
+ switch (perfilNormalizado) {
+     case 'E':
+     case 'D':
+     case 'V':
+     case 'A':
+         perfilReconocido = true;
+         break;
+     default:
+         perfilReconocido = false;
+         break;
+ }
+
```

```
+ String descripcionRol    = obtenerDescripcionPerfil(perfilNormalizado);
+ String zonaDesignada     = determinarZonaAutorizada(perfilNormalizado);
+ int estanciaMaxPermitida = obtenerEstanciaMaximaPorPerfil(perfilNormalizado);
+ boolean esRetrasoLectivo = (perfilNormalizado == 'E') && tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD);
```

B. FOCO EN LA NOVEDAD: MÉTODOS MODULARES Y BIFURCACIÓN CON `switch`

Observa cómo aislamos la lógica de cada atributo del perfil en métodos independientes y cómo controlamos la puntualidad:

```
// ------------------------------------------------------------------------
// Métodos estáticos modulares gobernados por switch clásico
// ------------------------------------------------------------------------
public static String obtenerDescripcionPerfil(char perfil) {
    String descripcion;
    switch (perfil) {
        case 'E':
            descripcion = "Estudiante Oficial";
            break;
        case 'D':
            descripcion = "Docente / Claustro";
            break;
        case 'V':
            descripcion = "Visita Externa / Proveedor";
            break;
        case 'A':
            descripcion = "Personal de Administración y Servicios (PAS)";
            break;
        default:
            descripcion = "Perfil No Catalogado";
            break;
    }
    return descripcion;
}

public static String determinarZonaAutorizada(char perfil) {
    String zona;
    switch (perfil) {
        case 'E':
            zona = "Aulas Docentes y Talleres de Informática";
            break;
        case 'D':
            zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
            break;
        case 'V':
            zona = "Secretaría, Conserjería y Despachos de Dirección";
            break;
        case 'A':
            zona = "Área de Gestión Administrativa y Archivo Central";
            break;
        default:
            zona = "Acceso Denegado / Zona No Autorizada";
            break;
    }
    return zona;
}

public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
    int estanciaMinutos;
    switch (perfil) {
        case 'E':
            estanciaMinutos = 360; // 6 horas lectivas
            break;
        case 'D':
            estanciaMinutos = 480; // 8 horas jornada docente
            break;
        case 'V':
            estanciaMinutos = 120; // 2 horas límite visitas
            break;
        case 'A':
            estanciaMinutos = 420; // 7 horas jornada administrativa
            break;
        default:
            estanciaMinutos = 0;
            break;
    }
    return estanciaMinutos;
}

// ------------------------------------------------------------------------
// Dentro de main: Invocación e informe condicional de puntualidad
// ------------------------------------------------------------------------
// [Tras validar y autorizar el acceso...]
System.out.printf("ROL CLASIFICADO:      [%c] %-25s | Válido: %b%n", perfilNormalizado, descripcionRol, perfilReconocido);
System.out.printf("ZONA ASIGNADA:        %s%n", zonaDesignada);
System.out.printf("LÍMITE AUTORIZADO:    %d minutos (Tiempo previsto: %d min)%n", estanciaMaxPermitida, minutosEstanciaTotal);

if (esRetrasoLectivo && accesoConcedido) {
    System.out.println(">> AVISO PUNTUALIDAD: [RETRASO LECTIVO REGISTRADO - Entrada posterior a las 08:15]");
}
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.2)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.2:
 * Incorpora estructuras de selección múltiple mediante sentencias switch clásicas.
 * Despliega lógica diferenciada por rol (Estudiante, Docente, Visita, PAS).
 * Control de puntualidad y asignación dinámica de zonas y estancias máximas.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.2 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales de seguridad biométrica y capacidad (Sprint 3)
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control horario de puntualidad lectiva para estudiantes (Día 25)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    /**
     * Calcula la estancia neta en minutos entre dos marcas temporales usando Duration.
     */
    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    /**
     * Calcula el porcentaje de ocupación con precisión decimal aplicando casting explícito.
     */
    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    /**
     * Construye y concatena el token QR oficial parametrizado para cualquier monitor.
     */
    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    /**
     * Calcula la distancia euclídea (hipotenusa) a la pantalla mediante el teorema de Pitágoras.
     */
    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    /**
     * Retorna la denominación oficial del perfil mediante una sentencia switch clásica con break.
     */
    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
```

```
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    /**
     * Asigna la zona física autorizada del campus mediante una sentencia switch clásica.
     */
    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    /**
     * Determina el límite de minutos de permanencia permitidos según el perfil con switch clásico.
     */
    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360; // 6 horas lectivas
                break;
            case 'D':
                estanciaMinutos = 480; // 8 horas jornada docente
                break;
            case 'V':
                estanciaMinutos = 120; // 2 horas límite visitas
                break;
            case 'A':
                estanciaMinutos = 420; // 7 horas jornada administrativa
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODO PRINCIPAL: ORQUESTADOR CON IF-ELSE Y SELECCIÓN MÚLTIPLE SWITCH
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Generadores independientes en el Heap
        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();
        LocalTime horaEmisionQr = LocalTime.now();

        int terminalId;
        double tempVestibulo;

        String nombrePersonaBruto, nombrePersonaLimpio;
        String dniPersonaBruto, dniPersonaLimpio;
        char perfilPersonaBruto;
        char perfilNormalizado;
        boolean esLetraValida;
        boolean esEntrada = true;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;
```

```
        int segundosActividadTerminal;
        double coordenadaXMetros, coordenadaYMetros;

        // Captura de datos interactiva
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 2.2 Oficial (Sprint 3: Switch & Perfiles)");
        System.out.println("================================================");
        System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
        terminalId = Integer.parseInt(teclado.nextLine());

        System.out.print("Temperatura base del sensor (ºC): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Distancia horizontal a pantalla en metros (eje X): ");
        coordenadaXMetros = teclado.nextDouble();

        System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
        coordenadaYMetros = teclado.nextDouble();

        teclado.nextLine(); // Limpieza del buffer de entrada

        System.out.print("DNI de la persona: ");
        dniPersonaBruto = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersonaBruto = teclado.nextLine();

        // Normalización y limpieza con métodos de String y Character
        dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
        nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

        System.out.print("Perfil de acceso (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
        perfilPersonaBruto = teclado.next().charAt(0);
        perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
        esLetraValida = Character.isLetter(perfilNormalizado);

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();
        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();
        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        // Operaciones temporales
        LocalTime tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
        LocalTime tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);
        int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

        // Calibración ambiental y diagnóstico
        double tempCalibrada = tempVestibulo + (generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA);
        boolean sensorNfcOperativo = generadorCalibracion.nextBoolean();
        int codigoTestCalibracion  = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);

        int aforoTotal = personasEnCentro + aforoDisponible;

        // Desglose del documento
        boolean esLongitudDniCorrecta = (dniPersonaLimpio.length() == LONGITUD_DNI_ESTANDAR);
        String numeroDniSolo = dniPersonaLimpio.substring(0, Math.max(0, dniPersonaLimpio.length() - 1));
        char letraFinalDni   = dniPersonaLimpio.charAt(Math.max(0, dniPersonaLimpio.length() - 1));

        // ---------------------------------------------------------------------
        // 4. CLASIFICACIÓN CON SENTENCIA SWITCH CLÁSICA Y VALIDACIÓN (DÍA 25)
        // ---------------------------------------------------------------------
        // Verificación de reconocimiento del perfil mediante agrupación con fall-through
        boolean perfilReconocido;
        switch (perfilNormalizado) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                perfilReconocido = true;
                break;
            default:
                perfilReconocido = false;
                break;
        }

        String descripcionRol    = obtenerDescripcionPerfil(perfilNormalizado);
        String zonaDesignada     = determinarZonaAutorizada(perfilNormalizado);
        int estanciaMaxPermitida = obtenerEstanciaMaximaPorPerfil(perfilNormalizado);

        // Control de puntualidad específico para estudiantes
        boolean esRetrasoLectivo = (perfilNormalizado == 'E') && tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD);

        // Variables de decisión de acceso
        boolean accesoConcedido;
        String motivoDenegacion = "";

        // Reglas de negocio consolidadas
        boolean temperaturaApta = (tempVestibulo >= TEMP_CORPORAL_MIN_FALLO && tempVestibulo <= TEMP_CORPORAL_MAX_FIEBRE);
        boolean aforoSuficiente = (aforoDisponible > 0);

        if (!temperaturaApta) {
```

```
            accesoConcedido = false;
            motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
                    tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
        } else if (!aforoSuficiente) {
            accesoConcedido = false;
            motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
        } else if (!esLongitudDniCorrecta) {
            accesoConcedido = false;
            motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
        } else if (!perfilReconocido) {
            accesoConcedido = false;
            motivoDenegacion = "Perfil '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
        } else {
            accesoConcedido = true;
            // Mutación de estado condicionada
            personasEnCentro++;
            aforoDisponible--;
            idUltimoFichaje++;
        }

        // Generación de códigos de seguridad condicionados
        int codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        // Construcción de tokens mediante método modular propio
        String tokenResumenVestibulo   = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);
        String tokenResumenPuertaNorte = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, ID_TERMINAL_PUERTA_NORTE, codigoTokenPuertaNorte, idUltimoFichaje);
        String tokenFormatoBarras      = tokenResumenVestibulo.replace("-", "/");

        // Cálculos métricos auxiliares
        int aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);
        double porcentajeOcupacionReal = calcularPorcentajeOcupacion(aforoSeguroVisualizado, aforoTotal);
        int porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        double distanciaDiagonalAntena = calcularDistanciaDiagonal(coordenadaXMetros, coordenadaYMetros);
        int desviacionJornadaMinutos   = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        double tempSeguraAcotada       = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        // Descomposición temporal de servicio
        int horasUptime    = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        int minutosUptime  = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        int segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA CON TRAZA DE PERFIL (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:         #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);

        if (accesoConcedido) {
            System.out.println("------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO AUTORIZADO - TORNO ABIERTO]");
            System.out.printf(">> TOKEN QR EMITIDO:  %s%n", tokenResumenVestibulo);
            System.out.printf(">> TOKEN BARRAS:      %s%n", tokenFormatoBarras);
            System.out.printf(">> PUERTA NORTE LINK: %s%n", tokenResumenPuertaNorte);
        } else {
            System.out.println("------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO DENEGADO - TORNO BLOQUEADO]");
            System.out.printf(">> MOTIVO RECHAZO:    %s%n", motivoDenegacion);
            System.out.println(">> PROTOCOLO:         Diríjase a la conserjería central para verificación manual.");
        }

        System.out.println("------------------------------------------------------------------------");
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio, nombrePersonaLimpio.length());
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%c'%n", numeroDniSolo, letraFinalDni);
        System.out.printf("ROL CLASIFICADO:      [%c] %-25s | Válido: %b%n", perfilNormalizado, descripcionRol, perfilReconocido);
        System.out.printf("ZONA ASIGNADA:        %s%n", zonaDesignada);
        System.out.printf("LÍMITE AUTORIZADO:    %d minutos (Tiempo previsto: %d min)%n", estanciaMaxPermitida, minutosEstanciaTotal);

        // Advertencia condicional de puntualidad (si aplica)
        if (esRetrasoLectivo && accesoConcedido) {
            System.out.println(">> AVISO PUNTUALIDAD: [RETRASO LECTIVO REGISTRADO - Entrada posterior a las 08:15]");
        }

        System.out.println("------------------------------------------------------------------------");
        System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
        System.out.printf("PERMANENCIA EXACTA:   %03d minutos (Método modular calcularEstancia)%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round) | Exacta: %6.2f %%%n", porcentajeOcupacionRedondeado, porcentajeOcupacionReal);
        System.out.printf("AFORO DISPONIBLE:     %d plazas restantes de un total de %d%n", aforoDisponible, aforoTotal);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Método modular calcularDistancia)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor acotado: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("DIAGNÓSTICO TEST:     Patrón #%04d | Sensor NFC: Activo (%b)%n", codigoTestCalibracion, sensorNfcOperativo);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión práctica, cada alumno desarrollará directamente en Java estructuras de selección múltiple sobre el archivo maestro de su especialidad técnica.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). SELECCIÓN MÚLTIPLE CLÁSICA CON `switch` Y `break`

Abre tu archivo `pr/src/MiProyecto.java` e implementa una sentencia `switch` tradicional que controle la toma de decisiones en función de una variable discreta, asegurando la inclusión de `break` en cada caso y una cláusula `default`:

- **Aventura conversacional.** Evalúa mediante un `switch` clásico el carácter del comando táctico introducido por el jugador (`'A'`, `'D'`, `'H'`, `'U'`). Cada caso debe actualizar las variables de combate (modificador de ataque, defensa, probabilidad de huida o puntos de curación mediante poción) y finalizar con su correspondiente sentencia `break`, contemplando una cláusula `default` para comandos desconocidos que penalice el turno del jugador.
- **Motor de recomendación.** Implementa un `switch` clásico sobre el carácter identificador de la categoría de contenido (`'C'` para Cine, `'M'` para Música, `'V'` para Videojuegos, `'L'` para Literatura). En cada caso, asigna el umbral mínimo de afinidad porcentual exigido y una cadena con la etiqueta descriptiva del sector, incluyendo una cláusula `default` de seguridad para categorías no catalogadas.
- **Simulador de físicas 2D.** Estructura un `switch` clásico que discrimine según el carácter representativo del material del cuerpo impactado (`'M'` para Metal, `'G'` para Goma, `'B'` para Barro, `'H'` para Hielo). En cada rama, asigna el coeficiente de restitución elástica específico (entre 0.05 y 0.98) que multiplicará el vector de velocidad para calcular la pérdida de energía en el rebote, cerrando con un `default` que asigne un rebote genérico.
- **Bóveda de contraseñas.** Utiliza un `switch` clásico para evaluar el nivel de clasificación de seguridad del secreto (`'B'` para Básico, `'C'` para Confidencial, `'S'` para Secreto, `'T'` para Top Secret). Cada caso debe fijar la longitud mínima obligatoria de la clave (8, 12, 16 o 24 caracteres) y el periodo de caducidad en días (365, 90, 30 o 7 días), previendo una rama `default` que rechace niveles no homologados.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). TRANSICIÓN HACIA EL ESTÁNDAR MODERNO: LAS `switch expression` DE JAVA 21

La sintaxis clásica de `switch` arrastra desde los años 70 el riesgo de olvidar un `break`, provocando caídas involuntarias de flujo. Para resolverlo definitivamente, Java incorporó en sus versiones LTS modernas las **`switch expression`**.

1. **Sintaxis de flecha (`->`) y asignación directa.** La expresión `switch` no necesita `break` y puede devolver un valor que se asigna directamente a una variable:

```
// Comparativa: de 15 líneas a una expresión concisa y segura
String descripcion = switch (perfil) {
    case 'E' -> "Estudiante Oficial";
    case 'D' -> "Docente / Claustro";
    case 'V' -> "Visita Externa / Proveedor";
    case 'A' -> "Personal de Administración y Servicios (PAS)";
    default  -> "Perfil No Catalogado";
};
```

2. **Bloques multilínea con `yield`.** Si una rama de la flecha requiere ejecutar varias operaciones antes de producir su resultado, se encierra entre llaves `{ }` y se utiliza la palabra reservada `yield`:

```
int estanciaMinutos = switch (perfil) {
    case 'V' -> {
        System.out.println(">> Aviso de seguridad: activando protocolo de acreditación temporal.");
        yield 120;
    }
    case 'E' -> 360;
    case 'D' -> 480;
    case 'A' -> 420;
    default  -> 0;
};
```

3. **Reto de refactorización.** Toma la sentencia `switch` clásica que implementaste en la Kata 1 para tu proyecto (**Aventura conversacional**, **Motor de recomendación**, **Simulador de físicas 2D** o **Bóveda de contraseñas**) y refactorízala en un método estático independiente utilizando la sintaxis moderna de `switch expression` con flecha (`->`) y al menos una rama gobernada por `yield`.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). RADIOGRAFÍA DE BYTECODE EN LA JVM: `tableswitch` FRENTE A `lookupswitch`

Abre la terminal de comandos de IntelliJ en el directorio del proyecto y compila y desensambla el archivo `ControlAccesoQR.class`:

```
javap -c pr/src/ControlAccesoQR.class
```

Localiza en el volcado las instrucciones que implementan los métodos `obtenerDescripcionPerfil` o la validación del perfil. Comprobarás que la máquina virtual de Java compila los bloques `switch` mediante dos instrucciones bytecode bien diferenciadas:

- **`tableswitch`.** Se genera cuando los valores de los casos son números consecutivos o forman una secuencia muy compacta (por ejemplo, `1, 2, 3, 4`). La JVM construye una tabla de saltos indexada directa donde el acceso a la rama tiene una complejidad algorítmica constante **$O(1)$**.
- **`lookupswitch`.** Se utiliza cuando los valores numéricos de los casos están dispersos o alejados en la tabla ASCII/Unicode (como nuestros caracteres `'A'=65`, `'D'=68`, `'E'=69` y `'V'=86`). La JVM genera una lista ordenada de pares `(clave, offset)` y ejecuta una búsqueda binaria en la tabla con coste **$O(\log n)$**.

Reto técnico. Identifica en el volcado de tu desensamblador cuál de las dos instrucciones bytecode (`tableswitch` o `lookupswitch`) ha seleccionado el compilador `javac` para tu clase y redacta un breve comentario técnico en la cabecera del método justificando la elección en función de la dispersión de los caracteres analizados.

4.1.3 Día 26 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **miércoles 28 de octubre de 2026** en la sede de **AzaharTech**. Durante la sesión matinal de pruebas de la versión `v2.2`, el equipo de conserjería del **IES El Caminàs** estuvo testeando el terminal físico en el vestíbulo principal. En una de las pruebas, un técnico introdujo por error la palabra "*diez*" en lugar del dígito `10` al indicar la hora de entrada; en otra, el sensor térmico envió una cadena vacía al leer la temperatura.

El resultado fue inmediato y desastroso: la consola arrojó una traza roja de excepción (`NumberFormatException` / `InputMismatchException`), la máquina virtual abortó la ejecución y el torno quedó congelado a mitad de jornada sin emitir ningún informe.

Laia Claramunt convoca a **Alba Torres** y **Pau Ferrer**:

«Alba, Pau: la lógica que implementamos ayer con sentencias `switch` clásicas para catalogar a estudiantes, docentes, visitas y personal de administración es impecable, pero tiene un talón de Aquiles:

Las líneas 153 a 179 de la versión `v2.2` ejecutan de forma directa:

```
terminalId = Integer.parseInt(teclado.nextLine());
tempVestibulo = teclado.nextDouble();
...
tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
```

*Nuestro software **asume ciegamente que el usuario y los sensores nunca se equivocan**. En un entorno real, una aplicación profesional jamás puede desplomarse con un volcado de pila (*stack trace*) ante un fallo tipográfico o una lectura corrupta de hardware.*

Si alguien escribe texto en un campo numérico, o si introduce la hora 25 o el minuto 78, la aplicación debe interceptar el fallo en el instante exacto en que ocurre, contener el error, activar una bandera de anomalía y denegar el paso de forma controlada mediante nuestro árbol de decisión.

*Hoy blindamos el código mediante el **control de excepciones básico con `try-catch`**. Aprenderemos a capturar errores de conversión numérica (`NumberFormatException`) y desbordamientos temporales (`DateTimeException`), garantizando que el terminal del instituto continúe operativo bajo cualquier circunstancia».*

---

2. Fundamento teórico: control de excepciones básico (`try-catch`) y programación defensiva

Una **excepción** en Java es un objeto que representa una condición anómala acontecida durante la ejecución. Si el hilo de ejecución no intercepta la excepción, la máquina virtual finaliza abruptamente el método `main`.

```
flowchart TD
   subgraph TryBlock ["Bloque vigilado TRY { }"]
      L1["Integer.parseInt(teclado.nextLine())"]
      L2["Double.parseDouble(teclado.nextLine())"]
      L3["LocalTime.of(hora, minuto)"]
      L1 --> L2 --> L3
   end

   L1 -.->|Lanza NumberFormatException| Catch1["<b>CATCH (NumberFormatException e)</b><br/>datosCorrectos = false<br/>motivo = 'Formato numérico erróneo'"]
   L2 -.->|Lanza NumberFormatException| Catch1
   L3 -.->|Lanza DateTimeException| Catch2["<b>CATCH (DateTimeException e)</b><br/>datosCorrectos = false<br/>motivo = 'Hora fuera de límites'"]

   L3 -->|Sin errores| NormalFlow["Flujo normal: evaluar aforo y reglas"]
   Catch1 --> Convergencia["Flujo de seguridad: Torno Bloqueado"]
   Catch2 --> Convergencia
```

A. ANATOMÍA DEL BLOQUE `try-catch`

1. **Bloque `try`.** Delimita el conjunto de instrucciones vigiladas donde puede originarse una excepción (conversiones numéricas con `Integer.parseInt()` o instanciaciones de `LocalTime.of()`).
2. **Bloque `catch`.** Se ejecuta de forma inmediata si se produce una excepción del tipo vigilado. Permite contener el impacto, registrar el problema y asignar un estado de error sin tumbar el proceso.

B. EXCEPCIONES CRÍTICAS EN LA CAPTURA POR CONSOLA

- **`NumberFormatException`.** Se dispara al intentar convertir un texto alfanumérico no convertible en número entero o decimal.
- **`DateTimeException`.** Se produce si se pasan valores numéricos fuera de los rangos válidos del calendario o reloj (por ejemplo, horas fuera del rango 0–23 o minutos fuera de 0–59).

C. ÁMBITO DE VARIABLES LOCALES (*BLOCK SCOPE*) E INICIALIZACIÓN PREVIA

Cualquier variable declarada dentro del bloque `try` sólo existe dentro de sus llaves. Para poder emplear los datos en los bloques posteriores de decisión (`if-else`) e informe, las variables deben **declararse e inicializarse con valores seguros por defecto antes del `try`**:

```
int terminalId = 0; // Inicialización defensiva previa
boolean datosCorrectos = true;

try {
    terminalId = Integer.parseInt(teclado.nextLine().trim());
} catch (NumberFormatException e) {
    datosCorrectos = false;
    motivoDenegacion = "El identificador del terminal debe ser estrictamente numérico.";
}
```

D. ESTÁNDAR AZAHARTECH: PROHIBICIÓN DEL BLOQUE `catch` VACÍO (*EMPTY CATCH*)

Norma de calidad en AzaharTech: Dejar un bloque `catch` sin instrucciones (`catch (Exception e) {}`) está terminantemente prohibido. Silenciar excepciones oculta fallos de diseño y provoca estados incoherentes en memoria. Todo bloque `catch` debe activar una bandera de fallo, emitir una traza o documentar el motivo del error.

---

3. Evolución del caso guía a `ControlAccesoQR v2.3`

En esta versión envolvemos la captura de datos en un bloque vigilado `try-catch`, inicializamos las variables con valores defensivos por encima del bloque para preservar su ámbito e introducimos la bandera `datosSintacticamenteCorrectos` como primer filtro de decisión.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.2 (`diff`)

```
+ import java.time.DateTimeException;

  // ========================================================================
  // MÉTODO MAIN: Inicialización previa de variables (Block Scope)
  // ========================================================================
+ boolean datosSintacticamenteCorrectos = true;
+ String motivoDenegacion = "";
```

```
  // ========================================================================
  // CAPTURA VIGILADA CON TRY-CATCH
  // ========================================================================
- terminalId = Integer.parseInt(teclado.nextLine());
- tempVestibulo = teclado.nextDouble();
- tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
+ try {
+     terminalId = Integer.parseInt(teclado.nextLine().trim());
+     tempVestibulo = Double.parseDouble(teclado.nextLine().trim());
+     // ... [Lecturas restantes] ...
+     tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
+     tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);
+ } catch (NumberFormatException e) {
+     datosSintacticamenteCorrectos = false;
+     motivoDenegacion = "Error de formato de entrada: se introdujo texto en un campo numérico.";
+ } catch (DateTimeException e) {
+     datosSintacticamenteCorrectos = false;
+     motivoDenegacion = "Error temporal cronológico: hora (0-23) o minuto (0-59) fuera de límites.";
+ }

  // ========================================================================
  // TOMA DE DECISIONES DEFENSIVA
  // ========================================================================
+ if (!datosSintacticamenteCorrectos) {
+     accesoConcedido = false; // Denegación inmediata sin evaluar más reglas
+ } else if (!temperaturaApta) { ...
```

B. FOCO EN LA NOVEDAD: BLOQUE VIGILADO Y CORTOCIRCUITO DE SEGURIDAD

Observa cómo se estructura la captura protegida y cómo la bandera de integridad actúa de escudo en el árbol de decisión:

```
// ------------------------------------------------------------------------
// Inicialización defensiva de variables en el ámbito de main
// ------------------------------------------------------------------------
int terminalId = 0;
double tempVestibulo = 0.0;
int horaEntrada = 0, minutoEntrada = 0;
int horaSalida = 0, minutoSalida = 0;
LocalTime tiempoEntrada = LocalTime.of(8, 0);
LocalTime tiempoSalida  = LocalTime.of(14, 0);

boolean datosSintacticamenteCorrectos = true;
String motivoDenegacion = "";

// ------------------------------------------------------------------------
// Captura vigilada interactiva
// ------------------------------------------------------------------------
try {
    System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
    terminalId = Integer.parseInt(teclado.nextLine().trim());

    System.out.print("Temperatura base del sensor (ºC): ");
    tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

    // ... [Lecturas de uptime, coordenadas, DNI, nombre y perfil] ...

    System.out.print("Hora de entrada (0-23): ");
    horaEntrada = Integer.parseInt(teclado.nextLine().trim());
    System.out.print("Minuto de entrada (0-59): ");
    minutoEntrada = Integer.parseInt(teclado.nextLine().trim());

    System.out.print("Hora de salida (0-23): ");
    horaSalida = Integer.parseInt(teclado.nextLine().trim());
    System.out.print("Minuto de salida (0-59): ");
    minutoSalida = Integer.parseInt(teclado.nextLine().trim());

    // Construcción cronológica vigilada
    tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
    tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

} catch (NumberFormatException e) {
    datosSintacticamenteCorrectos = false;
    motivoDenegacion = "Error de formato de entrada: se introdujo un valor alfanumérico en un campo numérico.";
} catch (DateTimeException e) {
    datosSintacticamenteCorrectos = false;
    motivoDenegacion = "Error temporal cronológico: hora (0-23) o minuto (0-59) fuera de límites válidos.";
}

// ------------------------------------------------------------------------
// Árbol de decisión defensivo
// ------------------------------------------------------------------------
boolean accesoConcedido;

if (!datosSintacticamenteCorrectos) {
    // Si la captura arrojó excepciones, se aborta la autorización de inmediato
    accesoConcedido = false;
} else if (!temperaturaApta) {
    accesoConcedido = false;
    motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
            tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
} else if (!aforoSuficiente) {
```

```
    accesoConcedido = false;
    motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
} else if (!esLongitudDniCorrecta) {
    accesoConcedido = false;
    motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
} else if (!perfilReconocido) {
    accesoConcedido = false;
    motivoDenegacion = "Perfil '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
} else {
    accesoConcedido = true;
    personasEnCentro++;
    aforoDisponible--;
    idUltimoFichaje++;
}
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.3)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.3:
 * Incorpora gestión básica de excepciones con bloques try-catch (Sprint 3: «El laberinto de la lógica»).
 * Protege el flujo ante NumberFormatException y DateTimeException en la captura interactiva.
 * Integra la bandera de integridad de datos en el árbol de decisión para evitar caídas del terminal.
 * Mantiene la lógica de perfiles modularizada mediante sentencias switch clásicas.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.3 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales de seguridad biométrica y capacidad (Sprint 3)
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control horario de puntualidad lectiva para estudiantes (Día 25)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    /**
     * Calcula la estancia neta en minutos entre dos marcas temporales usando Duration.
     */
    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    /**
     * Calcula el porcentaje de ocupación con precisión decimal aplicando casting explícito.
     */
    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    /**
     * Construye y concatena el token QR oficial parametrizado para cualquier monitor.
     */
    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    /**
     * Calcula la distancia euclídea (hipotenusa) a la pantalla mediante el teorema de Pitágoras.
     */
    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    /**
     * Retorna la denominación oficial del perfil mediante una sentencia switch clásica con break.
     */
    public static String obtenerDescripcionPerfil(char perfil) {
```

```
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    /**
     * Asigna la zona física autorizada del campus mediante una sentencia switch clásica.
     */
    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    /**
     * Determina el límite de minutos de permanencia permitidos según el perfil con switch clásico.
     */
    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360; // 6 horas lectivas
                break;
            case 'D':
                estanciaMinutos = 480; // 8 horas jornada docente
                break;
            case 'V':
                estanciaMinutos = 120; // 2 horas límite visitas
                break;
            case 'A':
                estanciaMinutos = 420; // 7 horas jornada administrativa
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODO PRINCIPAL: CAPTURA PROTEGIDA CON TRY-CATCH Y TOMA DE DECISIONES
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Generadores independientes en el Heap
        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();
        LocalTime horaEmisionQr = LocalTime.now();

        // Inicialización de variables con valores seguros de contingencia
        int terminalId = 0;
        double tempVestibulo = 0.0;
        int segundosActividadTerminal = 0;
        double coordenadaXMetros = 0.0;
        double coordenadaYMetros = 0.0;

        String nombrePersonaBruto = "", nombrePersonaLimpio = "";
        String dniPersonaBruto = "", dniPersonaLimpio = "";
        char perfilPersonaBruto = ' ';
        char perfilNormalizado = ' ';
        boolean esLetraValida = false;
        boolean esEntrada = true;

        int horaEntrada = 0, minutoEntrada = 0;
```

```
        int horaSalida = 0, minutoSalida = 0;

        LocalTime tiempoEntrada = LocalTime.of(8, 0);
        LocalTime tiempoSalida  = LocalTime.of(14, 0);

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        // Bandera de integridad técnica para el bloque try-catch (Día 26)
        boolean datosSintacticamenteCorrectos = true;
        String motivoDenegacion = "";

        System.out.println("================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 2.3 Oficial (Sprint 3: Try-Catch Shield) ");
        System.out.println("================================================");

        // ------------------------------------------------------------------
        // BLOQUE VIGILADO: CAPTURA DE DATOS INTERACTIVA CON TRY-CATCH
        // ------------------------------------------------------------------
        try {
            System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
            terminalId = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Temperatura base del sensor (ºC): ");
            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

            System.out.print("Segundos de actividad del terminal (uptime): ");
            segundosActividadTerminal = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Distancia horizontal a pantalla en metros (eje X): ");
            coordenadaXMetros = Double.parseDouble(teclado.nextLine().trim());

            System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
            coordenadaYMetros = Double.parseDouble(teclado.nextLine().trim());

            System.out.print("DNI de la persona: ");
            dniPersonaBruto = teclado.nextLine();

            System.out.print("Nombre completo: ");
            nombrePersonaBruto = teclado.nextLine();

            // Normalización de texto
            dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
            nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

            System.out.print("Perfil de acceso (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
            String lineaPerfil = teclado.nextLine().trim();
            if (lineaPerfil.isEmpty()) {
                perfilPersonaBruto = '?';
            } else {
                perfilPersonaBruto = lineaPerfil.charAt(0);
            }
            perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
            esLetraValida = Character.isLetter(perfilNormalizado);

            System.out.print("Hora de entrada (0-23): ");
            horaEntrada = Integer.parseInt(teclado.nextLine().trim());
            System.out.print("Minuto de entrada (0-59): ");
            minutoEntrada = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Hora de salida (0-23): ");
            horaSalida = Integer.parseInt(teclado.nextLine().trim());
            System.out.print("Minuto de salida (0-59): ");
            minutoSalida = Integer.parseInt(teclado.nextLine().trim());

            // Construcción cronológica vigilada
            tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
            tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

        } catch (NumberFormatException e) {
            datosSintacticamenteCorrectos = false;
            motivoDenegacion = "Error de formato de entrada: se introdujo un valor alfanumérico en un campo numérico.";
        } catch (DateTimeException e) {
            datosSintacticamenteCorrectos = false;
            motivoDenegacion = "Error temporal cronológico: hora (0-23) o minuto (0-59) fuera de límites válidos.";
        }

        // Operaciones de permanencia
        int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

        // Calibración ambiental y diagnóstico
        double tempCalibrada = tempVestibulo + (generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA);
        boolean sensorNfcOperativo = generadorCalibracion.nextBoolean();
        int codigoTestCalibracion  = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);

        int aforoTotal = personasEnCentro + aforoDisponible;

        // Desglose del documento
        boolean esLongitudDniCorrecta = (dniPersonaLimpio.length() == LONGITUD_DNI_ESTANDAR);
        String numeroDniSolo = dniPersonaLimpio.isEmpty() ? "0" : dniPersonaLimpio.substring(0, Math.max(0, dniPersonaLimpio.length() - 1));
        char letraFinalDni   = dniPersonaLimpio.isEmpty() ? '?' : dniPersonaLimpio.charAt(Math.max(0, dniPersonaLimpio.length() - 1));

        // Clasificación con switch clásico
        boolean perfilReconocido;
        switch (perfilNormalizado) {
            case 'E':
            case 'D':
            case 'V':
```

```
            case 'A':
                perfilReconocido = true;
                break;
            default:
                perfilReconocido = false;
                break;
        }

        String descripcionRol    = obtenerDescripcionPerfil(perfilNormalizado);
        String zonaDesignada     = determinarZonaAutorizada(perfilNormalizado);
        int estanciaMaxPermitida = obtenerEstanciaMaximaPorPerfil(perfilNormalizado);

        // Control de puntualidad específico para estudiantes
        boolean esRetrasoLectivo = (perfilNormalizado == 'E') && tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD);

        // ------------------------------------------------------------------
        // 4. BIFURCACIÓN CONDICIONAL DEFENSIVA (DÍA 26)
        // ------------------------------------------------------------------
        boolean accesoConcedido;

        // Reglas de negocio
        boolean temperaturaApta = (tempVestibulo >= TEMP_CORPORAL_MIN_FALLO && tempVestibulo <= TEMP_CORPORAL_MAX_FIEBRE);
        boolean aforoSuficiente = (aforoDisponible > 0);

        if (!datosSintacticamenteCorrectos) {
            // Primer filtro: si falló el try-catch, se deniega sin evaluar nada más
            accesoConcedido = false;
        } else if (!temperaturaApta) {
            accesoConcedido = false;
            motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
                    tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
        } else if (!aforoSuficiente) {
            accesoConcedido = false;
            motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
        } else if (!esLongitudDniCorrecta) {
            accesoConcedido = false;
            motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
        } else if (!perfilReconocido) {
            accesoConcedido = false;
            motivoDenegacion = "Perfil '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
        } else {
            accesoConcedido = true;
            personasEnCentro++;
            aforoDisponible--;
            idUltimoFichaje++;
        }

        // Generación de códigos de seguridad
        int codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        // Construcción de tokens mediante métodos modulares
        String tokenResumenVestibulo   = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);
        String tokenResumenPuertaNorte = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, ID_TERMINAL_PUERTA_NORTE, codigoTokenPuertaNorte, idUltimoFichaje);
        String tokenFormatoBarras      = tokenResumenVestibulo.replace("-", "/");

        // Cálculos métricos auxiliares
        int aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);
        double porcentajeOcupacionReal = calcularPorcentajeOcupacion(aforoSeguroVisualizado, aforoTotal);
        int porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        double distanciaDiagonalAntena = calcularDistanciaDiagonal(coordenadaXMetros, coordenadaYMetros);
        int desviacionJornadaMinutos   = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        double tempSeguraAcotada       = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        // Descomposición temporal de servicio
        int horasUptime    = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        int minutosUptime  = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        int segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // ------------------------------------------------------------------
        // 5. SALIDA FORMATEADA CON CONTROL DE INCIDENCIAS (System.out.printf)
        // ------------------------------------------------------------------
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:               %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:         #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);

        if (accesoConcedido) {
            System.out.println("------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO AUTORIZADO - TORNO ABIERTO]");
            System.out.printf(">> TOKEN QR EMITIDO:  %s%n", tokenResumenVestibulo);
            System.out.printf(">> TOKEN BARRAS:      %s%n", tokenFormatoBarras);
            System.out.printf(">> PUERTA NORTE LINK: %s%n", tokenResumenPuertaNorte);
        } else {
            System.out.println("------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO DENEGADO - TORNO BLOQUEADO]");
            System.out.printf(">> MOTIVO RECHAZO:    %s%n", motivoDenegacion);
            System.out.println(">> PROTOCOLO:         Diríjase a la conserjería central para verificación manual.");
        }

        System.out.println("------------------------------------------------------------------------");
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio, nombrePersonaLimpio.length());
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%c'%n", numeroDniSolo, letraFinalDni);
        System.out.printf("ROL CLASIFICADO:      [%c] %-25s | Válido: %b%n", perfilNormalizado, descripcionRol, perfilReconocido);
        System.out.printf("ZONA ASIGNADA:        %s%n", zonaDesignada);
        System.out.printf("LÍMITE AUTORIZADO:    %d minutos (Tiempo previsto: %d min)%n", estanciaMaxPermitida, minutosEstanciaTotal);

        if (esRetrasoLectivo && accesoConcedido) {
```

```
            System.out.println(">> AVISO PUNTUALIDAD: [RETRASO LECTIVO REGISTRADO - Entrada posterior a las 08:15]");
        }

        System.out.println("--------------------------------------------------------------------------");
        System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
        System.out.printf("PERMANENCIA EXACTA:   %03d minutos (Método modular calcularEstancia)%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("--------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round) | Exacta: %6.2f %%%n", porcentajeOcupacionRedondeado, porcentajeOcupacionReal);
        System.out.printf("AFORO DISPONIBLE:     %d plazas restantes de un total de %d%n", aforoDisponible, aforoTotal);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Método modular calcularDistancia)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor acotado: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("DIAGNÓSTICO TEST:     Patrón #%04d | Sensor NFC: Activo (%b)%n", codigoTestCalibracion, sensorNfcOperativo);
        System.out.printf("ESTADO INTEGRIDAD:    Datos correctos: %b (Try-Catch Shield Activo)%n", datosSintacticamenteCorrectos);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("==========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta segunda sesión, cada estudiante integrará el blindaje de captura de datos mediante bloques `try-catch` en su proyecto propio.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). BLINDAJE INTERACTIVO CON `try-catch`

Abre el archivo `pr/src/MiProyecto.java` de tu proyecto asignado y protege mediante `try-catch` el punto de entrada más susceptible de fallar ante entradas de usuario malformadas, declarando e inicializando variables por encima del bloque para preservar el ámbito:

- **Aventura conversacional.** Protege la lectura numérica del turno táctico o la selección de identificadores de inventario contra errores de tipo `NumberFormatException`. Si el jugador introduce texto en lugar de un número entero, intercepta el fallo, activa una bandera de acción errónea y muestra una narración que indique que el personaje pierde la concentración en este turno, garantizando que la partida no finalice por una excepción no controlada.
- **Motor de recomendación.** Envuelve la lectura de la valoración numérica del contenido (puntuación del 1 al 10 en formato decimal) con un bloque `try-catch` que intercepte entradas erróneas o separadores incompatibles (`NumberFormatException`). En caso de error, activa un estado de corrección defensiva asignando una puntuación intermedia por defecto (5.0) y notificando al usuario del ajuste automático.
- **Simulador de físicas 2D.** Blinda la captura de los parámetros iniciales del móvil (como la masa del cuerpo o la velocidad inicial de lanzamiento) ante valores alfanuméricos no numéricos mediante `try-catch`. Si se produce una excepción en la conversión, intercepta el error, restaura los parámetros cinemáticos estándar del simulador y emite una advertencia de contingencia en consola.
- **Bóveda de contraseñas.** Protege la lectura del PIN numérico de desbloqueo o del número de iteraciones del algoritmo criptográfico contra `NumberFormatException`. Si el operador teclea letras o caracteres especiales en el campo numérico, captura la excepción, computa un intento fallido en el contador de seguridad y bloquea el acceso preventivamente sin interrumpir la ejecución del servicio.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). CAPTURA MÚLTIPLE (*MULTI-CATCH*) Y SANEAMIENTO DE ERRORES

1. **Agrupación de excepciones con el operador `|`.** En lugar de duplicar bloques de captura idénticos, Java permite asociar múltiples excepciones disjuntas en una sola cláusula `catch`:

```
try {
    int valor = Integer.parseInt(teclado.nextLine().trim());
    LocalTime hora = LocalTime.of(valor, 0);
} catch (NumberFormatException | DateTimeException e) {
    System.err.println(">> Error de entrada detectado: " + e.getClass().getSimpleName());
    System.err.println(">> Detalle: " + e.getMessage());
}
```

2. **Refactorización defensiva.** Incorpora en tu proyecto un bloque multi-catch que diferencie entre fallos tipográficos y violaciones de límites lógicos, garantizando que el usuario reciba un mensaje claro y que nunca aparezca una traza de depuración técnica en la consola.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). RADIOGRAFÍA DE BYTECODE EN LA JVM: LA TABLA INTERNA DE EXCEPCIONES

Abre la consola de comandos de IntelliJ en el directorio raíz del proyecto y analiza el bytecode compilado de `ControlAccesoQR.class` con el desensamblador oficial en modo detallado:

```
javap -c -v pr/src/ControlAccesoQR.class
```

Localiza dentro del método `main` la estructura interna denominada **`Exception table`**:

```
Exception table:
   from    to  target type
      87   164   167   Class java/lang/NumberFormatException
      87   164   185   Class java/time/DateTimeException
```

Reto técnico: redacta un comentario técnico en la cabecera de tu clase explicando:

1. El significado exacto de las columnas **`from`**, **`to`** y **`target`** en la tabla de excepciones de la JVM (rango de direcciones de instrucciones vigiladas y dirección del salto hacia el manejador de la excepción).
2. Por qué la gestión de excepciones en la arquitectura moderna de la JVM sigue el modelo de **coste cero (*Zero-Cost Exceptions*)**, donde el bloque `try` no introduce ninguna sobrecarga en la CPU mientras no se produzca un error real en tiempo de ejecución.

4.1.4 Día 27 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **jueves 29 de octubre de 2026** en la sede de **AzaharTech**. Los equipos de desarrollo se preparan para culminar la primera semana del **Sprint 3**. Tras haber blindado ayer el terminal con bloques `try-catch` ante errores de entrada (`NumberFormatException`, `DateTimeException`), el equipo somete la versión `v2.3` a una auditoría estricta de calidad y mantenibilidad.

Laia Claramunt proyecta el método `main` y abre el debate con **Alba Torres** y **Pau Ferrer**:

*«Equipo, hemos conseguido un terminal estable que ya no colapsa ante datos corruptos y clasifica los roles mediante sentencias `switch`. Sin embargo, al revisar el código antes de cerrar la semana, detectamos un síntoma evidente de **deuda técnica**: nuestro método `main` está saturado.*

Las comprobaciones de negocio en las líneas 195 a 210 acumulan expresiones booleanas compuestas cada vez más densas:

```
boolean temperaturaApta = (tempVestibulo >= TEMP_CORPORAL_MIN_FALLO && tempVestibulo <= TEMP_CORPORAL_MAX_FIEBRE);
boolean aforoSuficiente = (aforoDisponible > 0);
...
if (!datosSintacticamenteCorrectos) { ... }
else if (!temperaturaApta) { ... }
else if (!aforoSuficiente) { ... }
```

Si mañana el instituto añade un control para validar que el DNI no solo tenga 9 caracteres sino que su último dígito sea una letra mayúscula, o si se modifica el rango de temperatura según la estación del año, nos veremos obligados a seguir inflando el `main` con condiciones enrevesadas.

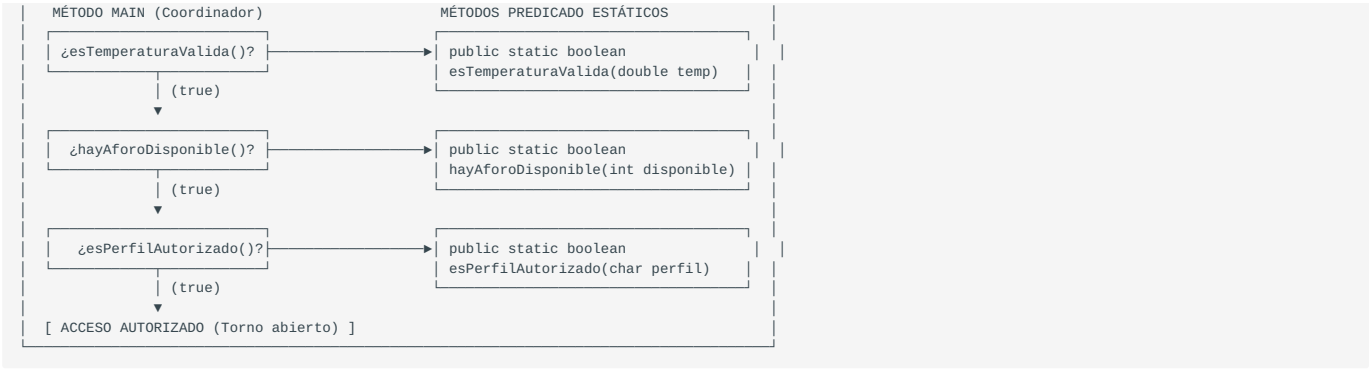
*Hoy cerramos el bloque de estructuras de selección completando el diseño del terminal antes de abordar los bucles la próxima semana. Aplicaremos **refactorización mediante métodos predicado estáticos**, simplificaremos la lógica booleana empleando las **Leyes de De Morgan** y auditaremos la **complejidad ciclomática** para asegurar que cada regla de negocio resida en un método especializado, testeable y limpio».*

---

2. Fundamento teórico: refactorización de la selección, métodos predicado y álgebra booleana

El objetivo de un orquestador principal (`main`) es coordinar las fases del programa (capturar, validar, decidir e informar), no contener los detalles de bajo nivel de cada validación.

```
┌──────────────────────────────────────────────────────────────────────┐
│              DESACOPLO DEL ÁRBOL DE DECISIÓN MEDIANTE MÉTODOS PREDICADO              │
├──────────────────────────────────────────────────────────────────────┤
```



A. MÉTODOS PREDICADO BOOLEANOS (`boolean`)

Un **método predicado** es una función modular pura que recibe uno o más parámetros de entrada, evalúa una regla de negocio y devuelve exclusivamente un valor booleano (`true` o `false`).

- **Claridad semántica.** Reemplaza expresiones crípticas como `temp >= 35.0 && temp <= 37.5` por llamadas autoexplicativas como `esTemperaturaSegura(tempVestibulo)` .
- **Aislamiento de cambios.** Si la jefatura de estudios modifica los márgenes térmicos, el cambio se confina al método especializado sin riesgo de alterar el flujo central de la aplicación.

B. SIMPLIFICACIÓN BOOLEANA: LEYES DE DE MORGAN

Al construir validaciones defensivas complejas, los desarrolladores suelen recurrir a negaciones sobre expresiones compuestas, lo que dificulta la comprensión del código. Las **Leyes de De Morgan** permiten simplificar y transformar estas expresiones:

1. `!(A && B)` es estrictamente equivalente a `(!A || !B)`
2. `!(A || B)` es estrictamente equivalente a `(!A && !B)`

```
// Expresión negativa compleja:
if (!(dniLimpio.length() == 9 && Character.isLetter(dniLimpio.charAt(8)))) { ... }

// Equivalente simplificado y directo mediante método predicado afirmativo:
if (!esFormatoDniCorrecto(dniLimpio)) { ... }
```

C. ANÁLISIS DE COMPLEJIDAD CICLOMÁTICA (*CYCLOMATIC COMPLEXITY*)

La complejidad ciclomática de McCabe mide el número de caminos linealmente independientes a través del grafo de flujo de un programa: $$V(G) = P + 1$$ Donde P es el número de puntos de decisión (`if` , `else if` , `case` , operadores lógicos `&&` , `||`). * **Estándar AzaharTech.** Un método `main` no debe superar un valor de $V(G) \le 10$. La extracción de bifurcaciones hacia métodos predicado independientes fragmenta la complejidad total en unidades pequeñas y manejables.

---

3. Evolución del caso guía a `ControlAccesoQR v2.4` (Java nativo)

En esta versión extraemos las reglas de validación biométrica, documental y de perfil hacia métodos predicado estáticos (`boolean`), aligerando el árbol de decisión de `main` y reduciendo notablemente su complejidad ciclomática.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.3 (`diff`)

```
  // ========================================================================
  // MÉTODOS PREDICADO BOOLEANOS (Extracción modular de reglas de negocio)
  // ========================================================================
+ public static boolean esTemperaturaCorporalSegura(double temperatura) {
+     return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
+ }
+
+ public static boolean esDocumentoValido(String dni) {
+     return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
+ }
+
+ public static boolean esPerfilAutorizado(char perfil) {
+     boolean autorizado;
+     switch (perfil) {
+         case 'E': case 'D': case 'V': case 'A': autorizado = true; break;
+         default: autorizado = false; break;
+     }
```

```
+      return autorizado;
+ }

   // ============================================================================
   // MÉTODO MAIN: Árbol de decisión desacoplado
   // ============================================================================
- boolean esLongitudDniCorrecta = (dniPersonaLimpio.length() == LONGITUD_DNI_ESTANDAR);
+ boolean esLongitudDniCorrecta = esDocumentoValido(dniPersonaLimpio);
- boolean perfilReconocido;
- switch (perfilNormalizado) { ... } // Se traslada al método predicado
+ boolean perfilReconocido = esPerfilAutorizado(perfilNormalizado);

   if (!datosSintacticamenteCorrectos) {
       accesoConcedido = false;
- } else if (!temperaturaApta) {
+ } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
       accesoConcedido = false;
       motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
               tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
   } else if (aforoDisponible <= 0) {
```

B. FOCO EN LA NOVEDAD: MÉTODOS PREDICADO Y ÁRBOL DE DECISIÓN LIMPIO

Observa la claridad semántica que adquiere el orquestador `main` al apoyarse en funciones que responden preguntas booleanas concretas:

```
// ----------------------------------------------------------------------------
// Métodos predicado estáticos puros
// ----------------------------------------------------------------------------
public static boolean esTemperaturaCorporalSegura(double temperatura) {
    return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
}

public static boolean esDocumentoValido(String dni) {
    return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
}

public static boolean esPerfilAutorizado(char perfil) {
    boolean autorizado;
    switch (perfil) {
        case 'E':
        case 'D':
        case 'V':
        case 'A':
            autorizado = true;
            break;
        default:
            autorizado = false;
            break;
    }
    return autorizado;
}

// ----------------------------------------------------------------------------
// Árbol de decisión limpio en main
// ----------------------------------------------------------------------------
boolean accesoConcedido;

if (!datosSintacticamenteCorrectos) {
    accesoConcedido = false;
} else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
    accesoConcedido = false;
    motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
            tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
} else if (aforoDisponible <= 0) {
    accesoConcedido = false;
    motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
} else if (!esLongitudDniCorrecta) {
    accesoConcedido = false;
    motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
} else if (!perfilReconocido) {
    accesoConcedido = false;
    motivoDenegacion = "Perfil '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
} else {
    accesoConcedido = true;
    personasEnCentro++;
    aforoDisponible--;
    idUltimoFichaje++;
}
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.4)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.4:
 * Consolidación de estructuras de selección y toma de decisiones.
 * Refactorización de reglas de negocio en métodos predicado estáticos booleanos.
 * Optimización de la legibilidad lógica y reducción de la complejidad ciclomática en main.
 * Preserva el blindaje interactivo con try-catch y la modularización con switch clásico.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.4 (Octubre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales de seguridad biométrica y capacidad (Sprint 3)
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control horario de puntualidad lectiva para estudiantes (Día 25)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    /**
     * Calcula la estancia neta en minutos entre dos marcas temporales usando Duration.
     */
    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    /**
     * Calcula el porcentaje de ocupación con precisión decimal aplicando casting explícito.
     */
    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    /**
     * Construye y concatena el token QR oficial parametrizado para cualquier monitor.
     */
    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    /**
     * Calcula la distancia euclídea (hipotenusa) a la pantalla mediante el teorema de Pitágoras.
     */
    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    /**
     * Retorna la denominación oficial del perfil mediante una sentencia switch clásica con break.
     */
    public static String obtenerDescripcionPerfil(char perfil) {
```

```
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    /**
     * Asigna la zona física autorizada del campus mediante una sentencia switch clásica.
     */
    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    /**
     * Determina el límite de minutos de permanencia permitidos según el perfil con switch clásico.
     */
    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360; // 6 horas lectivas
                break;
            case 'D':
                estanciaMinutos = 480; // 8 horas jornada docente
                break;
            case 'V':
                estanciaMinutos = 120; // 2 horas límite visitas
                break;
            case 'A':
                estanciaMinutos = 420; // 7 horas jornada administrativa
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (NOVEDAD DÍA 27: REFACTORIZACIÓN MODULAR)
    // ------------------------------------------------------------------------

    /**
     * Predicado que valida si una lectura de temperatura está en el rango biométrico seguro.
     */
    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    /**
     * Predicado que valida la integridad formal de longitud del documento de identidad.
     */
    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    /**
     * Predicado que comprueba si un código de rol está homologado por el centro.
     */
    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
```

```
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: ORQUESTADOR DESACOPLADO MEDIANTE MÉTODOS PREDICADO
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Generadores independientes en el Heap
        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();
        LocalTime horaEmisionQr = LocalTime.now();

        // Inicialización de variables con valores de contingencia
        int terminalId = 0;
        double tempVestibulo = 0.0;
        int segundosActividadTerminal = 0;
        double coordenadaXMetros = 0.0;
        double coordenadaYMetros = 0.0;

        String nombrePersonaBruto = "", nombrePersonaLimpio = "";
        String dniPersonaBruto = "", dniPersonaLimpio = "";
        char perfilPersonaBruto = ' ';
        char perfilNormalizado = ' ';
        boolean esLetraValida = false;
        boolean esEntrada = true;

        int horaEntrada = 0, minutoEntrada = 0;
        int horaSalida = 0, minutoSalida = 0;

        LocalTime tiempoEntrada = LocalTime.of(8, 0);
        LocalTime tiempoSalida  = LocalTime.of(14, 0);

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        // Bandera de integridad técnica para el bloque try-catch (Día 26)
        boolean datosSintacticamenteCorrectos = true;
        String motivoDenegacion = "";

        System.out.println("=====================================================");
        System.out.println("   AZAHARTECH - TERMINAL " + NOMBRE_CENTRO);
        System.out.println("   Versión 2.4 Oficial (Sprint 3: Predicados & Clean)");
        System.out.println("=====================================================");

        // ------------------------------------------------------------------------
        // BLOQUE VIGILADO: CAPTURA DE DATOS INTERACTIVA CON TRY-CATCH
        // ------------------------------------------------------------------------
        try {
            System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
            terminalId = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Temperatura base del sensor (ºC): ");
            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

            System.out.print("Segundos de actividad del terminal (uptime): ");
            segundosActividadTerminal = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Distancia horizontal a pantalla en metros (eje X): ");
            coordenadaXMetros = Double.parseDouble(teclado.nextLine().trim());

            System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
            coordenadaYMetros = Double.parseDouble(teclado.nextLine().trim());

            System.out.print("DNI de la persona: ");
            dniPersonaBruto = teclado.nextLine();

            System.out.print("Nombre completo: ");
            nombrePersonaBruto = teclado.nextLine();

            // Normalización de texto
            dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
            nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

            System.out.print("Perfil de acceso (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
            String lineaPerfil = teclado.nextLine().trim();
            if (lineaPerfil.isEmpty()) {
                perfilPersonaBruto = '?';
            } else {
                perfilPersonaBruto = lineaPerfil.charAt(0);
            }
            perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
            esLetraValida = Character.isLetter(perfilNormalizado);

            System.out.print("Hora de entrada (0-23): ");
            horaEntrada = Integer.parseInt(teclado.nextLine().trim());
            System.out.print("Minuto de entrada (0-59): ");
            minutoEntrada = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Hora de salida (0-23): ");
```

```
            horaSalida = Integer.parseInt(teclado.nextLine().trim());
            System.out.print("Minuto de salida (0-59): ");
            minutoSalida = Integer.parseInt(teclado.nextLine().trim());

            // Construcción cronológica vigilada
            tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
            tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

        } catch (NumberFormatException e) {
            datosSintacticamenteCorrectos = false;
            motivoDenegacion = "Error de formato de entrada: se introdujo un valor alfanumérico en un campo numérico.";
        } catch (DateTimeException e) {
            datosSintacticamenteCorrectos = false;
            motivoDenegacion = "Error temporal cronológico: hora (0-23) o minuto (0-59) fuera de límites válidos.";
        }

        // Operaciones de permanencia
        int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

        // Calibración ambiental y diagnóstico
        double tempCalibrada = tempVestibulo + (generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA);
        boolean sensorNfcOperativo = generadorCalibracion.nextBoolean();
        int codigoTestCalibracion  = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);

        int aforoTotal = personasEnCentro + aforoDisponible;

        // Desglose del documento
        boolean esLongitudDniCorrecta = esDocumentoValido(dniPersonaLimpio);
        String numeroDniSolo = dniPersonaLimpio.isEmpty() ? "0" : dniPersonaLimpio.substring(0, Math.max(0, dniPersonaLimpio.length() - 1));
        char letraFinalDni   = dniPersonaLimpio.isEmpty() ? '?' : dniPersonaLimpio.charAt(Math.max(0, dniPersonaLimpio.length() - 1));

        // Clasificación con switch clásico
        boolean perfilReconocido = esPerfilAutorizado(perfilNormalizado);
        String descripcionRol    = obtenerDescripcionPerfil(perfilNormalizado);
        String zonaDesignada     = determinarZonaAutorizada(perfilNormalizado);
        int estanciaMaxPermitida = obtenerEstanciaMaximaPorPerfil(perfilNormalizado);

        // Control de puntualidad específico para estudiantes
        boolean esRetrasoLectivo = (perfilNormalizado == 'E') && tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD);

        // ---------------------------------------------------------------------
        // 5. ÁRBOL DE DECISIÓN REFACTORIZADO CON MÉTODOS PREDICADO (DÍA 27)
        // ---------------------------------------------------------------------
        boolean accesoConcedido;

        if (!datosSintacticamenteCorrectos) {
            accesoConcedido = false;
        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
            accesoConcedido = false;
            motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
                    tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
        } else if (aforoDisponible <= 0) {
            accesoConcedido = false;
            motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
        } else if (!esLongitudDniCorrecta) {
            accesoConcedido = false;
            motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
        } else if (!perfilReconocido) {
            accesoConcedido = false;
            motivoDenegacion = "Perfil '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
        } else {
            accesoConcedido = true;
            personasEnCentro++;
            aforoDisponible--;
            idUltimoFichaje++;
        }

        // Generación de códigos de seguridad
        int codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        // Construcción de tokens mediante métodos modulares
        String tokenResumenVestibulo   = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);
        String tokenResumenPuertaNorte = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, ID_TERMINAL_PUERTA_NORTE, codigoTokenPuertaNorte, idUltimoFichaje);
        String tokenFormatoBarras      = tokenResumenVestibulo.replace("-", "/");

        // Cálculos métricos auxiliares
        int aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);
        double porcentajeOcupacionReal = calcularPorcentajeOcupacion(aforoSeguroVisualizado, aforoTotal);
        int porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        double distanciaDiagonalAntena = calcularDistanciaDiagonal(coordenadaXMetros, coordenadaYMetros);
        int desviacionJornadaMinutos   = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        double tempSeguraAcotada       = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        // Descomposición temporal de servicio
        int horasUptime    = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        int minutosUptime  = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        int segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // ---------------------------------------------------------------------
        // 6. SALIDA FORMATEADA CON CONTROL DE INCIDENCIAS (System.out.printf)
        // ---------------------------------------------------------------------
        System.out.println("\n=========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                      ");
        System.out.println("=========================================================================");
        System.out.printf("CENTRO:               %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:         #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);

        if (accesoConcedido) {
```

```
            System.out.println("--------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO AUTORIZADO - TORNO ABIERTO]");
            System.out.printf(">> TOKEN QR EMITIDO:  %s%n", tokenResumenVestibulo);
            System.out.printf(">> TOKEN BARRAS:      %s%n", tokenFormatoBarras);
            System.out.printf(">> PUERTA NORTE LINK: %s%n", tokenResumenPuertaNorte);
        } else {
            System.out.println("--------------------------------------------------------------------------");
            System.out.println(">> VEREDICTO:         [ACCESO DENEGADO - TORNO BLOQUEADO]");
            System.out.printf(">> MOTIVO RECHAZO:    %s%n", motivoDenegacion);
            System.out.println(">> PROTOCOLO:         Diríjase a la conserjería central para verificación manual.");
        }

        System.out.println("--------------------------------------------------------------------------");
        System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio, nombrePersonaLimpio.length());
        System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%c'%n", numeroDniSolo, letraFinalDni);
        System.out.printf("ROL CLASIFICADO:      [%c] %-25s | Válido: %b%n", perfilNormalizado, descripcionRol, perfilReconocido);
        System.out.printf("ZONA ASIGNADA:        %s%n", zonaDesignada);
        System.out.printf("LÍMITE AUTORIZADO:    %d minutos (Tiempo previsto: %d min)%n", estanciaMaxPermitida, minutosEstanciaTotal);

        if (esRetrasoLectivo && accesoConcedido) {
            System.out.println(">> AVISO PUNTUALIDAD: [RETRASO LECTIVO REGISTRADO - Entrada posterior a las 08:15]");
        }

        System.out.println("--------------------------------------------------------------------------");
        System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
        System.out.printf("PERMANENCIA EXACTA:   %03d minutos (Método modular calcularEstancia)%n", minutosEstanciaTotal);
        System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
        System.out.println("--------------------------------------------------------------------------");
        System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round) | Exacta: %6.2f %%%n", porcentajeOcupacionRedondeado, porcentajeOcupacionReal);
        System.out.printf("AFORO DISPONIBLE:     %d plazas restantes de un total de %d%n", aforoDisponible, aforoTotal);
        System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Método modular calcularDistancia)%n", distanciaDiagonalAntena);
        System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor acotado: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime, tempSeguraAcotada);
        System.out.printf("DIAGNÓSTICO TEST:     Patrón #%04d | Sensor NFC: Activo (%b)%n", codigoTestCalibracion, sensorNfcOperativo);
        System.out.printf("ESTADO INTEGRIDAD:    Datos correctos: %b (Try-Catch Shield Activo)%n", datosSintacticamenteCorrectos);
        System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
        System.out.println("==========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión, cada alumno cerrará la semana refactorizando la lógica condicional de su proyecto asignado mediante métodos predicado estáticos y simplificación lógica.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). REFACTORIZACIÓN A MÉTODOS PREDICADO EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e identifica las expresiones condicionales complejas que evalúan el estado de la aplicación. Extrae dichas expresiones a **métodos estáticos de tipo predicado (`public static boolean`)**, limpiando el flujo principal:

- **Aventura conversacional.** Encapsula las comprobaciones de supervivencia del protagonista y de viabilidad de combate en métodos predicado independientes. Define métodos que determinen si el héroe sigue con vida en función de sus puntos actuales, si dispone de puntos de acción suficientes para ejecutar una habilidad especial o si el adversario es susceptible a un golpe crítico, invocando estos métodos desde la estructura condicional de decisión del asalto.
- **Motor de recomendación.** Extrae los cálculos de afinidad a un método predicado especializado que valide si una puntuación calculada supera el corte de recomendación prioritaria. Añade otro método predicado que compruebe si los metadatos del contenido cumplen con las restricciones de edad o perfil del usuario, aligerando el bloque principal de decisión.
- **Simulador de físicas 2D.** Aísla las condiciones geométricas perimetrales en métodos predicado específicos para la detección de colisiones. Crea métodos que comprueben si la coordenada horizontal del móvil sobrepasa las fronteras laterales y si la altura vertical contacta con la superficie de impacto, delegando en ellos la activación del recálculo de la trayectoria cinemática.
- **Bóveda de contraseñas.** Modula la evaluación de robustez de las credenciales mediante métodos predicado que verifiquen los requisitos de seguridad de la clave. Implementa un método que dictamine si la longitud es conforme al estándar corporativo y otro que compruebe si el nivel de criticidad asignado pertenece a los perfiles homologados por la política del sistema.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). SIMPLIFICACIÓN LÓGICA CON LAS LEYES DE DE MORGAN

1. **Refactorización de expresiones negativas.** Analiza expresiones condicionales complejas con operadores lógicos mezclados y aplica De Morgan para transformarlas en condiciones afirmativas directas:
2. Transformar expresiones del tipo `if (!(esValido && hayEspacio))` en su forma equivalente desacoplada `if (!esValido || !hayEspacio)`.
3. Evaluar cómo la inversión de operadores relacionales (`<` frente a `>=`) mejora la legibilidad cuando se combina con nombres de métodos predicado claros.
4. **Aplicación en el proyecto propio.** Localiza en tu código cualquier condición que utilice más de dos operadores lógicos combinados con negaciones y simplifícala aplicando equivalencias de De Morgan, comprobando que las pruebas de ejecución arrojen idéntico resultado funcional.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). MÉTRICAS DE CALIDAD: COMPLEJIDAD CICLOMÁTICA (MCCABE) EN EL FLUJO DE CONTROL

Abre la herramienta de inspección de código o calcula manualmente la complejidad ciclomática $V(G)$ de tu método `main` y de tus métodos predicado:

$$V(G) = P + 1$$

Donde P representa el número de predicados decisionales (`if` , `case` , operadores lógicos condicionales `&&` , `||`).

Reto técnico: redacta un comentario técnico en la cabecera de tu clase documentando: 1. El cálculo exacto de la complejidad ciclomática de tu clase antes y después de extraer los métodos predicado. 2. Por qué mantener la complejidad ciclomática de cualquier método por debajo de 10 ($V(G) \le 10$) reduce exponencialmente la probabilidad de defectos lógicos y facilita la cobertura completa de pruebas unitarias (*branch coverage*) antes de abordar las estructuras de repetición en la Semana 8.

4.2 Semana 8. Estructuras de repetición e iteración (bucles `while` y `do-while`)

4.2.1 Día 28 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **lunes 2 de noviembre de 2026**. Los equipos de desarrollo de **AzaharTech** inician la segunda semana del **Sprint 3: «El laberinto de la lógica»**. Tras la refactorización modular del pasado jueves, la versión `v2.4` del terminal dispone de un árbol de decisión limpio y blindado ante excepciones. Sin embargo, el software aún presenta una limitación operativa que impide su despliegue en el mundo real.

Pau Ferrer muestra la pantalla de la terminal a **Alba Torres** y **Laia Claramunt**:

*«Alba, Laia: el terminal valida con precisión la temperatura, comprueba el DNI con métodos predicado y clasifica los roles con el `switch` . Pero fijaos en lo que ocurre nada más imprimir el informe: **el programa termina y el proceso muere en el sistema operativo**.*

*Esta mañana, el personal de conserjería del **IES El Caminàs** nos trasladó su frustración: a primera hora entran cientos de personas en cuestión de minutos. Es inviable tener que volver a lanzar la aplicación por consola para cada alumno o profesor que pasa por el torno. Además, al cerrarse la JVM, **las variables de memoria se reinician**: el aforo vuelve a ser 1.200, el último fichaje vuelve a ser el 1.042 y se pierde por completo la trazabilidad de la jornada.*

*Un terminal físico de control de presencia debe funcionar como un **demonio o servicio continuo**: se inicia a primera hora, se mantiene a la escucha procesando fichajes de forma indefinida, actualiza dinámicamente las plazas libres en el campus y solo se detiene cuando un operador introduce un comando de cierre de sesión».*

Laia Claramunt toma el marcador y traza el objetivo de la Semana 8:

*«Llegó el momento de dar el salto a las **estructuras de repetición**. Hoy arrancamos con el bucle post-test `do-while`.*

*Aprenderemos a gobernar el ciclo del terminal mediante una **variable centinela**, mantendremos el estado del aforo mutando iteración tras iteración en memoria y diseñaremos un cierre de auditoría que recopile el total de accesos autorizados y denegados al término del servicio».*

---

2. Fundamento teórico: estructuras de repetición iterativas (`while` y `do-while`)

Las estructuras de control de repetición (o bucles) permiten ejecutar de forma reiterada un conjunto de sentencias mientras una condición lógica sea evaluada como verdadera. A cada una de las ejecuciones del bloque se le denomina **iteración**.

```
flowchart TD
    subgraph PreTest ["Bucle PRE-TEST: while"]
        InicioWhile([Inicio]) --> CondWhile{"¿Condición cierta?<br/>(while)"}
        CondWhile -- true --> CuerpoWhile["Ejecuta cuerpo { }"]
        CuerpoWhile --> CondWhile
        CondWhile -- false --> FinWhile([Flujo convergente])
    end

    subgraph PostTest ["Bucle POST-TEST: do-while"]
        InicioDo([Inicio]) --> CuerpoDo["Ejecuta cuerpo { }<br/><b>(Garantizado al menos 1 vez)</b>"]
        CuerpoDo --> CondDo{"¿Condición cierta?<br/>(while)"}
        CondDo -- true --> CuerpoDo
        CondDo -- false --> FinDo([Flujo convergente])
    end
```

A. COMPARATIVA CONCEPTUAL: `while` FRENTE A `do-while`

Característica	Bucle pre-test (`while`)	Bucle post-test (`do-while`)
Evaluación de la condición	Antes de entrar al cuerpo (*pre-test*)	Al finalizar cada vuelta (*post-test*)
Garantía de ejecución mínima	**0 veces** (si la condición inicial es falsa)	**Al menos 1 vez** (incondicionalmente)
Punto y coma de cierre	No lleva tras la cabecera: `while (c) { }`	**Obligatorio al final**: `do { } while (c);`
Caso de uso idóneo	Recorridos condicionados, espera de recursos	Menús interactivos, demonios de servicio continuo

B. GESTIÓN DE ESTADO: CENTINELAS, CONTADORES Y ACUMULADORES

El control de un bucle profesional descansa sobre tres patrones fundamentales:

1. **Variable centinela (*Sentinel flag*):** Variable booleana (`boolean sesionActiva = true;`) cuyo cambio de estado indica que el bucle debe concluir (por ejemplo, cuando el operador pulsa una tecla distinta de `'S'`).
2. **Contador de eventos:** Variable entera que se incrementa en una unidad fija en cada iteración (`totalFichajesProcesados++`).
3. **Acumulador dinámico:** Variable que almacena el balance acumulado de magnitudes mutables a lo largo del tiempo (por ejemplo, el recuento acumulativo de autorizaciones y denegaciones).

C. CICLO DE VIDA Y PERSISTENCIA EN MEMORIA (*STACK ALLOCATION*)

- **Variables de sesión (fuera del bucle):** `personasEnCentro`, `aforoDisponible`, `totalAutorizados`. Se declaran antes del `do` para que mantengan su estado y acumulen cambios a lo largo de las sucesivas iteraciones.
- **Variables transitorias (dentro del bucle):** `dniPersona`, `tempVestibulo`, `accesoConcedido`. Nacen y mueren en cada vuelta en la memoria de pila (*Stack*), garantizando que los datos de un usuario no contaminen los del siguiente.

---

3. Evolución del caso guía a `ControlAccesoQR v2.5` (Java nativo)

Evolucionamos el código de la versión `v2.4` a la **`v2.5`**. Conservamos intactas las constantes inmutables, la captura protegida con `try-catch`, los métodos modulares de cálculo y los métodos predicado booleanos, transformando la aplicación en un servicio desatendido.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.4 (`diff`)

```
   // ========================================================================
   // MÉTODO MAIN: Estado global persistente (fuera del bucle)
   // ========================================================================
   int personasEnCentro = 1200;
   int aforoDisponible = 800;
   int idUltimoFichaje = 1042;
+  int totalFichajesProcesados = 0;
+  int totalFichajesAutorizados = 0;
+  int totalFichajesDenegados = 0;
+  boolean sesionActiva = true;

+  do {
+      totalFichajesProcesados++;
       // ... [Captura protegida con try-catch y validaciones de v2.4] ...

       if (accesoConcedido) {
+          totalFichajesAutorizados++;
           personasEnCentro++;
           aforoDisponible--;
           idUltimoFichaje++;
       } else {
+          totalFichajesDenegados++;
       }

       // ... [Salida formateada del informe individual] ...

+      // Evaluación de continuidad mediante centinela
+      System.out.print("\n¿Desea registrar otro acceso? (S = Sí / cualquier otra = Salir): ");
+      sesionActiva = teclado.nextLine().trim().equalsIgnoreCase("S");
+  } while (sesionActiva);

+  // ========================================================================
+  // INFORME DE AUDITORÍA CONSOLIDADA (Tras la desconexión del servicio)
+  // ========================================================================
+  System.out.println("AUDITORÍA FINAL DE LA SESIÓN...");
```

B. FOCO EN LA NOVEDAD: BUCLE DE SERVICIO Y ACUMULADORES DE SESIÓN

Observa cómo las variables de balance se declaran en el ámbito exterior para no reiniciarse en cada vuelta, mientras que la bandera centinela `sesionActiva` gobierna la permanencia en el ciclo:

```java
// ------------------------------------------------------------------------
// Estado global y acumuladores (persisten entre iteraciones sucesivas)
// ------------------------------------------------------------------------
int personasEnCentro = 1200;
int aforoDisponible = 800;
int idUltimoFichaje = 1042;

int totalFichajesProcesados = 0;
int totalFichajesAutorizados = 0;
int totalFichajesDenegados = 0;

boolean sesionActiva = true;

// ------------------------------------------------------------------------
// Bucle continuo de servicio desatendido
// ------------------------------------------------------------------------
do {
    totalFichajesProcesados++;

    // [Captura con try-catch y toma de decisiones mediante predicados...]
    if (accesoConcedido) {
        totalFichajesAutorizados++;
        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;
    } else {
        totalFichajesDenegados++;
    }

    // [Emisión de informe individual de acceso...]

    // Consulta de continuidad que gobierna la condición centinela
    System.out.print("\n¿Desea registrar otro acceso en el terminal? (S = Sí / cualquier otra tecla = Cerrar sesión): ");
    String respuestaContinuar = teclado.nextLine().trim();
    sesionActiva = respuestaContinuar.equalsIgnoreCase("S");

} while (sesionActiva);

// ------------------------------------------------------------------------
```

```
// Auditoría final consolidada (se ejecuta una sola vez tras salir del bucle)
// ------------------------------------------------------------------------
System.out.println("\n========================================================================");
System.out.println("          AUDITORÍA FINAL DE LA SESIÓN · IES EL CAMINÀS               ");
System.out.println("========================================================================");
System.out.printf("TOTAL FICHAJES PROCESADOS:    %d%n", totalFichajesProcesados);
System.out.printf("ACCESOS AUTORIZADOS (TORNO):  %d (%.1f %%)%n", totalFichajesAutorizados,
        totalFichajesProcesados > 0 ? ((double) totalFichajesAutorizados / totalFichajesProcesados) * 100 : 0.0);
System.out.printf("ACCESOS DENEGADOS (BLOQUEO):  %d (%.1f %%)%n", totalFichajesDenegados,
        totalFichajesProcesados > 0 ? ((double) totalFichajesDenegados / totalFichajesProcesados) * 100 : 0.0);
System.out.printf("OCUPACIÓN FINAL DEL CAMPUS:   %d personas (Plazas disponibles: %d)%n", personasEnCentro, aforoDisponible);
System.out.println("ESTADO DEL TERMINAL:          Servicio finalizado ordenadamente por el operador.");
System.out.println("========================================================================");
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.5)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.5:
 * Incorpora estructuras de repetición iterativas mediante bucle post-test do-while (Sprint 3).
 * Mantiene el terminal en servicio continuo desatendido procesando múltiples fichajes.
 * Gestiona la persistencia de estado del aforo en memoria y acumuladores de sesión.
 * Emite una auditoría final consolidada al desconectar el servicio.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.5 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales de seguridad biométrica y capacidad (Sprint 3)
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control horario de puntualidad lectiva para estudiantes (Día 25)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    /**
     * Calcula la estancia neta en minutos entre dos marcas temporales usando Duration.
     */
    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    /**
     * Calcula el porcentaje de ocupación con precisión decimal aplicando casting explícito.
     */
    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    /**
     * Construye y concatena el token QR oficial parametrizado para cualquier monitor.
     */
    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    /**
     * Calcula la distancia euclídea (hipotenusa) a la pantalla mediante el teorema de Pitágoras.
     */
    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    /**
     * Retorna la denominación oficial del perfil mediante una sentencia switch clásica con break.
     */
    public static String obtenerDescripcionPerfil(char perfil) {
```

```
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    /**
     * Asigna la zona física autorizada del campus mediante una sentencia switch clásica.
     */
    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    /**
     * Determina el límite de minutos de permanencia permitidos según el perfil con switch clásico.
     */
    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360; // 6 horas lectivas
                break;
            case 'D':
                estanciaMinutos = 480; // 8 horas jornada docente
                break;
            case 'V':
                estanciaMinutos = 120; // 2 horas límite visitas
                break;
            case 'A':
                estanciaMinutos = 420; // 7 horas jornada administrativa
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (modularización de reglas de negocio)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
```

```
    // 4. MÉTODO PRINCIPAL: SERVICIO CONTINUO DESATENDIDO (DO-WHILE)
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Generadores en el Heap (persisten durante toda la sesión)
        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // ------------------------------------------------------------------
        // ESTADO GLOBAL Y ACUMULADORES (Persisten entre iteraciones sucesivas)
        // ------------------------------------------------------------------
        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;

        boolean sesionActiva = true;

        System.out.println("========================================================================");
        System.out.println("   AZAHARTECH · SISTEMA DESATENDIDO DE ACCESOS - IES EL CAMINÀS       ");
        System.out.println("   Versión 2.5 Oficial (Sprint 3: Servicio Continuo do-while Activo)  ");
        System.out.println("========================================================================");

        // ------------------------------------------------------------------
        // BUCLE PRINCIPAL DE SERVICIO: ITERACIÓN POR CADA FICHAJE
        // ------------------------------------------------------------------
        do {
            LocalTime horaEmisionQr = LocalTime.now();

            // Variables transitorias de cada ciclo
            int terminalId = 0;
            double tempVestibulo = 0.0;
            int segundosActividadTerminal = 0;
            double coordenadaXMetros = 0.0;
            double coordenadaYMetros = 0.0;

            String nombrePersonaBruto = "", nombrePersonaLimpio = "";
            String dniPersonaBruto = "", dniPersonaLimpio = "";
            char perfilPersonaBruto = ' ';
            char perfilNormalizado = ' ';
            boolean esLetraValida = false;
            boolean esEntrada = true;

            int horaEntrada = 0, minutoEntrada = 0;
            int horaSalida = 0, minutoSalida = 0;

            LocalTime tiempoEntrada = LocalTime.of(8, 0);
            LocalTime tiempoSalida  = LocalTime.of(14, 0);

            boolean datosSintacticamenteCorrectos = true;
            String motivoDenegacion = "";

            totalFichajesProcesados++; // Conteo de transacciones

            System.out.println("\n------------------------------------------------------------------------");
            System.out.printf(">>> REGISTRO N.º %04d EN PROCESO DE VALIDACIÓN <<<%n", totalFichajesProcesados);
            System.out.println("------------------------------------------------------------------------");

            // Captura interactiva protegida con try-catch
            try {
                System.out.print("Introduce identificador del terminal en texto (ej. 101): ");
                terminalId = Integer.parseInt(teclado.nextLine().trim());

                System.out.print("Temperatura base del sensor (ºC): ");
                tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

                System.out.print("Segundos de actividad del terminal (uptime): ");
                segundosActividadTerminal = Integer.parseInt(teclado.nextLine().trim());

                System.out.print("Distancia horizontal a pantalla en metros (eje X): ");
                coordenadaXMetros = Double.parseDouble(teclado.nextLine().trim());

                System.out.print("Altura de la pantalla de TV en metros (eje Y): ");
                coordenadaYMetros = Double.parseDouble(teclado.nextLine().trim());

                System.out.print("DNI de la persona: ");
                dniPersonaBruto = teclado.nextLine();

                System.out.print("Nombre completo: ");
                nombrePersonaBruto = teclado.nextLine();

                dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
                nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

                System.out.print("Perfil de acceso (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
                String lineaPerfil = teclado.nextLine().trim();
                if (lineaPerfil.isEmpty()) {
                    perfilPersonaBruto = '?';
                } else {
                    perfilPersonaBruto = lineaPerfil.charAt(0);
                }
                perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
                esLetraValida = Character.isLetter(perfilNormalizado);
```

```
            System.out.print("Hora de entrada (0-23): ");
            horaEntrada = Integer.parseInt(teclado.nextLine().trim());
            System.out.print("Minuto de entrada (0-59): ");
            minutoEntrada = Integer.parseInt(teclado.nextLine().trim());

            System.out.print("Hora de salida (0-23): ");
            horaSalida = Integer.parseInt(teclado.nextLine().trim());
            System.out.print("Minuto de salida (0-59): ");
            minutoSalida = Integer.parseInt(teclado.nextLine().trim());

            tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
            tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

        } catch (NumberFormatException e) {
            datosSintacticamenteCorrectos = false;
            motivoDenegacion = "Error de formato de entrada: valor numérico ilegible.";
        } catch (DateTimeException e) {
            datosSintacticamenteCorrectos = false;
            motivoDenegacion = "Error cronológico: hora (0-23) o minuto (0-59) fuera de rango.";
        }

        int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

        double tempCalibrada = tempVestibulo + (generadorCalibracion.nextDouble() * MAX_VARIACION_TERMICA);
        boolean sensorNfcOperativo = generadorCalibracion.nextBoolean();
        int codigoTestCalibracion  = BASE_TOKEN_SEGURIDAD + generadorCalibracion.nextInt(RANGO_TOKEN_SEGURIDAD);

        int aforoTotal = personasEnCentro + aforoDisponible;

        boolean esLongitudDniCorrecta = esDocumentoValido(dniPersonaLimpio);
        String numeroDniSolo = dniPersonaLimpio.isEmpty() ? "0" : dniPersonaLimpio.substring(0, Math.max(0, dniPersonaLimpio.length() - 1));
        char letraFinalDni   = dniPersonaLimpio.isEmpty() ? '?' : dniPersonaLimpio.charAt(Math.max(0, dniPersonaLimpio.length() - 1));

        boolean perfilReconocido = esPerfilAutorizado(perfilNormalizado);
        String descripcionRol    = obtenerDescripcionPerfil(perfilNormalizado);
        String zonaDesignada     = determinarZonaAutorizada(perfilNormalizado);
        int estanciaMaxPermitida = obtenerEstanciaMaximaPorPerfil(perfilNormalizado);

        boolean esRetrasoLectivo = (perfilNormalizado == 'E') && tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD);

        // ------------------------------------------------------------------
        // TOMA DE DECISIONES Y MUTACIÓN DE ESTADO ACUMULATIVO
        // ------------------------------------------------------------------
        boolean accesoConcedido;

        if (!datosSintacticamenteCorrectos) {
            accesoConcedido = false;
            totalFichajesDenegados++;
        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
            accesoConcedido = false;
            motivoDenegacion = String.format("Lectura térmica anómala (%.1f ºC fuera del rango %.1f-%.1f ºC)",
                    tempVestibulo, TEMP_CORPORAL_MIN_FALLO, TEMP_CORPORAL_MAX_FIEBRE);
            totalFichajesDenegados++;
        } else if (aforoDisponible <= 0) {
            accesoConcedido = false;
            motivoDenegacion = "Aforo de recinto saturado. No hay plazas libres disponibles.";
            totalFichajesDenegados++;
        } else if (!esLongitudDniCorrecta) {
            accesoConcedido = false;
            motivoDenegacion = "Documento irregular. El DNI debe contener exactamente 9 caracteres.";
            totalFichajesDenegados++;
        } else if (!perfilReconocido) {
            accesoConcedido = false;
            motivoDenegacion = "Perfil '" + perfilNormalizado + "' no autorizado por jefatura de estudios.";
            totalFichajesDenegados++;
        } else {
            accesoConcedido = true;
            totalFichajesAutorizados++;
            // Mutaciones persistentes en memoria que sobreviven para el siguiente usuario
            personasEnCentro++;
            aforoDisponible--;
            idUltimoFichaje++;
        }

        int codigoTokenVestibulo   = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
        int codigoTokenPuertaNorte = BASE_TOKEN_SEGURIDAD + generadorPuertaNorte.nextInt(RANGO_TOKEN_SEGURIDAD);

        String tokenResumenVestibulo   = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);
        String tokenResumenPuertaNorte = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, ID_TERMINAL_PUERTA_NORTE, codigoTokenPuertaNorte, idUltimoFichaje);
        String tokenFormatoBarras      = tokenResumenVestibulo.replace("-", "/");

        int aforoSeguroVisualizado = Math.min(personasEnCentro, aforoTotal);
        double porcentajeOcupacionReal = calcularPorcentajeOcupacion(aforoSeguroVisualizado, aforoTotal);
        int porcentajeOcupacionRedondeado = (int) Math.round(porcentajeOcupacionReal);

        double distanciaDiagonalAntena = calcularDistanciaDiagonal(coordenadaXMetros, coordenadaYMetros);
        int desviacionJornadaMinutos   = Math.abs(minutosEstanciaTotal - JORNADA_BASE_MINUTOS);
        double tempSeguraAcotada       = Math.max(TEMP_MIN_SEGURA, Math.min(tempCalibrada, TEMP_MAX_SEGURA));

        int horasUptime    = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        int minutosUptime  = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        int segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        // Salida de informe de la persona actual
        System.out.println("\n========================================================================");
        System.out.println("             INFORME OFICIAL DE ACCESO EN VESTÍBULO                     ");
        System.out.println("========================================================================");
        System.out.printf("CENTRO:              %-30s | FECHA: %s%n", NOMBRE_CENTRO, fechaActual);
        System.out.printf("REGISTRO N.º:        #%05d | HORA EMISIÓN QR: %s%n", idUltimoFichaje, horaEmisionQr);
```

```
            if (accesoConcedido) {
                System.out.println("------------------------------------------------------------------------");
                System.out.println(">> VEREDICTO:         [ACCESO AUTORIZADO - TORNO ABIERTO]");
                System.out.printf(">> TOKEN QR EMITIDO:  %s%n", tokenResumenVestibulo);
                System.out.printf(">> TOKEN BARRAS:      %s%n", tokenFormatoBarras);
                System.out.printf(">> PUERTA NORTE LINK: %s%n", tokenResumenPuertaNorte);
            } else {
                System.out.println("------------------------------------------------------------------------");
                System.out.println(">> VEREDICTO:         [ACCESO DENEGADO - TORNO BLOQUEADO]");
                System.out.printf(">> MOTIVO RECHAZO:    %s%n", motivoDenegacion);
                System.out.println(">> PROTOCOLO:         Diríjase a la conserjería central para verificación manual.");
            }

            System.out.println("------------------------------------------------------------------------");
            System.out.printf("PERSONA:              %-30s | LONGITUD: %d%n", nombrePersonaLimpio, nombrePersonaLimpio.length());
            System.out.printf("DNI DESGLOSADO:       Número: %s | Letra: '%c'%n", numeroDniSolo, letraFinalDni);
            System.out.printf("ROL CLASIFICADO:      [%c] %-25s | Válido: %b%n", perfilNormalizado, descripcionRol, perfilReconocido);
            System.out.printf("ZONA ASIGNADA:        %s%n", zonaDesignada);
            System.out.printf("LÍMITE AUTORIZADO:    %d minutos (Tiempo previsto: %d min)%n", estanciaMaxPermitida, minutosEstanciaTotal);

            if (esRetrasoLectivo && accesoConcedido) {
                System.out.println(">> AVISO PUNTUALIDAD: [RETRASO LECTIVO REGISTRADO - Entrada posterior a las 08:15]");
            }

            System.out.println("------------------------------------------------------------------------");
            System.out.printf("HORARIO FORMAL:       Entrada %s | Salida %s (LocalTime)%n", tiempoEntrada, tiempoSalida);
            System.out.printf("PERMANENCIA EXACTA:   %03d minutos (Método modular calcularEstancia)%n", minutosEstanciaTotal);
            System.out.printf("DESVIACIÓN JORNADA:   %03d min respecto a jornada completa (480 min).%n", desviacionJornadaMinutos);
            System.out.println("------------------------------------------------------------------------");
            System.out.printf("OCUPACIÓN REDONDEADA: %d %% (Math.round) | Exacta: %6.2f %%%n", porcentajeOcupacionRedondeado, porcentajeOcupacionReal);
            System.out.printf("AFORO ACTUALIZADO:    %d plazas libres (Ocupación: %d personas)%n", aforoDisponible, personasEnCentro);
            System.out.printf("DISTANCIA ENFOQUE TV: %.2f metros (Método modular calcularDistancia)%n", distanciaDiagonalAntena);
            System.out.printf("ACTIVO:               %02dh %02dm %02ds (Sensor acotado: %.1f ºC)%n", horasUptime, minutosUptime, segundosUptime,
tempSeguraAcotada);
            System.out.printf("DIAGNÓSTICO TEST:     Patrón #%04d | Sensor NFC: Activo (%b)%n", codigoTestCalibracion, sensorNfcOperativo);
            System.out.printf("ESTADO INTEGRIDAD:    Datos correctos: %b (Try-Catch Shield Activo)%n", datosSintacticamenteCorrectos);
            System.out.printf("REGISTRO LOG:         %s%n", RUTA_LOGS);
            System.out.println("========================================================================");

            // ------------------------------------------------------------------
            // EVALUACIÓN DE CONTINUIDAD MEDIANTE VARIABLE CENTINELA
            // ------------------------------------------------------------------
            System.out.print("\n¿Desea registrar otro acceso en el terminal? (S = Sí / cualquier otra tecla = Cerrar sesión): ");
            String respuestaContinuar = teclado.nextLine().trim();
            sesionActiva = respuestaContinuar.equalsIgnoreCase("S");

        } while (sesionActiva);

        // ------------------------------------------------------------------
        // 5. INFORME DE AUDITORÍA CONSOLIDADA (TRAS SALIR DEL BUCLE)
        // ------------------------------------------------------------------
        System.out.println("\n========================================================================");
        System.out.println("          AUDITORÍA FINAL DE LA SESIÓN · IES EL CAMINÀS               ");
        System.out.println("========================================================================");
        System.out.printf("TOTAL FICHAJES PROCESADOS:    %d%n", totalFichajesProcesados);
        System.out.printf("ACCESOS AUTORIZADOS (TORNO):  %d (%.1f %%)%n", totalFichajesAutorizados,
                totalFichajesProcesados > 0 ? ((double) totalFichajesAutorizados / totalFichajesProcesados) * 100 : 0.0);
        System.out.printf("ACCESOS DENEGADOS (BLOQUEO):  %d (%.1f %%)%n", totalFichajesDenegados,
                totalFichajesProcesados > 0 ? ((double) totalFichajesDenegados / totalFichajesProcesados) * 100 : 0.0);
        System.out.printf("OCUPACIÓN FINAL DEL CAMPUS:   %d personas (Plazas disponibles: %d)%n", personasEnCentro, aforoDisponible);
        System.out.println("ESTADO DEL TERMINAL:          Servicio finalizado ordenadamente por el operador.");
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión, cada estudiante integrará la iteración continua en su proyecto asignado, preservando contadores entre vueltas y explorando pruebas automatizadas sin necesidad de desensamblar bytecode.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). BUCLE DE SERVICIO Y PERSISTENCIA DE SESIÓN

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa una estructura iterativa de repetición (`while` o `do-while`), definiendo acumuladores fuera del bucle para conservar el balance global al término de la ejecución:

- **Aventura conversacional.** Envuelve la secuencia de turnos en un bucle pre-test `while` condicionado a que los puntos de vida del héroe y del monstruo sean mayores que cero. Fuera del bucle, declara acumuladores para el recuento total de turnos disputados, el daño global infligido por el jugador y el daño total recibido. En cada vuelta, ejecuta la acción táctica y contraataque, y al expirar el bucle por derrota de algún contendiente, imprime un cuadro de honor con las estadísticas finales del combate.
- **Motor de recomendación.** Implementa un bucle post-test `do-while` que permita al analista evaluar múltiples ítems de forma continuada, consultando al final de cada análisis si desea procesar otro contenido mediante una respuesta afirmativa o negativa. Mantén variables acumuladoras antes del bucle para registrar el número de recomendaciones emitidas, el recuento de contenidos calificados como de alta afinidad y el promedio de compatibilidad obtenido en la sesión.
- **Simulador de físicas 2D.** Diseña un bucle de avance temporal continuo `while` que calcule el vuelo parabólico del proyectil mientras la altura vertical sea mayor o igual a cero y el tiempo no supere el límite de la simulación. En cada iteración, computa el avance incremental de la posición y velocidad actualizando un contador de impactos perimetrales y registrando la altura máxima alcanzada por el móvil a lo largo de toda la trayectoria.
- **Bóveda de contraseñas.** Implementa un bucle interactivo `do-while` de autenticación que restrinja el acceso a un máximo prefijado de tentativas (por ejemplo, tres intentos). Fuera del bucle, declara un contador de fallos acumulados y una bandera booleana de desbloqueo. En cada iteración, solicita el PIN numérico; si es erróneo, decrementa las tentativas restantes y, si el usuario agota sus opciones sin éxito, activa el bloqueo definitivo de la bóveda informando del incidente.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). BUCLE DE SANEAMIENTO DE ENTRADA (*INPUT CLEANSING LOOP*)

Uno de los problemas más frecuentes en aplicaciones de consola es que un fallo de formato provoca que el programa termine o que el flujo continúe con variables corruptas.

1. **El patrón de reintento local.** En lugar de abortar la iteración completa cuando un usuario introduce un dato erróneo, se utiliza un bucle `do-while` circunscrito exclusivamente a la lectura de ese campo, combinándolo con un bloque `try-catch` y una bandera local de validación.
2. **Implementación de saneamiento.** Diseña un bucle que solicite reiteradamente un valor numérico positivo hasta que el usuario teclee una cifra correcta, absorbiendo posibles `NumberFormatException` sin saltar al siguiente paso ni reiniciar el proceso global.
3. **Refactorización en tu proyecto.** Aplica este patrón en el dato interactivo más crítico de tu especialidad (la elección de acción en la aventura, la puntuación en el recomendador, el ángulo de lanzamiento en las físicas o el código secreto en la bóveda).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). PRUEBAS POR LOTES MEDIANTE REDIRECCIÓN DE ENTRADA (*BATCH STREAM FEEDING*)

En el desarrollo corporativo, los ingenieros no prueban los bucles tecleando manualmente los datos una y otra vez por consola. En su lugar, automatizan la entrada simulando la llegada de un lote masivo de usuarios mediante **redirección de flujos estándar (`stdin`)**.

1. **Construcción del archivo de casos de prueba.**
   Crea en la raíz del proyecto un archivo de texto llamado `lote_pruebas.txt` que contenga una batería completa de entradas simuladas para alimentar el bucle de tu aplicación (incluyendo entradas correctas, fallos tipográficos para probar el `try-catch`, valores límite y el carácter final de cierre de sesión). Cada línea del fichero representará una respuesta enviada a `teclado.nextLine()`.
2. **Inyección por tubería desde la terminal integrada de IntelliJ.**
   Ejecuta tu programa redirigiendo el archivo hacia la entrada estándar:
3. En terminales Unix / macOS / Git Bash:

```
java pr.src.ControlAccesoQR < lote_pruebas.txt
```

4. En terminales Windows PowerShell:

```
Get-Content lote_pruebas.txt | java pr.src.ControlAccesoQR
```

5. **Reto de auditoría y análisis.**
   Comprueba que el bucle procesa secuencialmente todas las solicitudes en milisegundos, que los acumuladores de sesión reflejan exactamente el número de casos inyectados en el fichero de texto y que el programa finaliza imprimiendo la auditoría final sin que el terminal quede bloqueado en una espera infinita de teclado (*hang state*). Documenta en un comentario de la clase los resultados obtenidos en el procesamiento automatizado del lote.

4.2.2 Día 29 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **martes 3 de noviembre de 2026**. En la sede de **AzaharTech**, el equipo de desarrollo analiza la telemetría recogida durante la primera mañana de pruebas de la versión `v2.5` en el **IES El Caminàs**.

El bucle `do-while` implantado ayer resolvió la continuidad del servicio: el terminal ya no se apaga tras cada fichaje y mantiene los contadores vivos en memoria. Sin embargo, **Pau Ferrer** proyecta un vídeo grabado a las 07:58 h en el torno de la entrada principal que evidencia un nuevo reto:

*«Fijaos en lo que ocurre a primera hora: no llega un único usuario aislado, sino una **cola continua de estudiantes** esperando para pasar su tarjeta QR.*

En la versión `v2.5`, tras cada lectura individual, el terminal se detiene y pregunta al operador si desea registrar otro acceso. En plena hora punta, tener que confirmar manualmente cada paso vuelve a generar retenciones inasumibles. Pero hay un peligro todavía mayor:

*Si el aforo libre del centro está a punto de agotarse (por ejemplo, quedan solo 3 plazas) y en la puerta hay una fila de 15 personas, nuestro sistema no debe procesar a ciegas. Necesitamos un **modo de procesamiento de colas por ráfaga (Burst Mode)** gobernado por un bucle pre-test `while`: Mientras sigan quedando personas en la fila física Y el centro disponga de aforo (`personasEnCola > 0 && aforoDisponible > 0`), el terminal valida y franquea el paso de forma fluida. En el instante exacto en que las plazas se agotan, el bucle **debe abortar antes de permitir el siguiente intento**, bloqueando la cola restante y activando el protocolo de espera».*

Alba Torres y **Laia Claramunt** asienten y concretan la pauta técnica:

*«Esa es la diferencia conceptual clave entre las estructuras de repetición. El `do-while` de ayer garantizaba que el servicio se ejecutara al menos una vez para atender al operador. El bucle `while` de hoy actúa como una **barrera de guardia previa**: comprueba las condiciones antes de dar cada paso.*

*Hoy aprenderemos a anidar un bucle pre-test `while` dentro del ciclo general de servicio, dominaremos las **condiciones compuestas de parada**, implementaremos la gestión de colas desatendidas y mediremos la telemetría de procesamiento por ráfaga».*

---

2. Fundamento teórico: el bucle pre-test `while` como estructura de guardia

A diferencia de las estructuras post-test, el bucle `while` evalúa su expresión booleana en la cabecera, **antes de ejecutar ninguna instrucción de su cuerpo**. Si la condición resulta falsa en la primera comprobación, el flujo desvía la ejecución inmediatamente y el interior del bucle no se ejecuta jamás.

```
flowchart TD
    Inicio([Llegada de lote de personas]) --> CondGuardia{"¿Condición de guardia?<br/><b>(personasEnCola > 0 && aforoDisponible > 0)</b>"}

    CondGuardia -- true --> Cuerpo["<b>Procesar siguiente en cola:</b><br/>• Validar credenciales y temperatura<br/>• Mutar contadores: aforoDisponible--<br/>• Reducir cola: personasEnCola--"]
    Cuerpo --> CondGuardia

    CondGuardia -- false --> Fin([<b>Fin de ráfaga:</b><br/>Cola vacía o Aforo agotado])
```

A. ANATOMÍA Y GARANTÍAS DEL BUCLE PRE-TEST

La sintaxis del bucle `while` responde al patrón de guardia condicional:

```
while (condicionDeContinuidad) {
    // Cuerpo del bucle: se ejecuta mientras la condición sea true
}
```

- **Evaluación previa estricta:** Si una ráfaga llega con `personasEnCola = 0` o con `aforoDisponible = 0`, el bucle no realiza ningún cálculo ni altera los contadores.
- **Garantía de terminación:** Para evitar bucles infinitos, el cuerpo del `while` debe contener al menos una sentencia que aproxime las variables de control hacia la condición de parada (en nuestro caso, el decremento `personasEnCola--` y `aforoDisponible--`).

B. EVALUACIÓN EN CORTOCIRCUITO EN LA CABECERA DEL BUCLE

Cuando enlazamos múltiples condiciones mediante el operador lógico `&&`, Java aplica la evaluación en cortocircuito:

```
while (personasEnCola > 0 && aforoDisponible > 0) { ... }
```

1. Si `personasEnCola > 0` es falso (la fila se ha vaciado), Java **no evalúa** el estado del aforo y sale del ciclo de inmediato.
2. Si `personasEnCola > 0` es verdadero pero `aforoDisponible > 0` es falso (el instituto se ha llenado), la condición global evalúa a falso y el bucle se detiene antes de registrar un acceso indebido.

C. COMPARATIVA DE INVARIANTES DE CONTROL

Criterio	Bucle de sesión (`do-while`)	Bucle de cola/ráfaga (`while`)
Rol en el sistema	Mantiene encendido el terminal desatendido	Procesa ráfagas de usuarios en espera
Punto de evaluación	Al final de la transacción (*post-test*)	Antes de procesar a cada persona (*pre-test*)
Comportamiento límite	Siempre procesa al menos un comando	Si no hay aforo o cola, no ejecuta nada (0 veces)
Gobernado por	Variable centinela del operador (`sesionActiva`)	Expresión compuesta de estado (`cola > 0 && aforo > 0`)

---

3. Evolución del caso guía a `ControlAccesoQR v2.6` (Java nativo)

Evolucionamos la versión `v2.5` a la **`v2.6`**. Mantenemos intactas todas las constantes corporativas, los métodos modulares y la captura con `try-catch`, incorporando: 1. Simulación de **cola de espera en vestíbulo** (`personasEnCola`). 2. Un bucle interno **`while`** que procesa la ráfaga de accesos de forma continua mientras haya personas esperando y queden plazas libres en el campus. 3. Detección automática de **parada anticipada por saturación de aforo** en plena ráfaga. 4. Actualización en tiempo real del balance de la cola y emisión de alerta si quedan usuarios bloqueados fuera del centro.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.5 (**`diff`**)

```
  do {
-     totalFichajesProcesados++;
-     // ... [Procesamiento exclusivo de un único usuario aislado] ...

+     // ========================================================================
+     // NOVEDAD DÍA 29: Captura de lote de personas en espera (cola física)
+     // ========================================================================
+     System.out.print("Personas esperando en la cola del torno (1 = individual, N = ráfaga): ");
+     int personasEnCola = Integer.parseInt(teclado.nextLine().trim());
+
+     // Bucle pre-test de guardia: se ejecuta mientras haya cola Y quede aforo libre
+     while (personasEnCola > 0 && aforoDisponible > 0) {
+         totalFichajesProcesados++;
+         personasEnCola--; // Una persona avanza desde la fila hacia el sensor
+
+         // ... [Lectura de credenciales, predicados y verificación térmica] ...
+
+         if (accesoConcedido) {
+             totalFichajesAutorizados++;
+             personasEnCentro++;
+             aforoDisponible--;
+             idUltimoFichaje++;
+         } else {
+             totalFichajesDenegados++;
+         }
+     }
+
+     // Alerta de parada anticipada si la cola no se pudo vaciar por falta de plazas
+     if (personasEnCola > 0 && aforoDisponible == 0) {
+         System.out.println(">> ALERTA TORNO: Aforo saturado. Personas retenidas en espera: " + personasEnCola);
+     }

      System.out.print("\n¿Desea registrar otro lote en el terminal? (S/N): ");
      sesionActiva = teclado.nextLine().trim().equalsIgnoreCase("S");
  } while (sesionActiva);
```

B. FOCO EN LA NOVEDAD: BUCLE **`while`** DE RÁFAGA Y CONDICIÓN DE GUARDIA

Observa cómo el bucle pre-test `while` se anida dentro del ciclo general de sesión para gobernar el flujo de la cola:

```java
// ------------------------------------------------------------------------
// Captura del volumen de la cola en el vestíbulo
// ------------------------------------------------------------------------
System.out.print("\nPersonas esperando en la fila del torno: ");
int personasEnCola = 0;
try {
    personasEnCola = Integer.parseInt(teclado.nextLine().trim());
} catch (NumberFormatException e) {
    System.err.println(">> Entrada inválida. Se asume 1 persona en espera.");
    personasEnCola = 1;
}

// ------------------------------------------------------------------------
// Bucle pre-test while: barrera de guardia por aforo y disponibilidad
// ------------------------------------------------------------------------
while (personasEnCola > 0 && aforoDisponible > 0) {
    totalFichajesProcesados++;
    personasEnCola--; // Decremento de la fila hacia el torno

    // [Captura y validación con métodos predicado...]
    if (accesoConcedido) {
        totalFichajesAutorizados++;
        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;
    } else {
        totalFichajesDenegados++;
    }

    System.out.printf(">> PASO REGISTRADO: #%05d | Quedan en cola: %d | Plazas libres: %d%n",
            idUltimoFichaje, personasEnCola, aforoDisponible);
}

// ------------------------------------------------------------------------
// Diagnóstico de parada de la ráfaga
// ------------------------------------------------------------------------
if (personasEnCola > 0 && aforoDisponible == 0) {
```

```
    System.out.println("----------------------------------------------------------------------");
    System.out.println(">> CORTE DE EMERGENCIA: Aforo completo al 100%.");
    System.out.printf(">> RETENCIÓN EN VESTÍBULO: %d personas no han podido acceder.%n", personasEnCola);
    System.out.println("----------------------------------------------------------------------");
}
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.6)

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.6:
 * Incorpora procesamiento de colas por ráfaga mediante bucle pre-test while (Sprint 3).
 * Anida la gestión de cola dentro del servicio continuo gobernado por do-while.
 * Aplica condiciones compuestas de parada anticipada por agotamiento de aforo.
 * Mantiene la persistencia de sesión, predicados modulares y captura con try-catch.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.6 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales de seguridad biométrica y capacidad (Sprint 3)
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control horario de puntualidad lectiva para estudiantes (Día 25)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
```

```
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: SESIÓN CONTINUA (DO-WHILE) CON COLA POR RÁFAGA (WHILE)
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo   = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
        int personasEnCentro = 1198;
        int aforoDisponible = 2; // Simulación con solo 2 plazas disponibles para verificar parada
        int idUltimoFichaje = 1042;

        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;

        boolean sesionActiva = true;
```

```
        System.out.println("========================================================================");
        System.out.println("   AZAHARTECH · CONTROL DE ACCESO DESATENDIDO - IES EL CAMINÀS        ");
        System.out.println("   Versión 2.6 Oficial (Sprint 3: Procesamiento de Colas con While)   ");
        System.out.println("========================================================================");

        // BUCLE EXTERNO DE SESIÓN DEL OPERADOR
        do {
            System.out.println("\n------------------------------------------------------------------------");
            System.out.printf("ESTADO ACTUAL: %d personas dentro | %d plazas disponibles%n", personasEnCentro, aforoDisponible);
            System.out.print("Introduce número de personas esperando en la fila del torno: ");

            int personasEnCola = 0;
            try {
                personasEnCola = Integer.parseInt(teclado.nextLine().trim());
                if (personasEnCola < 0) {
                    personasEnCola = 0;
                }
            } catch (NumberFormatException e) {
                System.err.println(">> Entrada inválida. Se procesará 1 persona por defecto.");
                personasEnCola = 1;
            }

            // BUCLE INTERNO PRE-TEST WHILE: PROCESAMIENTO DE LA COLA MIENTRAS QUEDE AFORO
            while (personasEnCola > 0 && aforoDisponible > 0) {
                totalFichajesProcesados++;
                personasEnCola--; // Avanza una persona de la fila

                LocalTime horaEmisionQr = LocalTime.now();

                int terminalId = 0;
                double tempVestibulo = 0.0;
                int segundosActividadTerminal = 0;
                double coordenadaXMetros = 0.0;
                double coordenadaYMetros = 0.0;

                String nombrePersonaBruto = "", nombrePersonaLimpio = "";
                String dniPersonaBruto = "", dniPersonaLimpio = "";
                char perfilPersonaBruto = ' ';
                char perfilNormalizado = ' ';
                boolean esLetraValida = false;
                boolean esEntrada = true;

                int horaEntrada = 8, minutoEntrada = 10;
                int horaSalida = 14, minutoSalida = 0;

                LocalTime tiempoEntrada = LocalTime.of(horaEntrada, minutoEntrada);
                LocalTime tiempoSalida  = LocalTime.of(horaSalida, minutoSalida);

                boolean datosSintacticamenteCorrectos = true;
                String motivoDenegacion = "";

                System.out.printf("%n>>> PROCESANDO ACCESO EN COLA [Restantes en espera: %d] <<<%n", personasEnCola);

                try {
                    System.out.print("ID Terminal (ej. 101): ");
                    terminalId = Integer.parseInt(teclado.nextLine().trim());

                    System.out.print("Temperatura sensor (ºC): ");
                    tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

                    System.out.print("DNI de la persona: ");
                    dniPersonaBruto = teclado.nextLine();

                    System.out.print("Nombre completo: ");
                    nombrePersonaBruto = teclado.nextLine();

                    dniPersonaLimpio = dniPersonaBruto.trim().toUpperCase();
                    nombrePersonaLimpio = nombrePersonaBruto.trim().toUpperCase();

                    System.out.print("Perfil (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
                    String lineaPerfil = teclado.nextLine().trim();
                    perfilPersonaBruto = lineaPerfil.isEmpty() ? '?' : lineaPerfil.charAt(0);
                    perfilNormalizado = Character.toUpperCase(perfilPersonaBruto);
                    esLetraValida = Character.isLetter(perfilNormalizado);

                } catch (NumberFormatException e) {
                    datosSintacticamenteCorrectos = false;
                    motivoDenegacion = "Error de formato en datos numéricos.";
                }

                int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);
                int aforoTotal = personasEnCentro + aforoDisponible;

                boolean esLongitudDniCorrecta = esDocumentoValido(dniPersonaLimpio);
                boolean perfilReconocido = esPerfilAutorizado(perfilNormalizado);
                String descripcionRol = obtenerDescripcionPerfil(perfilNormalizado);
                String zonaDesignada = determinarZonaAutorizada(perfilNormalizado);
                int estanciaMaxPermitida = obtenerEstanciaMaximaPorPerfil(perfilNormalizado);

                boolean esRetrasoLectivo = (perfilNormalizado == 'E') && tiempoEntrada.isAfter(HORA_LIMITE_PUNTUALIDAD);

                // DECISIÓN CONDICIONAL DEFENSIVA
                boolean accesoConcedido;

                if (!datosSintacticamenteCorrectos) {
                    accesoConcedido = false;
                    totalFichajesDenegados++;
                } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                    accesoConcedido = false;
```

```
                    motivoDenegacion = String.format("Temperatura anómala (%.1f ºC fuera de rango)", tempVestibulo);
                    totalFichajesDenegados++;
                } else if (aforoDisponible <= 0) {
                    accesoConcedido = false;
                    motivoDenegacion = "Aforo de recinto saturado.";
                    totalFichajesDenegados++;
                } else if (!esLongitudDniCorrecta) {
                    accesoConcedido = false;
                    motivoDenegacion = "DNI no reglamentario.";
                    totalFichajesDenegados++;
                } else if (!perfilReconocido) {
                    accesoConcedido = false;
                    motivoDenegacion = "Perfil no catalogado.";
                    totalFichajesDenegados++;
                } else {
                    accesoConcedido = true;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;
                }

                // Generación de identificadores de paso
                int codigoTokenVestibulo = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                String tokenResumenVestibulo = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoTokenVestibulo, idUltimoFichaje);

                // Salida simplificada durante ráfagas
                if (accesoConcedido) {
                    System.out.printf(">> PASO AUTORIZADO: Torno abierto para %s [Token: %s] | Plazas libres restantes: %d%n",
                            nombrePersonaLimpio, tokenResumenVestibulo, aforoDisponible);
                } else {
                    System.out.printf(">> PASO DENEGADO: Torno bloqueado para %s [Motivo: %s]%n",
                            nombrePersonaLimpio, motivoDenegacion);
                }
            } // FIN DEL BUCLE WHILE DE COLA

            // COMPROBACIÓN DE PARADA ANTICIPADA POR AFORO
            if (personasEnCola > 0 && aforoDisponible == 0) {
                System.out.println("\n--------------------------------------------------------------------");
                System.out.println(">> CORTE DE EMERGENCIA: Aforo completo del centro alcanzado.");
                System.out.printf(">> RETENCIÓN EN VESTÍBULO: Quedan %d personas en fila sin poder acceder.%n", personasEnCola);
                System.out.println("--------------------------------------------------------------------");
            }

            // Consulta de continuidad de sesión
            System.out.print("\n¿Desea mantener activo el terminal para nuevos accesos? (S = Sí / cualquier otra = Cerrar): ");
            sesionActiva = teclado.nextLine().trim().equalsIgnoreCase("S");

        } while (sesionActiva);

        // AUDITORÍA FINAL
        System.out.println("\n====================================================================");
        System.out.println("          AUDITORÍA FINAL CONSOLIDADA · IES EL CAMINÀS               ");
        System.out.println("====================================================================");
        System.out.printf("TOTAL FICHAJES ATENDIDOS:     %d%n", totalFichajesProcesados);
        System.out.printf("ACCESOS AUTORIZADOS:          %d (%.1f %%)%n", totalFichajesAutorizados,
                totalFichajesProcesados > 0 ? ((double) totalFichajesAutorizados / totalFichajesProcesados) * 100 : 0.0);
        System.out.printf("ACCESOS DENEGADOS:            %d (%.1f %%)%n", totalFichajesDenegados,
                totalFichajesProcesados > 0 ? ((double) totalFichajesDenegados / totalFichajesProcesados) * 100 : 0.0);
        System.out.printf("OCUPACIÓN FINAL REGISTRADA:   %d personas (Plazas libres: %d)%n", personasEnCentro, aforoDisponible);
        System.out.println("====================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión, cada alumno implementará un bucle pre-test `while` de guardia con condiciones compuestas en su proyecto asignado.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). BUCLE `while` DE GUARDIA EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa una estructura `while` cuya cabecera evalúe al menos dos condiciones compuestas mediante el operador `&&`:

- **Aventura conversacional.** Diseña un bucle de combate automático o ráfaga de asaltos que se ejecute mientras el héroe conserve puntos de salud Y le queden pociones o puntos de energía táctica en su inventario. En cada iteración, consume un recurso, aplica el efecto cinemático y actualiza el estado de los contendientes, garantizando que si el inventario se vacía antes de vencer, el bucle se detiene inmediatamente.
- **Motor de recomendación.** Implementa un bucle `while` que procese una cola de contenidos sugeridos mientras queden elementos pendientes de clasificar Y el saldo de créditos publicitarios asignado a la sesión sea superior a cero. Cada iteración evaluará la afinidad de un ítem y descontará su coste de procesamiento; si el presupuesto se agota, el bucle debe concluir dejando constancia de los contenidos restantes sin procesar.
- **Simulador de físicas 2D.** Modela un bucle `while` de avance balístico que mantenga la simulación activa mientras la energía cinética del cuerpo supere un umbral mínimo de movimiento Y la posición no haya rebasado los límites físicos del escenario. En cada vuelta, aplica la pérdida de energía por rozamiento; cuando la velocidad caiga por debajo del umbral crítico, el bucle debe detener el móvil de forma natural.
- **Bóveda de contraseñas.** Construye un bucle `while` que simule una auditoría de robustez o ataque de fuerza bruta controlado sobre una credencial, ejecutándose mientras no se descubra la combinación correcta Y el contador de intentos no sobrepase el umbral de bloqueo de seguridad. En cada ciclo, evalúa un patrón de prueba y descuenta los intentos restantes; si se alcanza el tope sin éxito, el bucle expira bloqueando el acceso.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). CORTOCIRCUITO Y ORDEN DE PRECEDENCIA EN LA GUARDIA

El orden de los operandos en la cabecera de un bucle `while` es determinante para el rendimiento y la seguridad del programa:

1. **La regla de evaluación defensiva:** Si una condición comprueba la validez previa de una variable (por ejemplo, que una referencia no sea nula o que un contador sea positivo), debe colocarse siempre **a la izquierda** del operador `&&`:

```
// Correcto: si elementosRestantes es 0, no se evalúa la llamada al método
while (elementosRestantes > 0 && consultarDisponibilidad()) { ... }
```

2. **Refactorización en tu proyecto:** Revisa la condición de tu bucle `while` y reorganiza las expresiones de forma que las comprobaciones aritméticas más rápidas y restrictivas se evalúen en primer lugar, impidiendo que se ejecuten llamadas a métodos costosos cuando la condición general ya es falsa.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). TELEMETRÍA Y BENCHMARKING DE BUCLES CON `System.nanoTime()`

En el desarrollo de software de alto rendimiento, los ingenieros miden con precisión de nanosegundos el impacto de las estructuras de repetición:

1. **Instrumentación del bucle:**
   Envuelve la ejecución de tu bucle `while` con tomas de tiempo de alta resolución utilizando la API nativa de Java:

```
long tiempoInicio = System.nanoTime();

// Ejecución del bucle while bajo prueba...

long tiempoFin = System.nanoTime();
long duracionTotalNs = tiempoFin - tiempoInicio;
double duracionMs = duracionTotalNs / 1_000_000.0;
```

2. **Prueba de escalabilidad:**
   Ejecuta tu programa configurando una cola o ráfaga de 100 iteraciones y luego una de 10.000 iteraciones.
3. **Reto de análisis:**
   Calcula en un informe por consola el tiempo medio consumido por cada iteración individual (nanosegundos por ciclo). Documenta en un comentario de tu clase cómo influye el compilador JIT (*Just-In-Time*) de la máquina virtual, explicando por qué las iteraciones intermedias y finales suelen ejecutarse a mayor velocidad que las primeras vueltas debido a la optimización dinámica del código nativo.

4.2.3 Día 30 - 2 sesiones

1. Caso guía en AzaharTech

Son las 15:00 h del **miércoles 4 de noviembre de 2026**. En la sala técnica de **AzaharTech**, el equipo de desarrollo se reúne tras una mañana intensa de pruebas de campo en el **IES El Caminàs**.

La incorporación del bucle `while` para ráfagas en la versión `v2.6` permitió al personal de conserjería absorber la cola de primera hora sin bloqueos manuales. Sin embargo, a medida que la jornada avanza, la operativa del centro cambia: a media mañana se registran visitas aisladas, a las 11:00 h jefatura de estudios solicita consultar la ocupación en tiempo real y a mediodía conserjería necesita verificar si ha habido incidencias térmicas antes del recreo.

Pau Ferrer muestra la consola a **Alba Torres** y **Laia Claramunt**:

*«Alba, Laia: el terminal ahora mismo fuerza un flujo rígido: pide una ráfaga, procesa y luego pregunta si se desea continuar. En la vida real, el conserje no siempre quiere fichar alumnos: A veces necesita **consultar el aforo disponible sin alterar los contadores**, otras veces necesita **ver los picos de temperatura detectados** y otras necesita procesar una entrada urgente.*

*Para que nuestro software sea un producto profesional terminado, debemos estructurar el terminal mediante una **consola interactiva gobernada por un menú de opciones**: El terminal debe mantenerse en un bucle `do-while` continuo presentando un abanico claro de acciones (fichaje individual, ráfaga de cola, monitor de telemetría y salida). Además, debemos introducir **algoritmos de agregación estadística en memoria**: calcular en tiempo real la temperatura media, registrar el pico térmico máximo y mínimo detectado y totalizar las horas de estancia autorizadas sin necesidad de guardar listas de datos».*

Alba Torres resume la meta pedagógica de la sesión:

*«Hoy combinamos todo lo aprendido en el sprint: integraremos una sentencia `switch` dentro de un bucle `do-while`, gobernaremos el ciclo con una variable centinela de apagado y programaremos **algoritmos de agregación numérica en tiempo real** (contadores, acumuladores y detección de extremos con `Math.max` y `Math.min`)».*

2. Fundamento teórico: el patrón de menú interactivo y algoritmos de agregación

El diseño de aplicaciones de terminal profesionales descansa sobre dos pilares algorítmicos fundamentales: la **máquina de estados por menú interactivo** y los **acumuladores de agregación estadística**.

```
flowchart TD
    Inicio([Inicio de Jornada]) --> Menu["<b>Menú Principal (do-while)</b><br/>1. Registro de acceso individual<br/>2. Procesar ráfaga de cola (while)<br/>3. Consultar telemetría y aforo<br/>4. Apagar terminal (Centinela)"]

    Menu --> Opcion{"switch (opcion)"}

    Opcion -- "Caso 1" --> Fichaje["<b>Fichaje Individual</b><br/>Valida credenciales y actualiza aforo"]
    Opcion -- "Caso 2" --> Rafaga["<b>Modo Ráfaga (while)</b><br/>Procesa cola mientras quede aforo"]
    Opcion -- "Caso 3" --> Telemetria["<b>Monitor en Vivo</b><br/>Imprime aforo, promedios y extremos"]
    Opcion -- "Caso 4" --> Apagado["<b>Centinela: sesionActiva = false</b><br/>Prepara el cierre ordenado"]
    Opcion -- "default" --> Invalido["<b>Aviso de error</b><br/>Opción no reconocida"]

    Fichaje --> Retorno{"¿sesionActiva?"}
    Rafaga --> Retorno
    Telemetria --> Retorno
    Invalido --> Retorno
    Apagado --> Retorno

    Retorno -- true --> Menu
    Retorno -- false --> Auditoria([Auditoría Final y Fin])
```

A. EL PATRÓN DE MENÚ INTERACTIVO (`do-while` + `switch`)

La arquitectura estándar de una aplicación de consola interactiva combina tres piezas: 1. **Bucle post-test `do-while`:** Garantiza que el menú se dibuje en pantalla al menos una vez y permanezca activo hasta que el usuario elija conscientemente la opción de salida. 2. **Lectura protegida:** La selección de opción se lee como texto y se convierte a entero dentro de un bloque `try-catch`, evitando que el terminal colapse si el operador introduce una letra. 3. **Despacho mediante `switch`:** Dirige el puntero de ejecución hacia la rutina solicitada mediante etiquetas `case` limpias y una rama `default` para capturar opciones fuera de rango.

B. ALGORITMOS DE AGREGACIÓN ESTADÍSTICA EN TIEMPO REAL

En aplicaciones embebidas o de alto rendimiento, no es necesario almacenar miles de registros en memoria para calcular métricas globales. Basta con actualizar **variables de acumulación paso a paso** en cada iteración del bucle:

Patrón algorítmico	Inicialización previa	Mutación en cada iteración válida	Significado técnico
Contador simple	`int total = 0;`	`total++;`	Recuento de eventos discretos
Acumulador de suma	`double suma = 0.0;`	`suma += valorActual;`	Suma aritmética de magnitudes
Media aritmética	No se guarda fija	`media = suma / total;`	Promedio en tiempo real (evitar división por cero)
Búsqueda de máximo	`double max = -Double.MAX_VALUE;`	`max = Math.max(max, val);`	Registro del pico más alto detectado
Búsqueda de mínimo	`double min = Double.MAX_VALUE;`	`min = Math.min(min, val);`	Registro del valor más bajo detectado

---

3. Evolución del caso guía a `ControlAccesoQR v2.7` (Java nativo)

Evolucionamos la versión `v2.6` a la **`v2.7`**. Mantenemos intactas todas las constantes corporativas, los métodos modulares y los predicados booleanos, incorporando: 1. Menú principal interactivo gobernado por `do-while` y despachado con `switch`. 2. Variables de agregación estadística de sesión: picos térmicos (`tempMaximaRegistrada`, `tempMinimaRegistrada`) y estancia acumulada (`minutosEstanciaAcumulados`). 3. Opción de monitorización en vivo para consultar el estado del campus sin consumir fichajes ni alterar el aforo. 4. Desconexión controlada mediante la opción 4 del menú.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.6 (**`diff`**)

```
  // ========================================================================
  // MÉTODO MAIN: Acumuladores estadísticos globales
  // ========================================================================
+ double tempMaximaRegistrada = -1.0;
+ double tempMinimaRegistrada = 100.0;
+ int minutosEstanciaAcumulados = 0;

  do {
+     // Menú interactivo de opciones
+     System.out.println("1. Registro de acceso individual");
+     System.out.println("2. Procesar ráfaga de cola en vestíbulo (while)");
+     System.out.println("3. Consultar telemetría y aforo en tiempo real");
+     System.out.println("4. Cerrar jornada y apagar terminal");
+     int opcionMenu = Integer.parseInt(teclado.nextLine().trim());
+
+     switch (opcionMenu) {
+         case 1:
+             // Fichaje individual y actualización de agregados
+             if (accesoConcedido) {
+                 tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
+                 tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
+                 minutosEstanciaAcumulados += minutosEstanciaTotal;
+             }
+             break;
+         case 2:
+             // Modo ráfaga con bucle while...
+             break;
+         case 3:
+             // Monitorización en vivo sin mutar aforo...
+             break;
+         case 4:
+             sesionActiva = false; // Desconexión ordenada
+             break;
+         default:
+             System.err.println("Opción no válida.");
+             break;
+     }
  } while (sesionActiva);
```

B. FOCO EN LA NOVEDAD: MENÚ DE CONSOLA Y ACTUALIZACIÓN DE AGREGADOS

Observa cómo se despachan las opciones y cómo los extremos estadísticos se actualizan de forma transparente utilizando `Math.max` y `Math.min`:

```
// ------------------------------------------------------------------------
// Despacho de operaciones del terminal
// ------------------------------------------------------------------------
switch (opcionMenu) {
    case 1:
        // [Lectura de credenciales y evaluación con métodos predicado...]
        if (accesoConcedido) {
            totalFichajesAutorizados++;
            personasEnCentro++;
            aforoDisponible--;
            idUltimoFichaje++;

            // Actualización de agregados estadísticos en memoria
            tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
            tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
            minutosEstanciaAcumulados += minutosEstanciaTotal;

            System.out.println(">> VEREDICTO: [ACCESO AUTORIZADO - TORNO ABIERTO]");
        } else {
            totalFichajesDenegados++;
            System.out.println(">> VEREDICTO: [ACCESO DENEGADO - TORNO BLOQUEADO]");
        }
        break;

    case 2:
        // Modo ráfaga: bucle while de cola
        while (personasEnCola > 0 && aforoDisponible > 0) {
            // [Procesamiento secuencial de la cola...]
        }
        break;

    case 3:
        // Telemetría en tiempo real: consulta no destructiva
        double estanciaMedia = totalFichajesAutorizados > 0
                ? (double) minutosEstanciaAcumulados / totalFichajesAutorizados : 0.0;
        System.out.printf("OCUPACIÓN: %d personas | PLAZAS LIBRES: %d%n", personasEnCentro, aforoDisponible);
        System.out.printf("RANGO TÉRMICO SESIÓN: Mín: %.1f ºC | Máx: %.1f ºC%n",
                tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada, Math.max(0.0, tempMaximaRegistrada));
        System.out.printf("ESTANCIA MEDIA PREVISTA: %.1f minutos%n", estanciaMedia);
        break;

    case 4:
        System.out.println(">> Iniciando protocolo de apagado del terminal...");
        sesionActiva = false; // Centinela que provoca la salida del do-while
        break;

    default:
        System.err.println(">> Código de operación no reconocido por el sistema.");
        break;
}
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.7)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.7:
 * Incorpora menú interactivo multipropósito mediante do-while y switch (Sprint 3).
 * Despliega algoritmos de agregación estadística en tiempo real:
 * - Detección dinámica de picos térmicos máximos y mínimos con Math.max/min.
 * - Acumulación y cálculo de medias de estancia sin consumo de colecciones.
 * - Monitor de telemetría en vivo no destructivo y cierre ordenado por centinela.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.7 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales biométricos y capacidad
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control de puntualidad lectiva
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
```

```
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: CONSOLA INTERACTIVA CON MENÚ Y AGREGADOS EN VIVO
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
        int personasEnCentro = 1195;
        int aforoDisponible = 5;
        int idUltimoFichaje = 1042;

        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;
```

```
        // Acumuladores para análisis estadístico (Día 30)
        double tempMaximaRegistrada = -1.0;
        double tempMinimaRegistrada = 100.0;
        int minutosEstanciaAcumulados = 0;

        boolean sesionActiva = true;

        System.out.println("=========================================================================");
        System.out.println("   AZAHARTECH · TERMINAL MULTIPROPÓSITO - IES EL CAMINÀS              ");
        System.out.println("   Versión 2.7 Oficial (Sprint 3: Menú Interactivo y Métricas en Vivo) ");
        System.out.println("=========================================================================");

        // BUCLE PRINCIPAL DE GESTIÓN (MENÚ DE COMANDOS)
        do {
            System.out.println("\n-------------------------------------------------------------------------");
            System.out.println("                    PANEL DE OPERACIONES DEL TORNO                    ");
            System.out.println("-------------------------------------------------------------------------");
            System.out.println("  [1] Registrar acceso individual interactivo");
            System.out.println("  [2] Procesar cola en ráfaga (Modo Burst con bucle while)");
            System.out.println("  [3] Consultar telemetría y métricas de ocupación en vivo");
            System.out.println("  [4] Cerrar jornada y desconectar terminal");
            System.out.print("Seleccione una operación (1-4): ");

            int opcionMenu = 0;
            try {
                opcionMenu = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                opcionMenu = -1; // Fuerza la entrada en el default del switch
            }

            switch (opcionMenu) {
                case 1:
                    // OPERACIÓN 1: FICHAJE INDIVIDUAL INTERACTIVO
                    totalFichajesProcesados++;
                    LocalTime horaEmisionQr = LocalTime.now();

                    int terminalId = 0;
                    double tempVestibulo = 0.0;
                    String nombrePersonaLimpio = "";
                    String dniPersonaLimpio = "";
                    char perfilNormalizado = ' ';
                    boolean datosSintacticamenteCorrectos = true;
                    String motivoDenegacion = "";

                    System.out.println("\n--- [MODO 1: REGISTRO INDIVIDUAL] ---");
                    try {
                        System.out.print("ID Terminal (ej. 101): ");
                        terminalId = Integer.parseInt(teclado.nextLine().trim());

                        System.out.print("Temperatura sensor (ºC): ");
                        tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

                        System.out.print("DNI del usuario: ");
                        dniPersonaLimpio = teclado.nextLine().trim().toUpperCase();

                        System.out.print("Nombre completo: ");
                        nombrePersonaLimpio = teclado.nextLine().trim().toUpperCase();

                        System.out.print("Perfil (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
                        String lineaPerfil = teclado.nextLine().trim();
                        perfilNormalizado = lineaPerfil.isEmpty() ? '?' : Character.toUpperCase(lineaPerfil.charAt(0));

                    } catch (NumberFormatException e) {
                        datosSintacticamenteCorrectos = false;
                        motivoDenegacion = "Error de formato en campos numéricos.";
                    }

                    LocalTime tiempoEntrada = LocalTime.of(8, 10);
                    LocalTime tiempoSalida = LocalTime.of(14, 0);
                    int minutosEstanciaTotal = calcularEstanciaMinutos(tiempoEntrada, tiempoSalida);

                    boolean accesoConcedido;
                    if (!datosSintacticamenteCorrectos) {
                        accesoConcedido = false;
                        totalFichajesDenegados++;
                    } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                        accesoConcedido = false;
                        motivoDenegacion = String.format("Temperatura anómala (%.1f ºC fuera de rango)", tempVestibulo);
                        totalFichajesDenegados++;
                    } else if (aforoDisponible <= 0) {
                        accesoConcedido = false;
                        motivoDenegacion = "Aforo de campus saturado.";
                        totalFichajesDenegados++;
                    } else if (!esDocumentoValido(dniPersonaLimpio)) {
                        accesoConcedido = false;
                        motivoDenegacion = "Documento irregular (longitud distinta de 9).";
                        totalFichajesDenegados++;
                    } else if (!esPerfilAutorizado(perfilNormalizado)) {
                        accesoConcedido = false;
                        motivoDenegacion = "Perfil no catalogado por el centro.";
                        totalFichajesDenegados++;
                    } else {
                        accesoConcedido = true;
                        totalFichajesAutorizados++;
                        personasEnCentro++;
                        aforoDisponible--;
                        idUltimoFichaje++;

                        // Actualización de agregados estadísticos en tiempo real
                        tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
```

```
                        tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
                        minutosEstanciaAcumulados += minutosEstanciaTotal;
                    }

                    int codigoToken = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                    String token = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoToken, idUltimoFichaje);

                    if (accesoConcedido) {
                        System.out.printf(">> RESULTADO: [ACCESO AUTORIZADO] Torno abierto para %s [Token: %s]%n",
                                nombrePersonaLimpio, token);
                    } else {
                        System.out.printf(">> RESULTADO: [ACCESO DENEGADO] Torno bloqueado. Causa: %s%n", motivoDenegacion);
                    }
                    break;

                case 2:
                    // OPERACIÓN 2: MODO RÁFAGA CON BUCLE WHILE
                    System.out.println("\n--- [MODO 2: RÁFAGA DE COLA EN VESTÍBULO] ---");
                    System.out.printf("Plazas libres en el centro: %d%n", aforoDisponible);
                    System.out.print("Número de personas esperando en la fila: ");
                    int personasEnCola = 0;
                    try {
                        personasEnCola = Integer.parseInt(teclado.nextLine().trim());
                    } catch (NumberFormatException e) {
                        personasEnCola = 0;
                    }

                    while (personasEnCola > 0 && aforoDisponible > 0) {
                        totalFichajesProcesados++;
                        totalFichajesAutorizados++;
                        personasEnCentro++;
                        aforoDisponible--;
                        idUltimoFichaje++;
                        personasEnCola--;

                        double tempSimulada = 36.4 + (generadorVestibulo.nextDouble() * 0.6);
                        tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempSimulada);
                        tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempSimulada);
                        minutosEstanciaAcumulados += 360;

                        System.out.printf("  -> Fichaje #%05d validado de forma automática. Plazas restantes: %d%n",
                                idUltimoFichaje, aforoDisponible);
                    }

                    if (personasEnCola > 0 && aforoDisponible == 0) {
                        System.out.printf(">> AVISO: Aforo saturado. Quedan %d personas retenidas en espera.%n", personasEnCola);
                    } else {
                        System.out.println(">> Ráfaga procesada completamente con éxito.");
                    }
                    break;

                case 3:
                    // OPERACIÓN 3: TELEMETRÍA Y AGREGADOS EN VIVO (NO DESTRUCTIVA)
                    System.out.println("\n--- [MODO 3: MONITOR DE TELEMETRÍA EN VIVO] ---");
                    int aforoTotal = personasEnCentro + aforoDisponible;
                    double porcentajeOcupacion = calcularPorcentajeOcupacion(personasEnCentro, aforoTotal);
                    double estanciaMedia = totalFichajesAutorizados > 0
                            ? ((double) minutosEstanciaAcumulados / totalFichajesAutorizados) : 0.0;

                    System.out.printf("OCUPACIÓN ACTUAL:         %d/%d personas (%6.2f %%)%n", personasEnCentro, aforoTotal, porcentajeOcupacion);
                    System.out.printf("PLAZAS LIBRES:            %d vacantes disponibles%n", aforoDisponible);
                    System.out.printf("FICHAJES ATENDIDOS:       %d totales (%d autorizados, %d denegados)%n",
                            totalFichajesProcesados, totalFichajesAutorizados, totalFichajesDenegados);
                    System.out.printf("PICO TÉRMICO MÁXIMO:      %.1f ºC%n", Math.max(0.0, tempMaximaRegistrada));
                    System.out.printf("PICO TÉRMICO MÍNIMO:      %.1f ºC%n", tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada);
                    System.out.printf("ESTANCIA PROMEDIO:        %.1f minutos (Total acumulado: %d min)%n",
                            estanciaMedia, minutosEstanciaAcumulados);
                    break;

                case 4:
                    // OPERACIÓN 4: APAGADO Y SALIDA
                    System.out.println("\n>> Cerrando operaciones del terminal de accesos...");
                    sesionActiva = false; // Mutación del centinela que rompe el do-while
                    break;

                default:
                    System.err.println(">> Error: Opción numérica no contemplada en el menú. Seleccione entre 1 y 4.");
                    break;
            }

        } while (sesionActiva);

        // AUDITORÍA FINAL AL APAGAR
        System.out.println("\n======================================================================");
        System.out.println("          AUDITORÍA FINAL DE LA JORNADA · IES EL CAMINÀS              ");
        System.out.println("======================================================================");
        System.out.printf("TOTAL OPERACIONES GESTIONADAS: %d%n", totalFichajesProcesados);
        System.out.printf("ACCESOS AUTORIZADOS (TORNO):   %d%n", totalFichajesAutorizados);
        System.out.printf("ACCESOS DENEGADOS (BLOQUEO):   %d%n", totalFichajesDenegados);
        System.out.printf("OCUPACIÓN FINAL DEL RECINTO:   %d personas%n", personasEnCentro);
        System.out.println("ESTADO DEL DISPOSITIVO:        Terminal desconectado de forma segura.");
        System.out.println("======================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión práctica, cada estudiante integrará el patrón de consola interactiva con menú y agregados estadísticos dinámicos en su proyecto técnico.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). MENÚ INTERACTIVO Y ACUMULADORES EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa una consola interactiva estructurada con un bucle `do-while` y despacho mediante `switch`, incorporando acumuladores fuera del ciclo:

- **Aventura conversacional.** Construye un menú de combate interactivo con opciones tácticas (atacar, defender, usar objeto, consultar estado del héroe y huir). Mantén variables acumuladoras fuera del bucle para registrar el daño total causado, el número de asaltos disputados y el valor del golpe más contundente asestado durante la partida mediante `Math.max`.
- **Motor de recomendación.** Diseña un menú de gestión de catálogo (valorar nuevo ítem, procesar recomendaciones en lote, ver estadísticas de afinidad media y salir). En la opción estadística, calcula y muestra el promedio acumulado de compatibilidad y los picos máximos y mínimos de afinidad detectados a lo largo de las valoraciones de la sesión.
- **Simulador de físicas 2D.** Estructura un panel de control cinemático (lanzar nuevo móvil, simular caída con ráfaga temporal `while`, consultar telemetría balística y finalizar). Registra en variables de sesión la altura máxima alcanzada por los proyectiles y la velocidad de impacto más elevada registrada entre todos los lanzamientos.
- **Bóveda de contraseñas.** Implementa una consola administrativa de seguridad (comprobar clave individual, auditar lote de contraseñas en ráfaga, consultar métricas de robustez y bloquear bóveda). Mantén acumuladores que computen la longitud media de las claves evaluadas y el recuento de credenciales categorizadas como de alta criticidad.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). BÚSQUEDA DE EXTREMOS Y CÁLCULO DE MEDIAS SIN ARRAYS

El cálculo de magnitudes estadísticas sin utilizar estructuras de almacenamiento dinámicas requiere gestionar cuidadosamente los valores frontera:

1. **La trampa de la inicialización de extremos:**
   Si inicializamos `min = 0.0`, cualquier número positivo evaluado posteriormente será mayor que cero, por lo que el mínimo jamás se actualizará.
   Solución estándar: Inicializar la variable de mínimo con el valor más grande representable (`Double.MAX_VALUE`) y la de máximo con el más pequeño (`-Double.MAX_VALUE`), o inicializarlas con el valor exacto de la primera iteración válida.
2. **Prevención de división por cero:**
   Al calcular promedios (`media = suma / total`), comprueba siempre mediante el operador ternario que el contador de eventos sea estrictamente superior a cero (`total > 0 ? suma / total : 0.0`) para evitar anomalías como `NaN` (*Not a Number*).
3. **Refactorización en tu proyecto:** Revisa el cálculo de la media y de los valores pico de tu proyecto aplicando estas directrices defensivas.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). MATRIZ DE TRANSICIÓN DE ESTADOS Y PRUEBA DE CAJA BLANCA

En lugar de verificar la aplicación introduciendo datos al azar, diseña una **Matriz de Transición de Estados** para auditar que el menú de tu programa no corrompe la memoria ante secuencias operativas complejas:

1. **Diseño de la batería de transiciones:**
   En un comentario al final de tu clase, define una tabla de 5 pasos consecutivos que ponga a prueba el ciclo del menú:
2. *Paso 1:* Ejecutar una consulta estadística (opción 3) sin haber introducido ningún dato previo (verificar que no arroje división por cero).
3. *Paso 2:* Introducir un código de menú inexistente (por ejemplo, `opción = 99`) o una cadena de texto (verificar que el `default` y el `try-catch` mantengan vivo el bucle).
4. *Paso 3:* Ejecutar dos operaciones válidas consecutivas (opciones 1 y 2) comprobando que los acumuladores de picos máximos y sumas muten correctamente.
5. *Paso 4:* Volver a consultar la telemetría (opción 3) para contrastar que las métricas reflejen exactamente las operaciones previas.
6. *Paso 5:* Introducir el comando de salida (opción 4) y confirmar la desconexión limpia.
7. **Ejecución y certificación:**
   Ejecuta la secuencia en la consola de IntelliJ y documenta en el comentario si el comportamiento del programa superó la prueba de transición de estados sin fugas de control ni reinicios anómalos de las variables de sesión.

4.2.4 Día 31 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **jueves 5 de noviembre de 2026**. Con la sesión de hoy, los equipos de desarrollo de **AzaharTech** culminan la segunda semana del **Sprint 3: «El laberinto de la lógica»**. El terminal `v2.7` dispone de un menú consolidado que atiende fichajes individuales, ráfagas con `while` y métricas estadísticas en tiempo real sin arrays.

Sin embargo, durante la reunión de seguimiento con el equipo directivo del **IES El Caminàs**, el coordinador de seguridad del instituto plantea un escenario crítico:

*«El sistema funciona con fluidez para registrar entradas por la mañana, pero un torno de acceso es **bidireccional**: a las 14:00 h centenares de personas abandonan las instalaciones a la vez.*

*Hasta hoy, el software solo sabía sumar personas al campus (`personasEnCentro++`). Si alguien sale del instituto, el sistema debe registrar el sentido de la marcha, **reducir la ocupación y liberar plazas en el aforo** (`aforoDisponible++`), vigilando estrictamente que el contador nunca caiga por debajo de cero (*underflow*).*

*Además, la normativa autonómica exige que el terminal cuente con un **protocolo de evacuación de emergencia**: si se activa la alarma de incendios, el sistema debe suspender los fichajes ordinarios y ejecutar un vaciado progresivo del recinto mediante un bucle `while (personasEnCentro > 0)` que supervise la salida completa de los ocupantes hasta certificar el desalojo total del centro».*

Alba Torres y **Laia Claramunt** recogen el guante y marcan el objetivo del día:

*«Hoy cerramos la Semana 8 dominando los **patrones avanzados de control por centinela y banderas de estado (State Flags)**.*

*Aprenderemos a diferenciar entre un **valor centinela de escape** y una **bandera lógica de control**, programaremos bucles de vaciado condicional con protección ante desbordamientos negativos y dotaremos al terminal de un control bidireccional seguro antes de abordar el bucle `for` la próxima semana».*

---

2. Fundamento teórico: centinelas de escape, banderas de estado y bucles de vaciado

En ingeniería de software, gobernar la detención de un bucle requiere seleccionar con precisión el mecanismo de control en función de la naturaleza del evento.

```
flowchart TD
    Inicio([Inicio de Operación]) --> Seleccion{"Sentido del paso"}

    Seleccion -- "Entrada (E)" --> ValEntrada{"¿Aforo disponible?<br/>(aforoDisponible > 0)"}
    ValEntrada -- true --> Suma["personasEnCentro++<br/>aforoDisponible--"]
    ValEntrada -- false --> BloqueoEntrada["Torno bloqueado: Aforo saturado"]

    Seleccion -- "Salida (S)" --> ValSalida{"¿Hay ocupantes?<br/>(personasEnCentro > 0)"}
    ValSalida -- true --> Resta["personasEnCentro--<br/>aforoDisponible++"]
    ValSalida -- false --> BloqueoSalida["Torno bloqueado: Recinto vacío (Underflow guard)"]

    Seleccion -- "Evacuación (Simulacro)" --> BucleEvac{"while (personasEnCentro > 0)"}
    BucleEvac -- true --> Desalojo["Desalojar bloque de personas<br/>personasEnCentro -= lote<br/>aforoDisponible += lote"]
    Desalojo --> BucleEvac
    BucleEvac -- false --> Certificado([Certificado de Centro Evacuado])
```

A. VALORES CENTINELA FRENTE A BANDERAS DE ESTADO (*FLAGS*)

Aunque ambos mecanismos controlan la continuidad de las estructuras de repetición, responden a conceptos distintos:

Mecanismo	Definición técnica	Tipo de dato típico	Caso de uso en AzaharTech
Valor Centinela (*Sentinel Value*)	Dato especial fuera del rango de negocio que señala el fin de una secuencia	`String` (`"FIN"`), `int` (`-1`)	Teclear DNI `"SALIR"` para interrumpir una cola interactiva
Bandera de Estado (*State Flag*)	Variable lógica que almacena si se ha producido una condición crítica	`boolean` (`emergenciaActiva`)	Bandera que detiene las entradas si salta la alarma de incendios
Invariante de Bucle	Condición matemática que debe cumplirse antes, durante y después de iterar	Expresión booleana compuesta	Garantizar siempre que $0 \le \text{personasEnCentro} \le \text{AFORO_MAXIMO}$

B. PROTECCIÓN ANTE SUBDESBORDAMIENTO (*UNDERFLOW GUARD*)

Al decrementar contadores en un bucle de salida, es imprescindible aplicar una guardia condicional previa. Si se permite restar cuando `personasEnCentro == 0`, el sistema incurre en una inconsistencia física (número negativo de ocupantes dentro de un edificio). Toda operación de salida debe condicionarse estrictamente a `personasEnCentro > 0`.

C. BUCLES DE DRENAJE PROGRESIVO (*DRAINING LOOPS*)

Para modelar procesos de descarga (como vaciar una cola de impresión, purgar un búfer de red o evacuar un edificio), se emplea el patrón de **bucle de drenaje**:

```
while (personasEnCentro > 0) {
    int loteSalida = Math.min(personasEnCentro, CAPACIDAD_PUERTA);
    personasEnCentro -= loteSalida;
    aforoDisponible += loteSalida;
    // Registro de progreso...
}
```

El uso de `Math.min` garantiza que en la última iteración no se resten más personas de las que realmente quedan en el interior del recinto.

---

3. Evolución del caso guía a `ControlAccesoQR v2.8` (Java nativo)

Evolucionamos la versión `v2.7` a la **`v2.8`**, completando las operaciones de la Semana 8. Mantenemos todas las constantes inmutables, la captura con `try-catch` y los métodos predicado, incorporando: 1. Control bidireccional en el fichaje individual: selector de sentido del paso (`'E'` = Entrada, `'S'` = Salida) con protección ante *underflow*. 2. Opción 4 del menú: **Protocolo de**

evacuación de emergencia gobernado por un bucle `while` de vaciado progresivo. 3. Actualización de agregados en ambos sentidos: las salidas liberan aforo y computan en el histórico de tránsitos. 4. Desconexión del terminal reubicada en la opción 5.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.7 (`diff`)

```
  // ========================================================================
  // MENÚ DE OPERACIONES: Opción de evacuación y sentido de paso
  // ========================================================================
- System.out.println("  [4] Cerrar jornada y apagar terminal");
+ System.out.println("  [4] Activar protocolo de evacuación / desalojo rápido (while)");
+ System.out.println("  [5] Cerrar jornada y apagar terminal");

  switch (opcionMenu) {
      case 1:
          // Captura de sentido de paso ('E' / 'S')
+         System.out.print("Sentido del paso (E = Entrada, S = Salida): ");
+         char sentido = teclado.nextLine().trim().toUpperCase().charAt(0);
+         if (sentido == 'S') {
+             if (personasEnCentro > 0) {
+                 personasEnCentro--;
+                 aforoDisponible++;
+                 System.out.println(">> SALIDA REGISTRADA: Torno desbloqueado hacia el exterior.");
+             } else {
+                 System.out.println(">> BLOQUEO: No constan personas dentro del centro.");
+             }
+         } else {
+             // [Lógica previa de entrada y comprobación de aforo disponible...]
+         }
          break;

+     case 4:
+         // PROTOCOLO DE EVACUACIÓN CON BUCLE WHILE DE DRENAJE
+         System.out.println(">> ALERTA: Iniciando desalojo ordenado de emergencia...");
+         while (personasEnCentro > 0) {
+             int desalojadosLote = Math.min(personasEnCentro, 250); // Salida en oleadas
+             personasEnCentro -= desalojadosLote;
+             aforoDisponible += desalojadosLote;
+             System.out.printf("  -> Evacuadas %d personas. Quedan dentro: %d%n", desalojadosLote, personasEnCentro);
+         }
+         System.out.println(">> EVACUACIÓN COMPLETADA: Recinto 100% desalojado.");
+         break;

-     case 4: sesionActiva = false; break;
+     case 5: sesionActiva = false; break;
```

B. FOCO EN LA NOVEDAD: TRÁNSITO BIDIRECCIONAL Y VACIADO DE EMERGENCIA

Observa cómo se gestiona el sentido de paso y cómo el bucle `while` de evacuación garantiza matemáticamente que el campus quede vacío sin valores negativos:

```
// ------------------------------------------------------------------------
// Gestión bidireccional en el fichaje individual
// ------------------------------------------------------------------------
System.out.print("Sentido del tránsito (E = Entrada, S = Salida): ");
String lineaSentido = teclado.nextLine().trim().toUpperCase();
char sentidoTransito = lineaSentido.isEmpty() ? 'E' : lineaSentido.charAt(0);

if (sentidoTransito == 'S') {
    // Protección de subdesbordamiento: no se puede salir si el centro está vacío
    if (personasEnCentro > 0) {
        personasEnCentro--;
        aforoDisponible++;
        totalFichajesAutorizados++;
        System.out.printf(">> SALIDA AUTORIZADA: Buen viaje, %s. Plazas libres: %d%n",
                nombrePersonaLimpio, aforoDisponible);
    } else {
        totalFichajesDenegados++;
        System.err.println(">> ALERTA DE SEGURIDAD: Imposible registrar salida; el recinto ya figura vacío.");
    }
} else {
    // Proceso ordinario de entrada (verificación de aforo y temperatura...)
}

// ------------------------------------------------------------------------
// Operación 4: Bucle while de drenaje para evacuación de emergencia
// ------------------------------------------------------------------------
case 4:
    System.out.println("\n--- [PROTOCOLO DE EMERGENCIA: EVACUACIÓN GENERAL] ---");
    if (personasEnCentro == 0) {
        System.out.println(">> El campus ya se encuentra completamente vacío.");
    } else {
        int loteEvacuacion = 300; // Capacidad de desagüe simultánea de las puertas de emergencia
        int oleada = 1;

        while (personasEnCentro > 0) {
            int personasQueSalen = Math.min(personasEnCentro, loteEvacuacion);
            personasEnCentro -= personasQueSalen;
            aforoDisponible += personasQueSalen;
```

```
            System.out.printf("  -> Oleada #%d: Evacuadas %d personas por salidas de emergencia (Restantes: %d)%n",
                    oleada, personasQueSalen, personasEnCentro);
            oleada++;
        }
        System.out.println(">> CERTIFICACIÓN DE SEGURIDAD: Campus 100% desalojado. Aforo libre restaurado.");
    }
    break;
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.8)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.8:
 * Cierre de la Semana 8 (Estructuras de repetición iterativas y centinelas).
 * Incorpora tránsito bidireccional (Entrada/Salida) con protección ante underflow.
 * Despliega protocolo de evacuación de emergencia mediante bucle while de drenaje.
 * Mantiene consola interactiva, acumuladores estadísticos y blindaje try-catch.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.8 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales biométricos y capacidad
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control de puntualidad lectiva
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
```

```
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: CONSOLA BIDIRECCIONAL Y PROTOCOLO DE EVACUACIÓN
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
        int personasEnCentro = 1190;
        int aforoDisponible = 10;
        int idUltimoFichaje = 1042;

        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;

        // Acumuladores estadísticos
```

```
        double tempMaximaRegistrada = -1.0;
        double tempMinimaRegistrada = 100.0;
        int minutosEstanciaAcumulados = 0;

        boolean sesionActiva = true;

        System.out.println("========================================================================");
        System.out.println("   AZAHARTECH · CONTROL BIDIRECCIONAL DE ACCESOS - IES EL CAMINÀS    ");
        System.out.println("   Versión 2.8 Oficial (Sprint 3: Tránsito Bidireccional y Evacuación) ");
        System.out.println("========================================================================");

        // BUCLE PRINCIPAL DE GESTIÓN (MENÚ DE OPERACIONES)
        do {
            System.out.println("\n------------------------------------------------------------------------");
            System.out.println("                    PANEL DE OPERACIONES DEL TORNO                     ");
            System.out.println("------------------------------------------------------------------------");
            System.out.println("  [1] Registrar tránsito individual (Entrada / Salida)");
            System.out.println("  [2] Procesar cola en ráfaga (Modo Burst con bucle while)");
            System.out.println("  [3] Consultar telemetría y métricas de ocupación en vivo");
            System.out.println("  [4] Activar protocolo de evacuación de emergencia (Draining while)");
            System.out.println("  [5] Cerrar jornada y apagar terminal");
            System.out.print("Seleccione una operación (1-5): ");

            int opcionMenu = 0;
            try {
                opcionMenu = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                opcionMenu = -1;
            }

            switch (opcionMenu) {
                case 1:
                    // OPERACIÓN 1: TRÁNSITO INDIVIDUAL BIDIRECCIONAL
                    totalFichajesProcesados++;

                    System.out.println("\n--- [MODO 1: TRÁNSITO INDIVIDUAL] ---");
                    System.out.print("Sentido del tránsito (E = Entrada, S = Salida): ");
                    String lineaSentido = teclado.nextLine().trim().toUpperCase();
                    char sentidoTransito = lineaSentido.isEmpty() ? 'E' : lineaSentido.charAt(0);

                    System.out.print("Nombre completo: ");
                    String nombrePersonaLimpio = teclado.nextLine().trim().toUpperCase();

                    if (sentidoTransito == 'S') {
                        // SALIDA: Guardia de subdesbordamiento (Underflow Guard)
                        if (personasEnCentro > 0) {
                            personasEnCentro--;
                            aforoDisponible++;
                            totalFichajesAutorizados++;
                            idUltimoFichaje++;

                            System.out.println(">> VEREDICTO: [SALIDA AUTORIZADA - TORNO EXTERIOR ABIERTO]");
                            System.out.printf(">> PASO REGISTRADO: Hasta pronto, %s. Ocupación campus: %d%n",
                                    nombrePersonaLimpio, personasEnCentro);
                        } else {
                            totalFichajesDenegados++;
                            System.err.println(">> ALERTA: Imposible autorizar salida; no constan ocupantes dentro del centro.");
                        }
                    } else {
                        // ENTRADA: Validación biométrica y de aforo
                        int terminalId = 0;
                        double tempVestibulo = 0.0;
                        String dniPersonaLimpio = "";
                        char perfilNormalizado = ' ';
                        boolean datosSintacticamenteCorrectos = true;
                        String motivoDenegacion = "";

                        try {
                            System.out.print("ID Terminal (ej. 101): ");
                            terminalId = Integer.parseInt(teclado.nextLine().trim());

                            System.out.print("Temperatura sensor (ºC): ");
                            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());

                            System.out.print("DNI del usuario: ");
                            dniPersonaLimpio = teclado.nextLine().trim().toUpperCase();

                            System.out.print("Perfil (E=Estudiante, D=Docente, V=Visita, A=Admin): ");
                            String lineaPerfil = teclado.nextLine().trim();
                            perfilNormalizado = lineaPerfil.isEmpty() ? '?' : Character.toUpperCase(lineaPerfil.charAt(0));

                        } catch (NumberFormatException e) {
                            datosSintacticamenteCorrectos = false;
                            motivoDenegacion = "Error de formato numérico en los datos introducidos.";
                        }

                        boolean accesoConcedido;
                        if (!datosSintacticamenteCorrectos) {
                            accesoConcedido = false;
                            totalFichajesDenegados++;
                        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                            accesoConcedido = false;
                            motivoDenegacion = String.format("Temperatura anómala (%.1f ºC fuera de rango)", tempVestibulo);
                            totalFichajesDenegados++;
                        } else if (aforoDisponible <= 0) {
                            accesoConcedido = false;
                            motivoDenegacion = "Aforo de campus saturado.";
                            totalFichajesDenegados++;
                        } else if (!esDocumentoValido(dniPersonaLimpio)) {
                            accesoConcedido = false;
```

```
                    motivoDenegacion = "Documento irregular (longitud no reglamentaria de 9).";
                    totalFichajesDenegados++;
                } else if (!esPerfilAutorizado(perfilNormalizado)) {
                    accesoConcedido = false;
                    motivoDenegacion = "Perfil no catalogado por el centro.";
                    totalFichajesDenegados++;
                } else {
                    accesoConcedido = true;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;

                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
                    minutosEstanciaAcumulados += 360;
                }

                int codigoToken = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                String token = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoToken, idUltimoFichaje);

                if (accesoConcedido) {
                    System.out.printf(">> VEREDICTO: [ENTRADA AUTORIZADA] Torno desbloqueado para %s [Token: %s]%n",
                            nombrePersonaLimpio, token);
                } else {
                    System.out.printf(">> VEREDICTO: [ENTRADA DENEGADA] Torno bloqueado. Causa: %s%n", motivoDenegacion);
                }
            }
            break;

        case 2:
            // OPERACIÓN 2: MODO RÁFAGA CON BUCLE WHILE
            System.out.println("\n--- [MODO 2: RÁFAGA DE COLA EN VESTÍBULO] ---");
            System.out.printf("Plazas libres disponibles: %d%n", aforoDisponible);
            System.out.print("Número de personas esperando en la fila: ");
            int personasEnCola = 0;
            try {
                personasEnCola = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                personasEnCola = 0;
            }

            while (personasEnCola > 0 && aforoDisponible > 0) {
                totalFichajesProcesados++;
                totalFichajesAutorizados++;
                personasEnCentro++;
                aforoDisponible--;
                idUltimoFichaje++;
                personasEnCola--;

                double tempSimulada = 36.3 + (generadorVestibulo.nextDouble() * 0.7);
                tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempSimulada);
                tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempSimulada);
                minutosEstanciaAcumulados += 360;

                System.out.printf("  -> Fichaje #%05d validado en ráfaga. Plazas restantes: %d%n",
                        idUltimoFichaje, aforoDisponible);
            }

            if (personasEnCola > 0 && aforoDisponible == 0) {
                System.out.printf(">> AVISO: Aforo saturado. %d personas permanecen retenidas en espera.%n", personasEnCola);
            } else {
                System.out.println(">> Ráfaga completada con éxito.");
            }
            break;

        case 3:
            // OPERACIÓN 3: TELEMETRÍA Y AGREGADOS EN VIVO
            System.out.println("\n--- [MODO 3: MONITOR DE TELEMETRÍA EN VIVO] ---");
            int aforoTotal = personasEnCentro + aforoDisponible;
            double porcentajeOcupacion = calcularPorcentajeOcupacion(personasEnCentro, aforoTotal);
            double estanciaMedia = totalFichajesAutorizados > 0
                    ? ((double) minutosEstanciaAcumulados / totalFichajesAutorizados) : 0.0;

            System.out.printf("OCUPACIÓN ACTUAL:         %d/%d personas (%6.2f %%)%n", personasEnCentro, aforoTotal, porcentajeOcupacion);
            System.out.printf("PLAZAS LIBRES:            %d vacantes disponibles%n", aforoDisponible);
            System.out.printf("TRANSACCIONES TOTALES:    %d registros (%d autorizados, %d denegados)%n",
                    totalFichajesProcesados, totalFichajesAutorizados, totalFichajesDenegados);
            System.out.printf("PICO TÉRMICO MÁXIMO:      %.1f ºC%n", Math.max(0.0, tempMaximaRegistrada));
            System.out.printf("PICO TÉRMICO MÍNIMO:      %.1f ºC%n", tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada);
            System.out.printf("ESTANCIA PROMEDIO:        %.1f minutos (Total acumulado: %d min)%n",
                    estanciaMedia, minutosEstanciaAcumulados);
            break;

        case 4:
            // OPERACIÓN 4: PROTOCOLO DE EVACUACIÓN CON BUCLE WHILE DE DRENAJE
            System.out.println("\n--- [MODO 4: PROTOCOLO DE EVACUACIÓN DE EMERGENCIA] ---");
            if (personasEnCentro == 0) {
                System.out.println(">> El campus ya se encuentra completamente vacío.");
            } else {
                int loteSalidaSimultanea = 300; // Capacidad de evacuación de los portones de emergencia
                int oleada = 1;

                while (personasEnCentro > 0) {
                    int evacuadosLote = Math.min(personasEnCentro, loteSalidaSimultanea);
                    personasEnCentro -= evacuadosLote;
                    aforoDisponible += evacuadosLote;

                    System.out.printf("  -> Oleada #%d: Desalojadas %d personas por salidas de emergencia (Ocupantes restantes: %d)%n",
                            oleada, evacuadosLote, personasEnCentro);
```

```
                        oleada++;
                    }
                    System.out.println(">> CERTIFICACIÓN DE SEGURIDAD: Recinto 100% desalojado. Aforo íntegro restaurado.");
                }
                break;

            case 5:
                // OPERACIÓN 5: APAGADO Y SALIDA
                System.out.println("\n>> Desconectando terminal de presencia de forma ordenada...");
                sesionActiva = false; // Mutación del centinela que rompe el do-while
                break;

            default:
                System.err.println(">> Error: Opción no válida. Seleccione una operación entre 1 y 5.");
                break;
        }

    } while (sesionActiva);

    // AUDITORÍA FINAL AL APAGAR
    System.out.println("\n================================================================================");
    System.out.println("          AUDITORÍA FINAL DE LA JORNADA · IES EL CAMINÀS              ");
    System.out.println("================================================================================");
    System.out.printf("TOTAL OPERACIONES GESTIONADAS: %d%n", totalFichajesProcesados);
    System.out.printf("ACCESOS AUTORIZADOS (TOTAL):   %d%n", totalFichajesAutorizados);
    System.out.printf("ACCESOS DENEGADOS (BLOQUEO):   %d%n", totalFichajesDenegados);
    System.out.printf("OCUPACIÓN FINAL REGISTRADA:    %d personas%n", personasEnCentro);
    System.out.println("ESTADO DEL DISPOSITIVO:        Terminal desconectado de forma segura.");
    System.out.println("================================================================================");

    teclado.close();
  }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión práctica de cierre de semana, cada alumno implementará patrones avanzados de centinela, flujo bidireccional y guardias de subdesbordamiento en su proyecto.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). CONTROL BIDIRECCIONAL Y VACIADO EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa una rutina de drenaje progresivo con bucle `while` combinada con protección ante subdesbordamiento:

- **Aventura conversacional.** Diseña una mecánica de combate con puntos de escudo defensivo. Permite al jugador regenerar o perder escudo según la acción; implementa una rutina de vaciado progresivo mediante un bucle `while (escudoActual > 0)` que simule un ataque de drenaje mágico masivo del enemigo, absorbiendo el escudo en paquetes mediante `Math.min` sin permitir que el valor caiga en cifras negativas.
- **Motor de recomendación.** Modela un sistema de catálogo dinámico con gestión bidireccional (añadir nuevos productos recomendados y dar de baja contenidos obsoletos con guardia ante lista vacía). Implementa una opción de purga total mediante un bucle `while (itemsPendientes > 0)` que libere la cola de recomendaciones por lotes emitiendo el balance final.
- **Simulador de físicas 2D.** Incorpora una rutina de amortiguación o frenada gradual mediante un bucle `while (velocidadActual > 0.0)`. En cada ciclo, aplica un coeficiente de frenado cinemático que reduzca la velocidad sin sobrepasar el reposo absoluto (`0.0`), registrando la distancia total recorrida hasta la detención completa del móvil.
- **Bóveda de contraseñas.** Construye un procedimiento de formateo seguro o autodestrucción de credenciales ante brecha de seguridad. Emplea un bucle `while (credencialesActivas > 0)` que sobreescriba y desaloje las claves almacenadas en bloques, garantizando que el contador de registros descienda limpiamente hasta cero antes de certificar el bloqueo de la bóveda.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). EL PATRÓN DE VALOR CENTINELA DE ESCAPE (*SENTINEL VALUE*)

A diferencia de las banderas booleanas, un **valor centinela** es un dato del mismo tipo que las entradas normales pero con un valor reservado que señala el fin de la transmisión:

1. **Lectura iterativa hasta centinela:** Diseña un bucle `while` que solicite lecturas continuadas por consola hasta que el usuario introduzca una palabra reservada o valor centinela (por ejemplo, teclear el texto `"FIN"` o un valor numérico `-1`):

```
System.out.print("Introduce identificador de usuario (o 'FIN' para terminar el lote): ");
String entrada = teclado.nextLine().trim();

while (!entrada.equalsIgnoreCase("FIN")) {
    // Procesar entrada válida...

    System.out.print("Introduce identificador de usuario (o 'FIN' para terminar): ");
    entrada = teclado.nextLine().trim();
}
```

2. **Refactorización defensiva:** Asegúrate de que el valor centinela no sea contabilizado erróneamente como un dato real de negocio ni altere los acumuladores de medias o extremos estadísticos de tu aplicación.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). AUDITORÍA DE INVARIANTES Y PRUEBA DE ESFUERZO DE SUBDESBORDAMIENTO

En el desarrollo de software crítico, los invariantes de clase y de bucle garantizan la integridad matemática del estado en memoria:

1. **Definición de la prueba de esfuerzo (*Stress Test*):**
Diseña un caso de prueba en tu programa que intente ejecutar reiteradamente operaciones de decremento o salida cuando el contador ya haya alcanzado el valor cero (por ejemplo, solicitar 10 salidas consecutivas en un centro con 0 ocupantes, o drenar un recurso agotado).
2. **Verificación de la condición invariante:**
Comprueba que tu guardia condicional intercepta el intento en el 100 % de los casos, impidiendo que la variable de estado decremente hacia valores negativos (*underflow*).
3. **Certificación técnica:**
Redacta un comentario técnico en la cabecera de tu clase formalizando el **invariante de bucle** de tu sistema (por ejemplo, $\forall t \in \text{Ejecución}, \; 0 \le \text{recurso}(t) \le \text{LÍMITE}$), explicando por qué la verificación previa al decremento es la base de la seguridad concurrente y transaccional en sistemas de producción.

4.3 Semana 9. Bucles for, control por banderas y depuración

4.3.1 Día 32 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **lunes 9 de noviembre de 2026**. Los equipos de desarrollo de **AzaharTech** inician la tercera y última semana del **Sprint 3: «El laberinto de la lógica»**. Durante la Semana 8 se consolidaron los bucles iterativos orientados a eventos: el ciclo continuo de servicio (`do-while`), el procesamiento de colas inciertas (`while`) y los protocolos de drenaje por evacuación.

Sin embargo, a primera hora de esta mañana, el equipo de mantenimiento de hardware del **IES El Caminàs** ha remitido una nueva necesidad técnica:

*«Equipo: cada día a las 07:45 h, antes de abrir las puertas a los estudiantes, el terminal debe ejecutar una **rutina de autodiagnóstico secuencial de los 8 sensores infrarrojos** del torno (del sensor 1 al sensor 8) y calcular una **tabla de proyección horaria de aforo** para las 6 horas lectivas de la jornada (de 08:00 a 14:00).*

*En estos procesos, **el número de iteraciones se conoce de antemano con absoluta precisión**: sabemos con exactitud cuántos sensores hay y cuántas horas dura el turno.*

Gestionar estas secuencias deterministas mediante un `while` resulta farragoso y peligroso: hay que declarar un contador fuera del bucle, acordarse de incrementarlo en la última línea y vigilar que ninguna rama condicional interna salte la actualización, arriesgándonos a provocar un bucle infinito por descuido.

*Para recorrer rangos numéricos acotados, contadores exactos y secuencias deterministas, el estándar indiscutible de la industria es el bucle **`for` clásico**».*

Alba Torres y **Laia Claramunt** conectan la consola y definen la arquitectura de la sesión:

*«Hoy estructuramos el bucle **`for`** en Java. Analizaremos su cabecera compacta de tres secciones (inicialización, condición y paso), entenderemos el **ámbito léxico estricto de la variable de control** y dotaremos al terminal de un módulo de diagnóstico y proyección horaria automatizado».*

---

2. Fundamento teórico: el bucle contador `for` clásico

El bucle `for` es una estructura de repetición compacta diseñada para iterar un número conocido de veces o recorrer rangos numéricos determinados. A diferencia de `while`, centraliza en una única línea la creación, evaluación y mutación de la variable de control.

```
flowchart TD
    Inicio([Entrada al bucle for]) --> Init["<b>1. Inicialización</b><br/>int i = 1 (Se ejecuta una sola vez)"]
    Init --> Cond{"<b>2. Condición</b><br/>¿i <= LÍMITE?"}

    Cond -- true --> Cuerpo["<b>3. Cuerpo de sentencias</b><br/>Ejecuta tareas de la iteración"]
    Cuerpo --> Paso["<b>4. Actualización / Paso</b><br/>i++ (Incremento o decremento)"]
    Paso --> Cond

    Cond -- false --> Fin([Salida del bucle for])
```

A. LAS TRES SECCIONES DE LA CABECERA DEL BUCLE

La sintaxis del bucle `for` tradicional se divide en tres cláusulas separadas por punto y coma:

```
for (inicialización; condición; actualización) {
    // Cuerpo que se ejecuta en cada iteración
}
```

1. **Inicialización (`int i = 0`):** Se ejecuta **una única vez** al comenzar el bucle. Permite declarar e inicializar una o más variables de control.
2. **Condición de continuidad (`i < limite`):** Expresión booleana evaluada antes de cada vuelta (*pre-test*). Si resulta `true`, se ejecuta el cuerpo; si es `false`, el bucle expira de inmediato.
3. **Paso o actualización (`i++`):** Sentencia ejecutada automáticamente al término de cada iteración, justo antes de volver a evaluar la condición.

B. ÁMBITO LÉXICO DE LA VARIABLE DE CONTROL (*LOOP-SCOPED VARIABLE*)

Una de las grandes ventajas de ingeniería del bucle `for` es el confinamiento de variables. Si la variable índice se declara dentro de la cabecera (`for (int i = 0; ...)`), **nace y muere dentro del bucle**. No contamina el espacio de nombres de la función ni puede ser modificada accidentalmente tras el ciclo:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Paso: " + i);
}
// System.out.println(i); // ¡ERROR DE COMPILACIÓN!: cannot find symbol variable i
```

C. COMPARATIVA ARQUITECTÓNICA ENTRE ESTRUCTURAS DE REPETICIÓN

Estructura	Mecanismo de control	Momento de evaluación	Cuándo utilizarla en AzaharTech
`for`	Variable contador o rango determinista	Pre-test (antes de entrar)	Recorridos acotados, tablas horarias, diagnósticos de N elementos
`while`	Expresión de estado o guardia compleja	Pre-test (antes de entrar)	Colas de longitud incierta, vaciado de recursos, eventos externos
`do-while`	Variable centinela o interacción	Post-test (al finalizar)	Menús interactivos, demonios de servicio que exigen al menos 1 paso

---

3. Evolución del caso guía a `ControlAccesoQR v2.9` (Java nativo)

Evolucionamos la versión `v2.8` a la **`v2.9`**. Conservamos íntegras las constantes corporativas, los métodos modulares y la captura protegida, incorporando: 1. Constantes para los sensores de paso (`NUM_SENSORES_TORNO = 8`) y el turno (`HORAS_TURNO_LECTIVO = 6`). 2. Opción 4 del menú: **Autodiagnóstico de hardware** mediante un bucle `for` ascendente. 3. **Proyección horaria de aforo** mediante un segundo bucle `for` que calcula la ocupación estimada hora a hora. 4. Menú reestructurado con 6 operaciones ordenadas.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.8 (`diff`)

```
   // ========================================================================
   // CONSTANTES DE HARDWARE Y PLANIFICACIÓN LECTIVA
   // ========================================================================
+ final static int NUM_SENSORES_TORNO = 8;
+ final static int HORAS_TURNO_LECTIVO = 6;

   // ========================================================================
   // MENÚ DE OPERACIONES: Nueva opción de diagnóstico y proyección (for)
   // ========================================================================
+ System.out.println("  [4] Diagnóstico de sensores y proyección horaria (bucles for)");
- System.out.println("  [4] Activar protocolo de evacuación...");
+ System.out.println("  [5] Activar protocolo de evacuación de emergencia");
- System.out.println("  [5] Cerrar jornada...");
+ System.out.println("  [6] Cerrar jornada y apagar terminal");

+ case 4:
+     // AUTODIAGNÓSTICO SECUENCIAL MEDIANTE BUCLE FOR
+     for (int s = 1; s <= NUM_SENSORES_TORNO; s++) {
+         double latenciaMs = 1.2 + (s * 0.4);
+         System.out.printf("  -> Sensor #%d: ONLINE [Latencia: %.1f ms]%n", s, latenciaMs);
+     }
+
+     // PROYECCIÓN HORARIA MEDIANTE BUCLE FOR
+     for (int h = 1; h <= HORAS_TURNO_LECTIVO; h++) {
+         int horaReloj = 8 + h;
+         System.out.printf("  -> Tramo %02d:00 h: Estimación carga lectiva%n", horaReloj);
+     }
+     break;
```

B. FOCO EN LA NOVEDAD: ITERACIÓN DETERMINISTA CON BUCLE `for`

Observa cómo la inicialización, la cota superior y el incremento unitario quedan empaquetados de forma limpia en la cabecera:

```java
// ------------------------------------------------------------------------
// Operación 4: Diagnóstico secuencial y proyección con bucles for
// ------------------------------------------------------------------------
case 4:
    System.out.println("\n--- [MODO 4: DIAGNÓSTICO SECUENCIAL DE SENSORES (BUCLE FOR)] ---");
    System.out.printf("Verificando los %d sensores infrarrojos del bastidor del torno...%n", NUM_SENSORES_TORNO);

    // Bucle for: recorrido determinista de 1 a NUM_SENSORES_TORNO
    for (int sensor = 1; sensor <= NUM_SENSORES_TORNO; sensor++) {
        double latenciaTestMs = 0.8 + (sensor * 0.25);
        boolean sensorCalibrado = (latenciaTestMs < 4.0);
        System.out.printf("  [Sensor IR #%02d]: Estado OK (%b) | Latencia respuesta: %4.2f ms%n",
                sensor, sensorCalibrado, latenciaTestMs);
    }
    System.out.println(">> CERTIFICACIÓN HARDWARE: Los 8 sensores operan en rango nominal.");

    System.out.println("\n--- [PROYECCIÓN HORARIA DE OCUPACIÓN (BUCLE FOR)] ---");
    System.out.println("Hora tramo | Ocupación estimada | Margen disponible");
    System.out.println("-----------------------------------------------");

    // Bucle for: cálculo de estimación horaria para las 6 horas de jornada
```

```
    for (int tramo = 1; tramo <= HORAS_TURNO_LECTIVO; tramo++) {
        int horaPaso = 8 + tramo;
        // Simulación de fluctuación de ocupación según el avance del día
        int ocupacionEstimada = Math.min(1200, personasEnCentro + (tramo * 2) - (tramo == 3 ? 15 : 0));
        int vacantesEstimadas = Math.max(0, 1200 - ocupacionEstimada);

        System.out.printf("  %02d:00 h   |    %4d personas    |    %4d plazas%n",
                horaPaso, ocupacionEstimada, vacantesEstimadas);
    }
    System.out.println("-------------------------------------------------");
    break;
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.9)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.9:
 * Apertura de la Semana 9 (Bucles for, control por banderas y depuración).
 * Incorpora el bucle contador for clásico para secuencias deterministas:
 * - Autodiagnóstico secuencial de sensores de paso de hardware.
 * - Tabla de proyección horaria de aforo para la jornada lectiva.
 * Mantiene la consola interactiva, tránsito bidireccional y blindaje try-catch.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.9 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales biométricos y capacidad
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control de puntualidad y hardware (Semana 9)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);
    final static int NUM_SENSORES_TORNO = 8;
    final static int HORAS_TURNO_LECTIVO = 6;

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
```

```
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: CONSOLA DE SERVICIO CON BUCLES FOR (DÍA 32)
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
        int personasEnCentro = 1190;
        int aforoDisponible = 10;
        int idUltimoFichaje = 1042;

        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
```

```
        int totalFichajesDenegados = 0;

        double tempMaximaRegistrada = -1.0;
        double tempMinimaRegistrada = 100.0;
        int minutosEstanciaAcumulados = 0;

        boolean sesionActiva = true;

        System.out.println("========================================================================");
        System.out.println("   AZAHARTECH · CONTROL DE ACCESOS Y HARDWARE - IES EL CAMINÀS       ");
        System.out.println("   Versión 2.9 Oficial (Sprint 3: Bucles for Deterministas Activos)  ");
        System.out.println("========================================================================");

        // BUCLE PRINCIPAL DE GESTIÓN (MENÚ DE OPERACIONES)
        do {
            System.out.println("\n------------------------------------------------------------------------");
            System.out.println("                    PANEL DE OPERACIONES DEL TORNO                    ");
            System.out.println("------------------------------------------------------------------------");
            System.out.println("  [1] Registrar tránsito individual (Entrada / Salida)");
            System.out.println("  [2] Procesar cola en ráfaga (Modo Burst con bucle while)");
            System.out.println("  [3] Consultar telemetría y métricas de ocupación en vivo");
            System.out.println("  [4] Diagnóstico de sensores y proyección horaria (bucles for)");
            System.out.println("  [5] Activar protocolo de evacuación de emergencia");
            System.out.println("  [6] Cerrar jornada y apagar terminal");
            System.out.print("Seleccione una operación (1-6): ");

            int opcionMenu = 0;
            try {
                opcionMenu = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                opcionMenu = -1;
            }

            switch (opcionMenu) {
                case 1:
                    // TRÁNSITO INDIVIDUAL
                    totalFichajesProcesados++;
                    System.out.println("\n--- [MODO 1: TRÁNSITO INDIVIDUAL] ---");
                    System.out.print("Sentido del tránsito (E = Entrada, S = Salida): ");
                    String lineaSentido = teclado.nextLine().trim().toUpperCase();
                    char sentidoTransito = lineaSentido.isEmpty() ? 'E' : lineaSentido.charAt(0);

                    System.out.print("Nombre completo: ");
                    String nombrePersonaLimpio = teclado.nextLine().trim().toUpperCase();

                    if (sentidoTransito == 'S') {
                        if (personasEnCentro > 0) {
                            personasEnCentro--;
                            aforoDisponible++;
                            totalFichajesAutorizados++;
                            idUltimoFichaje++;
                            System.out.println(">> VEREDICTO: [SALIDA AUTORIZADA] Torno desbloqueado al exterior.");
                        } else {
                            totalFichajesDenegados++;
                            System.err.println(">> ALERTA: Imposible autorizar salida; recinto vacío.");
                        }
                    } else {
                        int terminalId = 0;
                        double tempVestibulo = 0.0;
                        String dniPersonaLimpio = "";
                        char perfilNormalizado = ' ';
                        boolean datosSintacticamenteCorrectos = true;
                        String motivoDenegacion = "";

                        try {
                            System.out.print("ID Terminal: ");
                            terminalId = Integer.parseInt(teclado.nextLine().trim());
                            System.out.print("Temperatura (ºC): ");
                            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());
                            System.out.print("DNI: ");
                            dniPersonaLimpio = teclado.nextLine().trim().toUpperCase();
                            System.out.print("Perfil (E/D/V/A): ");
                            String lineaPerfil = teclado.nextLine().trim();
                            perfilNormalizado = lineaPerfil.isEmpty() ? '?' : Character.toUpperCase(lineaPerfil.charAt(0));
                        } catch (NumberFormatException e) {
                            datosSintacticamenteCorrectos = false;
                            motivoDenegacion = "Error de formato numérico.";
                        }

                        boolean accesoConcedido;
                        if (!datosSintacticamenteCorrectos) {
                            accesoConcedido = false;
                            totalFichajesDenegados++;
                        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                            accesoConcedido = false;
                            motivoDenegacion = "Temperatura anómala.";
                            totalFichajesDenegados++;
                        } else if (aforoDisponible <= 0) {
                            accesoConcedido = false;
                            motivoDenegacion = "Aforo completo.";
                            totalFichajesDenegados++;
                        } else if (!esDocumentoValido(dniPersonaLimpio)) {
                            accesoConcedido = false;
                            motivoDenegacion = "DNI no reglamentario.";
                            totalFichajesDenegados++;
                        } else if (!esPerfilAutorizado(perfilNormalizado)) {
                            accesoConcedido = false;
                            motivoDenegacion = "Perfil no catalogado.";
                            totalFichajesDenegados++;
                        } else {
```

```
                    accesoConcedido = true;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;

                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
                    minutosEstanciaAcumulados += 360;
                }

                int codigoToken = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                String token = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoToken, idUltimoFichaje);
                System.out.printf(">> VEREDICTO: %s [Token: %s]%n",
                        accesoConcedido ? "[ENTRADA AUTORIZADA]" : "[ENTRADA DENEGADA - " + motivoDenegacion + "]", token);
            }
            break;

        case 2:
            // RÁFAGA DE COLA CON WHILE
            System.out.println("\n--- [MODO 2: RÁFAGA CON BUCLE WHILE] ---");
            System.out.print("Personas en cola: ");
            int personasEnCola = 0;
            try {
                personasEnCola = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                personasEnCola = 0;
            }

            while (personasEnCola > 0 && aforoDisponible > 0) {
                totalFichajesProcesados++;
                totalFichajesAutorizados++;
                personasEnCentro++;
                aforoDisponible--;
                idUltimoFichaje++;
                personasEnCola--;

                double tempSimulada = 36.4 + (generadorVestibulo.nextDouble() * 0.6);
                tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempSimulada);
                tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempSimulada);
                minutosEstanciaAcumulados += 360;
            }
            System.out.printf(">> Ráfaga procesada. Plazas disponibles: %d%n", aforoDisponible);
            break;

        case 3:
            // TELEMETRÍA EN VIVO
            System.out.println("\n--- [MODO 3: MONITOR DE TELEMETRÍA] ---");
            int aforoTotal = personasEnCentro + aforoDisponible;
            double porcentajeOcupacion = calcularPorcentajeOcupacion(personasEnCentro, aforoTotal);
            System.out.printf("OCUPACIÓN: %d/%d (%6.2f %%) | VACANTES: %d%n",
                    personasEnCentro, aforoTotal, porcentajeOcupacion, aforoDisponible);
            System.out.printf("PICO MÁX: %.1f ºC | PICO MÍN: %.1f ºC%n",
                    Math.max(0.0, tempMaximaRegistrada), tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada);
            break;

        case 4:
            // DIAGNÓSTICO Y PROYECCIÓN CON BUCLES FOR (DÍA 32)
            System.out.println("\n--- [MODO 4: AUTODIAGNÓSTICO SECUENCIAL DE SENSORES (FOR)] ---");
            for (int sensor = 1; sensor <= NUM_SENSORES_TORNO; sensor++) {
                double latenciaTestMs = 0.8 + (sensor * 0.35);
                System.out.printf("  [Sensor IR #%02d]: ONLINE | Latencia: %4.2f ms | Estado: OK%n", sensor, latenciaTestMs);
            }
            System.out.println(">> CERTIFICACIÓN HARDWARE: 8/8 sensores validados.");

            System.out.println("\n--- [TABLA HORARIA PROYECTADA DE OCUPACIÓN (FOR)] ---");
            System.out.println(" Tramo lectivo | Carga estimada | Margen libre ");
            System.out.println("-----------------------------------------------");
            for (int tramo = 1; tramo <= HORAS_TURNO_LECTIVO; tramo++) {
                int horaReloj = 8 + tramo;
                int cargaSimulada = Math.min(1200, personasEnCentro + (tramo * 2) - (tramo == 3 ? 12 : 0));
                int margenLibre = Math.max(0, 1200 - cargaSimulada);
                System.out.printf("   %02d:00 h    |  %4d personas |  %4d plazas%n", horaReloj, cargaSimulada, margenLibre);
            }
            System.out.println("-----------------------------------------------");
            break;

        case 5:
            // EVACUACIÓN CON WHILE
            System.out.println("\n--- [MODO 5: EVACUACIÓN DE EMERGENCIA] ---");
            while (personasEnCentro > 0) {
                int evacuados = Math.min(personasEnCentro, 300);
                personasEnCentro -= evacuados;
                aforoDisponible += evacuados;
                System.out.printf("  -> Desalojadas %d personas. Restantes: %d%n", evacuados, personasEnCentro);
            }
            System.out.println(">> RECINTO EVACUADO AL 100%.");
            break;

        case 6:
            // SALIDA
            System.out.println("\n>> Desconectando terminal...");
            sesionActiva = false;
            break;

        default:
            System.err.println(">> Opción no válida (1-6).");
            break;
    }
```

```
        } while (sesionActiva);

        // AUDITORÍA FINAL
        System.out.println("\n========================================================================");
        System.out.println("          AUDITORÍA FINAL DE LA JORNADA · IES EL CAMINÀS              ");
        System.out.println("========================================================================");
        System.out.printf("OPERACIONES TOTALES:        %d%n", totalFichajesProcesados);
        System.out.printf("PASOS AUTORIZADOS:          %d%n", totalFichajesAutorizados);
        System.out.printf("PASOS DENEGADOS:            %d%n", totalFichajesDenegados);
        System.out.printf("OCUPACIÓN FINAL:            %d personas%n", personasEnCentro);
        System.out.println("ESTADO:                      Terminal apagado con éxito.");
        System.out.println("========================================================================");

        teclado.close();
    }
}
```

---

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión práctica, cada estudiante integrará la iteración determinista mediante bucles `for` en su proyecto asignado.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). ITERACIÓN SECUENCIAL CON BUCLE `for`

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa un bucle `for` clásico para resolver un requerimiento de tamaño conocido:

- **Aventura conversacional.** Modela un ataque de ráfaga mágica o lluvia de proyectiles que ejecute un número prefijado de impactos consecutivos (por ejemplo, 4 impactos). En cada vuelta del bucle `for`, calcula el daño individual de cada proyectil, descuéntalo de la salud del enemigo y emite la secuencia del impacto en pantalla.
- **Motor de recomendación.** Implementa un bucle `for` que recorra y evalúe una escala fija de 5 niveles de afinidad o categorías principales del catálogo. Para cada nivel, calcula el factor de ponderación proporcional a su posición en la escala y genera una tabla comparativa con las puntuaciones normalizadas.
- **Simulador de físicas 2D.** Diseña un bucle `for` para realizar un muestreo cinemático a intervalos regulares de tiempo fijo (por ejemplo, muestrear la posición del cuerpo durante los primeros 10 segundos exactos, de $t = 1$ a $t = 10$). Imprime una tabla tabular con la posición y velocidad en cada segundo exacto de la trayectoria.
- **Bóveda de contraseñas.** Construye un bucle `for` que genere una máscara de protección visual imprimiendo asteriscos (`*`) en función de la longitud exacta de la clave introducida, o que genere una secuencia criptográfica de prueba de N caracteres iterando exactamente el tamaño solicitado.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). EL BUCLE `for` REGRESIVO Y DESBORDAMIENTO DE LÍMITES

El diseño de bucles regresivos (hacia atrás) requiere prestar especial atención a los operadores relacionales para no omitir el valor frontera:

1. **La cuenta atrás:** Diseña un bucle `for` que realice una cuenta regresiva desde 10 hasta 0 con decremento unitario (`i--`):

```
for (int i = 10; i >= 0; i--) {
    System.out.println("T-menos: " + i);
}
```

2. **El error de omisión del cero:** Analiza por qué cambiar la condición a `i > 0` excluye la iteración final del valor cero (*Off-by-one error*).
3. **Refactorización en tu proyecto:** Aplica un bucle `for` regresivo en un proceso de tu aplicación (por ejemplo, una cuenta atrás de reactivación en la aventura, un decremento de peso en el recomendador, un retroceso de fotogramas en las físicas o una cuenta atrás de bloqueo en la bóveda).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). AUDITORÍA DE LÍMITES Y PRUEBA DE CAJA BLANCA (*OFF-BY-ONE TESTING*)

Uno de los errores más comunes en la historia de la informática es el fallo de límites por un elemento (*Off-By-One Error* o problema del poste de la cerca):

1. **Diseño de la prueba de límites:**
   En un comentario al final de tu clase, formaliza el cálculo matemático del número exacto de iteraciones que ejecuta un bucle `for`: $$\text{Total Iteraciones} = \text{Límite Superior} - \text{Límite Inferior} + 1 \quad (\text{usando } \le)$$ $$\text{Total Iteraciones} = \text{Límite Superior} - \text{Límite Inferior} \quad (\text{usando } <)$$
2. **Matriz de comprobación de frontera:**
   Verifica en tu código las siguientes situaciones extremas:
3. ¿Qué ocurre si el rango del bucle es nulo (por ejemplo, `for (int i = 1; i <= 0; i++)`)? Demuestra que el bucle se comporta como pre-test y no se ejecuta ninguna vez.
4. ¿Qué ocurre si el inicio y el fin coinciden (`i = 5; i <= 5`)? Comprueba que se ejecuta exactamente una vez.
5. **Certificación:**
   Añade un comentario en la cabecera del bucle documentando que el rango de iteraciones ha sido auditado contra errores de desfase de límites.

4.3.2 Día 33 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **martes 10 de noviembre de 2026**. En la sede de **AzaharTech**, el equipo de desarrollo continúa desplegando la última semana del **Sprint 3**. Tras dominar ayer el bucle contador `for` simple para inspeccionar secuencias lineales de sensores, un nuevo requerimiento de infraestructura llega desde el **IES El Caminàs**.

El coordinador TIC del instituto explica el despliegue físico del centro:

*«Nuestro campus no depende de un único torno aislado. Contamos con **3 accesos físicos perimetrales**: 1. Acceso 1: Torno Vestíbulo Central (Edificio Principal). 2. Acceso 2: Torno Puerta Norte (Ciclos Formativos de Informática). 3. Acceso 3: Torno Puerta Sur (Talleres y Cafetería).*

*Cada uno de estos 3 accesos dispone de **4 canales de paso bidireccionales** independientes con lectores ópticos. En conserjería necesitan visualizar en pantalla una **matriz de estado global del campus (3 accesos $\times$ 4 canales)** en forma de tabla visual bidimensional (filas y columnas) que represente el estado operativo de los 12 puntos de paso.*

Un bucle simple solo sabe avanzar en una única dimensión (una lista lineal). Para generar una cuadrícula o matriz bidimensional necesitamos que un bucle recorra las filas (los accesos) y, por cada fila, otro bucle recorra todas sus columnas (los canales de paso)».

Alba Torres toma el teclado y plantea el reto técnico a **Pau Ferrer**:

*«Pau, acabamos de entrar en el terreno de los **bucles anidados (Nested Loops)**.*

*Hoy aprenderemos cómo interactúan dos bucles `for` coordinados como las manecillas de un reloj: el bucle interior gira a toda velocidad completando todas las columnas antes de que el bucle exterior avance un solo paso a la siguiente fila. Analizaremos su **complejidad temporal cuadrática $O(N \times M)$** y generaremos el mapa visual matricial de los accesos del instituto».*

---

2. Fundamento teórico: bucles anidados y matrices lógicas en consola

Un **bucle anidado** es una estructura de control donde el cuerpo de una sentencia de repetición contiene en su interior otro bucle completo.

```
flowchart TD
    Inicio([Inicio de Matriz]) --> BucleExtInit["<b>Bucle Exterior: Filas (Acceso i = 1)</b>"]
```

```
    BucleExtInit --> CondExt{"¿i <= TOTAL_TORNOS?<br/>(i <= 3)"}

    CondExt -- true --> BucleIntInit["<b>Bucle Interior: Columnas (Canal j = 1)</b>"]
    BucleIntInit --> CondInt{"¿j <= CANALES?<br/>(j <= 4)"}

    CondInt -- true --> Cuerpo["<b>Imprimir celda [i, j]:</b><br/>System.out.printf(' [T%d-C%d: OK] ', i, j);"]
    Cuerpo --> PasoInt["j++ (Siguiente canal)"]
    PasoInt --> CondInt

    CondInt -- false --> SaltoFila["<b>Salto de línea:</b><br/>System.out.println();"]
    SaltoFila --> PasoExt["i++ (Siguiente torno)"]
    PasoExt --> CondExt

    CondExt -- false --> Fin([Fin de Matriz Bidimensional])
```

A. LA ANALOGÍA DEL RELOJ MECÁNICO

Para comprender la ejecución de dos bucles anidados, el modelo mental más intuitivo es el segundero y el minutero de un reloj: * El **bucle exterior** (i) representa las horas o los minutos: avanza lentamente, un solo paso cada vez. * El **bucle interior** (j) representa los segundos: por cada unidad que avanza el exterior, el interior debe dar **una vuelta completa de principio a fin**.

```
for (int i = 1; i <= 3; i++) {       // FILAS (Exterior)
    for (int j = 1; j <= 4; j++) {   // COLUMNAS (Interior)
        System.out.print("[" + i + "," + j + "] ");
    }
    System.out.println();             // Salto de línea al terminar cada fila
}
```

B. IMPRESIÓN DE CUADRÍCULAS: `print()` FRENTE A `println()`

La representación visual de matrices en consola exige coordinar dos métodos de salida: 1. `System.out.print(...)` / `System.out.printf(...)` : se utiliza **dentro del bucle interior** para imprimir los elementos de la fila uno al lado del otro en horizontal, separados por espacios. 2. `System.out.println()` : se invoca **en el bucle exterior, justo después de que el bucle interior termina**, para romper la línea y bajar el cursor a la siguiente fila.

C. ANÁLISIS DE COMPLEJIDAD TEMPORAL CUADRÁTICA

El número total de ejecuciones del cuerpo más interno es el **producto cartesiano** de las dimensiones:

$$\text{Total Iteraciones} = N \times M$$

Dimensión exterior (N filas)	Dimensión interior (M columnas)	Iteraciones totales	Complejidad algorítmica
3 tornos	4 canales de paso	**12 ejecuciones**	$O(N \times M)$
10 aulas	25 puestos informáticos	**250 ejecuciones**	$O(N \times M)$
1.000 registros	1.000 comparaciones	**1.000.000 ejecuciones**	$O(N^2)$ (Cuadrática)

---

3. Evolución del caso guía a `ControlAccesoQR v2.10` (Java nativo)

Evolucionamos la versión `v2.9` a la `v2.10` . Conservamos intactas todas las constantes corporativas, los métodos modulares y la captura protegida, incorporando: 1. Constantes para la arquitectura del campus: `TOTAL_TORNOS_CAMPUS = 3` y `CANALES_POR_TORNO = 4` . 2. Actualización de la **Opción 4 del menú**: ampliación del diagnóstico para generar la **Matriz visual de supervisión del campus (3 $\times$ 4)** mediante bucles `for` anidados. 3. Simulación de telemetría de canales cruzada con indicadores de paso en cuadrícula.

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.9 (`diff`)

```
   // ========================================================================
   // CONSTANTES DE ARQUITECTURA MULTI-ACCESO DEL CAMPUS
   // ========================================================================
   final static int NUM_SENSORES_TORNO = 8;
   final static int HORAS_TURNO_LECTIVO = 6;
+  final static int TOTAL_TORNOS_CAMPUS = 3;
+  final static int CANALES_POR_TORNO = 4;

   // ========================================================================
   // MENÚ DE OPERACIONES: Opción 4 ampliada con matriz bidimensional
   // ========================================================================
```

```
- System.out.println("  [4] Diagnóstico de sensores y proyección horaria (bucles for)");
+ System.out.println("  [4] Diagnóstico hardware y matriz perimetral del campus (bucles anidados)");

  case 4:
      // [Diagnóstico lineal de sensores de v2.9...]

+     // MATRIZ BIDIMENSIONAL DE ACCESOS Y CANALES (BUCLES ANIDADOS)
+     System.out.println("\n--- [MATRIZ DE SUPERVISIÓN PERIMETRAL: 3 TORNOS x 4 CANALES] ---");
+     for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS; torno++) {
+         System.out.printf("Torno #%d | ", torno);
+         for (int canal = 1; canal <= CANALES_POR_TORNO; canal++) {
+             boolean operativo = ((torno + canal) % 2 == 0);
+             System.out.printf("[C%d: %s] ", canal, operativo ? "OK" : "BUSY");
+         }
+         System.out.println(); // Salto de línea tras completar las columnas del torno
+     }
      break;
```

B. FOCO EN LA NOVEDAD: GENERACIÓN DE LA MATRIZ DE ACCESOS CON BUCLES ANIDADOS

Observa la coordinación entre el índice exterior `torno` y el interior `canal`, junto con el uso estricto de `print` horizontal y `println` de cierre de fila:

```
// ---------------------------------------------------------------------
// Operación 4 (Ampliación Día 33): Matriz perimetral con bucles for anidados
// ---------------------------------------------------------------------
System.out.println("\n--- [MAPA VISUAL PERIMETRAL: 3 TORNOS x 4 CANALES DE PASO] ---");
System.out.println("Ubicación campus      | Canal 1     Canal 2     Canal 3     Canal 4");
System.out.println("-----------------------------------------------------------------------");

// BUCLE EXTERIOR: Itera sobre los 3 tornos físicos del campus (FILAS)
for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS; torno++) {
    String nombreAcceso = switch (torno) {
        case 1 -> "Vestíbulo Central  | ";
        case 2 -> "Puerta Norte Ciclos| ";
        case 3 -> "Puerta Sur Talleres| ";
        default -> "Acceso Perimetral  | ";
    };
    System.out.print(nombreAcceso);

    // BUCLE INTERIOR: Itera sobre los 4 canales de cada torno (COLUMNAS)
    for (int canal = 1; canal <= CANALES_POR_TORNO; canal++) {
        // Simulación determinista de estado del canal mediante paridad de índices
        boolean canalDespejado = ((torno + canal) % 2 == 0);
        String estadoVisual = canalDespejado ? "[ACTIVO]  " : "[PASO]    ";

        // Salida horizontal sin salto de línea
        System.out.print(estadoVisual);
    }

    // Salto de línea obligatorio al completar todos los canales del torno actual
    System.out.println();
}
System.out.println("-----------------------------------------------------------------------");
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.10)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.10:
 * Consolidación de la Semana 9 (Bucles for, control por banderas y depuración).
 * Incorpora bucles for anidados para la generación de matrices bidimensionales:
 * - Mapa visual perimetral de supervisión del campus (3 tornos x 4 canales).
 * - Algoritmo de renderizado tabular horizontal (print) y salto de fila (println).
 * Mantiene la consola interactiva, tránsito bidireccional y blindaje try-catch.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.10 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales biométricos y capacidad
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control de puntualidad y hardware perimetral (Semana 9)
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);
    final static int NUM_SENSORES_TORNO = 8;
    final static int HORAS_TURNO_LECTIVO = 6;
    final static int TOTAL_TORNOS_CAMPUS = 3;
    final static int CANALES_POR_TORNO = 4;

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
```

```
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: CONSOLA DE SERVICIO CON BUCLES ANIDADOS (DÍA 33)
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
        int personasEnCentro = 1190;
        int aforoDisponible = 10;
        int idUltimoFichaje = 1042;
```

```
        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;

        double tempMaximaRegistrada = -1.0;
        double tempMinimaRegistrada = 100.0;
        int minutosEstanciaAcumulados = 0;

        boolean sesionActiva = true;

        System.out.println("========================================================================");
        System.out.println("   AZAHARTECH · CONTROL INTEGRAL DEL CAMPUS - IES EL CAMINÀS          ");
        System.out.println("   Versión 2.10 Oficial (Sprint 3: Bucles Anidados y Matriz de Accesos) ");
        System.out.println("========================================================================");

        // BUCLE PRINCIPAL DE GESTIÓN (MENÚ DE OPERACIONES)
        do {
            System.out.println("\n------------------------------------------------------------------------");
            System.out.println("                    PANEL DE OPERACIONES DEL TORNO                     ");
            System.out.println("------------------------------------------------------------------------");
            System.out.println("  [1] Registrar tránsito individual (Entrada / Salida)");
            System.out.println("  [2] Procesar cola en ráfaga (Modo Burst con bucle while)");
            System.out.println("  [3] Consultar telemetría y métricas de ocupación en vivo");
            System.out.println("  [4] Diagnóstico hardware y matriz perimetral del campus (bucles anidados)");
            System.out.println("  [5] Activar protocolo de evacuación de emergencia");
            System.out.println("  [6] Cerrar jornada y apagar terminal");
            System.out.print("Seleccione una operación (1-6): ");

            int opcionMenu = 0;
            try {
                opcionMenu = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                opcionMenu = -1;
            }

            switch (opcionMenu) {
                case 1:
                    // TRÁNSITO INDIVIDUAL
                    totalFichajesProcesados++;
                    System.out.println("\n--- [MODO 1: TRÁNSITO INDIVIDUAL] ---");
                    System.out.print("Sentido del tránsito (E = Entrada, S = Salida): ");
                    String lineaSentido = teclado.nextLine().trim().toUpperCase();
                    char sentidoTransito = lineaSentido.isEmpty() ? 'E' : lineaSentido.charAt(0);

                    System.out.print("Nombre completo: ");
                    String nombrePersonaLimpio = teclado.nextLine().trim().toUpperCase();

                    if (sentidoTransito == 'S') {
                        if (personasEnCentro > 0) {
                            personasEnCentro--;
                            aforoDisponible++;
                            totalFichajesAutorizados++;
                            idUltimoFichaje++;
                            System.out.println(">> VEREDICTO: [SALIDA AUTORIZADA] Torno exterior abierto.");
                        } else {
                            totalFichajesDenegados++;
                            System.err.println(">> ALERTA: Imposible autorizar salida; recinto vacío.");
                        }
                    } else {
                        int terminalId = 0;
                        double tempVestibulo = 0.0;
                        String dniPersonaLimpio = "";
                        char perfilNormalizado = ' ';
                        boolean datosSintacticamenteCorrectos = true;
                        String motivoDenegacion = "";

                        try {
                            System.out.print("ID Terminal: ");
                            terminalId = Integer.parseInt(teclado.nextLine().trim());
                            System.out.print("Temperatura (ºC): ");
                            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());
                            System.out.print("DNI: ");
                            dniPersonaLimpio = teclado.nextLine().trim().toUpperCase();
                            System.out.print("Perfil (E/D/V/A): ");
                            String lineaPerfil = teclado.nextLine().trim();
                            perfilNormalizado = lineaPerfil.isEmpty() ? '?' : Character.toUpperCase(lineaPerfil.charAt(0));
                        } catch (NumberFormatException e) {
                            datosSintacticamenteCorrectos = false;
                            motivoDenegacion = "Error de formato numérico.";
                        }

                        boolean accesoConcedido;
                        if (!datosSintacticamenteCorrectos) {
                            accesoConcedido = false;
                            totalFichajesDenegados++;
                        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                            accesoConcedido = false;
                            motivoDenegacion = "Temperatura anómala.";
                            totalFichajesDenegados++;
                        } else if (aforoDisponible <= 0) {
                            accesoConcedido = false;
                            motivoDenegacion = "Aforo completo.";
                            totalFichajesDenegados++;
                        } else if (!esDocumentoValido(dniPersonaLimpio)) {
                            accesoConcedido = false;
                            motivoDenegacion = "DNI no reglamentario.";
                            totalFichajesDenegados++;
                        } else if (!esPerfilAutorizado(perfilNormalizado)) {
                            accesoConcedido = false;
                            motivoDenegacion = "Perfil no catalogado.";
```

```
                    totalFichajesDenegados++;
                } else {
                    accesoConcedido = true;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;

                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
                    minutosEstanciaAcumulados += 360;
                }

                int codigoToken = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                String token = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoToken, idUltimoFichaje);
                System.out.printf(">> VEREDICTO: %s [Token: %s]%n",
                        accesoConcedido ? "[ENTRADA AUTORIZADA]" : "[ENTRADA DENEGADA - " + motivoDenegacion + "]", token);
            }
            break;

        case 2:
            // RÁFAGA DE COLA CON WHILE
            System.out.println("\n--- [MODO 2: RÁFAGA CON BUCLE WHILE] ---");
            System.out.print("Personas en cola: ");
            int personasEnCola = 0;
            try {
                personasEnCola = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                personasEnCola = 0;
            }

            while (personasEnCola > 0 && aforoDisponible > 0) {
                totalFichajesProcesados++;
                totalFichajesAutorizados++;
                personasEnCentro++;
                aforoDisponible--;
                idUltimoFichaje++;
                personasEnCola--;

                double tempSimulada = 36.4 + (generadorVestibulo.nextDouble() * 0.6);
                tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempSimulada);
                tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempSimulada);
                minutosEstanciaAcumulados += 360;
            }
            System.out.printf(">> Ráfaga procesada. Plazas disponibles: %d%n", aforoDisponible);
            break;

        case 3:
            // TELEMETRÍA EN VIVO
            System.out.println("\n--- [MODO 3: MONITOR DE TELEMETRÍA] ---");
            int aforoTotal = personasEnCentro + aforoDisponible;
            double porcentajeOcupacion = calcularPorcentajeOcupacion(personasEnCentro, aforoTotal);
            System.out.printf("OCUPACIÓN: %d/%d (%6.2f %%) | VACANTES: %d%n",
                    personasEnCentro, aforoTotal, porcentajeOcupacion, aforoDisponible);
            System.out.printf("PICO MÁX: %.1f ºC | PICO MÍN: %.1f ºC%n",
                    Math.max(0.0, tempMaximaRegistrada), tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada);
            break;

        case 4:
            // DIAGNÓSTICO Y MATRIZ PERIMETRAL CON BUCLES ANIDADOS (DÍA 33)
            System.out.println("\n--- [MODO 4: DIAGNÓSTICO HARDWARE Y MATRIZ PERIMETRAL] ---");
            for (int sensor = 1; sensor <= NUM_SENSORES_TORNO; sensor++) {
                double latenciaTestMs = 0.8 + (sensor * 0.35);
                System.out.printf("  [Sensor IR #%02d]: ONLINE | Latencia: %4.2f ms | Estado: OK%n", sensor, latenciaTestMs);
            }

            // GENERACIÓN DE MATRIZ BIDIMENSIONAL CON BUCLES FOR ANIDADOS
            System.out.println("\n--- [MAPA VISUAL PERIMETRAL: 3 TORNOS x 4 CANALES DE PASO] ---");
            System.out.println("Ubicación campus      | Canal 1     Canal 2     Canal 3     Canal 4");
            System.out.println("-------------------------------------------------------------------------");

            // BUCLE EXTERIOR: Filas (Tornos perimetrales)
            for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS; torno++) {
                String nombreAcceso = switch (torno) {
                    case 1 -> "Vestíbulo Central  | ";
                    case 2 -> "Puerta Norte Ciclos| ";
                    case 3 -> "Puerta Sur Talleres| ";
                    default -> "Acceso Perimetral  | ";
                };
                System.out.print(nombreAcceso);

                // BUCLE INTERIOR: Columnas (Canales de paso)
                for (int canal = 1; canal <= CANALES_POR_TORNO; canal++) {
                    boolean canalDespejado = ((torno + canal) % 2 == 0);
                    String estadoVisual = canalDespejado ? "[ACTIVO]  " : "[PASO]    ";
                    System.out.print(estadoVisual);
                }

                // Salto de línea obligatorio tras completar cada fila de la matriz
                System.out.println();
            }
            System.out.println("-------------------------------------------------------------------------");
            break;

        case 5:
            // EVACUACIÓN CON WHILE
            System.out.println("\n--- [MODO 5: EVACUACIÓN DE EMERGENCIA] ---");
            while (personasEnCentro > 0) {
                int evacuados = Math.min(personasEnCentro, 300);
                personasEnCentro -= evacuados;
```

```
                        aforoDisponible += evacuados;
                        System.out.printf("  -> Desalojadas %d personas. Restantes: %d%n", evacuados, personasEnCentro);
                    }
                    System.out.println(">> RECINTO EVACUADO AL 100%.");
                    break;

                case 6:
                    // SALIDA
                    System.out.println("\n>> Desconectando terminal...");
                    sesionActiva = false;
                    break;

                default:
                    System.err.println(">> Opción no válida (1-6).");
                    break;
            }

        } while (sesionActiva);

        // AUDITORÍA FINAL
        System.out.println("\n=========================================================================");
        System.out.println("          AUDITORÍA FINAL DE LA JORNADA · IES EL CAMINÀS              ");
        System.out.println("=========================================================================");
        System.out.printf("OPERACIONES TOTALES:        %d%n", totalFichajesProcesados);
        System.out.printf("PASOS AUTORIZADOS:          %d%n", totalFichajesAutorizados);
        System.out.printf("PASOS DENEGADOS:            %d%n", totalFichajesDenegados);
        System.out.printf("OCUPACIÓN FINAL:            %d personas%n", personasEnCentro);
        System.out.println("ESTADO:                      Terminal apagado con éxito.");
        System.out.println("=========================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión práctica, cada estudiante implementará bucles anidados para generar representaciones bidimensionales o matrices de cálculo en su proyecto técnico.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). MATRICES BIDIMENSIONALES EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa una estructura de dos bucles `for` anidados para modelar una cuadrícula o tabla de doble entrada:

- **Aventura conversacional.** Genera en consola un mapa bidimensional de la mazmorra o sala de combate (por ejemplo, una cuadrícula de 5×5 celdas). El bucle exterior recorrerá las filas y el interior las columnas; imprime celdas vacías (`.`) y sitúa condicionalmente mediante las coordenadas $(fila, columna)$ la posición del héroe (`@`) y del adversario (`M`).
- **Motor de recomendación.** Modela una matriz de afinidad cruzada de contenidos. Define una tabla de 4×4 donde las filas representen perfiles de usuario tipo y las columnas representen categorías temáticas; calcula en cada intersección el coeficiente de recomendación multiplicativo e imprímelo en formato de tabla alineada.
- **Simulador de físicas 2D.** Dibuja en la consola una cuadrícula cartesiana espacial (por ejemplo, 8×8 caracteres) donde el plano represente el área de impacto. En cada celda de la matriz (x, y), comprueba si coincide con la posición redondeada del proyectil para imprimir el carácter de impacto (`*`) o el espacio vacío (`·`).
- **Bóveda de contraseñas.** Construye una tarjeta de coordenadas criptográfica de autenticación de doble factor. Mediante bucles anidados, genera una cuadrícula de 4×5 celdas alfanuméricas con cabeceras de filas (letras A–D) y columnas (números 1–5), imprimiendo en cada casilla un código de seguridad pseudoaleatorio de dos dígitos.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). BUCLES ANIDADOS DEPENDIENTES Y PATRONES GEOMÉTRICOS

Un **bucle anidado dependiente** ocurre cuando la cota o límite del bucle interior está condicionada por el valor actual del índice del bucle exterior:

1. **El patrón triangular:** Analiza el siguiente fragmento donde el número de columnas impresas crece en cada fila:

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

2. **Cálculo de iteraciones acumuladas:** A diferencia de una matriz rectangular ($N \times M$), en un bucle triangular el número total de ejecuciones responde a la serie aritmética de Gauss: $$\text{Total Iteraciones} = \frac{N \times (N + 1)}{2}$$
3. **Refactorización en tu proyecto:** Aplica un bucle anidado dependiente en tu especialidad (por ejemplo, un escalado progresivo de daño por combos en la aventura, un árbol de pesos jerárquico en el recomendador o una pirámide de complejidad de contraseñas en la bóveda).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). AUDITORÍA DE EXPLOSIÓN COMBINATORIA Y ANÁLISIS TEMPORAL $O(N \TIMES M)$

En el desarrollo de software corporativo, el anidamiento de bucles es una de las causas principales de degradación de rendimiento (*Performance Bottleneck*):

1. **Modelado formal de la complejidad:**
   En un comentario al final de tu clase, calcula cuántas iteraciones se ejecutarían si la cuadrícula de tu programa escalara de las dimensiones de prueba ($3 \times 4 = 12$) a un entorno masivo de datos:
2. Matriz 100×100: 10.000 iteraciones.
3. Matriz 1.000×1.000: $1.000.000$ iteraciones.
4. Matriz tridimensional ($N \times M \times K$ con 3 bucles anidados) para $N=1.000$: $1.000.000.000$ iteraciones ($O(N^3)$).
5. **Prueba de estrés de reloj:**
   Implementa una toma de tiempo con `System.currentTimeMillis()` alrededor de una simulación matricial a gran escala y comprueba experimentalmente cómo el tiempo de respuesta se multiplica de forma cuadrática al duplicar el tamaño de las filas y columnas.
6. **Certificación de arquitectura:**
   Redacta un comentario técnico documentando por qué en los estándares de calidad de AzaharTech se prohíbe anidar más de dos bucles en el hilo principal de ejecución, exigiendo delegar procesamientos de orden superior a algoritmos de búsqueda indexada o procesamiento concurrente.

4.3.3 Día 34 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **miércoles 11 de noviembre de 2026**. En la sala de ingeniería de **AzaharTech**, el equipo de desarrollo afronta la penúltima jornada del **Sprint 3: «El laberinto de la lógica»**. La matriz bidimensional de supervisión perimetral desplegada ayer en la versión `v2.10` permitió al **IES El Caminàs** visualizar el estado de los 3 tornos y sus 4 canales de paso.

Sin embargo, durante las pruebas de integración en el campus surgen dos situaciones operativas que exigen ajustar el comportamiento de los bucles:

«Equipo, nuestros bucles actuales recorren rígidamente todas las celdas hasta el final sin considerar incidencias intermedias: 1. * **Canal en mantenimiento:** Si el canal 2 de la Puerta Norte está desconectado por revisión técnica, no debemos ejecutar su telemetría ordinaria; debemos detectarlo y procesar directamente la rama alternativa, manteniendo el flujo lineal del código.* 2. * **Parada anticipada por seguridad perimetral:** Si durante el barrido de los tornos se detecta una alerta crítica de intrusión o avería grave, el escaneo debe concluir de inmediato. No tiene sentido seguir comprobando el resto de canales ni de accesos.*

En muchos entornos de programación no reglada se recurre a sentencias de salto incondicional como `break` *o* `continue` *para interrumpir los bucles a la fuerza. Sin embargo, en AzaharTech aplicamos los principios más sólidos de la* ***Programación Estructurada****: el principio* ***SESE (Single Entry, Single Exit)****.*

Todo bucle debe tener ***un único punto de entrada y un único punto de salida****, gobernado limpiamente por su* ***condición de cabecera****. Hoy aprenderemos a detener bucles simples y anidados mediante* ***banderas booleanas de estado (State Flags)*** *y a filtrar situaciones especiales mediante bifurcaciones condicionales directas, preservando la trazabilidad absoluta del programa».*

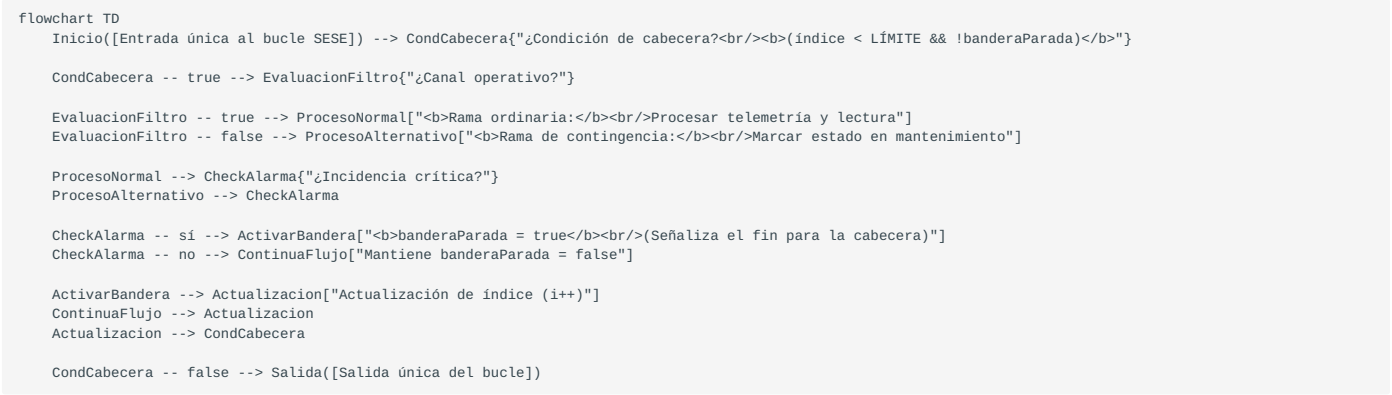
Alba Torres y **Laia Claramunt** abren el entorno de desarrollo y plantean la pauta técnica:

«Al gobernar los bucles exclusivamente a través de su cabecera, el código resulta 100 % predecible, los invariantes matemáticos se cumplen en todo momento y eliminamos cualquier riesgo de provocar efectos secundarios imprevistos en la máquina virtual».

---

2. Fundamento teórico: el principio SESE y control de bucles por banderas de estado

El principio **SESE (*Single Entry, Single Exit*)** es uno de los pilares fundacionales de la programación estructurada (formalizado a partir del teorema de Böhm-Jacopini y los trabajos de Edsger Dijkstra). Establece que cualquier bloque de control (sea condicional o iterativo) debe comenzar por un único punto y finalizar ordenadamente por su condición de salida natural.



A. PARADA ANTICIPADA MEDIANTE EXPRESIONES COMPUESTAS EN CABECERA

En Java, la cabecera de un bucle `for` no está limitada a comparar un número: admite **expresiones booleanas compuestas** con la misma flexibilidad que un bucle `while`.

Si deseamos que un bucle se detenga en cuanto se produzca un evento crítico (por ejemplo, una alerta de seguridad o haber encontrado un dato buscado), introducimos una **bandera booleana de estado (*State Flag*)** en la propia condición de corte:

```
boolean paradaRequerida = false;

// El bucle avanza mientras no supere el límite Y no se haya levantado la bandera
for (int i = 0; i < total && !paradaRequerida; i++) {
    if (hayPeligro(i)) {
        paradaRequerida = true; // Mutación que provocará la salida limpia en la cabecera
    } else {
        procesarElemento(i);
    }
}
```

B. ESCAPE COORDINADO DE BUCLES ANIDADOS SIN SALTOS ABRUPTOS

Cuando trabajamos con dos bucles anidados (filas y columnas), la misma bandera de estado se evalúa en las cabeceras de **ambos bucles**. En cuanto el bucle interior detecta la condición de corte y muta la bandera, el bucle interior finaliza ordenadamente y el bucle exterior también detecta la bandera en su siguiente evaluación, terminando ambos sin requerir instrucciones de salto:

```
boolean incidenciaCritica = false;

for (int fila = 1; fila <= MAX_FILAS && !incidenciaCritica; fila++) {
    for (int col = 1; col <= MAX_COLS && !incidenciaCritica; col++) {
        if (celdaCorrupta(fila, col)) {
            incidenciaCritica = true; // Señaliza la parada a ambos niveles
        } else {
            inspeccionar(fila, col);
        }
    }
}
```

C. FILTRADO SECUENCIAL FRENTE A INTERRUPCIONES

Para omitir el procesamiento de elementos en revisión o no computables, no es necesario saltar instrucciones a mitad de camino. La solución estructurada consiste en utilizar una **bifurcación condicional limpia (`if-else`)**: la rama afirmativa procesa el dato regular y la rama alternativa notifica la excepción técnica, convergiendo ambas de forma natural al cierre de la iteración.

Criterio	Enfoque no estructurado (con saltos)	Enfoque estructurado SESE (con banderas)
Puntos de salida	Múltiples salidas dispersas (`break` interior)	**Único punto de salida** en la cabecera del bucle
Control de la parada	Ocurre de forma invisible dentro del cuerpo	**Explícito y visible** en la condición del bucle
Omisión de tareas	Salto incondicional (`continue`)	Bifurcación condicional legible (`if-else`)
Trazabilidad y depuración	Compleja: el cursor salta líneas de golpe	Predecible: flujo descendente continuo

---

3. Evolución del caso guía a `ControlAccesoQR v2.11` (Java nativo)

Evolucionamos la versión `v2.10` a la **`v2.11`** . Mantenemos íntegras las constantes corporativas, los métodos modulares y la captura con `try-catch` , incorporando: 1. **Control de ráfaga estructurado**: el bucle `while` de cola evalúa una bandera booleana `vetoJudicialDetectado` en su propia cabecera para suspender el paso de forma inmediata ante credenciales vetadas, sin emplear `break` . 2. **Matriz perimetral gobernada por SESE**: los dos bucles `for` anidados evalúan la bandera `alarmaSabotajeDetectada` en sus respectivas cabeceras para detener el barrido perimetral ante emergencias. 3. **Filtrado secuencial de canales**: el canal 2 de la Puerta Norte se gestiona mediante una bifurcación `if-else` afirmativa sin usar `continue` .

A. COMPARATIVA DE CAMBIOS RESPECTO A V2.10 (`diff`)

```
  // ========================================================================
  // MODO 2: Ráfaga de cola gobernada por bandera booleana en cabecera
  // ========================================================================
+ boolean vetoJudicialDetectado = false;
- while (personasEnCola > 0 && aforoDisponible > 0) {
+ while (personasEnCola > 0 && aforoDisponible > 0 && !vetoJudicialDetectado) {
      if (idUltimoFichaje == 1044) {
          System.out.println(">> ALERTA: Credencial en lista de veto judicial.");
+         vetoJudicialDetectado = true; // Detención estructurada en cabecera
      } else {
          totalFichajesProcesados++;
          // ... [Procesamiento ordinario del paso] ...
      }
  }

  // ========================================================================
  // MODO 4: Matriz perimetral con parada coordinada SESE (sin saltos)
  // ========================================================================
+ boolean alarmaSabotajeDetectada = false;
- for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS; torno++) {
+ for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS && !alarmaSabotajeDetectada; torno++) {
      System.out.print(nombreAcceso);
-     for (int canal = 1; canal <= CANALES_POR_TORNO; canal++) {
+     for (int canal = 1; canal <= CANALES_POR_TORNO && !alarmaSabotajeDetectada; canal++) {
```

```
+          if (torno == 2 && canal == 2) {
+              System.out.print("[REVISIÓN] "); // Filtrado estructurado sin continue
+          } else if (alarmaSabotajeDetectada) {
+              // Detección de incidencia crítica
+          } else {
              System.out.print("[ONLINE]   ");
+          }
       }
       System.out.println();
   }
```

B. FOCO EN LA NOVEDAD: PARADA ANTICIPADA Y FILTRADO SIN SALTOS

Observa cómo la bandera `alarmaSabotajeDetectada` se propaga a las cabeceras de ambos bucles para garantizar un escape limpio bajo el estándar SESE:

```
// ------------------------------------------------------------------------
// Operación 4: Matriz perimetral estructurada (SESE) sin sentencias de salto
// ------------------------------------------------------------------------
System.out.println("\n--- [INSPECCIÓN ESTRUCTURADA: MATRIZ PERIMETRAL SESE] ---");
boolean alarmaSabotajeDetectada = false; // Bandera de estado coordinada

// BUCLE EXTERIOR: Cabecera con comprobación de límite Y bandera de parada
for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS && !alarmaSabotajeDetectada; torno++) {
    System.out.printf("Torno #%d | ", torno);

    // BUCLE INTERIOR: Cabecera con comprobación de canal Y bandera de parada
    for (int canal = 1; canal <= CANALES_POR_TORNO && !alarmaSabotajeDetectada; canal++) {

        // 1. Filtrado estructurado con if-else (sin continue)
        if (torno == 2 && canal == 2) {
            System.out.print("[REVISIÓN] ");
        } else if (alarmaSabotajeDetectada) {
            // Si la alarma se hubiera activado, no se procesa la celda
        } else {
            // Simulación: detección de sabotaje en el canal 2 del torno 3
            boolean falloCriticoEnCanal = (torno == 3 && canal == 2 && false); // Simulación inactiva

            if (falloCriticoEnCanal) {
                System.out.print("[SABOTAJE] ");
                alarmaSabotajeDetectada = true; // Provoca la terminación limpia en ambas cabeceras
            } else {
                System.out.printf("[C%d: OK]   ", canal);
            }
        }
    }
    System.out.println(); // Fin de fila
}

if (alarmaSabotajeDetectada) {
    System.out.println(">> [ALERTA DE SEGURIDAD]: Barrido perimetral detenido por fallo crítico.");
}
System.out.println("-------------------------------------------------------------------------");
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v2.11)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 2.11:
 * Cierre técnico de estructuras de repetición bajo el principio SESE (Sprint 3).
 * Implementa control de bucles estructurado mediante banderas de estado:
 * - Parada de ráfagas en cabecera de while mediante bandera de veto judicial.
 * - Salida coordinada de matrices bidimensionales evaluando banderas en for.
 * - Filtrado secuencial de canales en mantenimiento con if-else (sin saltos).
 * Mantiene consola interactiva, tránsito bidireccional y blindaje try-catch.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 2.11 (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales biométricos y capacidad
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control de puntualidad y hardware perimetral
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);
    final static int NUM_SENSORES_TORNO = 8;
    final static int HORAS_TURNO_LECTIVO = 6;
    final static int TOTAL_TORNOS_CAMPUS = 3;
    final static int CANALES_POR_TORNO = 4;

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
                descripcion = "Docente / Claustro";
                break;
            case 'V':
```

```
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: CONSOLA DE SERVICIO ESTRUCTURADA SESE (DÍA 34)
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
        int personasEnCentro = 1190;
        int aforoDisponible = 10;
        int idUltimoFichaje = 1042;
```

```
        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;

        double tempMaximaRegistrada = -1.0;
        double tempMinimaRegistrada = 100.0;
        int minutosEstanciaAcumulados = 0;

        boolean sesionActiva = true;

        System.out.println("=================================================================");
        System.out.println("   AZAHARTECH · CONTROL ESTRUCTURADO SESE - IES EL CAMINÀS            ");
        System.out.println("   Versión 2.11 Oficial (Sprint 3: Control por Banderas de Estado)    ");
        System.out.println("=================================================================");

        // BUCLE PRINCIPAL DE GESTIÓN (MENÚ DE OPERACIONES)
        do {
            System.out.println("\n-----------------------------------------------------------------");
            System.out.println("                    PANEL DE OPERACIONES DEL TORNO                  ");
            System.out.println("-----------------------------------------------------------------");
            System.out.println("  [1] Registrar tránsito individual (Entrada / Salida)");
            System.out.println("  [2] Procesar cola en ráfaga (con guardia de veto en cabecera)");
            System.out.println("  [3] Consultar telemetría y métricas de ocupación en vivo");
            System.out.println("  [4] Diagnóstico perimetral estructurado SESE (3 tornos x 4 canales)");
            System.out.println("  [5] Activar protocolo de evacuación de emergencia");
            System.out.println("  [6] Cerrar jornada y apagar terminal");
            System.out.print("Seleccione una operación (1-6): ");

            int opcionMenu = 0;
            try {
                opcionMenu = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                opcionMenu = -1;
            }

            switch (opcionMenu) {
                case 1:
                    // TRÁNSITO INDIVIDUAL
                    totalFichajesProcesados++;
                    System.out.println("\n--- [MODO 1: TRÁNSITO INDIVIDUAL] ---");
                    System.out.print("Sentido del tránsito (E = Entrada, S = Salida): ");
                    String lineaSentido = teclado.nextLine().trim().toUpperCase();
                    char sentidoTransito = lineaSentido.isEmpty() ? 'E' : lineaSentido.charAt(0);

                    System.out.print("Nombre completo: ");
                    String nombrePersonaLimpio = teclado.nextLine().trim().toUpperCase();

                    if (sentidoTransito == 'S') {
                        if (personasEnCentro > 0) {
                            personasEnCentro--;
                            aforoDisponible++;
                            totalFichajesAutorizados++;
                            idUltimoFichaje++;
                            System.out.println(">> VEREDICTO: [SALIDA AUTORIZADA] Torno exterior abierto.");
                        } else {
                            totalFichajesDenegados++;
                            System.err.println(">> ALERTA: Imposible autorizar salida; recinto vacío.");
                        }
                    } else {
                        int terminalId = 0;
                        double tempVestibulo = 0.0;
                        String dniPersonaLimpio = "";
                        char perfilNormalizado = ' ';
                        boolean datosSintacticamenteCorrectos = true;
                        String motivoDenegacion = "";

                        try {
                            System.out.print("ID Terminal: ");
                            terminalId = Integer.parseInt(teclado.nextLine().trim());
                            System.out.print("Temperatura (ºC): ");
                            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());
                            System.out.print("DNI: ");
                            dniPersonaLimpio = teclado.nextLine().trim().toUpperCase();
                            System.out.print("Perfil (E/D/V/A): ");
                            String lineaPerfil = teclado.nextLine().trim();
                            perfilNormalizado = lineaPerfil.isEmpty() ? '?' : Character.toUpperCase(lineaPerfil.charAt(0));
                        } catch (NumberFormatException e) {
                            datosSintacticamenteCorrectos = false;
                            motivoDenegacion = "Error de formato numérico.";
                        }

                        boolean accesoConcedido;
                        if (!datosSintacticamenteCorrectos) {
                            accesoConcedido = false;
                            totalFichajesDenegados++;
                        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                            accesoConcedido = false;
                            motivoDenegacion = "Temperatura anómala.";
                            totalFichajesDenegados++;
                        } else if (aforoDisponible <= 0) {
                            accesoConcedido = false;
                            motivoDenegacion = "Aforo completo.";
                            totalFichajesDenegados++;
                        } else if (!esDocumentoValido(dniPersonaLimpio)) {
                            accesoConcedido = false;
                            motivoDenegacion = "DNI no reglamentario.";
                            totalFichajesDenegados++;
                        } else if (!esPerfilAutorizado(perfilNormalizado)) {
                            accesoConcedido = false;
```

```
                    motivoDenegacion = "Perfil no catalogado.";
                    totalFichajesDenegados++;
                } else {
                    accesoConcedido = true;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;

                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
                    minutosEstanciaAcumulados += 360;
                }

                int codigoToken = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                String token = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoToken, idUltimoFichaje);
                System.out.printf(">> VEREDICTO: %s [Token: %s]%n",
                        accesoConcedido ? "[ENTRADA AUTORIZADA]" : "[ENTRADA DENEGADA - " + motivoDenegacion + "]", token);
            }
            break;

        case 2:
            // RÁFAGA CON PARADA ESTRUCTURADA POR BANDERA EN CABECERA (SIN BREAK)
            System.out.println("\n--- [MODO 2: RÁFAGA CON PARADA ESTRUCTURADA EN CABECERA] ---");
            System.out.print("Personas en cola para esta ráfaga: ");
            int personasEnCola = 0;
            try {
                personasEnCola = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                personasEnCola = 0;
            }

            boolean vetoJudicialDetectado = false; // Bandera de estado en cabecera

            // La condición de continuidad gobierna el ciclo de forma exclusiva
            while (personasEnCola > 0 && aforoDisponible > 0 && !vetoJudicialDetectado) {
                if (idUltimoFichaje == 1044) {
                    System.out.println(">> [ALERTA JUDICIAL]: Credencial bloqueada. Suspendiendo cola en cabecera...");
                    vetoJudicialDetectado = true; // Provoca la terminación limpia en la siguiente evaluación
                } else {
                    totalFichajesProcesados++;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;
                    personasEnCola--;

                    double tempSimulada = 36.4 + (generadorVestibulo.nextDouble() * 0.5);
                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempSimulada);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempSimulada);
                    minutosEstanciaAcumulados += 360;
                }
            }
            System.out.printf(">> Estado tras ráfaga. Plazas disponibles: %d | Restantes en cola: %d%n",
                    aforoDisponible, personasEnCola);
            break;

        case 3:
            // TELEMETRÍA EN VIVO
            System.out.println("\n--- [MODO 3: MONITOR DE TELEMETRÍA] ---");
            int aforoTotal = personasEnCentro + aforoDisponible;
            double porcentajeOcupacion = calcularPorcentajeOcupacion(personasEnCentro, aforoTotal);
            System.out.printf("OCUPACIÓN: %d/%d (%6.2f %%) | VACANTES: %d%n",
                    personasEnCentro, aforoTotal, porcentajeOcupacion, aforoDisponible);
            System.out.printf("PICO MÁX: %.1f ºC | PICO MÍN: %.1f ºC%n",
                    Math.max(0.0, tempMaximaRegistrada), tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada);
            break;

        case 4:
            // DIAGNÓSTICO PERIMETRAL SESE CON BANDERAS EN CABECERA (DÍA 34)
            System.out.println("\n--- [MODO 4: INSPECCIÓN PERIMETRAL ESTRUCTURADA (SESE)] ---");
            boolean alarmaSabotajeDetectada = false; // Bandera compartida en ambas cabeceras

            // BUCLE EXTERIOR: Evaluación de índice Y bandera de estado
            for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS && !alarmaSabotajeDetectada; torno++) {
                String nombreAcceso = switch (torno) {
                    case 1 -> "Vestíbulo Central  | ";
                    case 2 -> "Puerta Norte Ciclos| ";
                    case 3 -> "Puerta Sur Talleres| ";
                    default -> "Acceso Perimetral  | ";
                };
                System.out.print(nombreAcceso);

                // BUCLE INTERIOR: Evaluación de canal Y bandera de estado
                for (int canal = 1; canal <= CANALES_POR_TORNO && !alarmaSabotajeDetectada; canal++) {

                    // Filtrado secuencial estructurado con if-else (sin continue)
                    if (torno == 2 && canal == 2) {
                        System.out.print("[REVISIÓN] ");
                    } else {
                        // Comprobación ordinaria de telemetría
                        boolean falloCritico = (torno == 3 && canal == 2 && false); // Simulación inactiva

                        if (falloCritico) {
                            System.out.print("[SABOTAJE] ");
                            alarmaSabotajeDetectada = true; // Finaliza limpiamente ambos bucles en cabecera
                        } else {
                            System.out.printf("[C%d: OK]   ", canal);
                        }
                    }
```

```
                        }
                        System.out.println(); // Fin de fila
                    }

                    if (alarmaSabotajeDetectada) {
                        System.out.println(">> [ALERTA DE SEGURIDAD]: Barrido perimetral detenido por fallo crítico.");
                    }
                    System.out.println("-------------------------------------------------------------------");
                    break;

                case 5:
                    // EVACUACIÓN CON WHILE
                    System.out.println("\n--- [MODO 5: EVACUACIÓN DE EMERGENCIA] ---");
                    while (personasEnCentro > 0) {
                        int evacuados = Math.min(personasEnCentro, 300);
                        personasEnCentro -= evacuados;
                        aforoDisponible += evacuados;
                        System.out.printf("  -> Desalojadas %d personas. Restantes: %d%n", evacuados, personasEnCentro);
                    }
                    System.out.println(">> RECINTO EVACUADO AL 100%.");
                    break;

                case 6:
                    // SALIDA
                    System.out.println("\n>> Desconectando terminal...");
                    sesionActiva = false;
                    break;

                default:
                    System.err.println(">> Opción no válida (1-6).");
                    break;
            }

        } while (sesionActiva);

        // AUDITORÍA FINAL
        System.out.println("\n===================================================================");
        System.out.println("          AUDITORÍA FINAL DE LA JORNADA · IES EL CAMINÀS            ");
        System.out.println("===================================================================");
        System.out.printf("OPERACIONES TOTALES:        %d%n", totalFichajesProcesados);
        System.out.printf("PASOS AUTORIZADOS:          %d%n", totalFichajesAutorizados);
        System.out.printf("PASOS DENEGADOS:            %d%n", totalFichajesDenegados);
        System.out.printf("OCUPACIÓN FINAL:            %d personas%n", personasEnCentro);
        System.out.println("ESTADO:                      Terminal apagado con éxito.");
        System.out.println("===================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión práctica, cada estudiante aplicará el principio SESE para gobernar la parada y el filtrado de bucles exclusivamente a través de banderas booleanas en cabecera y bifurcaciones estructuradas.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). PARADA EN CABECERA Y FILTRADO EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad e implementa una rutina de bucle gobernada por una bandera booleana en su cabecera y filtrado con `if-else` directo, sin utilizar sentencias de salto:

- **Aventura conversacional.** Modela una secuencia de combate automático o ráfaga de asaltos gobernada por la cabecera `while (vidaHeroe > 0 && vidaMonstruo > 0 && !huidaExitosa)`. Si el jugador activa un objeto de escape, muta la bandera `huidaExitosa = true` para finalizar el bucle por su propia condición; gestiona los turnos en los que el monstruo está aturdido mediante un bloque `if (!adversarioAturdido) { contraatacar(); } else { notificarAturdimiento(); }`.
- **Motor de recomendación.** En la rutina de procesamiento por lotes de contenidos, define la cabecera del bucle evaluando `while (quedanItems && recomendacionesEmitidas < CUPO_MAXIMO)`. El bucle debe concluir naturalmente en cuanto se alcance la cuota sin forzar interrupciones; gestiona los productos descatalogados dentro del cuerpo mediante una bifurcación `if (itemEnStock) { analizarAfinidad(); } else { registrarDescatalogado(); }`.
- **Simulador de físicas 2D.** Estructura el bucle de simulación cinemática condicionando su cabecera a `while (tiempoActual < TIEMPO_MAX && !impactoCritico)`. Si la posición del proyectil detecta un obstáculo indestructible, activa `impactoCritico = true` para concluir la simulación limpiamente; gestiona las pérdidas de aceleración menores con un bloque `if (aceleracionRelevante) { recalcularVectores(); }`.
- **Bóveda de contraseñas.** Construye el algoritmo de auditoría de credenciales mediante un bucle `for` que incorpore en su cabecera `for (int i = 0; i < password.length() && !patronInseguroDetectado; i++)`. Si se detecta un carácter prohibido o secuencia maliciosa, activa la bandera booleana para salir por cabecera, procesando los caracteres válidos en la rama afirmativa del condicional.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). BÚSQUEDA LINEAL ESTRUCTURADA CON BANDERA EN CABECERA

El algoritmo clásico de búsqueda lineal es el mejor ejemplo para contrastar la programación estructurada frente a los escapes incondicionales:

1. **La cabecera compuesta:** Diseña un bucle `for` que busque un valor clave en un rango numérico sin usar `break`:

```
boolean elementoEncontrado = false;
int posicionHallada = -1;

// La cabecera garantiza la terminación por fin de rango O por éxito en la búsqueda
for (int i = 0; i < TAMANIO && !elementoEncontrado; i++) {
    if (coincideCriterio(i)) {
        elementoEncontrado = true;
        posicionHallada = i;
    }
}
```

2. **Ventaja SESE:** Tras el bucle, el flujo converge en un único punto predecible donde una simple comprobación (`if (elementoEncontrado)`) permite gestionar el resultado sin incertidumbres sobre el estado de la variable índice.
3. **Refactorización en tu proyecto:** Aplica este patrón en la búsqueda de elementos de tu especialidad (localizar una celda libre en la aventura, un contenido afín en el recomendador, un punto de colisión en las físicas o una clave coincidente en la bóveda).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). AUDITORÍA DE INVARIANTES Y VERIFICACIÓN DE FLUJO SESE

La programación estructurada permite verificar formalmente el comportamiento de un bucle mediante sus invariantes matemáticas:

1. **Definición de la invariante:**
En un comentario al final de tu clase, formaliza la invariante de tu bucle principal: demuestra que antes de entrar, en cada paso y al salir, la expresión lógica de cabecera gobierna con precisión el ciclo de vida de los datos: $$\text{Condición de parada} = \neg (\text{índice} < \text{LÍMITE} \land \neg \text{banderaParada})$$ Aplicando De Morgan: $$\text{Condición de parada} = (\text{índice} \ge \text{LÍMITE}) \lor \text{banderaParada}$$
2. **Prueba de caja blanca:**
Rastrea el valor de la bandera booleana en una ejecución donde se fuerce la parada anticipada y demuestra que el bucle no realiza ninguna iteración adicional tras mutar la bandera a `true`.
3. **Certificación:**
Documenta en un comentario de tu clase cómo el respeto al principio SESE reduce la deuda técnica, elimina caminos muertos (*Dead Code*) y simplifica el análisis de cobertura de pruebas unitarias (*Branch Coverage*) de cara a la sesión de depuración final del sprint.

4.3.4 Día 35 - 2 sesiones

---

1. Caso guía en AzaharTech

Son las 15:00 h del **jueves 12 de noviembre de 2026**. Los equipos de desarrollo de **AzaharTech** en Castellón de la Plana afrontan la última jornada lectiva del **Sprint 3: «El laberinto de la lógica»**. Tras 12 sesiones intensivas, el software de control de accesos para el **IES El Caminàs** ha dejado de ser un script lineal para convertirse en una aplicación corporativa robusta, interactiva y estructurada.

En la pantalla central, **Laia Claramunt**, acompañada de **Alba Torres** y **Pau Ferrer**, proyecta el hito de entrega del sprint:

«Equipo, hemos completado la travesía de este sprint. Si miramos atrás, el lunes 26 de octubre partíamos de un programa ciego que siempre abría el torno. Hoy entregamos la versión ***`v3.0 Oficial`*** *:* * Un sistema gobernado por estructuras de selección (`if-else` , `switch`) y predicados modulares. * Blindado defensivamente ante fallos de formato con `try-catch` . * Un demonio de servicio continuo estructurado con `do-while` y centinela de apagado. * Procesamiento de colas por ráfaga con `while` y protocolos de evacuación de emergencia. * Autodiagnóstico secuencial de sensores con `for` y supervisión perimetral matricial con bucles anidados. * Y todo ello respetando escrupulosamente el principio **SESE (*Single Entry, Single Exit*)**, gobernando cada iteración exclusivamente a través de banderas booleanas en cabecera, sin un solo `break` ni `continue` descontrolado en los bucles.

Hoy dedicamos la sesión al paso más importante de cualquier entrega profesional: la ***auditoría de integración, depuración guiada con puntos de ruptura (breakpoints) en IntelliJ*** *y la certificación de la versión final antes de etiquetar el commit definitivo en Git».*

---

2. Fundamento teórico: depuración formal, puntos de ruptura e inspección de memoria

La **depuración (*Debugging*)** es el proceso metódico de ejecutar un programa paso a paso bajo la supervisión de un depurador para verificar que las variables en memoria mutan exactamente según lo previsto.

```
flowchart TD
    Inicio([Lanzar aplicación en Modo Debug: Shift + F9]) --> BP["<b>Punto de ruptura (Breakpoint):</b><br/>La JVM congela el hilo de ejecución"]

    BP --> Inspeccion["<b>Inspección de memoria:</b><br/>• Pestaña Variables: valores en el Stack<br/>• Watch / Evaluate: evaluar expresiones al vuelo"]

    Inspeccion --> Accion{"Acción del desarrollador"}

    Accion -- "F8 (Step Over)" --> LineaSig["<b>Avanzar línea a línea</b><br/>Ejecuta la instrucción sin entrar en métodos"]
    Accion -- "F7 (Step Into)" --> EntraMetodo["<b>Entrar al método</b><br/>Salta al interior de predicados modulares"]
    Accion -- "F9 (Resume)" --> SiguienteBP["<b>Reanudar ejecución</b><br/>Avanza hasta el siguiente Breakpoint o fin"]

    LineaSig --> Inspeccion
    EntraMetodo --> Inspeccion
    SiguienteBP --> Fin([Certificación de calidad del software])
```

A. TIPOS DE PUNTOS DE RUPTURA (*BREAKPOINTS*) EN INTELLIJ IDEA

1. **Punto de ruptura de línea (*Line Breakpoint*):** Congela el programa justo antes de ejecutar la línea seleccionada. Permite examinar el estado de las variables locales en la memoria de pila (*Stack Frame*).
2. **Punto de ruptura condicional (*Conditional Breakpoint*):** Solo detiene la ejecución si se cumple una expresión booleana (por ejemplo, `aforoDisponible == 0` o `idUltimoFichaje == 1044`). Evita tener que avanzar manualmente por cientos de vueltas irrelevantes de un bucle.

B. NAVEGACIÓN EN DEPURACIÓN

Atajo en IntelliJ	Comando	Función técnica
`F8`	*Step Over*	Ejecuta la línea actual y pasa a la siguiente sin entrar en el detalle de los métodos invocados.
`F7`	*Step Into*	Salta al interior del método invocado (ideal para auditar la lógica de un método predicado).
`Shift + F8`	*Step Out*	Completa la ejecución del método actual y regresa a la línea llamante.
`F9`	*Resume Program*	Reanuda la ejecución continua hasta encontrar el siguiente *breakpoint*.
`Alt + F8`	*Evaluate Expression*	Abre una consola interactiva para calcular expresiones lógicas en caliente sin modificar el código.

C. LA TABLA DE AUDITORÍA DE VARIABLES (*TRACE MATRIX*)

Durante la depuración formal de un bucle, los ingenieros registran en una **tabla de traza** cómo mutan los acumuladores y banderas en cada ciclo, verificando que la condición de salida evalúa a `false` en el momento matemático exacto:

Iteración	Variable índice	`personasEnCentro`	`aforoDisponible`	Bandera de estado	Evaluación cabecera
0 (Inicio)	—	1.195	5	`parada = false`	—
1	`lote = 1`	1.196	4	`parada = false`	`true`
2	`lote = 2`	1.197	3	`parada = false`	`true`
…	…	…	…	…	…
5 (Límite)	`lote = 5`	1.200	0	`parada = true`	`false` $ \rightarrow$ **Salida limpia SESE**

---

3. Evolución del caso guía a `ControlAccesoQR v3.0` (Versión Final Sprint 3)

Alcanzamos el hito `v3.0 Oficial`. Esta versión consolida toda la arquitectura del Sprint 3 en una única clase robusta, limpia y completamente operativa.

A. CONSOLIDACIÓN RESPECTO A V2.11 (`diff`)

```
  /**
   * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
-  * Versión 2.11: Consolidación de estructuras de salto
+  * Versión 3.0 Oficial: Cierre y entrega final de Sprint 3 («El laberinto de la lógica»)
   *
-  * @version 2.11 (Noviembre 2026)
+  * @version 3.0 Oficial (Noviembre 2026)
   */

  // ============================================================================
  // ARQUITECTURA FINAL CONSOLIDADA (Sprint 3 Release)
  // ============================================================================
  // • Estructuras de selección múltiple (if-else jerárquico + switch clásico)
  // • Blindaje defensivo ante excepciones tipográficas (try-catch)
  // • Servicio continuo por menú interactivo (do-while + centinela de apagado)
  // • Procesamiento por ráfaga y evacuación progresiva (while)
  // • Diagnóstico secuencial y matriz perimetral (for simple y for anidado)
  // • Principio SESE riguroso: parada exclusiva por banderas en cabecera
```

B. FOCO EN LA CONSOLIDACIÓN: ARQUITECTURA MODULAR INTEGRAL

Observa cómo el método `main` coordina limpiamente todas las operaciones del terminal delegando las reglas en métodos predicado, mientras los bucles gestionan la persistencia y la salida ordenada:

```
// ------------------------------------------------------------------------
// Orquestación del menú interactivo consolidado v3.0
// ------------------------------------------------------------------------
do {
    // 1. Despliegue del panel de control multipropósito (opciones 1 a 6)
    // 2. Captura vigilada de opción con try-catch
    // 3. Despacho modular mediante sentencia switch:
    //    - Opción 1: Tránsito individual bidireccional (Entrada/Salida con guardias)
    //    - Opción 2: Ráfaga de cola en vestíbulo (bucle while con bandera judicial)
    //    - Opción 3: Monitor de telemetría y extremos térmicos (Math.max / Math.min)
    //    - Opción 4: Diagnóstico secuencial y matriz perimetral (bucles for anidados)
    //    - Opción 5: Protocolo de evacuación de emergencia (bucle while de drenaje)
    //    - Opción 6: Cierre ordenado de jornada (sesionActiva = false)
} while (sesionActiva);

// 4. Emisión de informe consolidado de auditoría final de sesión
```

C. CÓDIGO FUENTE COMPLETO INTEGRADO

Desplegar archivo completo ControlAccesoQR.java (v3.0 Oficial - Release Sprint 3)

```java
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 3.0 Oficial:
 * Cierre y consolidación final del Sprint 3 («El laberinto de la lógica»).
 * Integra la totalidad de estructuras de control de flujo de la plataforma:
 * - Selección condicional jerárquica if-else y múltiple con switch clásico.
 * - Programación defensiva con captura de excepciones mediante try-catch.
 * - Ciclo continuo de servicio desatendido mediante bucle post-test do-while.
 * - Procesamiento de colas por ráfaga y vaciado de emergencia con bucle while.
 * - Autodiagnóstico secuencial y matriz perimetral (3x4) con bucles for anidados.
 * - Cumplimiento estricto del principio SESE (paradas mediante banderas en cabecera).
 * - Algoritmos de agregación en tiempo real: picos térmicos, medias y aforo.
 *
 * @author Equipo AzaharTech (Alba Torres, Pau Ferrer, Laia Claramunt)
 * @version 3.0 Oficial (Noviembre 2026)
 * @since JDK 21 LTS
 *
 */

import java.util.Scanner;
import java.util.Random;
import java.time.LocalTime;
import java.time.LocalDate;
import java.time.Duration;
import java.time.DateTimeException;

public class ControlAccesoQR {

    // ------------------------------------------------------------------------
    // 1. CONSTANTES INMUTABLES DEL SISTEMA (configuración corporativa)
    // ------------------------------------------------------------------------
    final static String NOMBRE_CENTRO = "IES El Caminàs (Castellón)";
    final static String PREFIJO_CENTRO = "CAMINAS";
    final static String RUTA_LOGS = ".\\caminas\\terminal\\logs";

    final static int MINUTOS_POR_HORA = 60;
    final static int SEGUNDOS_POR_HORA = 3600;
    final static int SEGUNDOS_POR_MINUTO = 60;
    final static int JORNADA_BASE_MINUTOS = 480; // 8 horas
    final static double FACTOR_PORCENTAJE = 100.0;

    final static int ID_TERMINAL_PUERTA_NORTE = 102;
    final static int BASE_TOKEN_SEGURIDAD = 1000;
    final static int RANGO_TOKEN_SEGURIDAD = 9000;
    final static long SEMILLA_CALIBRACION = 987654321L;

    final static double TEMP_MIN_SEGURA = 15.0;
    final static double TEMP_MAX_SEGURA = 35.0;
    final static double MAX_VARIACION_TERMICA = 0.5;
    final static int LONGITUD_DNI_ESTANDAR = 9;

    // Umbrales biométricos y capacidad
    final static double TEMP_CORPORAL_MAX_FIEBRE = 37.5;
    final static double TEMP_CORPORAL_MIN_FALLO = 35.0;

    // Control de puntualidad y hardware perimetral
    final static LocalTime HORA_LIMITE_PUNTUALIDAD = LocalTime.of(8, 15);
    final static int NUM_SENSORES_TORNO = 8;
    final static int HORAS_TURNO_LECTIVO = 6;
    final static int TOTAL_TORNOS_CAMPUS = 3;
    final static int CANALES_POR_TORNO = 4;

    // ------------------------------------------------------------------------
    // 2. MÉTODOS ESTÁTICOS PROPIOS (modularización de la lógica del sistema)
    // ------------------------------------------------------------------------

    public static int calcularEstanciaMinutos(LocalTime entrada, LocalTime salida) {
        return (int) Duration.between(entrada, salida).toMinutes();
    }

    public static double calcularPorcentajeOcupacion(int personas, int aforoTotal) {
        return ((double) personas / aforoTotal) * FACTOR_PORCENTAJE;
    }

    public static String construirTokenQR(String prefijo, String dni, int terminalId, int codigoSec, int idFichaje) {
        String token = prefijo + "-" + dni;
        token += "-T" + terminalId;
        token += "-SEC#" + codigoSec;
        token += "-REG" + idFichaje;
        return token;
    }

    public static double calcularDistanciaDiagonal(double x, double y) {
        return Math.sqrt(Math.pow(x, 2.0) + Math.pow(y, 2.0));
    }

    public static String obtenerDescripcionPerfil(char perfil) {
        String descripcion;
        switch (perfil) {
            case 'E':
                descripcion = "Estudiante Oficial";
                break;
            case 'D':
```

```
                descripcion = "Docente / Claustro";
                break;
            case 'V':
                descripcion = "Visita Externa / Proveedor";
                break;
            case 'A':
                descripcion = "Personal de Administración y Servicios (PAS)";
                break;
            default:
                descripcion = "Perfil No Catalogado";
                break;
        }
        return descripcion;
    }

    public static String determinarZonaAutorizada(char perfil) {
        String zona;
        switch (perfil) {
            case 'E':
                zona = "Aulas Docentes y Talleres de Informática";
                break;
            case 'D':
                zona = "Departamentos Didácticos, Aulas y Sala de Profesores";
                break;
            case 'V':
                zona = "Secretaría, Conserjería y Despachos de Dirección";
                break;
            case 'A':
                zona = "Área de Gestión Administrativa y Archivo Central";
                break;
            default:
                zona = "Acceso Denegado / Zona No Autorizada";
                break;
        }
        return zona;
    }

    public static int obtenerEstanciaMaximaPorPerfil(char perfil) {
        int estanciaMinutos;
        switch (perfil) {
            case 'E':
                estanciaMinutos = 360;
                break;
            case 'D':
                estanciaMinutos = 480;
                break;
            case 'V':
                estanciaMinutos = 120;
                break;
            case 'A':
                estanciaMinutos = 420;
                break;
            default:
                estanciaMinutos = 0;
                break;
        }
        return estanciaMinutos;
    }

    // ------------------------------------------------------------------------
    // 3. MÉTODOS PREDICADO BOOLEANOS (reglas de negocio desacopladas)
    // ------------------------------------------------------------------------

    public static boolean esTemperaturaCorporalSegura(double temperatura) {
        return (temperatura >= TEMP_CORPORAL_MIN_FALLO && temperatura <= TEMP_CORPORAL_MAX_FIEBRE);
    }

    public static boolean esDocumentoValido(String dni) {
        return (dni != null && dni.length() == LONGITUD_DNI_ESTANDAR);
    }

    public static boolean esPerfilAutorizado(char perfil) {
        boolean autorizado;
        switch (perfil) {
            case 'E':
            case 'D':
            case 'V':
            case 'A':
                autorizado = true;
                break;
            default:
                autorizado = false;
                break;
        }
        return autorizado;
    }

    // ------------------------------------------------------------------------
    // 4. MÉTODO PRINCIPAL: CONSOLA INTEGRAL v3.0 (RELEASE SPRINT 3)
    // ------------------------------------------------------------------------
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        Random generadorVestibulo = new Random();
        Random generadorPuertaNorte = new Random();
        Random generadorCalibracion = new Random(SEMILLA_CALIBRACION);

        LocalDate fechaActual = LocalDate.now();

        // Estado persistente del campus
```

```
        int personasEnCentro = 1190;
        int aforoDisponible = 10;
        int idUltimoFichaje = 1042;

        int totalFichajesProcesados = 0;
        int totalFichajesAutorizados = 0;
        int totalFichajesDenegados = 0;

        // Acumuladores estadísticos
        double tempMaximaRegistrada = -1.0;
        double tempMinimaRegistrada = 100.0;
        int minutosEstanciaAcumulados = 0;

        boolean sesionActiva = true;

        System.out.println("========================================================================");
        System.out.println("   AZAHARTECH · CONTROL INTEGRAL DE ACCESOS - IES EL CAMINÀS          ");
        System.out.println("   Versión 3.0 Oficial (Release Sprint 3: Arquitectura Consolidada)   ");
        System.out.println("========================================================================");

        // BUCLE PRINCIPAL DE GESTIÓN (MENÚ DE OPERACIONES)
        do {
            System.out.println("\n------------------------------------------------------------------------");
            System.out.println("                    PANEL DE OPERACIONES DEL TORNO                      ");
            System.out.println("------------------------------------------------------------------------");
            System.out.println("  [1] Registrar tránsito individual (Entrada / Salida)");
            System.out.println("  [2] Procesar cola en ráfaga (Modo Burst con parada en cabecera)");
            System.out.println("  [3] Consultar telemetría y métricas de ocupación en vivo");
            System.out.println("  [4] Diagnóstico perimetral estructurado SESE (3 tornos x 4 canales)");
            System.out.println("  [5] Activar protocolo de evacuación de emergencia");
            System.out.println("  [6] Cerrar jornada y apagar terminal");
            System.out.print("Seleccione una operación (1-6): ");

            int opcionMenu = 0;
            try {
                opcionMenu = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                opcionMenu = -1;
            }

            switch (opcionMenu) {
                case 1:
                    // TRÁNSITO INDIVIDUAL BIDIRECCIONAL
                    totalFichajesProcesados++;
                    System.out.println("\n--- [MODO 1: TRÁNSITO INDIVIDUAL] ---");
                    System.out.print("Sentido del tránsito (E = Entrada, S = Salida): ");
                    String lineaSentido = teclado.nextLine().trim().toUpperCase();
                    char sentidoTransito = lineaSentido.isEmpty() ? 'E' : lineaSentido.charAt(0);

                    System.out.print("Nombre completo: ");
                    String nombrePersonaLimpio = teclado.nextLine().trim().toUpperCase();

                    if (sentidoTransito == 'S') {
                        if (personasEnCentro > 0) {
                            personasEnCentro--;
                            aforoDisponible++;
                            totalFichajesAutorizados++;
                            idUltimoFichaje++;
                            System.out.println(">> VEREDICTO: [SALIDA AUTORIZADA] Torno exterior abierto.");
                        } else {
                            totalFichajesDenegados++;
                            System.err.println(">> ALERTA: Imposible autorizar salida; recinto vacío.");
                        }
                    } else {
                        int terminalId = 0;
                        double tempVestibulo = 0.0;
                        String dniPersonaLimpio = "";
                        char perfilNormalizado = ' ';
                        boolean datosSintacticamenteCorrectos = true;
                        String motivoDenegacion = "";

                        try {
                            System.out.print("ID Terminal: ");
                            terminalId = Integer.parseInt(teclado.nextLine().trim());
                            System.out.print("Temperatura (ºC): ");
                            tempVestibulo = Double.parseDouble(teclado.nextLine().trim());
                            System.out.print("DNI: ");
                            dniPersonaLimpio = teclado.nextLine().trim().toUpperCase();
                            System.out.print("Perfil (E/D/V/A): ");
                            String lineaPerfil = teclado.nextLine().trim();
                            perfilNormalizado = lineaPerfil.isEmpty() ? '?' : Character.toUpperCase(lineaPerfil.charAt(0));
                        } catch (NumberFormatException e) {
                            datosSintacticamenteCorrectos = false;
                            motivoDenegacion = "Error de formato numérico.";
                        }

                        boolean accesoConcedido;
                        if (!datosSintacticamenteCorrectos) {
                            accesoConcedido = false;
                            totalFichajesDenegados++;
                        } else if (!esTemperaturaCorporalSegura(tempVestibulo)) {
                            accesoConcedido = false;
                            motivoDenegacion = "Temperatura anómala.";
                            totalFichajesDenegados++;
                        } else if (aforoDisponible <= 0) {
                            accesoConcedido = false;
                            motivoDenegacion = "Aforo completo.";
                            totalFichajesDenegados++;
                        } else if (!esDocumentoValido(dniPersonaLimpio)) {
                            accesoConcedido = false;
```

```
                    motivoDenegacion = "DNI no reglamentario.";
                    totalFichajesDenegados++;
                } else if (!esPerfilAutorizado(perfilNormalizado)) {
                    accesoConcedido = false;
                    motivoDenegacion = "Perfil no catalogado.";
                    totalFichajesDenegados++;
                } else {
                    accesoConcedido = true;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;

                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempVestibulo);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempVestibulo);
                    minutosEstanciaAcumulados += 360;
                }

                int codigoToken = BASE_TOKEN_SEGURIDAD + generadorVestibulo.nextInt(RANGO_TOKEN_SEGURIDAD);
                String token = construirTokenQR(PREFIJO_CENTRO, dniPersonaLimpio, terminalId, codigoToken, idUltimoFichaje);
                System.out.printf(">> VEREDICTO: %s [Token: %s]%n",
                        accesoConcedido ? "[ENTRADA AUTORIZADA]" : "[ENTRADA DENEGADA - " + motivoDenegacion + "]", token);
            }
            break;

        case 2:
            // RÁFAGA DE COLA CON GUARDIA EN CABECERA (PRINCIPIO SESE)
            System.out.println("\n--- [MODO 2: RÁFAGA CON PARADA ESTRUCTURADA EN CABECERA] ---");
            System.out.print("Personas en cola para esta ráfaga: ");
            int personasEnCola = 0;
            try {
                personasEnCola = Integer.parseInt(teclado.nextLine().trim());
            } catch (NumberFormatException e) {
                personasEnCola = 0;
            }

            boolean vetoJudicialDetectado = false;

            while (personasEnCola > 0 && aforoDisponible > 0 && !vetoJudicialDetectado) {
                if (idUltimoFichaje == 1044) {
                    System.out.println(">> [ALERTA JUDICIAL]: Credencial bloqueada. Suspendiendo cola en cabecera...");
                    vetoJudicialDetectado = true;
                } else {
                    totalFichajesProcesados++;
                    totalFichajesAutorizados++;
                    personasEnCentro++;
                    aforoDisponible--;
                    idUltimoFichaje++;
                    personasEnCola--;

                    double tempSimulada = 36.4 + (generadorVestibulo.nextDouble() * 0.5);
                    tempMaximaRegistrada = Math.max(tempMaximaRegistrada, tempSimulada);
                    tempMinimaRegistrada = Math.min(tempMinimaRegistrada, tempSimulada);
                    minutosEstanciaAcumulados += 360;
                }
            }
            System.out.printf(">> Estado tras ráfaga. Plazas disponibles: %d | Restantes en cola: %d%n",
                    aforoDisponible, personasEnCola);
            break;

        case 3:
            // MONITOR DE TELEMETRÍA EN VIVO
            System.out.println("\n--- [MODO 3: MONITOR DE TELEMETRÍA EN VIVO] ---");
            int aforoTotal = personasEnCentro + aforoDisponible;
            double porcentajeOcupacion = calcularPorcentajeOcupacion(personasEnCentro, aforoTotal);
            double estanciaMedia = totalFichajesAutorizados > 0
                    ? ((double) minutosEstanciaAcumulados / totalFichajesAutorizados) : 0.0;

            System.out.printf("OCUPACIÓN ACTUAL:         %d/%d personas (%6.2f %%)%n", personasEnCentro, aforoTotal, porcentajeOcupacion);
            System.out.printf("PLAZAS LIBRES:            %d vacantes disponibles%n", aforoDisponible);
            System.out.printf("TRANSACCIONES TOTALES:    %d registros (%d autorizados, %d denegados)%n",
                    totalFichajesProcesados, totalFichajesAutorizados, totalFichajesDenegados);
            System.out.printf("PICO TÉRMICO MÁXIMO:      %.1f ºC%n", Math.max(0.0, tempMaximaRegistrada));
            System.out.printf("PICO TÉRMICO MÍNIMO:      %.1f ºC%n", tempMinimaRegistrada > 50.0 ? 0.0 : tempMinimaRegistrada);
            System.out.printf("ESTANCIA PROMEDIO:        %.1f minutos (Total acumulado: %d min)%n",
                    estanciaMedia, minutosEstanciaAcumulados);
            break;

        case 4:
            // DIAGNÓSTICO SECUENCIAL Y MATRIZ PERIMETRAL SESE
            System.out.println("\n--- [MODO 4: INSPECCIÓN PERIMETRAL ESTRUCTURADA (SESE)] ---");
            boolean alarmaSabotajeDetectada = false;

            for (int torno = 1; torno <= TOTAL_TORNOS_CAMPUS && !alarmaSabotajeDetectada; torno++) {
                String nombreAcceso = switch (torno) {
                    case 1 -> "Vestíbulo Central  | ";
                    case 2 -> "Puerta Norte Ciclos| ";
                    case 3 -> "Puerta Sur Talleres| ";
                    default -> "Acceso Perimetral  | ";
                };
                System.out.print(nombreAcceso);

                for (int canal = 1; canal <= CANALES_POR_TORNO && !alarmaSabotajeDetectada; canal++) {
                    if (torno == 2 && canal == 2) {
                        System.out.print("[REVISIÓN] ");
                    } else {
                        boolean falloCritico = (torno == 3 && canal == 2 && false);

                        if (falloCritico) {
                            System.out.print("[SABOTAJE] ");
```

```
                                alarmaSabotajeDetectada = true;
                            } else {
                                System.out.printf("[C%d: OK]   ", canal);
                            }
                        }
                    }
                    System.out.println();
                }

                if (alarmaSabotajeDetectada) {
                    System.out.println(">> [ALERTA DE SEGURIDAD]: Barrido perimetral detenido por fallo crítico.");
                }
                System.out.println("-----------------------------------------------------------------------");
                break;

            case 5:
                // PROTOCOLO DE EVACUACIÓN CON WHILE DE DRENAJE
                System.out.println("\n--- [MODO 5: PROTOCOLO DE EVACUACIÓN DE EMERGENCIA] ---");
                if (personasEnCentro == 0) {
                    System.out.println(">> El campus ya se encuentra completamente vacío.");
                } else {
                    int loteSalidaSimultanea = 300;
                    int oleada = 1;

                    while (personasEnCentro > 0) {
                        int evacuadosLote = Math.min(personasEnCentro, loteSalidaSimultanea);
                        personasEnCentro -= evacuadosLote;
                        aforoDisponible += evacuadosLote;

                        System.out.printf("  -> Oleada #%d: Desalojadas %d personas por salidas de emergencia (Ocupantes restantes: %d)%n",
                                oleada, evacuadosLote, personasEnCentro);
                        oleada++;
                    }
                    System.out.println(">> CERTIFICACIÓN DE SEGURIDAD: Recinto 100% desalojado. Aforo íntegro restaurado.");
                }
                break;

            case 6:
                // APAGADO Y SALIDA
                System.out.println("\n>> Desconectando terminal de presencia de forma ordenada...");
                sesionActiva = false;
                break;

            default:
                System.err.println(">> Error: Opción no válida. Seleccione una operación entre 1 y 6.");
                break;
            }

        } while (sesionActiva);

        // AUDITORÍA FINAL AL APAGAR
        System.out.println("\n=======================================================================");
        System.out.println("          AUDITORÍA FINAL DE LA JORNADA · IES EL CAMINÀS              ");
        System.out.println("=======================================================================");
        System.out.printf("TOTAL OPERACIONES GESTIONADAS: %d%n", totalFichajesProcesados);
        System.out.printf("ACCESOS AUTORIZADOS (TOTAL):   %d%n", totalFichajesAutorizados);
        System.out.printf("ACCESOS DENEGADOS (BLOQUEO):   %d%n", totalFichajesDenegados);
        System.out.printf("OCUPACIÓN FINAL REGISTRADA:    %d personas%n", personasEnCentro);
        System.out.println("ESTADO DEL DISPOSITIVO:        Terminal desconectado de forma segura.");
        System.out.println("=======================================================================");

        teclado.close();
    }
}
```

4. Segunda sesión: dojo de entrenamiento y katas de código

En esta sesión final de consolidación, cada estudiante auditará su proyecto técnico utilizando el depurador visual de IntelliJ y certificará la entrega formal de la versión `v3.0`.

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). CONSOLIDACIÓN Y CIERRE V3.0 EN PROYECTOS ASIGNADOS

Abre el archivo `pr/src/MiProyecto.java` de tu especialidad y unifica todas las estructuras de control desarrolladas durante las semanas 7, 8 y 9 en una versión final consolidada `v3.0`:

- **Aventura conversacional.** Integra el menú de combate interactivo (`do-while`), la gestión de turnos con ráfagas de daño continuado (`while`), el muestreo secuencial de impactos mágicos (`for`) y la parada limpia mediante banderas de vida o huida sin romper el principio SESE, emitiendo el cuadro de honor al término del duelo.
- **Motor de recomendación.** Consolida el menú de administración del catálogo, la auditoría de afinidades cruzadas con matrices bidimensionales (`for` anidados), el procesamiento de colas de ítems pendientes (`while`) y los acumuladores de picos y promedios de compatibilidad, presentando el informe consolidado al salir.
- **Simulador de físicas 2D.** Unifica el panel de control cinemático, la simulación temporal paso a paso (`while`), la cuadrícula espacial cartesiana para la representación del impacto (`for` anidados) y la amortiguación de energía con protección ante valores negativos de velocidad.
- **Bóveda de contraseñas.** Integra la consola de seguridad de la bóveda, la tarjeta de coordenadas criptográfica en matriz bidimensional, la auditoría de lotes de claves con parada defensiva ante secuencias maliciosas y el registro dinámico de métricas de entropía media sin colecciones.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). SESIÓN PRÁCTICA DE DEPURACIÓN FORMAL EN INTELLIJ IDEA

Abre tu proyecto en IntelliJ IDEA y ejecuta una sesión de depuración visual para certificar el comportamiento de la memoria:

1. **Colocación de puntos de ruptura (*Breakpoints*):**
   Coloca un punto de ruptura en la cabecera del bucle principal de tu menú interactivo y otro punto de ruptura condicional dentro de la rutina de procesamiento de cola o combate (haciendo clic derecho sobre el punto rojo e indicando una condición booleana, por ejemplo `contador == 3`).
2. **Navegación paso a paso:**
   Inicia la aplicación en modo depuración (`Shift + F9`):
3. Utiliza `F8` **(*Step Over*)** para avanzar línea a línea observando cómo cambian los valores de las variables en la pestaña *Variables* del depurador.
4. Utiliza `F7` **(*Step Into*)** cuando el cursor se sitúe sobre una llamada a un método predicado booleano, comprobando cómo se crea un nuevo marco en la pila (*Call Stack*) y cómo se evalúa la regla de negocio.
5. **Evaluación de expresiones al vuelo:**
   Durante la pausa en un breakpoint, pulsa `Alt + F8` **(*Evaluate Expression*)** y evalúa manualmente expresiones booleanas complejas (por ejemplo, comprobar si una condición compuesta evalúa a `true` o `false` antes de que la ejecute la CPU).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). CERTIFICACIÓN DE ENTREGA Y AUDITORÍA DOD (*DEFINITION OF DONE*)

Como cierre formal del Sprint 3, audita la calidad técnica de tu código conforme a la lista de verificación corporativa de AzaharTech:

1. **Lista de comprobación de calidad (*Definition of Done*):**
2. [ ] **Compilación limpia:** Cero errores y cero advertencias (*warnings*) de variables sin usar o tipos incompatibles.
3. [ ] **Principio SESE estricto:** Ningún bucle (`for` , `while` , `do-while`) contiene sentencias `break` o `continue` para forzar salidas irregulares.
4. [ ] **Protección ante caídas:** Todas las entradas interactivas por teclado están protegidas mediante bloques `try-catch` ante datos no numéricos.
5. [ ] **Complejidad contenida:** Los métodos auxiliares predicado descargan la lógica del método `main` , manteniendo la complejidad ciclomática controlada.
6. **Etiquetado formal en Git:**
   Abre la terminal integrada de IntelliJ y realiza el commit y etiquetado final de cierre del sprint:

```
git add pr/src/
git commit -m "feat(pr): entrega consolidada v3.0 - fin de sprint 3 (estructuras de control)"
git tag -a v3.0 -m "Release v3.0: Uso completo de estructuras de control y depuracion"
```

7. **Certificación:**
   Redacta un comentario técnico final en la cabecera de tu clase resumiendo los resultados de la auditoría de depuración y declarando el software listo para el pase al **Sprint 4 (Desarrollo y diseño de clases propias)**.